

Unit 10

DATA TYPES AND STRUCTURES

182 items · 1605 marks

Syllabus sections in this unit

- 10.1 Data Types and Records
- 10.2 Arrays
- 10.3 Files
- 10.4 Introduction to Abstract Data Types (ADT)

Unit 10: Data Types and Structures

1. M/J 26/P21/Q2/a(i)

2 A washing machine contains:

- two trays for the washing powder
- a user interface display
- five sensors.

The washing machine control program includes the following variables:

variable	description
MainsOn	TRUE when the mains power is on
WaterOn	TRUE when the water supply to the machine is turned on
DisplayOn	TRUE when the user interface display is active
Tray1	TRUE when it detects washing powder in tray 1
Tray2	TRUE when it detects washing powder in tray 2

(a) (i) Identify the data type used for the **five** variables.

[1]

2. M/J 26/P21/Q6/c

- 6** Fence parts are manufactured by a number of similar machines. The length of the part must fall within a certain range. Parts **not** within the length range are rejected.

The data for each rejected part forms a data packet made up of three data items, each separated by the '/' character delimiter. The format of a data packet is:

<length>/<machine ID>/<description of the fault>

- The length is 3 digits (in millimetres).
- The machine ID is a single character ('A', 'B', or 'C').
- The description of the fault is a variable length string.

A pseudocode program will process the data for several data packets.

The program contains the statement:

DataPacket ← "563/B/the fence length is too long"

- (a)** Complete the **two** pseudocode expressions which extract the appropriate data item.

PartLength ←

MachineID ←

[2]

- (b)** The program will create a module `RejectDescription()` to return the fault description.

Write the pseudocode for the module `RejectDescription()`

[4]

- (c)** The data packets for all rejected parts are saved in this format in a text file `Rejects.txt`. The file stores one data packet per line.

Write pseudocode to:

- find all the rejected parts with a length greater than 560 millimetres and that were produced on machine 'B' or machine 'C'
- output the description for each matching rejected part.

Your pseudocode will include a module call to `RejectDescription()` and the answers given for **6(a)**.

[6]

3. M/J 26/P21/Q7/a,b,c

- 7** Different data structures will be suitable for different computing applications.

- (a) (i)** Identify a computing application where a queue data structure is suitable.

[1]

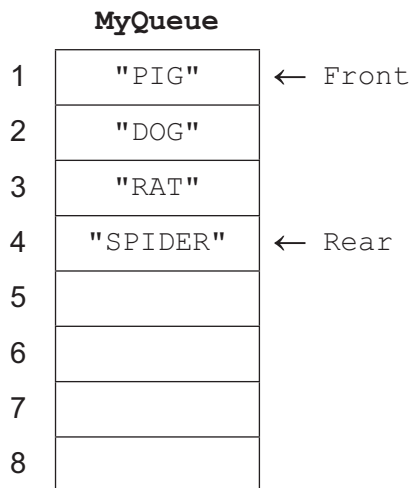
- (ii)** Justify why a queue is suitable for your application identified in **7(a)(i)**.

[2]

- (b)** A programmer designs a circular queue.

- The data items are stored in a 1D array, `MyQueue`
- The queue is managed using two pointers.

The diagram shows the circular queue populated with four items.



(i) Write **three** pseudocode declaration statements to implement this circular queue.

1

2

3

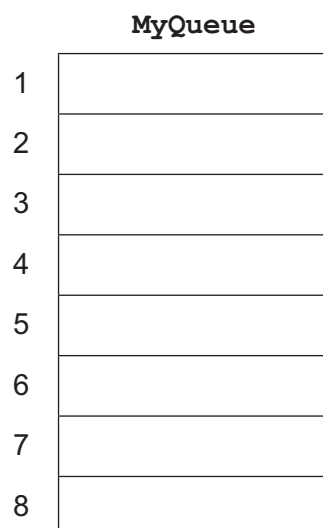
[2]

(ii) Two program modules are available to add and remove items to and from the queue.

module	description
EnQueue (<value>)	when <value> is added to the queue
DeQueue ()	when an item is removed from the queue

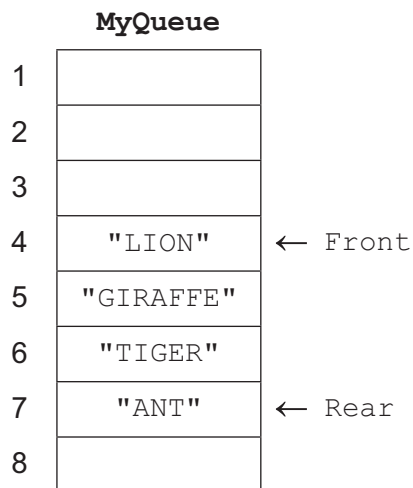
Complete the state of **MyQueue** after the following sequence of changes:

- DeQueue ()
- EnQueue ("HORSE")
- EnQueue ("CAT")



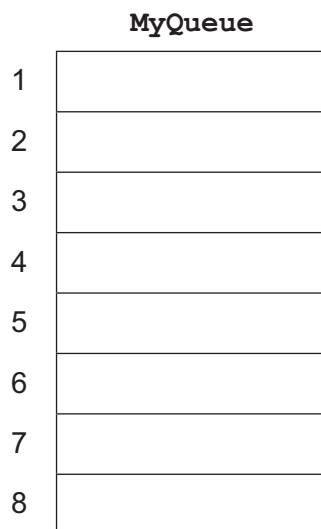
[2]

(iii) On a different occasion, the state of **MyQueue** is:



Complete the state of `MyQueue` after the following sequence of changes:

- `EnQueue ("GERBIL")`
- `EnQueue ("MONKEY")`
- `EnQueue ("SNAKE")`
- `DeQueue ()`



[2]

(c) `MyQueue` will be initially empty.

Write a pseudocode procedure `InitialiseQueue()` to set up the empty `MyQueue`, using a loop.

The initial array contents will be set to empty strings.

You are **not** required to repeat the declaration statements in 7(b)(i).

```
PROCEDURE InitialiseQueue()
ENDPROCEDURE
```

[3]

4. M/J 26/P21/Q9/a,b,c

- 9 A hotel has three floors numbered 0 to 2 (floor 0 is the ground floor). There are 15 rooms on each floor.

The hotel gets many enquiries from customers to check if a room is available that day.

The availability data for that day is stored in a 2D array `Version1`

- The first index is the floor number.
- The second index is the room number.
- On each of the three floors, the room numbers are 1 to 15.

Each cell of the array stores either:

- 'A' – the room is available
- 'B' – the room is already booked.

(a) Write the declaration statement for the 2D array `Version1`

[2]

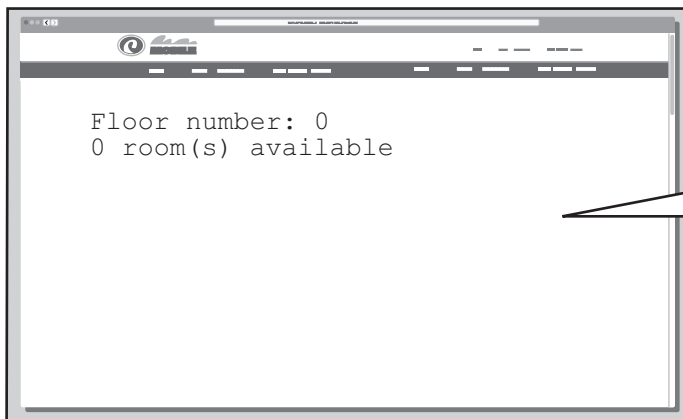
(b) An enquiry program will:

- input a floor number
- output all room numbers that are available on that floor
- output the number of rooms that are available.

The output must be in the format shown in the two examples.

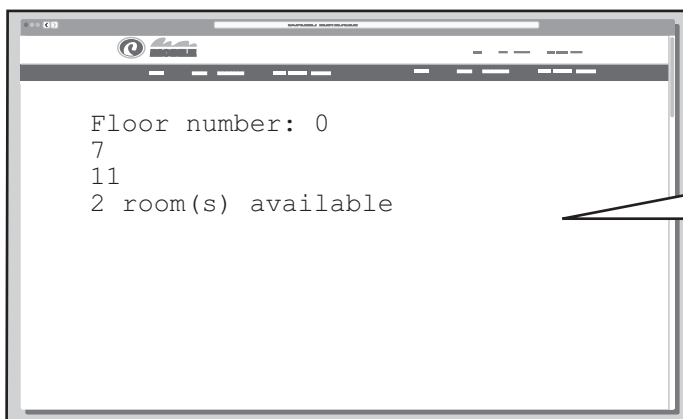
A wheelchair user makes an enquiry. She requires a room on the ground floor only.

Example program outputs:



```
Floor number: 0
0 room(s) available
```

There are no available rooms on the ground floor.



```
Floor number: 0
7
11
2 room(s) available
```

Shows two rooms available on the ground floor:

- rooms 7 and 11.

Write pseudocode for this enquiry program.

[6]

(c) Customers are becoming more selective with their enquiries when searching for an available room.

A new program design is required. It will create a record to define the data for a room.

Each room:

- may have a sea view
- is classified as: single room ('S'), double room ('D') or family room ('F')
- is either standard ('ST') or deluxe ('DL').

(i) Complete the pseudocode.

```
TYPE RoomData
```

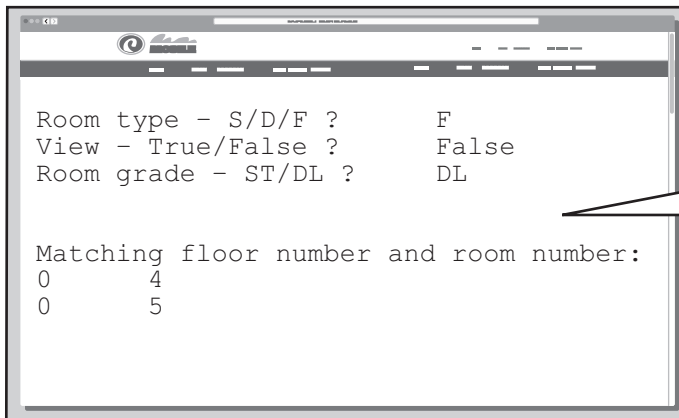
[3]

(ii) The room data is stored in a 2D array `Version2` of type `RoomData`

The new enquiry program uses this new data design and will:

- input the **three** customer requirements
- output the floor number(s) and room number(s) for all matching available rooms.

Example program output:



The customer asks for a deluxe family room; no need for a sea view.

The new enquiry program has found two matching rooms on the ground floor.

Write pseudocode for the new enquiry program.

[7]

5. M/J 26/P22/Q1/a(i)

- 1 A program is being developed to control a simple alarm system to be used in a house. The alarm has a control panel with a keypad and a screen to display text.

The keypad consists of 36 character keys:

- 26 upper-case letter keys, allowing A to Z to be entered
- 10 digit keys, allowing 0 to 9 to be entered.

- (a) When the alarm system is initially installed, a password must be created. When the password is initially entered, it is checked to ensure that it follows a set of validation rules. It is then permanently stored in the alarm system.

For a password to be valid it must follow these validation rules:

Rule 1: contain between six and eight characters inclusive.

Rule 2: contain at least one letter and one digit.

Rule 3: each character can only occur once.

The global variable `Pass` is used to store the password entered.

- (i) Complete the pseudocode to declare the variable to be used to hold the password.

DECLARE `Pass` :

[1]

6. M/J 26/P22/Q4/a,b

- 4 A desktop web browser stores the history of web addresses visited.

The web browser uses a back button and forward button to move between web pages recently visited.

Two stacks are used when each of these buttons are clicked:

- back stack
- forward stack.

The back button is clicked to return to a previously visited web page.

The back button operates as follows:

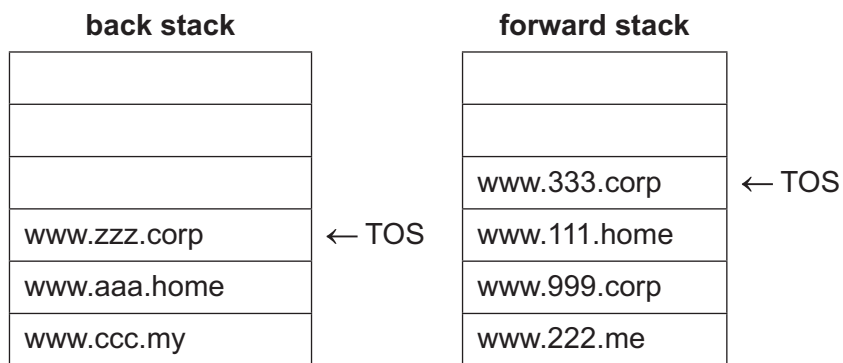
- The web address currently being displayed by the browser is pushed onto the forward stack.
- The web address from the top of the back stack is popped.
- The web address popped is loaded by the web browser.

The forward button is clicked to revisit a web page previously visited.

The forward button operates as follows:

- The web address currently being displayed by the browser is pushed onto the back stack.
- The web address from the top of the forward stack is popped.
- The web address popped is loaded by the web browser.

The current state of the two stacks:



The top of stack pointer is represented by TOS.

The current web address being displayed by the web browser is: www.bbb.home

The browsing session continues; the buttons are clicked in sequence in the three groups:

group 1:

back button

group 2:

forward button

group 3:

forward button

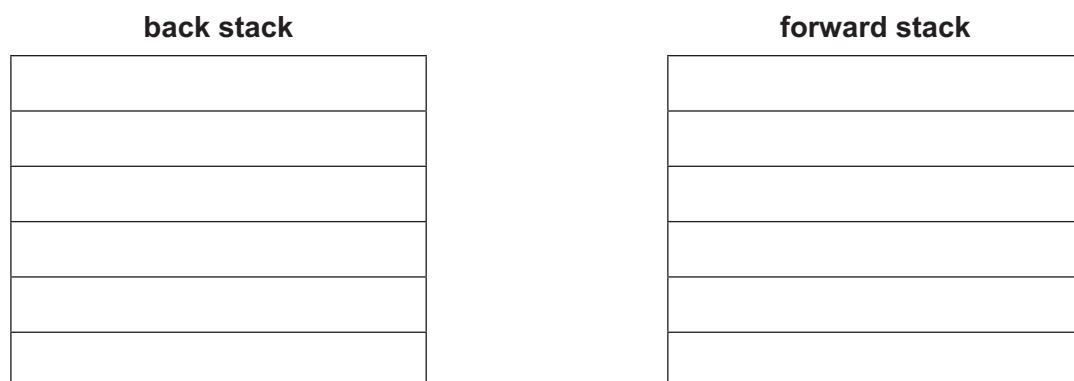
forward button

forward button

- (a) Complete the diagrams to show the state of both stacks after each group of operations has been performed.

Note: you must include the top of stack pointer (TOS) and the current web address being displayed for each group diagram.

after group 1



Web address currently displayed in the web browser:

after group 2

back stack

forward stack

Web address currently displayed in the web browser:

after group 3

back stack

forward stack

Web address currently displayed in the web browser:

[6]

(b) A stack is an Abstract Data Type (ADT)

Identify **one** other ADT and describe **two** features of the ADT.

ADT

feature 1

feature 2

[3]

7. M/J 26/P22/Q5/a,b

- 5 A programmer has defined a global 1D array `Numbers` of type `INTEGER`, which can store 1000 integers. The lower bound of the array `Numbers` is set to 1. Each element of the array has been initialised with a random integer in the range 200 to 99999.

The programmer has written a bubble sort algorithm to sort the array `Numbers` into ascending order.

Study algorithm A.

Assume all the variables have been correctly declared.

algorithm A:

```

FOR LoopCounter ← 1 TO 999                                //Outer Loop
  FOR Index ← 1 TO 999                                    //Inner Loop
    IF Numbers[Index] > Numbers[Index + 1] THEN
      //Start of block to swap neighbouring elements in the array
      Temp ← Numbers[Index]
      Numbers[Index] ← Numbers[Index + 1]
      Numbers[Index + 1] ← Temp
      //End of block to swap neighbouring elements in the array
    ENDIF
  NEXT Index
NEXT LoopCounter

```

- (a) Describe the change that would have to be made to the bubble sort algorithm (algorithm A) so that the `Numbers` array would be sorted into descending order.

[1]

- (b) The programmer rewrites the bubble sort algorithm to make it more efficient.

Study algorithm B.

Assume all the variables have been correctly declared.

algorithm B:

```

Boundary ← 999
REPEAT                                                    //Outer Loop
  NoSwaps ← TRUE
  FOR Index ← 1 TO Boundary                            //Inner Loop
    IF Numbers[Index] > Numbers[Index + 1] THEN
      //Start of block to swap neighbouring elements in the array
      Temp ← Numbers[Index]
      Numbers[Index] ← Numbers[Index + 1]
      Numbers[Index + 1] ← Temp
      //End of block to swap neighbouring elements in the array
    NoSwaps ← FALSE
  ENDIF
NEXT Index
Boundary ← Boundary - 1
UNTIL NoSwaps = TRUE

```

- (i) Explain why the second bubble sort algorithm (algorithm B) is more efficient than the first bubble sort algorithm (algorithm A). Your answer must refer to both inner and outer loops.

[4]

- (ii) State the situation that will result in the outer loop iterating for the minimum number of times when the second bubble sort algorithm (algorithm B) is used.

[1]

8. M/J 26/P22/Q6/b

- 6 A program is being developed to encrypt a message.

The following terms are used in the encryption technique to be used:

term	definition
plain text	The original message before it is encrypted.
cipher text	The encrypted version of the plain text.

The encryption method to be used is based on the algorithm for the Caesar Box Cipher. All plain text messages to be encrypted must be 25 characters in length or less. If a message is less than 25 characters in length, then spaces are added to the end to make the padded plain text the required length.

A square grid of five columns and five rows is used. The padded plain text message is put into the grid row by row. The cipher text is obtained by reading the grid column by column.

For example:

plain text: This is a message

The number of characters in the original plain text, including spaces, is 17.

plain text with added spaces at the end: This△is△a△message△△△△△△△△

The character △ is used to indicate a space in the padded plain text.

The grid filled, row by row, with the padded plain text.

T	h	i	s	△
i	s	△	a	△
m	e	s	s	a
g	e	△	△	△
△	△	△	△	△

The cipher text is obtained by reading the grid column by column.

cipher text: Timg△hsee△i△s△△sas△△△△a△△

The programmer has defined the first program module as:

module	description
AddPadding()	• user prompt for and input of plain text • check plain text contains between 1 and 25 characters inclusive • re-input of plain text until it contains between 1 and 25 characters • if required, additional space characters are added to the end of the plain text to ensure it contains exactly 25 characters • return of amended plain text

(b) The programmer has defined a global 2D array `Grid` to be used to create the cipher text.

(i) Complete the pseudocode statement used to declare the global 2D array `Grid`

DECLARE

(ii) The programmer has defined an additional program module as follows:

module	description
PopulateGrid()	- called with one parameter of type string representing the correctly padded plain text - populate the array <code>Grid</code> using the plain text so that it can be used at a later point to create the cipher text - output the contents of the array <code>Grid</code> so it is displayed row-by-row using the original example for the padded plain text: <code>ThisΔisΔaΔmessageΔΔΔΔΔΔΔΔΔ</code> the output would be: <code>This</code> <code>is a</code> <code>messa</code> <code>ge</code>

Write pseudocode for module `PopulateGrid()`

[8]

9. M/J 26/P23/Q2/a

2 A control program is being developed for a robot floor vacuum cleaner. The settings for the robot can be changed and include:

- the strength of the suction (numerical values: 0, 1, 2, 3, 4 or 5)
- the total vacuum cleaning time (a whole number of minutes).

The robot has a number of sensors.

The control program includes the variables:

variable	description
<code>MainsOn</code>	TRUE when the mains power is on
<code>DisplayOn</code>	TRUE when the user display is shown
<code>Suction</code>	TRUE when the program detects that the user has selected a non-zero value for the strength of the suction

(a) Identify the data type used for the **three** variables.

[1]

10. M/J 26/P23/Q5/b

- 5 Data is stored about people who have attended a job interview. Each interview is held at one of three office locations.

Each person's data is stored as a data packet made up of four data items:

<initial><shortened surname><office code><summary of the interview>

- The initial is a single character.
- The shortened surname is the first five characters of the surname.
 - You can assume all surnames are at least five characters.
- The office code is a single character 'A', 'B' or 'C'.
- The interview summary text is of variable length.

Example:

Peter Watkins was interviewed at office 'B'. After the interview it was decided to invite him for a second interview.

His data packet is stored by variable `Applicant`:

```
Applicant ← "PWatkiBSecond interview"
```

- (b) The data for all interviews is saved in this format in a text file `JobApps.txt`

Each line of the file stores one data packet.

A procedure `Search()` is written to search for a particular applicant.

If the applicant is found, the program will output 'Name found', otherwise it will output 'Name not found'.

Complete the pseudocode for procedure `Search()`

```
PROCEDURE Search(Initial : CHAR, ShortSurname : STRING)
  DECLARE ThisApplicant, ThisSurname : STRING
  DECLARE ThisInitial : CHAR
  DECLARE Found : BOOLEAN

  OPENFILE "JobApps.txt" .....
  Found ← FALSE
  REPEAT

    ..... "JobApps.txt", ThisApplicant
    ThisInitial ← LEFT(ThisApplicant, 1)

    ThisSurname ← .....

    IF ThisSurname = ShortSurname ..... THEN
      OUTPUT "Name found"
    ENDIF

  UNTIL Found = TRUE .....

  IF Found = FALSE THEN
    OUTPUT "Name not found"
  ENDIF

ENDPROCEDURE
```

11. M/J 26/P23/Q6/a,b,c,d

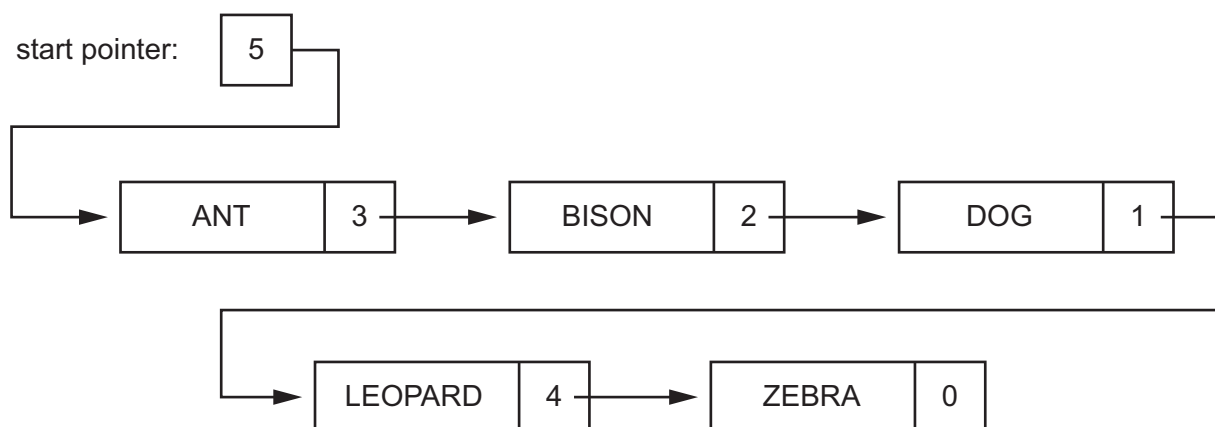
6 A linked list is an example of an Abstract Data Type (ADT).

A linked list is implemented using:

- a 1D array for the data items
- a 1D array for the link pointers
- a start pointer.

The linked list maintains the data items in alphabetical order.

(a) The state of the linked list after five data items have been added is:



Complete the diagram to show the `StartPointer` and the contents of the arrays `Data` and `Pointer`

`StartPointer`

Data		Pointer	
1		1	
2		2	
3		3	
4		4	
5		5	
6		6	
19		19	
20		20	

[3]

(b) State what the pointer value 0 (null pointer) indicates.

[1]

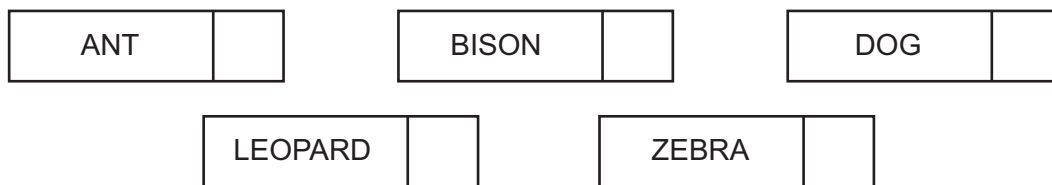
(c) To maintain the linked list, algorithms are required to:

- insert a new data item into the linked list
- delete a data item from the linked list.

- (i) The data item 'DOG' is to be deleted from the linked list given in **6(a)**.

Complete the diagram to show the new state of the linked list.

start pointer:



[1]

- (ii) To delete a data item, the algorithm will have special cases. One special case is the linked list is empty.

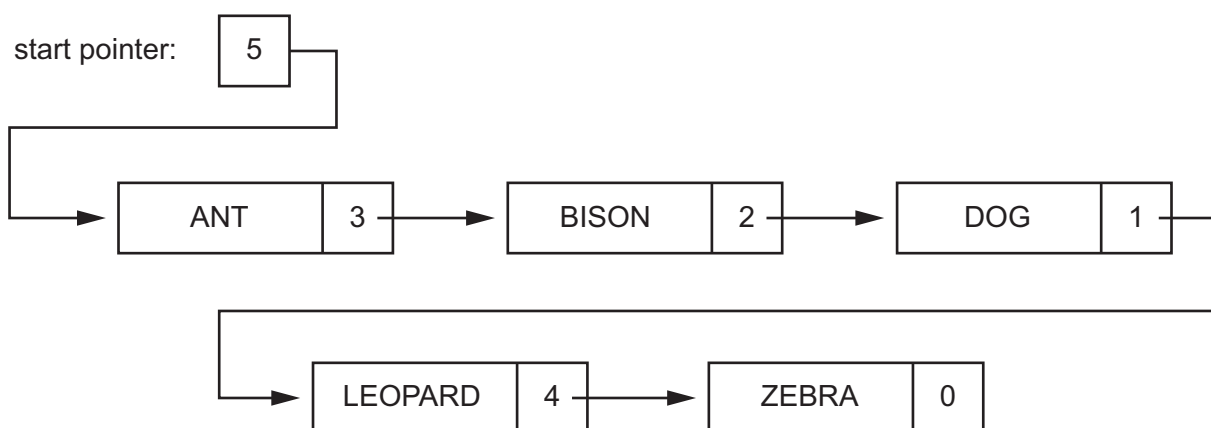
Describe **two** other special cases which must be considered when deleting a data item.

1

2

[2]

- (d) The linked list given in **6(a)** is repeated here. Data item 'BAT' is to be added to the linked list.



Describe how the data item 'BAT' is added to the linked list.

Do **not** attempt to write pseudocode.

[6]

12. M/J 26/P23/Q8/a,b

also Unit 12 (3 marks); this unit 16 marks

- 8** A warehouse stores stock in bins. A bin is a container which stores the stock items; each bin can store many identical items. The warehouse is organised on three different levels – level 0 (ground level), then level 1 and level 2. Each level has 120 bins, numbered 1 to 120.

A bin is identified by the level and a bin number. The data item stored for each bin is a Boolean value; this indicates whether the bin is in use (contains stock items) or not in use (contains no stock items).

- (a) (i) Complete the pseudocode to declare a 2D array `Bins`

DECLARE

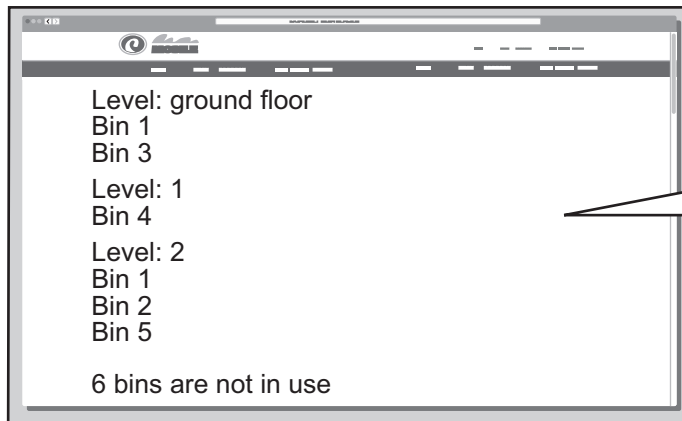
[2]

(ii) A pseudocode module `InactiveReport()` will produce a report that will show:

- a header for each level
- the bin number for each bin that is **not** in use on that level
- the total number of bins that are **not** in use on all levels.

The module is initially developed for only **five** bins on each level.

An example program output:



Shows:

- the header for each level
- a list of bin numbers for bins that are not in use on that level
- the total number of bins not in use.

Complete the pseudocode for the procedure `InactiveReport()`

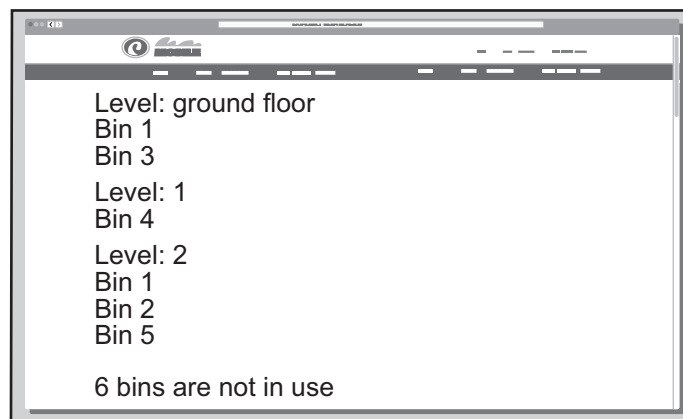
Assume that the `Bins` array is a global variable and has already been populated with data.

```
PROCEDURE InactiveReport()
```

```
ENDPROCEDURE
```

[5]

(iii) Test data were chosen to produce this output:



Complete the pseudocode test data which would produce the output given.
Reminder: The module is initially developed for only **five** bins on each level.

```
// Ground floor
Bins[0, 1] ← FALSE
Bins[0, 2] ← TRUE

Bins[0, 3] ← .....
Bins[0, 4] ← TRUE
Bins[0, 5] ← TRUE

// Level 1
Bins[1, 1] ← TRUE

Bins[1, 2] ← .....
Bins[1, 3] ← TRUE
Bins[1, 4] ← FALSE
Bins[1, 5] ← TRUE

// Level 2
```

[3]

- (b) It is decided to change the 2D array `Bins` so that more information is available about each bin.

Data items for each bin are:

- bin size (coded as 'Small', 'Medium' or 'Large')
- active – is the bin currently in use?
- stock item code for the stock item stored (four-digit string)
- stock level (0 to 500 items).

- (i) Complete the pseudocode to create a record structure with the identifier `BinData`

```
TYPE BinData
ENDTYPE
```

[2]

- (ii) A stock item is to be allocated to an unused bin. A function `FindBin()` is required that finds and returns the location of the first bin not in use which has a suitable size. The function will search on the ground floor first, then search level 1 and finally search level 2.

The function uses the global 2D array `Bins` using the revised record structure `BinData`

The function `FindBin()`:

- takes a parameter for the bin size required (coded as 'Small', 'Medium' or 'Large')
- searches for an available bin that matches the size required
- outputs the level and bin number (as a string) of the first unused bin with the matching size
 - for example, 'Level: 0 Bin: 4'
 - or if there are no available bins 'No bins available'.

Write the pseudocode function `FindBin()`

[7]

13. O/N 25/P21/Q3/a,b

- 3** A program is needed to manage individual rentals in a car-hire business.

The data items for each rental will be held in a record structure of type `RentalRecord`.
The programmer has started to define the items that will be needed:

Item	Example value	Comment
<code>RentalID</code>	"AB1234"	a unique alpha-numeric value
<code>CarID</code>	241	a numeric value used as an array index
<code>DisCode</code>	'S'	a letter indicating the type of discount offered
<code>Start</code>	13/06/2025	when the rental starts
<code>Duration</code>	7	the number of days of the rental
<code>Completed</code>	FALSE	TRUE when the car is returned and the rental charge paid

- (a) (i)** Write pseudocode to declare the record structure for type `RentalRecord`

[4]

- (ii)** A 1D array `Rental` containing 500 elements is used to store the data for all rental records.

Write pseudocode to declare the `Rental` array.

[2]

- (b)** State **three** benefits of using an array of records to store the data for all rentals.

1

2

3

[3]

14. O/N 25/P21/Q4/a,b

- 4** A program contains a global 1D array `Number` consisting of 20 elements of type `REAL`

A procedure `Store()` will input a sequence of up to 20 real values, one value at a time. These values will be assigned to elements of the array using four steps:

- Step 1: store the first value in the sequence in the first element of the array
Step 2: check each subsequent value input. If this value is larger than the previous value input, then assign the value to the next array element, otherwise go to **step 4**
Step 3: repeat from **step 2** unless the array is full
Step 4: output the count of the number of values stored in the array together with a suitable message.

- (a)** Complete the pseudocode for `Store()`

All variables used in the algorithm must be declared.

```
PROCEDURE Store()
```

```
ENDPROCEDURE
```

[6]

- (b)** The requirements of the program change:

- the number of values in the sequence is unknown, but may be higher than 20
- the values will need to be accessed by another program as data.

The data will be stored in a text file instead of an array.

- (i)** Give **two** benefits of using a text file instead of an array.

1

2

[2]

- (ii)** State any change that will need to be made before each value is written to the file.

[1]

15. O/N 25/P21/Q5/a

- 5** A program is being designed in pseudocode.

The program contains the following declaration for the global array `MyData`:

```
DECLARE MyData : ARRAY[1:10000] OF STRING
```

A function `FindFirst()` is written to search the array for a given string and to return the index of the first element where that string is found, or to return `-1` if the string is **not** found.

The function is written in pseudocode as shown:

```
FUNCTION FindFirst(SearchString : STRING) RETURNS INTEGER
  DECLARE Index, FoundAt : INTEGER
  FoundAt ← -1
  FOR Index ← 1 TO 10000
    IF MyData[Index] = SearchString THEN // outer conditional clause
      IF FoundAt = -1 THEN               // inner conditional clause
        FoundAt ← Index
      ENDIF
    ENDIF
  NEXT Index
  RETURN FoundAt
ENDFUNCTION
```

- (a) (i)** Comment on why the programmer chose `-1` as the initial value of `FoundAt`

[1]

- (ii)** Explain the purpose of the **outer** conditional clause.

[1]

- (iii)** The **inner** conditional clause ensures that only the index of the **first** matching element, if any, is returned.

Explain how this clause works.

[1]

16. O/N 25/P21/Q8/a,b

- 8 A program is being developed to manage student book loans from a college library. Students may borrow up to five books at a time from the library.

The programmer has defined a record type to define each loan.

The record data items are:

Data item	Data type	Comment
StudentID	STRING	the unique ID of the student who has borrowed the book
BookID	STRING	the unique ID of the book being borrowed
OnLoan	BOOLEAN	TRUE if the book has not been returned

The programmer has defined a global array `Loan` to store 5000 loan records.

There are more elements in the array than books in the library. Unused elements have the `StudentID` set to an empty string. These may occur anywhere in the array.

The programmer has defined the first program module:

Module	Description
<code>OKToBorrow()</code>	- called with a parameter of type <code>STRING</code> representing a <code>StudentID</code> - search the array for loan records for the specified student - output a suitable message to say whether the student may, or may not, borrow another book

- (a) Write efficient pseudocode for the module `OKToBorrow()`

[7]

- (b) A second module is defined:

Module	Description
<code>ReturnBook()</code>	- called with two parameters of type <code>STRING</code> representing a <code>StudentID</code> and a <code>BookID</code> - searches the array for the relevant loan record - when found, sets <code>OnLoan</code> to <code>FALSE</code> and returns <code>TRUE</code> - if a loan record is not found, or the book has already been returned, then returns <code>FALSE</code>

Write efficient pseudocode for the module `ReturnBook()`

Assume that each student is only allowed to borrow each book only once. That means that there will be no more than **one** loan record for a given combination of student and book.

[8]

17. O/N 25/P22/Q1/b

(b) Complete the table by giving the appropriate data type.

Variable	Example data value	Data type
Result	5.42	
MonthLetter	"JFMAMJJASOND"	
Birthday	15/11/2009	

[3]

18. O/N 25/P22/Q3/b,c

- 3 A text file `OldFile.txt` contains IDs, names and email addresses for members of a club. Three information items are stored for each member and each item is stored on a separate line.

The example shows the information for the first two members in the first six lines of the file:

Line in file	Information item	Example data
1	member 1 ID	"AB1234"
2	member 1 name	"Freddie Jones"
3	member 1 email address	"FreddieJ909@Cambridge.org"
4	member 2 ID	"BC2345"
5	member 2 name	"Sue Smith"
6	member 2 email address	"Sue1024@Cambridge.org"

The member ID string is always two letters followed by four digits.

The file design is to be changed so that information for each user is stored as a single line of the file, with the character '`\`' used as a separator between data items.

For example, the single line for member 1 will be:

```
"AB1234\Freddie Jones\FreddieJ909@Cambridge.org"
```

An algorithm will produce a new file `NewFile.txt` from the contents of `OldFile.txt`

Assume:

- `LineX`, `LineY` and `LineZ` are of type `STRING` and are used to store the three items of information for each member
- `NewString` is a temporary variable of type `STRING`
- `OldFile.txt` exists and contains valid data.

(b) The character '`\`' has been chosen as a separator.

Explain why this is a suitable character.

[1]

- (c) Two new information items are to be stored for each member. Both new items are encrypted; they can each contain any character (including ' \ ') and can be of any length up to a maximum of 99 characters.

For example:

Information item	Example data
member 1 ID	"AB1234"
member 1 name	"Freddie Jones"
member 1 email address	"FreddieJ909@Cambridge.org"
member 1 new information 1	"En/98&*(?\/D7iP"
member 1 new information 2	"23\CoboL"

Explain the additional file design changes needed so that:

- all the information items for each member are stored on a single line of `NewFile.txt`
- it is possible to extract each information item after the line is read.

Assume the ' \ ' character is not used in any email address.

[3]

19. O/N 25/P22/Q4/a,b

4 A quiz has nine questions. There are:

- five easy questions each worth 3 points
- four hard questions each worth 5 points.

At the end of the quiz the points for each correctly answered question are added to give a total score.

A check is made to test that the total score is valid using a 1D array `CheckTotal` of type `BOOLEAN`. Each index value of the array represents a possible total score. The corresponding element value is `TRUE` if the index value represents a valid total score and `FALSE` otherwise.

The first nine rows of the array are:

Index value	Element value	Comment
0	TRUE	valid total score (no correct answers)
1	FALSE	invalid total score
2	FALSE	invalid total score
3	TRUE	valid total score (one 3-point question correct)
4	FALSE	invalid total score
5	TRUE	valid total score (one 5-point question correct)
6	TRUE	valid total score (two 3-point questions correct)
7	FALSE	invalid total score
8	TRUE	valid total score (one 3-point question and one 5-point question correct)

For example, a total score of 6 points is valid; the value of the array element at index value 6 is `TRUE`

(a) Write pseudocode to declare `CheckTotal` and to set all elements of the array to `FALSE`

All variables used must be declared.

[4]

(b) The pseudocode represents an algorithm to set the appropriate elements of the array to `TRUE`

Complete the pseudocode:

```

DECLARE EasyQ, HardQ : INTEGER
FOR EasyQ ← ..... TO 15 STEP .....
    FOR HardQ ← 0 TO ..... STEP .....
        CheckTotal[.....] ← TRUE
    NEXT HardQ
NEXT EasyQ

```

[5]

20. O/N 25/P22/Q8/a,b,c

8 A program is being developed to manage student book loans from a college library.

The programmer has defined a record type to define each loan.

The data items are:

Data item	Data type	Comment
StudentID	STRING	the unique ID of the student who has borrowed the book
BookID	STRING	the unique ID of the book being borrowed
OnLoan	BOOLEAN	TRUE if the book has not been returned

The programmer has defined a global array `Loan` to store 7000 loan records.

There are more elements in the array than books in the library. Unused elements have the `StudentID` set to an empty string. These may occur anywhere in the array.

The programmer has defined a program module:

Module	Description
<code>CountLoans()</code>	- called with a parameter of type <code>STRING</code> representing a <code>StudentID</code> - counts the number of books currently on loan to the specified student - counts the number of books that the student has already returned - output both counts together with a suitable message

(a) Write pseudocode for module `CountLoans()`

[7]

(b) As a reminder, a global array `Loan` stores 7000 loan records with data items for each loan record:

Data item	Data type	Comment
StudentID	STRING	the unique ID of the student who has borrowed the book
BookID	STRING	the unique ID of the book being borrowed
OnLoan	BOOLEAN	TRUE if the book has not been returned

A new module is defined:

Module	Description
<code>NewLoan()</code>	- called with two parameters of type <code>STRING</code> representing a <code>StudentID</code> and a <code>BookID</code> - searches the array for an unused loan record - if found, updates the loan record and returns <code>TRUE</code>, otherwise returns <code>FALSE</code>

Write efficient pseudocode for `NewLoan()`

[7]

(c) When a book is returned, the loan record will have its `OnLoan` data set to `FALSE`

A new procedure `Archive()` will mark these records as unused after first saving them for future reference.

Explain how these records can be saved for future reference.

[2]

21. O/N 25/P23/Q3

- 3 A program uses a stack to hold up to 30 integer values. The stack is implemented using a global integer variable and a global 1D array.

The array is declared in pseudocode:

```
DECLARE ThisStack : ARRAY[1:30] OF INTEGER
```

Stack design notes:

- The global variable `SP` acts as a stack pointer. `SP` contains the array index of the last value pushed onto the stack.
- If the stack is empty, then `SP` is assigned the value zero.
- The first item added to the stack will be stored in `ThisStack[1]`
- `SP` is incremented each time an item is added to the stack.

A function `Pop()` is written to remove an item from `ThisStack`. The function returns an item of type `PopData` which is defined in pseudocode:

```
TYPE PopData
  DECLARE Data    : INTEGER
  DECLARE Exists  : BOOLEAN
ENDTYPE
```

The value removed from the stack is assigned to `Data` and `Exists` is set to `TRUE`. If it is **not** possible to remove a value from the stack, then `Exists` is set to `FALSE`

Complete the pseudocode for `Pop()`

```
FUNCTION Pop() RETURNS PopData
  DECLARE ThisPop : .....
  IF ..... THEN
    PopData.Exists ← ..... // Stack is empty
  ELSE
    PopData.Data ← .....
    PopData.Exists ← .....
    SP ← .....
  ENDIF
  RETURN ThisPop
ENDFUNCTION
```

[5]

22. O/N 25/P23/Q4

- 4 A program contains a global 1D array `Data` containing 20 elements of type `INTEGER`

A global string `NumString` represents a sequence of three-digit numbers, separated by commas.
For example:

"101,456,219,754,328"

The string contains at least **four** three-digit numbers. The total number of three-digit numbers in the string is unknown.

A procedure `Store()` will:

- extract one three-digit number at a time from `NumString`
- convert each of the three-digit numbers extracted to an integer and assign this to the next array element, starting from index 1
- end when all three-digit numbers have been stored, or when the array is full.

Complete the pseudocode for `Store()`

All local variables used must be declared.

PROCEDURE `Store()`

ENDPROCEDURE

[6]

23. O/N 25/P23/Q8/a,b,c

- 8 A program is being developed to manage student book loans from a college library.

The programmer has defined a record type to define each loan.

The data items are:

Data item	Data type	Comment
<code>StudentID</code>	<code>STRING</code>	the unique ID of the student who has borrowed the book The first three characters of a <code>StudentID</code> represent a tutor ID. Each student has one tutor.
<code>BookID</code>	<code>STRING</code>	the unique ID of the book being borrowed
<code>OnLoan</code>	<code>BOOLEAN</code>	<code>TRUE</code> if the book has not been returned

The programmer has defined a global array `Loan` to store 8000 loan records.

There are more elements in the array than books in the library. Unused elements have the `StudentID` set to an empty string. These may occur anywhere in the array.

The programmer has defined a program module:

Module	Description
<code>LoanStatus()</code>	- called with two parameters of type <code>STRING</code> representing a <code>StudentID</code> and a <code>BookID</code> - outputs a message saying whether a given loan has been returned or not - outputs a warning message if a record of the given loan is not found

(a) Write efficient pseudocode for the module `LoanStatus()`

Assume that each combination of `StudentID` and `BookID` can occur only once.

[8]

(b) A second module is defined:

Module	Description
<code>LoansPerTutor()</code>	- called with a parameter of type <code>STRING</code> representing a tutor ID (as a reminder, the first three characters of a <code>StudentID</code> represent a tutor ID) - returns an integer value representing the number of books currently on loan to students who have the given tutor

Reminder: unused elements have the `StudentID` set to an empty string. These may occur anywhere in the array.

Write pseudocode for the module `LoansPerTutor()`

[7]

(c) It is decided to mark as unused all records for book loans that have been returned. The data for these records will first be written to a new text file for archive purposes.

(i) The archive program will automatically generate a meaningful filename each time it is run.

Outline a meaningful format to use for the filename.

[2]

(ii) There is a problem that will need to be overcome before the data items can be written to a text file.

As a reminder, the data items are:

Data item	Data type	Comment
StudentID	STRING	the unique ID of the student who has borrowed the book The first three characters of a StudentID represent a tutor ID. Each student has one tutor.
BookID	STRING	the unique ID of the book being borrowed
OnLoan	BOOLEAN	TRUE if the book has not been returned

Explain the problem.

[1]

- (iii) One way of storing the data items in a text file is to store each data item on a separate line.

Identify **one** benefit and **one** drawback of this way of storing the data.

Benefit

Drawback

[2]

24. M/J 25/P21/Q5

5 Stacks and queues are both abstract data types.

A stack uses a top-of-stack pointer to indicate the location of the last item added to the stack.

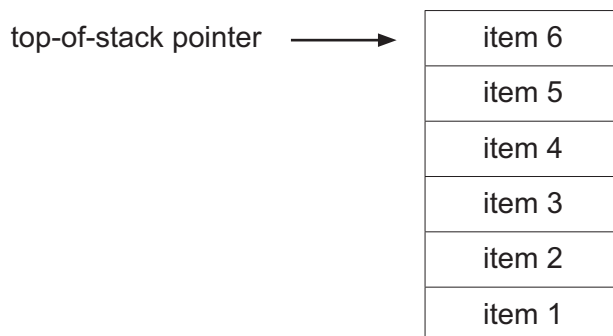
A queue uses two pointers:

- a front pointer to indicate the location of the next item to be removed from the queue
- a rear pointer to indicate the location of the next item to be added to the queue.

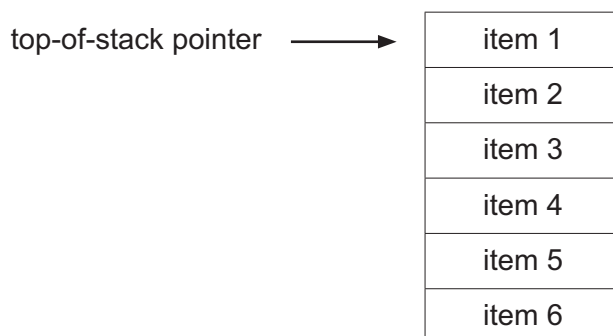
A queue can be used to reverse the items stored on a stack.

For example, if a stack contains six items:

Initial state of the stack:



Final state of the stack when the items have been reversed:



Describe how the queue could be used to reverse the items that are currently stored on the stack.

Your description must include how the pointers are used in both the stack and queue.

Assume:

- The stack initially contains an unknown number of items.
- The queue can store all the items currently stored on the stack.
- The queue is initially empty.

25. M/J 25/P21/Q6/b

- 6** A program monitors the speed of vehicles as they move around a large building site.

Each vehicle contains a sensor which reads an integer value that represents the speed of the vehicle. The value is expected to be in the range 0 to 60 inclusive.

The sensors cannot read values less than 0.

A program module has been written to validate the values read by the sensors.

- (b)** Each sensor used in the system has a unique sensor ID number in the range 1 to 50.

The program that is used to monitor the speed of each vehicle reads each sensor value every second and stores this value along with the sensor's unique ID into a global 2D array `Reading`

The global array `Reading` has been declared as follows:

```
DECLARE Reading : ARRAY[1:2000, 1:2] OF INTEGER
```

The array contains 4000 elements organised as 2000 rows and 2 columns.

Column 1 contains the sensor value, and column 2 contains the sensor ID.

When 2000 sensor readings have been taken, the array is full and the system stops taking any more sensor readings until the array has been processed.

A procedure `Sort()` is needed to sort the array into ascending order of sensor value using an efficient bubble sort algorithm.

Write **efficient** pseudocode for the procedure `Sort()`

[8]

26. M/J 25/P21/Q7/b(i)

- 7** A program is being developed to implement a customer loyalty scheme for a coffee shop.

The programmer has decided that the following data items need to be stored for each customer:

Data item	Description
customer ID	a unique six-digit string
points	an integer value that is increased by one for every cup of coffee the customer orders

When a customer visits the shop and orders coffee, the scheme operates as follows:

- The total number of points is increased by the number of coffees ordered.
- If just one cup of coffee is ordered and the number of points goes above 10, then:
 - the cup of coffee they have just ordered is given to them free of charge
 - the number of points is reduced by 11.
- If the order is for multiple coffees and the number of points goes above 10, then:
 - they get one coffee free of charge for every 11 points
 - the number of points is reduced by 11 for each free coffee.

For example, the:

- customer currently has 9 points
- customer orders 16 cups of coffee
- total number of points now becomes 25
- customer gets 2 free coffees and now has 3 points left.

The programmer has defined a program module that is called every time a customer places an order:

Module	Description
<code>CustomerOrder()</code>	- called with two integer parameters: - the number of coffees ordered - the current number of points - output a suitable message giving the number of free coffees - return a value for the new points total.

- (b) A text file `Loyalty.txt` will be used to store the data items for the loyalty scheme. The data items for each customer will be stored on a separate line of the text file where each data item is separated by a comma:

`<CustomerID>,<Points>`

The contents of the text file `Loyalty.txt` will always be stored in ascending order by customer ID.

When the data items are read from or written to the text file `Loyalty.txt`, they may need to be converted to the appropriate data type.

Each customer has a unique customer ID starting at "100001" with this value increasing by one each time a new customer joins the loyalty scheme.

When a customer joins the loyalty scheme, they are assigned the next customer ID and value of points is set to 0.

For example, if the loyalty scheme has 204 customers and a new customer joins the loyalty scheme, the following line is added to the text file `Loyalty.txt`:

`"100205,0"`

You can assume that the number of customers in the loyalty scheme will never be more than 9000.

The programmer has defined a program module as follows:

Module	Description
<code>AddNewCustomers()</code>	- called with an integer parameter representing the number of new customers to be added to the loyalty scheme - adds a new line, containing the required information, to the text file <code>Loyalty.txt</code> for each customer added to the loyalty scheme - outputs each new customer ID added to the loyalty scheme

- (i) Write pseudocode for module `AddNewCustomers()`

Assume that there is at least **one** customer already in the loyalty scheme.

27. M/J 25/P22/Q1/c,d

1 A program is being developed to control the production line in a factory.

(c) Pseudocode has been used to design modules for the program to control the production line.

The table shows **four** valid pseudocode expressions.

Complete the table by giving the data type of the evaluated expression.

Expression	Data type
RIGHT (MachineCode, 4)	
Speed * 2.5	
NOT Status	
IS_NUM(Check)	

[4]

(d) A global array `Product` is used as part of the pseudocode design being developed to control the production line. `Product` is used to store the number of rejected items each day.

The following pseudocode statement is used to assign a value to an element of the array:

`Product[x, y] ← 23`

The lower and upper bound values are shown in the table:

Variable	Lower bound	Upper bound
x	0	99
y	0	9

(i) State the number of dimensions of `Product`

[1]

(ii) State the total number of elements in array `Product`

[1]

(iii) Give the pseudocode declaration for the array `Product`

[2]

28. M/J 25/P22/Q5/b

- 5 An automated digital camera system is used to take a sequence of pictures of animals in a garden. During the design of the system, a state-transition diagram is produced.

The table details the states in the automated digital camera system along with the events which cause the states to change.

The system starts in standby mode. The sequence of pictures is taken when in active mode.

Current state	Event	Next state
standby mode	turn on	detect mode
detect mode	turn off	standby mode
detect mode	movement detected	active mode
active mode	20 seconds elapsed	sequence complete
active mode	turn off	standby mode
sequence complete	time saved	detect mode

- (b) At the end of each sequence of pictures, the time is saved as a string in the format <HH><MM><SS> where:

- HH represents the hours using two digits
- MM represents the minutes using two digits
- SS represents the seconds using two digits.

For example:

- "081230" is stored to represent the time 8:12:30, in the morning
- "152235" is stored to represent the time 15:22:35, in the afternoon.

Each string is stored on a new line in the text file `TimeTaken.txt`

An algorithm is required to process the content of the text file `TimeTaken.txt` once it has been transferred to the computer.

For each hour when pictures are taken, output a suitable message showing the hour and the total number of sequences of pictures taken within that hour.

Example outputs:

```
Hour : 15   Total : 32
Hour : 18   Total : 1
```

Write pseudocode for this algorithm.

Assume the text file `TimeTaken.txt` contains at least **one** line.

29. M/J 25/P22/Q6/b

- 6** Two arrays `Data` and `Pointer` are accessed by the procedure `Place()`

`Data` and `Pointer` are both global arrays of type `INTEGER`

The contents of these two arrays are shown:

Data		Pointer	
1	1018	1	7
2	1007	2	3
3	1010	3	1
4	1056	4	6
5	1092	5	-1
6	1062	6	5
7	1034	7	4
8	0	8	9
9	0	9	10
10	0	10	-1

Study the pseudocode:

```

PROCEDURE Place(Value : INTEGER, Start : INTEGER, Unused : INTEGER)
  DECLARE New, Current, Last : INTEGER
  CONSTANT NullPointer = -1
  New ← Unused
  Last ← NullPointer
  Current ← Start
  WHILE Current <> NullPointer AND Data[Current] < Value
    Last ← Current
    Current ← Pointer[Current]
  ENDWHILE
  Pointer[New] ← Pointer[Last]
  Pointer[Last] ← New
  Data[New] ← Value
ENDPROCEDURE

```

- (b)** The operation carried out by procedure `Place()` together with the arrays form part of the implementation of an Abstract Data Type (ADT).

Identify the ADT and state the operation carried out by procedure `Place()`

[2]

30. M/J 25/P22/Q7/a,b,c

- 7** A program is being developed to implement a customer loyalty scheme for a coffee shop.

Each customer has a unique customer ID starting at 10001 with this value increasing by one each time a new customer joins the loyalty scheme.

For example, the third customer who joins the loyalty scheme is given the customer ID 10003

The loyalty scheme is limited to 1000 customers.

A customer is awarded a loyalty point every time they buy a coffee.

The programmer has decided to use a global 2D array `Loyalty` of type `INTEGER`. The array `Loyalty` is made up of 1000 rows and 2 columns. Each row relates to one customer; column 1 contains the unique customer ID and column 2 contains the number of customer loyalty points.

Rows in the array `Loyalty` that are not currently being used have the value of Column 1 set to 99999

The array is sorted in ascending order by customer ID.

The programmer has defined a program module:

Module	Description
<code>FindCustomer()</code>	- called with parameter of type <code>INTEGER</code> representing a customer ID - searches the <code>Loyalty</code> array for this customer ID - the search will stop as soon as the customer ID is found - the search should efficiently deal with the situation when the customer ID is not stored in the <code>Loyalty</code> array - if the customer ID is found, return an integer value representing the loyalty points, otherwise return <code>-1</code>

(a) Write efficient pseudocode for module `FindCustomer()`

[8]

(b) A customer can claim a free coffee for every 11 loyalty points.

The programmer has defined a second program module:

Module	Description
<code>PointsReport()</code>	- output the customer ID for each customer who has 11 or more loyalty points - output the average loyalty points for all customers in the <code>Loyalty</code> array along with an appropriate message

Write efficient pseudocode for module `PointsReport()`

Assume the array contains the data for at least **one** customer.

[7]

(c) The programmer decides to amend the customer ID; it will be stored as a `STRING` instead of an `INTEGER`. This means that the 2D array `Loyalty` can no longer be used.

(i) Explain why the 2D array `Loyalty` can no longer be used.

[1]

(ii) Explain how a 1D array could be used to store both the loyalty points and the amended customer ID.

[2]

31. M/J 25/P23/Q1/c

1 A program is being developed for the management of a sports centre.

(c) The table lists some of the variables used in the program.

Complete the table by writing the most appropriate data type for each variable.

Variable	Use of variable	Data type
MemberCount	stores how many members have used the sports centre each day	
TotalTakings	stores the total amount of money taken each day	
BookingConfirmed	stores the state of a booking for an exercise class; a booking is either confirmed or not confirmed	
MemberDOB	to calculate when to send an email with a birthday message to a member	

[4]

32. M/J 25/P23/Q2/a,b,c

2 A programmer uses a number of Abstract Data Types (ADT) in the programs that she is developing.

(a) A stack using memory locations 400 to 409 is used in one program.

The diagram represents the current state of the stack.

A variable `TopOfStack` pointer indicates the last value added to the stack.

Stack

Memory location	Value	
409		
408		
407		
406		
405		
404		
403	'P'	← TopOfStack
402	'X'	
401	'D'	
400	'B'	

(i) Complete the answer column in the following table:

Answer

the memory location of the value that has been on the stack for the longest time	
the number of consecutive push operations that will result in the <code>TopOfStack</code> variable containing 409	

[2]

- (ii) The diagram shows the current state of the stack.

Stack	
Memory location	Value
409	
408	
407	'A'
406	'C'
405	'F'
404	'K'
403	'B'
402	'S'
401	'R'
400	'D'

← TopOfStack

The following sequence of operations are performed:

PUSH 'T'
 POP
 POP
 PUSH 'Z'
 PUSH 'X'
 POP
 PUSH 'Y'

Complete the following diagram to show the state of the stack after the operations have been performed.

Stack	
Memory location	Value
409	
408	
407	
406	
405	
404	
403	
402	
401	
400	

[3]

- (b) In a different program, the programmer decides to implement a linked list using an array of records.

- (i) The array is represented in the diagram.

A pointer value of -1 indicates the end of the linked list.

Complete the pointer field to produce a linked list that is in alphabetical order.

Index	Data	Pointer
0	"Neptune"	
1	"Jupiter"	
2	"Saturn"	
3	"Earth"	
4	"Mercury"	
5	"Uranus"	

[3]

- (ii) The variable `StartPointer` contains the index of the first item in the linked list.

State the value of the variable `StartPointer`

[1]

- (c) A third program is also being created to manage a queue.

Describe **two** features of a queue.

Feature one

Feature two

[2]

33. M/J 25/P23/Q7/b(i)

7 A program is being developed to implement a customer loyalty scheme for a coffee shop.

The programmer has decided that the following data items need to be stored for each customer:

Data item	Description
customer ID	a unique six-digit string
loyalty points	an integer value that depends on how often the customer visits the coffee shop
last visit date	the date the customer last visited the coffee shop

The programmer has defined a program module:

Module	Description
UpdateVisit()	- called with two parameters: - loyalty points of type <code>INTEGER</code> - last visit date of type <code>DATE</code> - change the loyalty points: - increase by four if the last visit date and the current date are in the same month - otherwise increase by one - change the last visit date to the current date - the changed values must be available to the code that follows the call to <code>UpdateVisit()</code>

(b) A text file `Loyalty.txt` will be used to store the data items for the loyalty scheme.

The text file `Loyalty.txt` contains only one line of data for each customer.

Each data item is separated by a comma:

`<CustomerID>,<LoyaltyPointsString>,<LastVisitDateString>`

`LastVisitDateString` is always eight characters in length in the format: `DDMMYYYY`.

An example of how a line of data will be stored in the text file `Loyalty.txt` is:

`"100123,132,05032025"`

This example shows that the customer with an ID of 100123 had 132 loyalty points following their last visit to the coffee shop on 05/03/2025.

If a customer visits the coffee shop on a Monday, the value of their loyalty points is increased by 10.

The programmer has defined **two** more program modules:

Module	Description
FindCustomer() (is already written)	- called with a parameter of type <code>STRING</code> that represents a customer ID - returns the line of data read from the text file <code>Loyalty.txt</code> containing the data items for that customer
MondayCheck()	- called with a parameter of type <code>STRING</code> that represents the customer ID of a customer - calls <code>FindCustomer()</code> - extracts the loyalty points from the string returned by <code>FindCustomer()</code> - increases the loyalty points for the current customer by 10 if the day visited is a Monday - returns an integer value representing the loyalty points

(i) Write pseudocode for module `MondayCheck()`

The module `FindCustomer()` must be used and assume it returns a valid string for the current customer.

[8]

34. O/N 24/P21/Q1/c

1 A program will calculate the tax payable based on the cost of an item.

Calculations will occur at many places in the program and these involve the use of one of three tax rates.

Tax rate values represent a percentage. For example, a tax rate value of 5.23 represents 5.23%. In this case, the tax payable on an item costing \$100 would be \$5.23.

Tax rate values are used at several places within the program. One example is given in pseudocode as follows:

```

HighRate ← FALSE
CASE OF ItemCost
    ≤ 50 : TaxRate ← 3.75          // tax rate of 3.75%
    ≤ 200 : TaxRate ← 5.23        // tax rate of 5.23%
    > 200 : TaxRate ← 6.25        // tax rate of 6.25%
        HighRate ← TRUE
ENDCASE
TaxPayable ← ItemCost * TaxRate // tax payable

```

(c) Give the **appropriate** data type for the variables in the following table, as used in the pseudocode:

Variable name	Data type
HighRate	
TaxPayable	

[2]

35. O/N 24/P21/Q6/a

- 6 A factory produces food items. The items must be used within a certain number of days after their production date. The number of days is known as the shelf life. It is different for each type of item but is always a whole number in the range 1 to 21 (inclusive).

The latest date that an item can be used is called the 'use-by' date.

A program is needed to produce labels which show the 'use-by' date.

Part of the program is a function `GetDate()` which will:

- take two parameters: a production date and a value representing the shelf life
- return the corresponding 'use-by' date.

The program contains a global 1D array `DaysInMonth` of type integer which stores the number of days in each month (index 1 is January):

Index	Value
1	31
2	28
3	31
4	30
...	...
11	30
12	31

Note: Leap years are **not** considered

- (a) An algorithm uses the array `DaysInMonth` to calculate a 'use-by' date. An alternative design would involve the use of multiple selection statements.

An array-based technique is often used when there is a large number of different values to check and where no pattern exists.

One advantage of using an array-based technique is the speed of execution compared to the use of multiple selection statements.

Give **two other** advantages of using an array for this type of operation rather than a solution based on multiple selection statements.

1

2

[2]

36. O/N 24/P21/Q8/b,c

- 8 An exam paper has a maximum of 75 marks. One of five pass grades (A to E) is assigned, depending on the mark obtained. The lowest mark for a given grade is known as the grade boundary. For example, if the grade boundary for an A grade is 65 marks, then any candidate who achieves a mark of 65 or above will be awarded an A. A grade of U is awarded for marks below the E grade boundary.

The five grade boundaries are stored in a global 1D array `GradeBoundary` of type integer.

For example:

Element	Value	Comment
GradeBoundary[1]	65	The minimum mark for an A grade.
GradeBoundary[2]	57	The minimum mark for a B grade.
GradeBoundary[3]	43	The minimum mark for a C grade.
GradeBoundary[4]	35	The minimum mark for a D grade.
GradeBoundary[5]	27	The minimum mark for an E grade.

A global 2D array `Result` of type integer contains candidate marks for the exam. Each row relates to one candidate. Column 1 contains the candidate mark and column 2 contains the unique candidate ID.

For example, for the fourth and fifth candidates:

Element	Mark	Element	ID
Result[4, 1]	56	Result[4, 2]	1074832
Result[5, 1]	54	Result[5, 2]	2573839

There are more rows in the array than candidates who sit the exam. Any unused rows will be at the end of the array.

Candidate papers that are given a mark within two marks of any grade boundary must be checked.

For example, given the values in the example grade boundaries above, any paper that was awarded between 41 and 45 marks (inclusive) would need to be checked.

A program is being written to identify papers that need to be checked.

The programmer has defined the first program module as follows:

Module	Description
<code>CheckMark()</code>	- called with a parameter of type integer representing a candidate mark - returns <code>TRUE</code> if the mark is within 2 of any of the five grade boundaries, otherwise returns <code>FALSE</code>

(b) A second module is defined:

Module	Description
<code>CheckAll()</code>	- called with a parameter of type integer representing the number of candidate marks in the <code>Result</code> array - uses <code>CheckMark()</code> to check each candidate mark - for each paper that needs to be checked, write the corresponding candidate ID on a separate line in a new file named <code>GRLIST.txt</code> - outputs a message with a count of how many papers need to be checked

Write pseudocode for module `CheckAll()`.

`CheckMark()` must be used to check each individual mark.

[8]

- (c) The requirement changes. Instead of a new file, the module described in part (b) needs to add the corresponding candidate ID for each paper that needs to be checked to an **existing** file.

Explain the change that will need to be made to `CheckAll()`.

[1]

37. O/N 24/P22/Q1/d

- 1 A program has been developed and released for general use. After a few months of use an error is detected where under certain circumstances it outputs an unexpected value.

- (c) To solve the error a programmer decides to create a new module.

The design of the new module has been completed and the module is being coded.

Identify **two** features of an IDE that will help during the coding of this new module.

1

2

[2]

- (d) The new module referred to in part (c) introduces **three** new variables.

Complete the following table by giving the appropriate data type for each.

Variable name	Used to store	Data type
Name	A customer name.	
Index	An array index.	
Result	The result of the division of any two non-zero numbers.	

[3]

38. O/N 24/P22/Q2/b(i)

2 A program is being developed to process bank card information.

When a card number is displayed, all the characters **except** the last four are replaced with the asterisk character ' * '.

Card numbers are stored as strings. The strings are between 10 and 20 characters in length.

The function `Conceal()` will take a string representing a card number and return a modified string.

Example strings:

Original string	Modified string
"1234567890"	"*****7890"
"1234567897652"	"*****7652"
"1234567890123456"	"*****3456"

(b) The requirements have been changed. `Conceal()` will now be written as a procedure which will process 100 card numbers each time it is called.

The card numbers will be stored in a 2D array `CardNumber`. The original string will be stored in column one and the modified string in column two.

(i) Write pseudocode to declare the array.

[2]

39. O/N 24/P22/Q3/a,b

- 3 A program uses a stack to hold up to 60 numeric values.
The stack is implemented using two integer variables and a 1D array.

The array is declared in pseudocode as shown:

```
DECLARE ThisStack : ARRAY[1:60] OF REAL
```

The stack operates as follows:

- Global variable `SP` acts as a stack pointer that points to the next available stack location. The value of `SP` represents an array index.
- Global variable `OnStack` represents the number of values currently on the stack.
- The stack grows upwards from array element index 1.

- (a) (i) Give the initial values that should be assigned to the **two** variables.

`SP`

`OnStack`

[1]

- (ii) Explain why it is **not** necessary to initialise the array elements before the stack is used.

[2]

- (b) A function to add a value to `ThisStack` is expressed in pseudocode as shown.
The function will return a value to indicate whether the operation was successful or not.

Complete the pseudocode by filling in the gaps.

```
FUNCTION Push(ThisValue : REAL) RETURNS BOOLEAN
    DECLARE ReturnValue : BOOLEAN
    IF ..... THEN
        RETURN ..... // stack is already full
    ENDIF
    ..... ← ThisValue
    SP ← .....
    OnStack ← OnStack + 1
    RETURN TRUE
ENDFUNCTION
```

[4]

40. O/N 24/P22/Q6/a

- 6 A shop sells sandwiches and snacks. The owner chooses a 'daily special' sandwich which is displayed on a board outside the shop. Each 'daily special' has two different fillings and is made with one type of bread.

The owner wants a program to randomly choose the 'daily special' sandwich.

The program designer decides to store the possible sandwich fillings in a 1D array of type string.

The array is declared in pseudocode as follows:

```
DECLARE Filling : ARRAY [1:35] OF STRING
```

Each element contains the name of one filling.

An example of the first five elements is as follows:

Index	Element value
1	"Cheese"
2	"Onion"
3	"Salmon"
4	"Anchovies"
5	"Peanut Butter"

A second 1D array stores the possible bread used:

```
DECLARE Bread : ARRAY [1:10] OF STRING
```

Each element contains the name of one type of bread.

An example of the first three elements is as follows:

Index	Element value
1	"White"
2	"Brown"
3	"Pitta"

Both arrays may contain unused elements. The value of these will be an empty string and they may occur anywhere in each array.

A procedure `Special()` will output a message giving the 'daily special' sandwich made from **two** randomly selected different fillings and **one** randomly selected bread.

Unused array elements must **not** be used when creating the 'daily special' sandwich.

Using the above examples, the output could be:

```
"The daily special is Cheese and Onion on Brown bread."
```

(a) Complete the pseudocode for the procedure `Special()`.

Assume that both arrays are global.

```
PROCEDURE Special()
ENDPROCEDURE
```

[7]

41. O/N 24/P22/Q8/a,b,c,d

- 8 A program is being developed to implement a game for up to six players.

During the game, each player assembles a team of characters. At the start of the game there are 45 characters available.

Each character has four attributes, as follows:

Attribute	Examples	Comment
Player	0, 1, 3	The player the character is assigned to.
Role	Builder, Teacher, Doctor	The job that the character will perform in the game.
Name	Bill, Lee, Farah, Mo	The name of the character. Several characters may perform the same role, but they will each have a unique name.
Level	14, 23, 76	The skill level of the character. An integer in the range 0 to 100 inclusive.

The programmer has defined a record type to define each character.

The record type definition is shown in pseudocode as follows:

```

TYPE CharacterType
    DECLARE Player : INTEGER
    DECLARE Role : STRING
    DECLARE Name : STRING
    DECLARE Level : INTEGER
ENDTYPE

```

The `Player` field indicates the player to which the character is assigned (1 to 6). The field value is 0 if the character is **not** assigned to any player.

The programmer has defined a global array to store the character data as follows:

```

DECLARE Character : ARRAY[1:45] OF CharacterType

```

At the start of the game all record fields are initialised, and all `Player` field values are set to 0

The programmer has defined a program module as follows:

Module	Description
<code>Assign()</code>	- called with two parameters: - an integer representing a player - a string representing a character role - search the <code>Character</code> array for an unassigned character with the required role - If found, assign the character to the given player and output a confirmation message, for example: "Bill the Builder has been assigned to player 3" - If no unassigned character with the required role is found, output a suitable message.

(a) Write pseudocode for module `Assign()`.

[7]

(b) A new module will store the contents of the `Character` array in a text file.

The module is defined as follows:

Module	Description
Save ()	- form a string from each record with fields separated by the character '^' - write each string to a separate line of the new file named SaveFile.txt

Complete the pseudocode for module Save () .

PROCEDURE Save ()

ENDPROCEDURE

[7]

(c) The program is changed and the record type definition is modified as follows:

```
TYPE CharacterType
    DECLARE Player : INTEGER
    DECLARE Role : STRING
    DECLARE Name : STRING
    DECLARE Level : INTEGER
    DECLARE Status : BOOLEAN
ENDTYPE
```

Describe how the additional Boolean field may be stored with the rest of the fields on one line of a text file.

[2]

(d) The save operation is to be extended so that multiple files may be saved as the game progresses. This will allow the user to restore the game from any saved position. The filename must reflect the sequence in which the files are saved.

Describe a method that would allow multiple files to be saved **and** give an example of **two** consecutive filenames.

Method

Example

[2]

42. O/N 24/P23/Q1/b

(b) Complete the table by giving the appropriate data type in each case.

Variable	Example data value	Data type
Available	TRUE	
Received	"18/04/2021"	
Index	100	

[3]

43. O/N 24/P23/Q3/a

- 3 The implementation of a linked list uses an integer variable and a 1D array `List` of type `Node`.

Record type `Node` is declared in pseudocode as follows:

```
TYPE Node
  DECLARE Data : STRING
  DECLARE Pointer : INTEGER
ENDTYPE
```

The array `List` is declared in pseudocode as follows:

```
DECLARE List : ARRAY[1:200] OF Node
```

The linked list will operate as follows:

- Integer variable `HeadPointer` will store the array index for the first node in the linked list.
- The `Pointer` field of a node contains the index value of the next array element (the next node) in the linked list.
- The value 0 is used as a null pointer.

- (a) (i) State why the value 0 has been selected as the null pointer.

[1]

- (ii) Give the range of valid values that could be assigned to variable `HeadPointer`.

[1]

44. O/N 24/P23/Q4/a,b

- 4 An examination paper has a maximum of 75 marks. One of five pass grades (A to E) is assigned, depending on the mark obtained. The lowest mark for a given grade is known as the grade boundary.

A program is being written to process examination marks.

The five grade boundaries are stored in a global 1D array `GB` of type `INTEGER`, for example:

Index	Value	Comment
1	65	The minimum mark for an A grade.
2	57	The minimum mark for a B grade.
3	43	The minimum mark for a C grade.
4	35	The minimum mark for a D grade.
5	27	The minimum mark for an E grade.

Any paper that achieves a mark within 2 marks of a grade boundary must be checked. Using the given table, a paper with 45 marks would need to be checked.

- (a) The pseudocode algorithm to determine whether a paper should be checked is as shown. The mark for the paper is stored in variable `Mark`. Global variables `Mark`, `Index`, `Upper` and `Lower` are declared as integers.

Complete the pseudocode.

```
FOR Index ← 1 TO .....
    Lower ← GB[Index] - 2
    Upper ← .....
    IF Mark ..... AND Mark ..... THEN
        OUTPUT "Check this paper"
    ENDIF
NEXT Index
```

[4]

- (b) An alternative algorithm to determine if a paper needs to be checked uses a global 1D array `Check`, containing 76 elements of type `BOOLEAN`. The indices of the array are from 0 to 75 (inclusive), corresponding to the range of possible marks.

An element value in `Check` is `TRUE` if the index is within 2 marks of a grade boundary. For example, in the case where the C grade boundary is 43 the corresponding part of the `Check` array would be as follows:

Index	Value
40	FALSE
41	TRUE
42	TRUE
43	TRUE
44	TRUE
45	TRUE
46	FALSE

← The grade boundary for a C grade

- (i) The mark for a given paper is stored in variable `Mark`.

Describe how an algorithm would use the `Check` array to determine whether this paper should be checked.

[2]

- (ii) A procedure `GBInitialise()` will initialise the `Check` array using values from the `GB` array.

Note it can be assumed that the maximum grade boundary value for A is 70 and the minimum value for E is 15.

Write pseudocode for the procedure.

[6]

45. O/N 24/P23/Q8/a,b,c

- 8 A program is being developed to implement a game for up to six players.

During the game, each player assembles a team of characters. At the start of the game there are 45 characters available.

Each character has four attributes, as follows:

Attribute	Examples	Comment
Player	0, 1, 3	The player the character is assigned to.
Role	Builder, Teacher, Doctor	The job that the character will perform in the game.
Name	Bill, Lee, Farah, Mo	The name of the character. Several characters may perform the same role, but they will each have a unique name.
Skill level	14, 23, 76	An integer in the range 0 to 100, inclusive.

The programmer has defined a record type to define each character. The record type definition is shown in pseudocode as follows:

```

TYPE CharacterType
    DECLARE Player : INTEGER
    DECLARE Role : STRING
    DECLARE Name : STRING
    DECLARE SkillLevel : INTEGER
ENDTYPE

```

The `Player` field indicates the player to which the character is assigned (1 to 6). This field value is 0 if the character is **not** assigned to any player.

The programmer has defined a global array to store the character data, as follows:

```

DECLARE Character : ARRAY[1:45] OF CharacterType

```

At the start of the game all record fields are initialised, and all `Player` fields are set to 0

The programmer has defined a program module as follows:

Module	Description
<code>Count()</code>	- called with two parameters: - an integer representing a player - a string representing a character role - searches the <code>Character</code> array for characters with the given role that are assigned to the given player - counts the number of assigned characters and sums their total skill level - outputs the result of the search if characters with the given role are found, for example: <pre> "Player 3 has 4 characters with the role of Teacher and the total skill level is 65" </pre> - if no characters with the given role are found, outputs: <pre> "No characters with that role are assigned to this player" </pre>

- (a) Complete the pseudocode for module `Count()`.

```

PROCEDURE Count(ThisPlayer : INTEGER, ThisRole : STRING)

```

ENDPROCEDURE

[7]

- (b) The `Character` array data has been saved in the text file `SaveFile.txt`. Each line of the file contains one element of the array (one record).

New modules are defined:

Module	Description
<code>Extract()</code> (already written)	- called with two parameters: - a string representing a complete line from the text file - an integer representing a field number (see structure below) - returns a string representing the required field
<code>Restore()</code>	- opens the text file <code>SaveFile.txt</code> - reads lines from the file and assigns values to each record in the <code>Character</code> array using data from each line of the file

As a reminder, the record structure is repeated here:

```

TYPE CharacterType
    DECLARE Player : INTEGER           //Field number 1
    DECLARE Role : STRING              //Field number 2
    DECLARE Name : STRING              //Field number 3
    DECLARE SkillLevel : INTEGER       //Field number 4
ENDTYPE
  
```

Write pseudocode for module `Restore()`.

You must use the module `Extract()`.

[7]

- (c) The game can last for several days and users often find that they have to close and rerun the game program many times in order to complete it.

Describe the benefit of using the file `SaveFile.txt` as described in part (b).

[2]

46. M/J 24/P21/Q1/a

- 1 An algorithm is developed in pseudocode before being coded in a programming language.

- (a) The following table shows four valid pseudocode assignment statements.

Complete the table by giving an appropriate data type to declare each of the variables A, B, C and D.

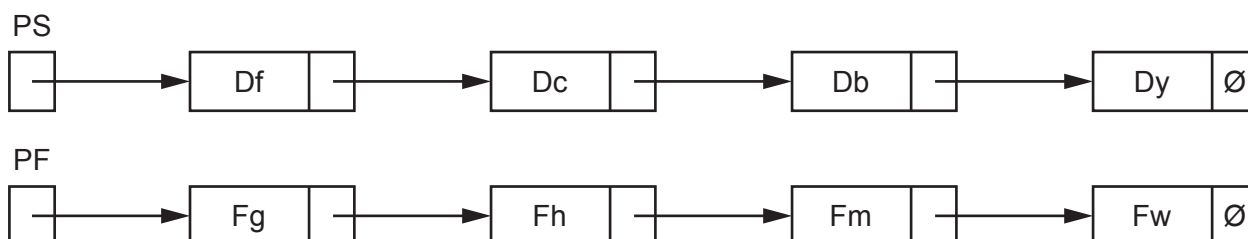
Assignment statement	Data type
<code>A ← LEFT(MyName, 1)</code>	
<code>B ← Total * 2</code>	
<code>C ← INT(ItemCost) / 3</code>	
<code>D ← "Odd OR Even"</code>	

[4]

47. M/J 24/P21/Q3/a,b

- 3 The diagram shows an Abstract Data Type (ADT) representation of a linked list after data items have been added.

- PS is the start pointer.
- PF is the free list pointer.
- Labels Df, Dc, Db and Dy represent the data items of nodes in the list.
- Labels Fg, Fh, Fm and Fw represent the data items of nodes in the free list.
- The symbol $\emptyset$ represents a null pointer.



- (a) Describe the linked list immediately after initialisation, before **any** data items are added.

[3]

- (b) A program will be written to include a linked list to store alphanumeric user IDs.

The design uses two variables and two 1D arrays to implement the linked list.
Each array element contains data of a single data type and **not** a record.

The statements below describe the design.

Complete the statements.

The two variables will be of type

The two variables will be used as to the arrays.

The values stored in the two variables will indicate

The first 1D array will be of type

The first 1D array will be used to

The second 1D array will be of type

The second 1D array will be used to

[5]

48. M/J 24/P21/Q4

4 A global 1D array `Data` contains 100 elements of type integer.

A function `Check()` will:

- total the element values in odd index locations (1, 3, 5 ... 97, 99)
- total the element values in even index locations (2, 4, 6 ... 98, 100)
- return one of three strings 'Odd', 'Even' or 'Same' to indicate which total is the greater, or whether the totals are the same.

Write pseudocode for the function `Check()`.

[6]

49. M/J 24/P21/Q8/b

- 8 A teacher is designing a program to process pseudocode projects written by her students.

Each student project is stored in a text file.

The process is split into a number of stages. Each stage performs a different task and creates a new file named as shown:

File name	Comment
MichaelAday_src.txt	student project file produced by student Michael Aday
MichaelAday_S1.txt	file produced by stage 1
MichaelAday_S2.txt	file produced by stage 2

The teacher has defined the first program module as follows:

Module	Description
DeleteComment()	- called with a parameter of type string representing a line of pseudocode from a student's project file - returns the line after removing any comments Note on comments: A comment starts with two forward slash characters and includes all the remaining characters on the line. The following example shows a string before and after the comment has been removed: Before: IF X2 > 13 THEN //check if limit exceeded After: IF X2 > 13 THEN

- (b) A second module is defined:

Module	Description
Stage_1()	- called with a parameter of type string representing a student name - creates a new stage 1 file - copies each line from the student's project file to the stage 1 file after removing any comment from each line - does not write blank lines to the stage 1 file - returns the number of lines written to the stage 1 file

Write pseudocode for module Stage_1().

Module DeleteComment() must be used in your solution.

50. M/J 24/P22/Q3/a,b

- 3** A factory needs a program to help manage its production of items.

Data will be stored about each item.

The data for each item will be held in a record structure of type `Component`.

The programmer has started to define the fields that will be needed as shown in the table.

Field	Example value	Comment
Item_Num	123478	a numeric value used as an array index
Reject	FALSE	TRUE if this item has been rejected
Stage	'B'	a letter to indicate the stage of production
Limit_1	13.5	any value in the range 0 to 100 inclusive
Limit_2	26.4	any value in the range 0 to 100 inclusive

- (a) (i)** Write pseudocode to declare the record structure for type `Component`.

[4]

- (ii)** A 1D array `Item` of 2000 elements will store the data for all items.

Write pseudocode to declare the `Item` array.

[2]

- (b)** State **three** benefits of using an array of records to store the data for all items.

1

2

3

[3]

51. M/J 24/P22/Q6/b

- 6** A music player stores music in a digital form and has a display which shows the track being played.

(b) Music is stored as a sequence of digital samples.

Each digital sample is a denary value in the range 0 to 99999999 (8 digits).

The samples are to be stored in a text file. Each sample is converted to a numeric string and 32 samples are concatenated (joined) to form a single line of the text file.

Each numeric string is 8 characters in length; leading '0' characters are added as required.

Example:

Sample	Denary value	String
1	456	"00000456"
2	48	"00000048"
3	37652	"00037652"
		
32	673	"00000673"

The example samples will be stored in the text file as a single line:

"000004560000004800037652...00000673"

- (i)** Identify one drawback of adding leading '0' characters to each numeric string.

[1]

- (ii)** Suggest an alternative method of storing the samples which does **not** involve adding leading '0' characters but which would still allow each individual sample to be extracted.

[1]

- (iii)** State **one** drawback of the alternative method given in part **(b)(ii)**.

[1]

52. M/J 24/P22/Q8/c

- 8** A teacher is designing a program to process pseudocode projects written by her students.

Each student project is stored in a text file.

The process is split into a number of stages. Each stage performs a different task and creates a new file.

For example:

File name	Comment
MichaelAday_src.txt	Student project file produced by student Michael Aday
MichaelAday_S1.txt	File produced by stage 1
MichaelAday_S2.txt	File produced by stage 2

(b) The teacher has defined the first program module as follows:

Module	Description
DeleteSpaces()	- called with a parameter of type string representing a line of pseudocode from a student's project file - returns the line after removing any leading space characters The following example shows a string before and after the leading spaces have been removed: Before: " IF X2 > 13 THEN" After: "IF X2 > 13 THEN"

Complete the pseudocode for module DeleteSpaces().

```
FUNCTION DeleteSpaces(Line : STRING) RETURNS STRING
```

```
ENDFUNCTION
```

[6]

(c) Two modules are defined:

Module	Description
DeleteComment() (already written)	- called with a parameter of type string representing a line of pseudocode from a student's project file - returns the line after removing any comment
Stage_2()	- called with two parameters: - a string representing an input file name - a string representing an output file name - copies each line from the input file to the existing output file having first removed all leading spaces and comments from that line - does not write blank lines to the output file - outputs a final message giving the number of blank lines removed

Write pseudocode for module Stage_2().

Modules DeleteComment() and DeleteSpaces() must be used in your solution.

[8]

53. M/J 24/P23/Q3/a

- 3** A record structure is declared to hold data relating to components being produced in a factory:

```
TYPE Component
  DECLARE Item_ID : STRING
  DECLARE Reject : BOOLEAN
  DECLARE Weight : REAL
ENDTYPE
```

The factory normally produces a batch (or set) of 1000 components at a time. A global array is declared to store 1000 records for a batch:

```
DECLARE Batch : ARRAY [1:1000] OF Component
```

Two global variables contain the minimum and maximum acceptable weight for each component. The values represent an inclusive range and are declared as:

```
DECLARE Min, Max : REAL
```

- (a) (i)** A program uses a variable `ThisIndex` as the array index to access a record.

Write a pseudocode clause to check whether or **not** the weight of an individual component is within the acceptable range.

[3]

- (ii)** When batches of less than 1000 components are processed, it is necessary to indicate that certain elements in the array are unused.

Suggest how an unused array element could be indicated.

[1]

54. M/J 24/P23/Q4/b

- 4** A procedure `TwoParts()` will input a sequence of real values, one at a time.

The procedure will:

- process the sequence in two parts
- form a first total by adding the values until the first zero
- form a second total by adding the values after the first zero until the second zero
- output the average of the two totals, together with a suitable message.

Values input in the first part are totalled using global variable `TotalA` and those input in the second part are totalled using global variable `TotalB`.

- (b)** The value zero denotes the split between the two parts of the sequence.

The requirement changes and now there may be up to 20 parts.

- (i)** Identify a suitable data structure that could be used to store the different total values.

[2]

- (ii)** Describe **three** benefits of using the data structure given in part **(b)(i)**.

1

2

3

[3]

55. M/J 24/P23/Q8/b

- 8 A teacher is designing a program to process pseudocode projects written by her students.

The program analyses a student project and extracts information about each module that is defined (each procedure or function). This information is stored in a global 2D array `ModInfo` of type string.

A module header is the first line of a module definition and starts with either of the keywords `PROCEDURE` or `FUNCTION`.

An example of part of the array is given below. Row 10 of the array shows that a procedure header occurs on line 27 and row 11 shows that a function header occurs on line 35. "P" represents a procedure and "F" represents a function:

	x = 1	x = 2	x = 3
<code>ModInfo[10, x]</code>	"27"	"P"	"MyProc(Z : CHAR)"
<code>ModInfo[11, x]</code>	"35"	"F"	"MyFun(Y : CHAR) RETURNS BOOLEAN"

The string stored in column 3 is called the module description. This is the module header **without** the keyword.

A valid module header will:

- be at least 13 characters long
- start with the keyword `PROCEDURE` or `FUNCTION`. The keyword may appear in either upper or lower case (or a mix of both) and **must** be followed by a space character.

The teacher has defined the first program module as follows:

Module	Description
<code>Header()</code>	• called with a parameter of type string representing a line of pseudocode • if the line is a valid procedure header, returns a string: "P<Module description>" • if the line is a valid function header, returns a string: "F<Module description>" • otherwise, returns an empty string For example, given the string: <pre>"FUNCTION Zap(X : INTEGER) RETURNS CHAR"</pre> <code>Header()</code> returns the string: <pre>"FZap(X : INTEGER) RETURNS CHAR"</pre>

(b) A new module is required:

Module	Description
FindModules()	- called with a parameter of type string representing a student project file name - uses module Header() to check each line of the project - assigns values to the ModInfo array for each module declaration in the student project

As a reminder, the previous example of part of the array is repeated below:

	x = 1	x = 2	x = 3
ModInfo[10, x]	"27"	"P"	"MyProc(Z : CHAR)"
ModInfo[11, x]	"35"	"F"	"MyFun(Y : CHAR) RETURNS BOOLEAN"

Write pseudocode for module FindModules().

Assume that the array contains enough rows for the number of modules in each project.

[8]

56. O/N 23/P21/Q1/d

1 The following pseudocode represents part of the algorithm for a program:

```

CASE OF ThisValue
    < 30      : Level ← "Low"           // less than 30
               Check ← 1
    < 20      : Level ← "Very Low"      // less than 20
               Check ← ThisValue / 2
    30 TO 40  : Level ← "Medium"        // between 30 and 40
               Check ← ThisValue / 3
               Data[ThisValue] ← Data[ThisValue] + 1
    > 40      : Level ← "High"
ENDCASE

```

(d) Give the appropriate data types for the variables ThisValue, Check and Level.

ThisValue

Check

Level

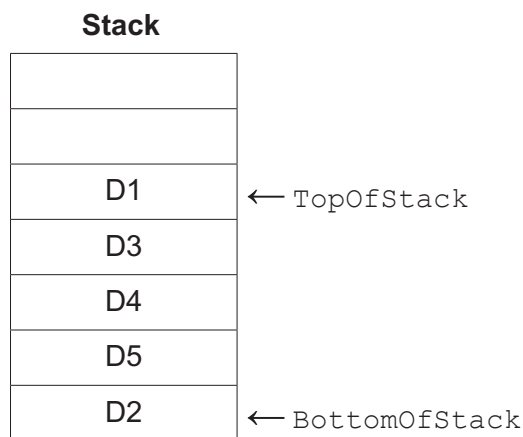
[3]

57. O/N 23/P21/Q3/a,b

3 The diagram represents an Abstract Data Type (ADT).

The operation of this stack may be summarised as follows:

- The `TopOfStack` pointer points to the last item added to the stack.
- The `BottomOfStack` pointer points to the first item on the stack.

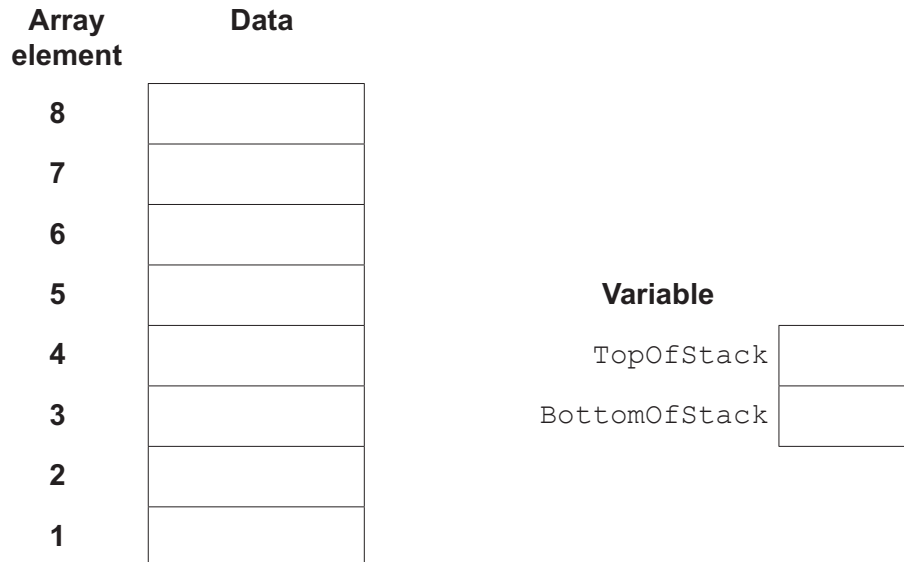


- (a) The stack is implemented using two variables and a 1D array of 8 elements as shown.

The variables are used to reference individual elements of the array, in such a way that:

- the array is filled from the lowest indexed element towards the highest
- all the elements of the array are available for the stack.

Complete the diagram to represent the state of the stack as shown above.



[3]

- (b) A function `Push()` will add a value onto the stack by manipulating the array and variables in part (a).

Before adding a value onto the stack, the algorithm will check that space is available.

If the value is added to the stack, the function will return `TRUE`, otherwise it will return `FALSE`.

The algorithm is expressed in five steps.

Complete the steps.

1. If then return FALSE
2. Otherwise TopOfStack
3. Use TopOfStack as an to the array.
4. Set the element at this to the being added.
5. Return

[5]

58. O/N 23/P21/Q4/a,b

- 4 A global array is declared in pseudocode as follows:

```
DECLARE Data : ARRAY[1:150] OF STRING
```

A function TooMany() will:

1. take two parameters:
 - a string (the search string)
 - an integer (the maximum value)
2. count the number of strings in the array that exactly match the search string
3. return TRUE if the count is greater than the maximum value, otherwise will return FALSE

- (a) Write pseudocode for the function TooMany() .

[6]

- (b) The global array is changed to a 2D array, organised as 150 rows by 2 columns. It is declared in pseudocode as follows:

```
DECLARE Data : ARRAY[1:150, 1:2] OF STRING
```

The algorithm for the function in **part (a)** is changed. Strings will only be counted if **both** of the following conditions are true:

- The current row is an even number.
- The search string exactly matches the value in **either** column.

Write pseudocode to check these conditions.

Assume that the row index is contained in variable Row and the search string in variable Search.

[3]

59. O/N 23/P21/Q8/a,b

- 8 A class of students are developing a program to send data between computers. Many computers are connected together to form a wired network. Serial ports are used to connect one computer to another.

Each computer:

- is assigned a unique three-digit ID
- has three ports, each identified by an integer value
- is connected to between one and three other computers.

Messages are sent between computers as a string of characters organised into fields as shown:

<STX><DestinationID><SourceID><Data><ETX>

Field number	Field name	Description
n/a	STX	a single character marking the start of the message (ASCII value 02)
1	DestinationID	three numeric characters that identify the destination computer
2	SourceID	three numeric characters that identify the source computer
3	Data	a variable length string containing the data being sent (Minimum length is 1 character)
n/a	ETX	a single character marking the end of the message (ASCII value 03)

For example, the following message contains the data "Hello Kevin" being sent from computer "101" to computer "232":

<STX>"232101Hello Kevin"<ETX>

Each computer will run a copy of the same program. Each program will contain a global variable, `MyID` of type string, that contains the unique ID of the computer in which the program is running.

The programmer has defined the first two program modules as follows:

Module	Description
<code>Transmit()</code> (already written)	• takes two parameters: ◦ a string containing a message ◦ an integer containing a port number • transmits the message using the given port
<code>SendFile()</code>	• takes three parameters: ◦ a string containing a text file name ◦ a string containing a Destination ID ◦ an integer containing a Port number • transmits the file one line at a time • transmits a final message with data string "*****"

- (a) Write pseudocode for module `SendFile()`.

Assume:

- module `Transmit()` has already been written and is used to transmit a message
- the value of `MyID` may be used as `SourceID`
- the file specified contains no blank lines
- the file specified does not contain the line "*****"

[7]

(b) Module `SendFile()` is used to copy a file from one computer to another.

A module within the program running on the destination computer will receive the data and write it to a new file.

Explain why module `SendFile()` transmits the message with data string "*****" after the last line of the file.

[2]

60. O/N 23/P22/Q1/c

- 1 A shop sells car accessories. A customer order is created if an item cannot be supplied from current stock. A program is being developed to create and manage the customer orders.

(a) The following identifier table shows some of the data that will be stored for each order.

Complete the identifier table by adding meaningful variable names and appropriate data types.

Example value	Explanation	Variable name	Data type
"Mr Khan"	The name of the customer		
3	The number of items in the order		
TRUE	To indicate whether this is a new customer		
15.75	The deposit paid by the customer		

[4]

(c) The data that needs to be stored for each customer order in part (a) is not all of the same type.

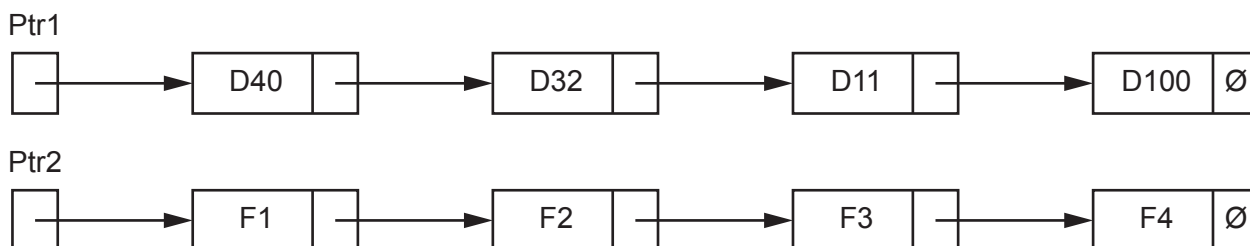
Describe an effective way of storing this data for many customer orders while the program is running.

[3]

61. O/N 23/P22/Q3/a,b

3 The diagram represents a linked list Abstract Data Type (ADT).

- Ptr1 is the start pointer. Ptr2 is the free list pointer.
- Labels D40, D32, D11 and D100 represent the data items of nodes in the list.
- Labels F1, F2, F3 and F4 represent the data items of nodes in the free list.
- The symbol $\emptyset$ represents a null pointer.



(a) The linked list is implemented using two variables and two 1D arrays as shown.

The pointer variables and the elements of the Pointer array store the indices (index numbers) of elements in the Data array.

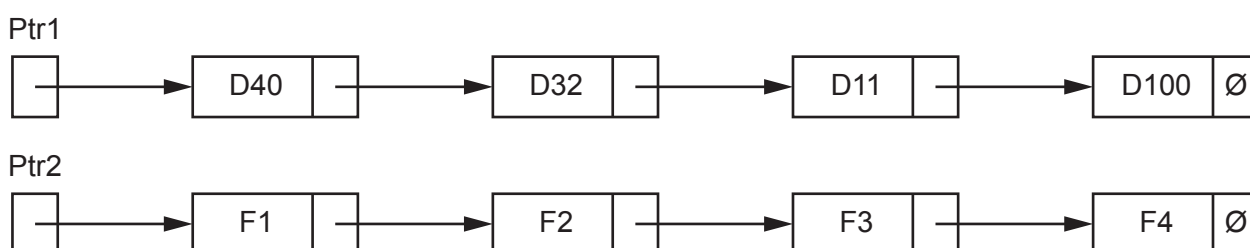
Complete the diagram to show how the linked list as shown above may be represented using the variables and arrays.

Variable		Value
Start_Pointer		
Free_List_Pointer		5

Index	Data array	Pointer array
1	D32	2
2		3
3		
4	D40	
5		
6	F2	7
7		
8		

[5]

(b) The original linked list is to be modified. A new node D6 is inserted between nodes D32 and D11.



The algorithm required is expressed in four steps as shown.

Complete the steps.

1. Assign the data item to
2. Set the of this node to point to
3. Set Ptr2 to point to
4. Set pointer of to point to

[4]

62. O/N 23/P22/Q6/a,b(iii)

6 (a) A procedure `CreateFiles()` will take two parameters:

- a string representing a file name
- an integer representing the number of files to be created.

The procedure will create the number of text files specified.

Each file is given a different name. Each file name is formed by concatenating the file name with a suffix based on the file number. The suffix is always three characters.

For example, the call `CreateFiles("TestData", 3)` would result in the creation of the three files, `TestData.001`, `TestData.002` and `TestData.003`.

Each file will contain a single line. For example, file `TestData.002` would contain the string:

```
This is File TestData.002
```

Write pseudocode for `CreateFiles()`.

Assume both parameters are valid and that the integer value is between 1 and 999, inclusive.

[6]

(b) A module `CheckFiles()` will count the number of files produced by `CreateFiles()` in part **(a)**.

`CheckFiles()` will take a string representing a file name and return the number of files found.

(iii) State the file mode that should be used in `CheckFiles()`.

[1]

63. O/N 23/P22/Q8/a,c

- 8 A class of students are developing a program to send data between computers. Many computers are connected together to form a wired network. Serial ports are used to connect one computer to another.

Each computer:

- is assigned a unique three-digit ID
- has three ports, each identified by an integer value
- is connected to between one and three other computers.

Data is sent as individual message strings.

Each string contains the destination ID (the ID of the computer that is to receive the message) followed by the data:

```
<DestinationID><Data>
```

Messages may pass through several computers on the way to their destination.

When a message arrives at a computer, that is **not** the destination, the program needs to forward it on to another computer using one of its serial ports.

The port to use is obtained from information that is stored in an array `RouteTable`.

`RouteTable` is a global 2D array of integers. It is declared in pseudocode as follows:

```
DECLARE RouteTable : ARRAY[1:6,1:3] OF INTEGER
```

The values in the first two columns of `RouteTable` define a range of ID values.

Column 3 gives the corresponding port number to use when forwarding the message to a computer with an ID within this range.

For example, the contents of `RouteTable` could be:

	Column 1	Column 2	Column 3
Row 1	100	199	1
Row 2	200	259	2
Row 3	-1	<undefined>	<undefined>
Row 4	260	399	2
Row 5	400	599	3
Row 6	600	999	1

In this example, a message that arrives with a `DestinationID` of "283" will be forwarded using port 2.

Row 3 in the example shows an unused row. These may occur anywhere. Unused rows have the column 1 element set to -1. The value of unused elements in the other two columns is undefined.

The programmer has defined the first program module as follows:

Module	Description
<code>GetPort()</code>	- takes a <code>DestinationID</code> as a parameter of type string - searches for the range corresponding to the <code>DestinationID</code> in the array - returns the port number, or returns <code>-1</code> if no corresponding range is found

(a) Write pseudocode for module `GetPort()`.

Assume `DestinationID` contains a valid three-digit string.

[7]

(b) Copies of the same program will run on each computer. The program contains a global variable `MyID` of type string, which contains the unique ID of the computer in which the program is running.

When messages are received, they are placed on one of two stacks. Stack 1 is used for messages that have reached their destination and stack 2 is used for messages that will be forwarded on to another computer.

Additional modules are defined:

Module	Description
<code>StackMsg()</code> (already written)	- takes two parameters: - a string representing a message - an integer representing the stack number - adds the message to the required stack - returns <code>TRUE</code> if the message is added to the required stack, otherwise returns <code>FALSE</code>
<code>ProcessMsg()</code>	- takes a message as a parameter of type string - ignores any message with a zero-length data field - extract the <code>DestinationID</code> from the message - checks whether the <code>DestinationID</code> is this computer or whether the message is to be forwarded - uses <code>StackMsg()</code> to add the message to the appropriate stack - outputs an error if the message could not be added to the stack

Write pseudocode for module `ProcessMsg()`.

Module `StackMsg()` must be used.

[7]

(c) The program contains a module `GetFile()` which receives text files sent from another computer.

Lines from the file are sent one at a time. Each message contains one line and `ProcessMsg()` from part (b) adds each message as it is received onto stack 1.

Module `GetFile()` removes messages from stack 1 and writes the data to a text file.

There is a problem. Under certain circumstances, the received file does not appear as expected.

Assume that while a file is being received `ProcessMsg()` receives only messages containing lines from the file.

- (i) Describe the circumstances and explain the problem.

Circumstances

Explanation

[3]

- (ii) Suggest a more appropriate Abstract Data Type that could be used to store the messages that would not have the same problem.

[1]

64. O/N 23/P23/Q1/a

- 1 A program is being developed in pseudocode before being converted into a programming language.

- (a) The following table shows four valid pseudocode assignment statements.

Complete the table by giving the data type that should be used to declare the variable underlined in each assignment statement.

Assignment statement	Data type
<u>MyVar1</u> ← Total1 / Total2	
<u>MyVar2</u> ← 27/10/2023	
<u>MyVar3</u> ← "Sum1 / Sum2"	
<u>MyVar4</u> ← Result1 AND Result2	

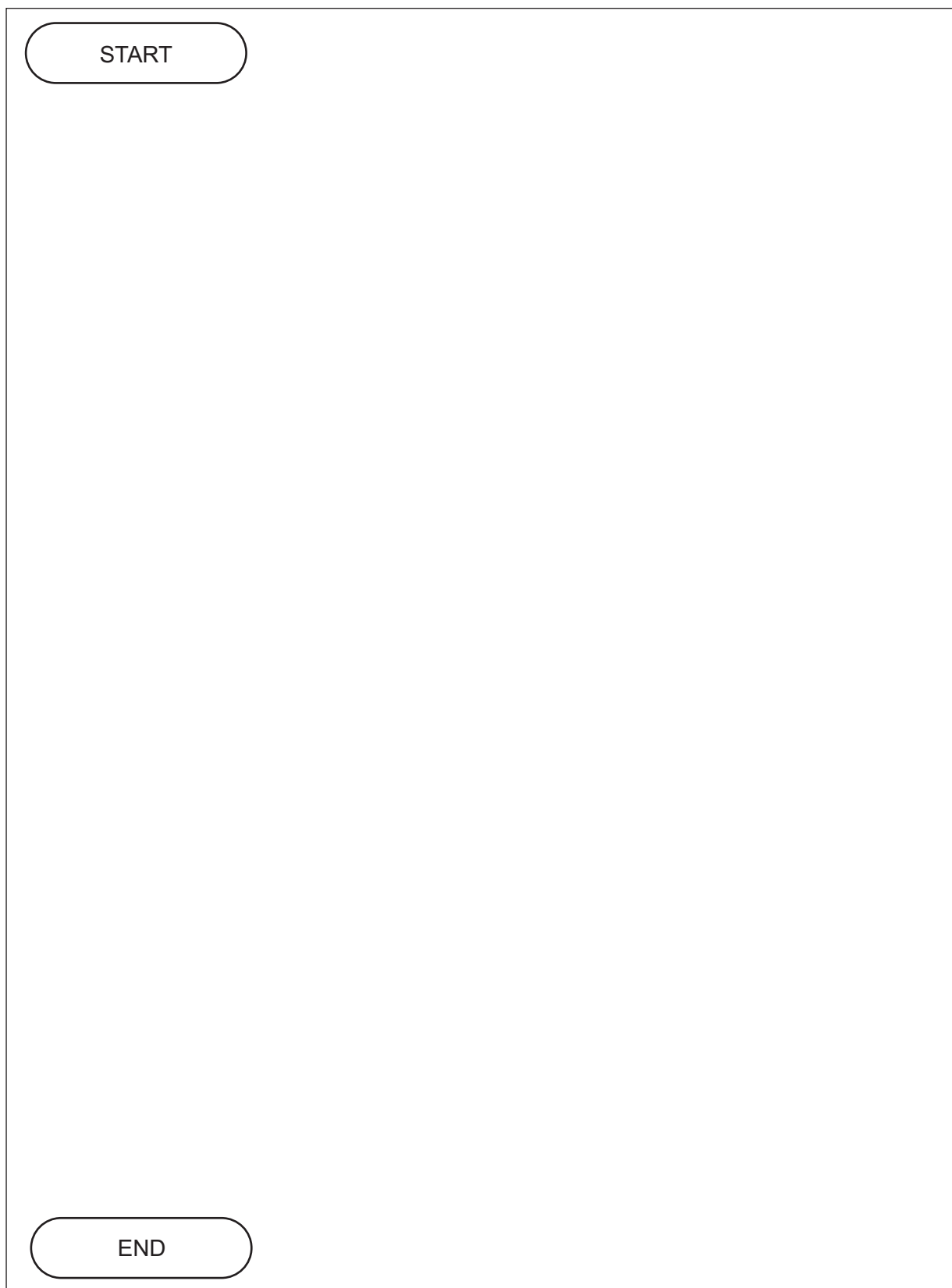
[4]

65. O/N 23/P23/Q2/b,c

- 2 Data is a 1D array of integers, containing 30 elements. All element values are unique.

- (a) An algorithm will output the index of the element with the smallest value.

Draw a program flowchart to represent the algorithm.



[5]

(b) The 30 data values could have been stored in separate variables rather than in an array.

Explain the benefits of using an array when designing a solution to part **(a)**.

[2]

- (c) The requirement changes. Array `Data` needs to hold 120 elements and each value may include a decimal place.

Write a pseudocode statement to declare the modified array.

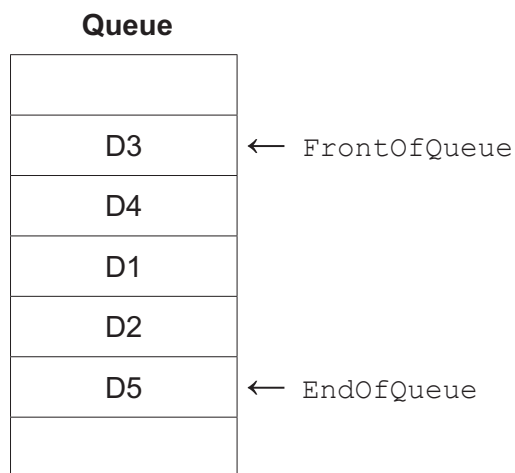
[2]

66. O/N 23/P23/Q3/a,b

- 3 The diagram represents a queue Abstract Data Type (ADT).

The organisation of this queue may be summarised as follows:

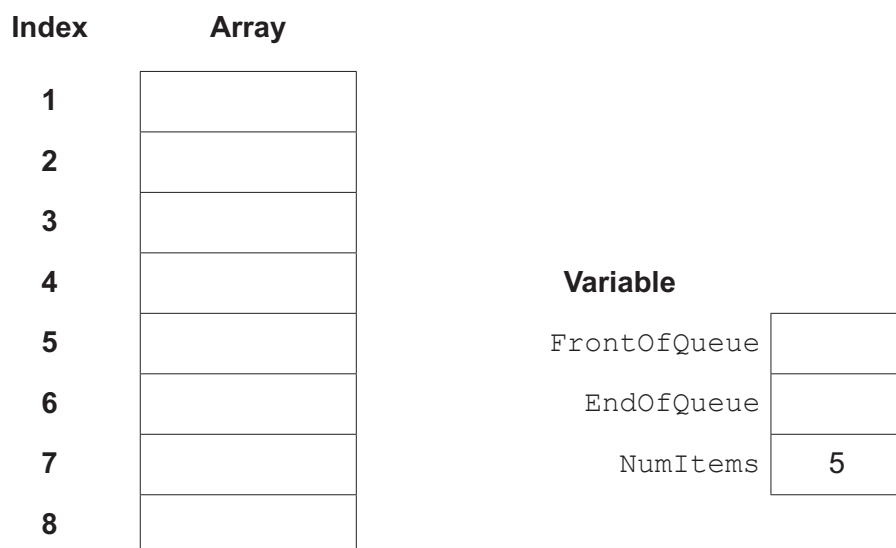
- The `FrontOfQueue` pointer points to the next data item to be removed.
- The `EndOfQueue` pointer points to the last data item added.



The queue is implemented using three variables and a 1D array of eight elements as shown. The variable `NumItems` stores the number of items in the queue.

The pointer variables store indices (index numbers) of the array.

- (a) Complete the diagram to represent the state of the queue as shown above.



[3]

- (b) A module `AddTo()` will add a value to the queue by manipulating the array and variables in part (a).

The queue implementation is circular. When pointers reach the end of the queue, they will 'wrap around' to the beginning.

Before a value can be added to the queue, it is necessary to check the queue is not full.

The algorithm to add a value to the queue is expressed in six steps.

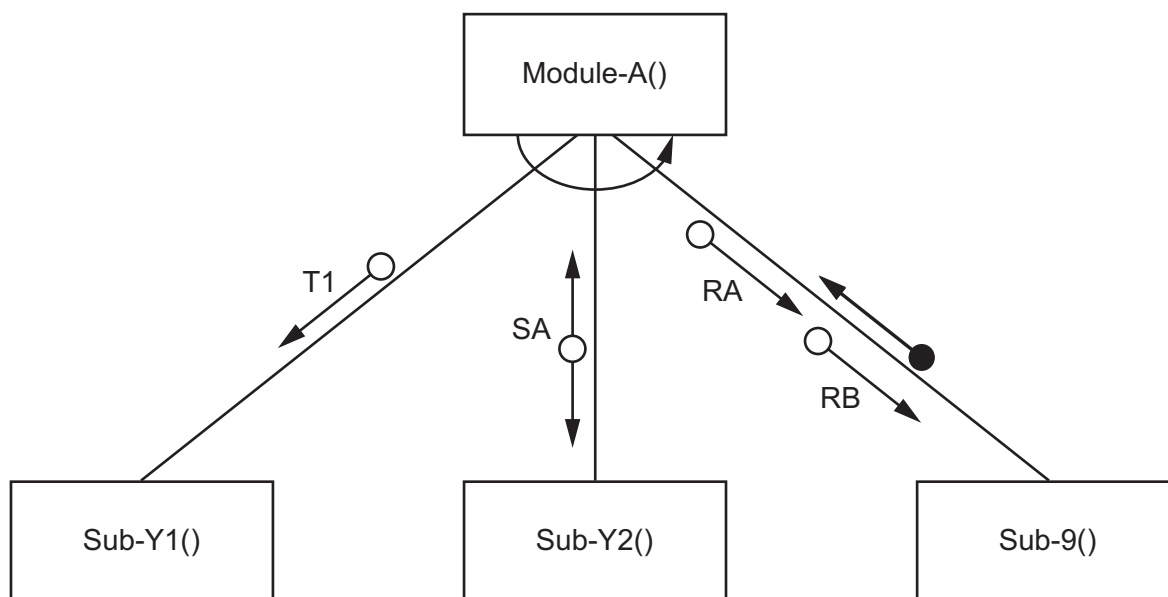
Complete the steps.

1. If NumItems then jump to step 6.
2. Increment
3. If then set EndOfQueue to
4. Increment
5. Set the at the index stored in to the being added.
6. Stop.

[6]

67. O/N 23/P23/Q7/b(i)

- 7 A structure chart shows the modular structure of a program:



- (b) The structure chart shows that Sub-9() is a function.

A Boolean value is returned by Sub-9() for processing by Module-A().

The original parameter RA is of type integer and RB is of type string.

A record type `MyType` will be defined with three fields to store the values passed between the two modules.

- (i) Write pseudocode to define `MyType`.

[3]

68. O/N 23/P23/Q8/a

- 8 A class of students are developing a program to send data between computers. Many computers are connected together to form a wired network. Serial ports are used to connect one computer to another.

Each computer:

- is assigned a unique three-digit ID
- has three ports, each identified by an integer value
- is connected to between one and three other computers.

Messages are sent between computers as a string of characters organised into fields as shown:

`<STX><DestinationID><SourceID><Data><ETX>`

Field name	Description
STX	a single character marking the start of the message (ASCII value 02)
DestinationID	three numeric characters identifying the destination computer
SourceID	three numeric characters identifying the source computer
Data	a variable length string containing the data being sent (Minimum length is 1 character)
ETX	a single character marking the end of the message (ASCII value 03)

For example, the following message contains the data "Hello Jack" being sent from computer "202" to computer "454":

`<STX>"454202Hello Jack"<ETX>`

Each computer will run a copy of the same program. Each program will contain a global variable `MyID` of type string which contains the unique ID of the computer in which the program is running.

The first two program modules are defined as follows:

Module	Description
<code>GetData()</code> (already written)	• returns the data field from a message that has been received • If no message is available, the module waits until one has been received.
<code>ReceiveFile()</code>	• takes a file name as a parameter of type string • creates a text file with the given file name (no checking required) • writes the data field returned by <code>GetData()</code> to the file • repeats until the data field is "*****", which is not written to the file • outputs a final message giving the total number of characters written to the file, for example: 132456 characters were written to newfile.txt

- (a) Write pseudocode for module `ReceiveFile()`.

Module `GetData()` has already been written and must be used.

[7]

69. M/J 23/P21/Q3/a

- 3** Customers collect points every time they make a purchase at a store.

A program is used to manage the points system and the table lists some of the information stored for one customer.

Information	Data type required
Name	String
Number of points collected	Integer
Date of birth	Date

- (a) (i)** Identify a suitable structure for storing the information for one customer. Explain the advantage of using this structure.

Structure

Advantage

[4]

- (ii)** Describe a data structure that could be used to store the information for **all** customers.

[2]

70. M/J 23/P21/Q6

- 6 A video-conferencing program supports up to six users. Speech from each user is sampled and digitised (converted from analogue to digital). Digitised values are stored in array `Sample`.

The array `Sample` consists of 6 rows by 128 columns and is of type integer. Each row contains 128 digitised sound samples from one user.

The digitised sound samples from each user are to be processed to produce a single value which will be stored in a 1D array `Result` of type integer. This process will be implemented by procedure `Mix()`.

A procedure `Mix()` will:

- calculate the average of each of the 6 sound samples in a column
- ignore sound sample values of 10 or less
- store the average value in the corresponding position in `Result`
- repeat for each column in array `Sample`

The diagram uses example values to illustrate the process:

		1	2	3	...	126	127	128
Sample:	1	20	20	20		30	30	2
	2	20	20	30		50	30	3
	3	20	20	40		40	40	4
	4	20	20	50		40	50	20
	5	20	3	5		6	60	4
	6	20	4	2		4	70	30
		↓	↓	↓		↓	↓	↓
Result:		20	20	35		40	46	25

Information for each item is stored as a single line in the `Stock.txt` file in the format:

`<ItemNum><SupplierCode><Description>`

Item information is as follows:

	Format	Comment
ItemNum	4 numeric characters	unique for each item in the range "0001" to "5999" inclusive
SupplierCode	5 alphabetic characters	to identify the supplier of the item
Description	a string	a minimum of 12 characters

The file is organised in ascending order of `ItemNum` and does **not** contain all possible values in the range.

A programmer has started to define program modules as follows:

Module	Description
<code>SuppExists()</code> (already written)	- called with a parameter of type string representing a supplier code - returns <code>TRUE</code> if the supplier code is already in use, otherwise returns <code>FALSE</code>
<code>IsNewSupp()</code>	- called with a parameter of type string representing a new supplier code - returns <code>TRUE</code> if the string only contains alphabetic characters (either upper or lower case) and the supplier code is not already in use, otherwise returns <code>FALSE</code>

(b) A new module has been defined:

Module	Description
<code>CheckNewItem()</code>	- called with a parameter of type string representing a line of item information - checks to see whether an item with the same <code>ItemNum</code> already exists in the file - returns <code>TRUE</code> if the <code>ItemNum</code> is not already in the file, otherwise returns <code>FALSE</code>

Write **efficient** pseudocode for module `CheckNewItem()`.

[7]

(d) A new module `AddItem()` will be used to add information to the `Stock.txt` file.

State the file mode that should be used for the algorithm within this module.

[1]

- (e) A new module `FindItem()` searches for a given item in the `Stock.txt` file, which is already organised in ascending order of `ItemNum`.

Describe how this organisation may improve the efficiency of the algorithm.

[3]

72. M/J 23/P22/Q1/c

- 1 A program calculates the postal cost based on the weight of the item and its destination. Calculations occur at various points in the program and these result in the choice of several possible postal costs. The programmer has built these postal costs into the program.

For example, the postal cost of \$3.75 is used in the following lines of pseudocode:

```
IF Weight < 250 AND ValidAddress = TRUE THEN
    ItemPostalCost ← 3.75      // set postal cost for item to $3.75
    ItemStatus ← "Valid"      // item can be sent
ENDIF
```

- (c) Give the **appropriate** data types for the following variables:

`ValidAddress`

`ItemPostalCost`

`ItemStatus`

[3]

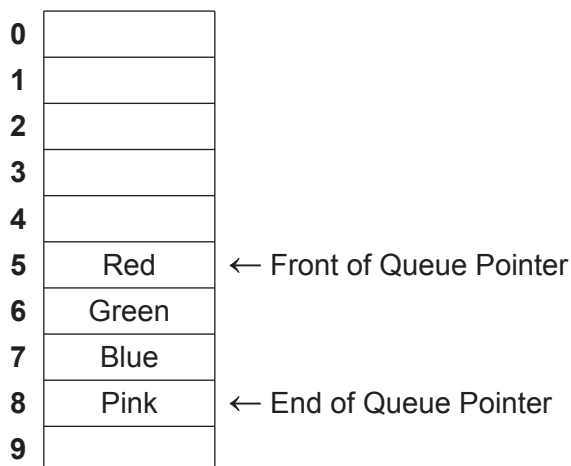
73. M/J 23/P22/Q3/a

3 A program stores data in a text file. When data is read from the file, it is placed in a queue.

- (a) The diagram below represents an Abstract Data Type (ADT) implementation of the queue. Each data item is stored in a separate location in the data structure. During initial design, the queue is limited to holding a maximum of 10 data items.

The operation of this queue may be summarised as follows:

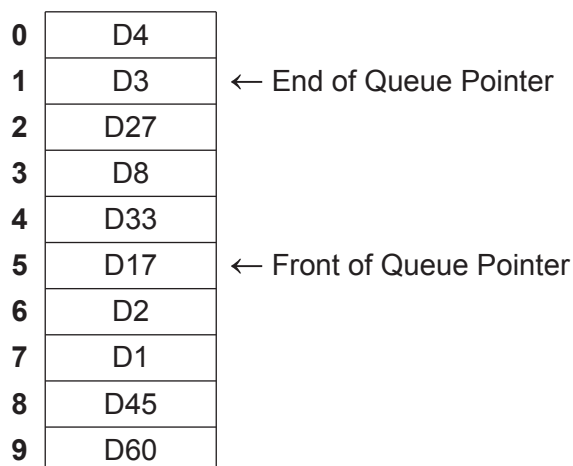
- The Front of Queue Pointer points to the next data item to be removed.
- The End of Queue Pointer points to the last data item added.
- The queue is circular so that locations can be reused.



- (i) Describe how the data items Orange and Yellow are added to the queue shown in the diagram.

[4]

- (ii) The following diagram shows the state of the queue after several operations have been performed. All queue locations have been used at least once.



State the number of data items in the queue.

[1]

74. M/J 23/P22/Q8/b

- 8 A computer shop assembles computers using items bought from several suppliers. A text file `Stock.txt` contains information about each item.

Information for each item is stored as a single line in the `Stock.txt` file in the format:

`<ItemNum><SupplierCode><Description>`

Valid item information is as follows:

	Format	Comment
ItemNum	4 numeric characters	unique number for each item in the range "0001" to "5999" inclusive
SupplierCode	3 alphabetic characters	to identify the supplier of the item
Description	a string	a minimum of 12 characters

The file is organised in ascending order of `ItemNum` and does **not** contain all possible values in the range.

A programmer has started to define program modules as follows:

Module	Description
<code>OnlyAlpha()</code> (already written)	- called with a parameter of type string - returns <code>TRUE</code> if the string contains only alphabetic characters, otherwise returns <code>FALSE</code>
<code>CheckInfo()</code>	- called with a parameter of type string representing a line of item information - checks to see whether the item information in the string is valid - returns <code>TRUE</code> if the item information is valid, otherwise returns <code>FALSE</code>

(b) A new module is defined as follows:

Module	Description
<code>AddItem()</code>	- called with a parameter of type string representing valid information for a new item that is not currently in the <code>Stock.txt</code> file - creates a new file <code>NewStock.txt</code> from the contents of the file <code>Stock.txt</code> and adds the new item information at the appropriate place in the <code>NewStock.txt</code> file

As a reminder, the file `Stock.txt` is organised in ascending order of `ItemNum` and does not contain all possible values in the range.

Write pseudocode for module `AddItem()`.

75. M/J 23/P23/Q1/a,c

- 1 The following pseudocode represents part of the algorithm for a program.

Line numbers are for reference only.

```

10  DECLARE Sheet4 : ARRAY[1:2, 1:50] OF INTEGER
...
100 FOR PCount ← 0 TO 49
101     Sheet4[1, PCount] ← 0
102     Sheet4[2, PCount] ← 47
103 NEXT PCount

```

- (a) The pseudocode contains references to an array.

Complete the table by writing the answer for each row.

Answer

The dimension of the array	
The name of the variable used as an array index	
The number of elements in the array	

[3]

- (c) The pseudocode does not include a declaration for PCount.

State the data type that should be used in the declaration.

[1]

76. M/J 23/P23/Q3/a,b(i)

- 3 A program processes data using a stack. The data is copied to a text file before the program ends.

- (a) The following diagram shows the current state of the stack.

The operation of this stack may be summarised as follows:

- The `TopOfStack` pointer points to the last item added to the stack.
- The `BottomOfStack` pointer points to the first item on the stack.
- The stack grows upwards when items are added.

Stack		Pointer
Memory location	Value	
506		
505	WWW	← TopOfStack
504	YYY	
503	XXX	
502	ZZZ	
501	NNN	
500	PPP	← BottomOfStack

- (i) An error will be generated if an attempt is made to POP a value when the stack is empty.

State the maximum number of consecutive POP operations that could be performed on the stack shown above **before** an error is generated.

[1]

- (ii) The following operations are performed:

1. POP and store value in variable `Data1`
2. POP and store value in variable `Data2`
3. PUSH value AAA
4. PUSH value BBB
5. POP and discard value
6. POP and store value in variable `Data2`

Complete the diagram to show the state of the stack and the variables **after** the given operations have been performed.

Stack		Pointer
Memory location	Value	
506		
505		
504		
503		
502		
501		
500		

Variable	Value
<code>Data1</code>	
<code>Data2</code>	

[4]

- (b) The data is copied to a text file before the program ends.

- (i) State an advantage of writing the data from the stack to a text file before the program ends.

[1]

77. M/J 23/P23/Q8/a,b

- 8 A computer shop assembles desktop computers, using items bought from several suppliers. A text file `Stock.txt` contains information about each item.

Information for each item is stored as a single line in the `Stock.txt` file in the format:

```
<ItemNum><SupplierCode><Description>
```

Item information is as follows:

	Format	Comment
ItemNum	4 numeric characters	unique number for each item in the range “0001” to “5999” inclusive
SupplierCode	3 alphabetic characters	code to identify the supplier of the item
Description	a string	a minimum of 12 characters

The file is organised in ascending order of `ItemNum` and does not contain all possible values in the range.

The programmer has defined the first program module as follows:

Module	Description
<code>ChangeSupp()</code>	- called with two parameters <code>Code1</code> and <code>Code2</code> of type string that represent valid supplier codes - creates a new file <code>NewStock.txt</code> from the contents of the file <code>Stock.txt</code> where any reference to <code>Code1</code> is replaced by <code>Code2</code> - returns a count of the number of items that have had their supplier code changed

- (a) Write pseudocode for module `ChangeSupp()`.

[8]

(b) A new module is required:

Module	Description
Report_1()	- takes a parameter of type string that represents a SupplierCode - searches the Stock.txt file for each line of item information that contains the given SupplierCode - produces a formatted report of items for the given SupplierCode, for example, for supplier DRG, the output could be: <pre> Report for Supplier: DRG Item Description 1234 USB Printer Cable 3m 1273 32GB USB Flash Drive 1350 Mouse Mat 320 x 240mm Number of items listed: 3 </pre>

Write pseudocode for module Report_1().

[6]

78. O/N 22/P21/Q1/b

(b) The following pseudocode is part of a program that stores names and test marks for use in other parts of the program.

```

DECLARE Name1, Name2, Name3 : STRING
DECLARE Mark1, Mark2, Mark3 : INTEGER
INPUT Name1
INPUT Mark1
INPUT Name2
INPUT Mark2
INPUT Name3
INPUT Mark3

```

(i) The pseudocode needs to be changed to allow for data to be stored for up to 30 students.

Explain why it would be good practice to use arrays to store the data.

[3]

(ii) The following pseudocode statement includes array references:

```
OUTPUT "Student ", Name[Count], " scored ", Mark[Count]
```

State the purpose of the variable Count and give its data type.

Purpose

Data type

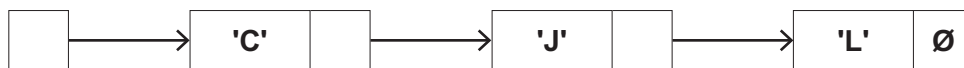
[2]

79. O/N 22/P21/Q4/a,b

- 4 (a) The following diagram shows an Abstract Data Type (ADT) representation of an ordered linked list. The data item stored in each node is a single character. The data will be accessed in alphabetical order.

The symbol $\emptyset$ represents a null pointer.

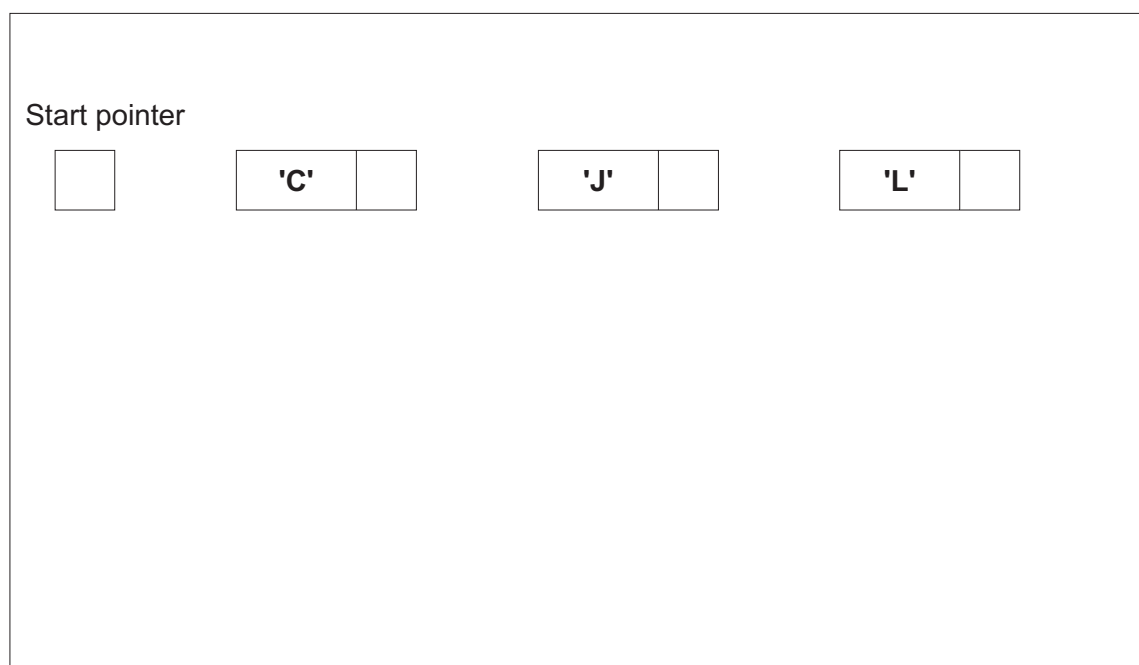
Start pointer



- (i) Nodes with data 'A' and 'K' are added to the linked list. Nodes with data 'J' and 'L' are deleted.

After the changes, the data items still need to be accessed in alphabetical order.

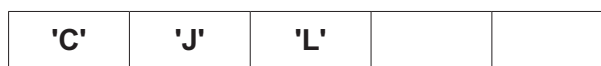
Complete the diagram to show the new state of the linked list.



[4]

- (ii) The original data could have been stored in a 1D array in which each element stores a character.

For example:



Explain the advantages of making the changes described in **part (a)(i)** when the data is stored in the linked list instead of an array.

[2]

- (iii) Explain the disadvantages of making the changes described in **part (a)(i)** when the data is stored in the linked list instead of an array.

[2]

- (b) A program will store data using a linked list like the one shown in **part (a)**.

Explain how the linked list can be implemented.

[4]

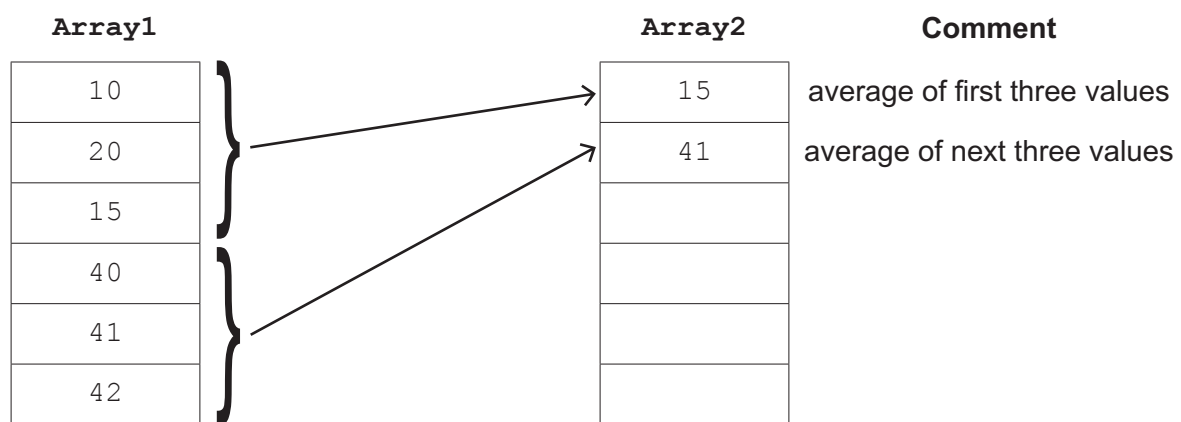
80. O/N 22/P21/Q5

- 5 A program uses two 1D arrays of type integer. `Array1` contains 600 elements and `Array2` contains 200 elements.

`Array1` contains sample values read from a sensor. The sensor always takes three consecutive samples and all of these values are stored in `Array1`.

A procedure `Summarise()` will calculate the average of three consecutive values from `Array1` and write the result to `Array2`. This will be repeated for all values in `Array1`.

The diagram below illustrates the process for the first six entries in `Array1`.



Write pseudocode for the procedure `Summarise()`.

[5]

81. O/N 22/P21/Q8/a,c

- 8 A teacher is designing a program to perform simple syntax checks on programs written by students. Student programs are submitted as text files, which are known as project files.

A project file may contain blank lines.

The teacher has defined the first program module as follows:

Module	Description
<code>CheckFile()</code>	• takes the name of an existing project file as a parameter of type string • returns <code>TRUE</code> if the file is valid (it contains at least 10 non-blank lines), otherwise returns <code>FALSE</code>

- (a) Write pseudocode for module `CheckFile()`.

[7]

Further modules are defined as follows:

Module	Description
<code>CheckLine()</code>	• takes a line from a project file as a parameter of type string • returns zero if the line is blank or contains no syntax error, otherwise returns an error number as an integer
<code>CountErrors()</code>	• takes two parameters: ◦ the name of a project file as a string ◦ the maximum number of errors as an integer • uses <code>CheckFile()</code> to test the project file. Outputs an error message and ends if the project file is not valid • calls <code>CheckLine()</code> for each line in the project file • counts the number of errors • outputs the number of errors or a warning message if the maximum number of errors is exceeded

- (c) Write pseudocode for module `CountErrors()`. Assume `CheckFile()` and `CheckLine()` have been written and can be used in your solution.

[8]

82. O/N 22/P22/Q3/a,b,c

- 3 A stack is used in a program to store string data which needs to be accessed in several modules.

- (a) A stack is an example of an Abstract Data Type (ADT).

Identify **one other** example of an ADT **and** describe its main features.

Example

Features

[3]

(b) Explain how the stack can be implemented using an array.

[5]

(c) A second stack is used in the program. The diagram below shows the initial state of this stack. Value X is at the top of the stack and was the last item added.

Upper-case letters are used to represent different data values.

Stack operations are performed in three groups as follows:

Group 1:

PUSH D
PUSH E

Group 2:

POP
POP
POP

Group 3:

PUSH A
PUSH B
POP
PUSH C

Complete the diagram to show the state of the stack **after** each group of operations has been performed.

Include the current stack pointer (SP) **after** each group.

Memory location	Initial state	After Group 1	After Group 2	After Group 3
957				
956				
955				
954				
953	X			
952	Y			
951	Z			
950	P			

← SP

[5]

83. O/N 22/P22/Q7/a,b,c

- 7 A teacher is designing a program to perform simple syntax checks on programs written by students.

Two global 1D arrays are used to store the syntax error data. Both arrays contain 500 elements.

- Array `ErrCode` contains integer values that represent an error number in the range 1 to 800.
- Array `ErrText` contains string values that represent an error description.

The following diagram shows an example of the arrays.

Index	ErrCode	ErrText
1	10	"Invalid identifier name"
2	20	"Bracket mismatch"
3	50	"Undeclared variable"
4	60	"Type mismatch in assignment"
...		
500	999	<Undefined>

Note:

- There may be less than 500 error numbers so corresponding elements in both arrays may be unused. Unused elements in `ErrCode` have the value 999. The value of unused elements in `ErrText` is undefined.
- Values in the `ErrCode` array are stored in ascending order but not all values may be present, for example, there may be no error code 31.

The teacher has defined two program modules as follows:

Module	Description
<code>OutputError()</code>	• takes two parameters as integers: ◦ a line number in the student's program ◦ an error number • searches for the error number in the <code>ErrCode</code> array: ◦ if found, outputs the corresponding error description and the line number, for example: "Bracket mismatch on line 34" ◦ if not found, outputs the line number and a warning, for example: "Unknown error on line 34"
<code>SortArrays()</code>	sorts the arrays into ascending order of <code>ErrCode</code>

(a) Write **efficient** pseudocode for module `OutputError()`.

[6]

(b) Write an **efficient** bubble sort algorithm in pseudocode for module `SortArrays()`.

[8]

(c) Two 1D arrays were described at the beginning of the question. Both arrays contain 500 elements.

- Array `ErrCode` contains integer values that represent an error number in the range 1 to 800.
- Array `ErrText` contains string values that represent an error description.

The two arrays will be replaced by a single array. A user-defined data type (record structure) has been declared as follows:

```
TYPE ErrorRec
    DECLARE ErrCode : STRING
    DECLARE ErrText : STRING
ENDTYPE
```

- (i) State the error in the record declaration.

[1]

- (ii) State **two** benefits of using the single array of the user-defined data type.

1

2

[2]

- (iii) Write the declaration for the single array in pseudocode.

[1]

84. O/N 22/P23/Q1/a

- 1 A program is required for a shopping website.

- (a) Part of the program requires four variables. The following table describes the use of each variable.

Complete the table by adding the most appropriate data type for each variable.

Variable use	Data type
Store the number of days in the current month	
Store the first letter of the customer's first name	
Store an indication of whether a year is a leap year	
Store the average amount spent per customer visit	

[4]

85. O/N 22/P23/Q4/a,b

- 4 (a)** A program contains a 1D array `DataItem` with 100 elements.

State the **one additional** piece of information required before the array can be declared.

[1]

- (b)** A programmer decides to implement a queue Abstract Data Type (ADT) in order to store characters received from the keyboard. The queue will need to store at least 10 characters and will be implemented using an array.

- (i)** Describe **two** operations that are typically required when implementing a queue. State the check that must be carried out before each operation can be completed.

Operation 1

Check 1

Operation 2

Check 2

[4]

- (ii)** Describe the declaration and initialisation of the variables and data structures used to implement the queue.

[5]

86. O/N 22/P23/Q5/b(i)

- 5 (a) A text string contains three data items concatenated as shown:

<StockID><Description><Cost>

Item lengths are:

Item	Length
StockID	5
Description	32
Cost	the remainder of the string

A procedure `Unpack()` takes four parameters of type string. One parameter is the original text string. The other three parameters are used to represent the three data items shown in the table and are assigned values within the procedure. These values will be used by the calling program after the procedure ends.

- (b) The design changes and a record structure is defined to store the three data items.

A user-defined data type `StockItem` is created as shown:

```
TYPE StockItem
  DECLARE StockID : STRING
  DECLARE Description : STRING
  DECLARE Cost : REAL
ENDTYPE
```

- (i) A variable `LineData` of type `StockItem` is declared.

Write the pseudocode statement to assign the value 12.99 to the `Cost` field of `LineData`.

[1]

87. O/N 22/P23/Q7/a,b(i)

- 7 A teacher is designing a program to perform simple syntax checks on programs written by students.

Two global 1D arrays are used to store the syntax error data. Both arrays contain 500 elements.

- Array `ErrCode` contains integer values that represent an error number in the range 1 to 800.
- Array `ErrText` contains string values that represent an error description.

The following diagram shows an example of the arrays.

Index	ErrCode	ErrText
1	10	"Invalid identifier name"
2	20	"Bracket mismatch"
3	50	" "
4	60	"Type mismatch in assignment"
...		
500	999	<Undefined>

Note:

- There are less than 500 error codes so corresponding elements in both arrays may be unused. Unused elements in `ErrCode` have the value 999. These will occur at the end of the array. The value of unused elements in `ErrText` is undefined.
- Values in the `ErrCode` array are stored in ascending order but not all values may be present. For example, there may be no error code 31.
- Some error numbers are undefined. In these instances, the `ErrCode` array will contain a valid error number but the corresponding `ErrText` element will contain an empty string.

The teacher has defined one program module as follows:

Module	Description
<code>OutputRange()</code>	• Prompts for input of two error numbers • Outputs a list of error numbers between the two numbers input (inclusive) together with the corresponding error description • Outputs a warning message when the error description is missing as for error number 50 in the example • Outputs a suitable header and a final count of error numbers found Output based on the example array data above: <pre>List of error numbers from 1 to 60 10 : Invalid identifier name 20 : Bracket mismatch 50 : Error Text Missing 60 : Type mismatch in assignment 4 error numbers output</pre>

- (a) Write pseudocode for module `OutputRange()`. Assume that the two numbers input represent a valid error number range.

(b) (i) Two additional modules are defined:

Module	Description
<code>SortArrays()</code>	Sorts the arrays into ascending order of <code>ErrCode</code>
<code>AddError()</code>	- Takes two parameters: - an error number as an integer - an error description as a string - Writes the error number and error description to the first unused element of the two arrays. Ensures the <code>ErrCode</code> array is still in ascending order - Returns the number of unused elements after the new error number has been added - Returns <code>-1</code> if the new error number could not be added

Write pseudocode for the module `AddError()`. Assume that the error code is **not** already in the `ErrCode` array.

[6]

88. O/N 21/P21/Q1/b

- 1 Sylvia is testing a program that has been written by her colleague. Her colleague tells her that the program does not contain any syntax errors.

(b) Complete the following table by giving the appropriate data type in each case.

Use of variable	Data type
The average mark in a class of 40 students	
An email address	
The number of students in the class	
To indicate whether an email has been read	

[4]

89. O/N 21/P21/Q3/a,b,c

- 3 (a) The diagram below represents a queue Abstract Data Type (ADT) that can hold a maximum of eight items.

The operation of this queue may be summarised as follows:

- The front of queue pointer points to the next item to be removed.
- The end of queue pointer points to the last item added.
- The queue is circular so that empty storage elements can be reused.

0	Frog	← Front of queue pointer
1	Cat	
2	Fish	
3	Elk	← End of queue pointer
4		
5		
6		
7		

- (i) Describe how “Octopus” is added to the given queue.

[2]

- (ii) Describe how the next item in the given queue is removed and stored in the variable `AnimalName`.

[2]

- (iii) Describe the state of the queue when the **front of queue** and the **end of queue** pointers have the same value.

[1]

- (b) Some operations are carried out on the original queue given in **part (a)**.

- (i) The current state of the queue is:

0	Frog
1	Cat
2	Fish
3	Elk
4	
5	
6	
7	

Complete the diagram to show the state of the queue after the following operations:

Add “Wasp”, “Bee” and “Mouse”, and then remove two data items.

[3]

(ii) The state of the queue after other operations are carried out is shown:

0	Frog	
1	Cat	
2	Fish	
3	Elk	← Front of queue pointer
4	Wasp	
5	Bee	
6	Mouse	← End of queue pointer
7	Ant	

Complete the following diagram to show the state of the queue after the following operations:

Remove one item, and then add “Dolphin” and “Shark”.

0	
1	
2	
3	
4	
5	
6	
7	

[2]

(c) The queue is implemented using a 1D array.

Describe the algorithm that should be used to modify the **end of queue pointer** when adding an item to the queue.

Your algorithm should detect any potential error conditions.

[3]

90. O/N 21/P21/Q5/a,c

- 5 The following data items will be recorded each time a student successfully logs on to the school network:

Data item	Example data
Student ID	"CJL404"
Host ID	"Lib01"
Time and date	"08:30, June 01, 2021"

The Student ID is six characters long. The other two data items are of variable length.

A single string will be formed by concatenating the three data items. A separator character will need to be inserted between items two and three.

For example:

"CJL404Lib01<separator>08:30, June 01, 2021"

Each string represents one log entry.

A programmer decides to store the concatenated strings in a 1D array `LogArray` that contains 2000 elements. Unused array elements will contain an empty string.

- (a) Suggest a suitable separator character **and** give a reason for your choice.

Character

Reason

[2]

- (c) A function `LogEvents()` will:

- take a Student ID as a parameter
- for each element in the array that matches the Student ID parameter:
 - add the value of the array element to the existing text file `LogFile`
 - assign an empty string to the array element
- count the number of lines added to the file
- return this count.

Write pseudocode for the function `LogEvents()`.

[7]

91. O/N 21/P21/Q6/a,b

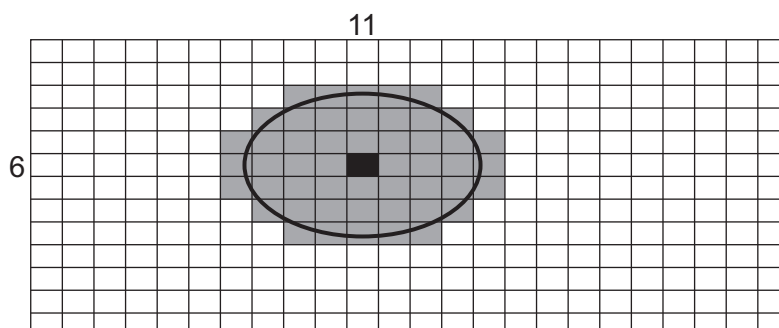
- 6 A mobile phone has a touchscreen. The screen is represented by a grid, divided into 800 rows and 1280 columns.

The grid is represented by a 2D array `Screen` of type `INTEGER`. An array element will be set to 0 unless the user touches that part of the screen.

Many array elements are set to 1 by a single touch of a finger or a stylus.

The following diagram shows a simplified touchscreen. The dark line represents a touch to the screen. All grid elements that are wholly or partly inside the outline will be set to 1. These elements are shaded.

The element shaded in black represents the centre point.



A program is needed to find the coordinates (the row and column) of the centre point. The centre point on the diagram above is row 6, column 11.

Assume:

- the user may only touch one area at a time
- screen rotation does not affect the touchscreen.

The programmer has started to define program modules as follows:

Module	Description
SetRow() (generates test data)	• Called with three parameters of type <code>INTEGER</code>: ◦ a row number ◦ the number of pixels to be skipped starting from column 1 ◦ the number of pixels that should be set to 1 • Sets the required number of pixels to 1 For example, <code>SetRow(3, 8, 5)</code> will give row 3 as in the diagram shown.
SearchInRow()	• Takes two parameters of type <code>INTEGER</code>: ◦ a row number ◦ a start column (1 or 1280) • Searches the given row from the start column (either left to right or right to left) for the first column that contains an element set to 1 • Returns the column number of the first element in the given row that is set to 1 • Returns -1 if no element is set to 1
SearchInCol()	• Takes two parameters of type <code>INTEGER</code>: ◦ a column number ◦ a start row (1 or 800) • Searches the given column from the start row (either up or down) for the first row that contains an element set to 1 • Returns the row number of the first element in the given column that is set to 1 • Returns -1 if no element is set to 1

(a) Write pseudocode to implement the module `SetRow()`.

(b) The module description of `SearchInRow()` is provided here for reference.

Module	Description
<code>SearchInRow()</code>	• Takes two parameters of type <code>INTEGER</code>: ◦ a row number ◦ a start column (1 or 1280) • Searches the given row from the start column (either left to right or right to left) for the first column that contains an element set to 1 • Returns the column number of the first element in the given row that is set to 1 • Returns -1 if no element is set to 1

Write pseudocode to implement the module `SearchInRow()`.

[8]

92. O/N 21/P22/Q1/b

(b) The following pseudocode assigns a value to an element of an array:

```
ThisArray[n] ← 42
```

Complete the following table by writing the answer for each row.

Answer

The number of dimensions of <code>ThisArray</code>	
The technical terms for minimum and maximum values that the variable <code>n</code> may take	
The technical term for the variable <code>n</code> in the pseudocode statement	

[3]

93. O/N 21/P22/Q3/a,b

3 A programmer is writing a program to help manage clubs in a school.

Data will be stored about each student in the school and each student may join up to three clubs.

The data will be held in a record structure of type `Student`.

The programmer has started to define the fields that will be needed as shown in the following table.

Field	Typical value	Comment
<code>StudentID</code>	"CF1234"	Unique to each student
<code>Email</code>	"Carmen47@xyzmail.com"	Contains letters, numbers and certain symbols
<code>Club_1</code>	1	Any value in the range 1 to 99 inclusive
<code>Club_2</code>	14	Any value in the range 1 to 99 inclusive
<code>Club_3</code>	27	Any value in the range 1 to 99 inclusive

(a) (i) Write pseudocode to declare the record structure for type `Student`.

[3]

(ii) A 1D array `Membership` containing 3000 elements will be used to store the student data.

Write pseudocode to declare the `Membership` array.

[2]

(iii) Some of the elements of the array will be unused.

Give an **appropriate** way of indicating an unused array element.

[1]

(iv) Some students are members of less than three clubs.

State **one** way of indicating an unused club field.

[1]

(b) A procedure `GetIDs()` will:

- prompt and input the number of a club
- output the `StudentID` of all the students who are members of that club
- output a count of all students in the given club.

Write pseudocode for the procedure `GetIDs()`.

[7]

94. O/N 21/P22/Q5/a,b

- 5 A company has several departments. Each department stores the name, email address and the status of each employee in that department in its own text file. All text files have the same format.

Employee details are stored as three separate data strings on three consecutive lines of the file. An example of the first six lines of one of the files is as follows:

File line	Comment
1	First employee name
2	First email address
3	First employee status
4	Second employee name
5	Second email address
6	Second employee status

A procedure `MakeNewFile()` will:

- take three parameters as strings:
 - an existing file name
 - a new file name
 - a search status value
- create a new text file using the new file name
- write all employee details to the new file where the employee status is **not** equal to the search status value
- count the number of sets of employee details that were in the original file
- count the number of sets of employee details that were written to the new file
- produce a summary output.

An example summary output is as follows:

```
File Marketing contained 54 employee details
52 employee sets of details were written to file NewMarketingList
```

- (a) Write pseudocode for the procedure `MakeNewFile()`.

[7]

- (b) An alternative format could be used for storing the data.

A text file will still be used.

- (i) Describe the alternative format.

[1]

- (ii) State **one** advantage **and** **one** disadvantage of the alternative format.

Advantage

Disadvantage

[2]

95. O/N 21/P22/Q6/a

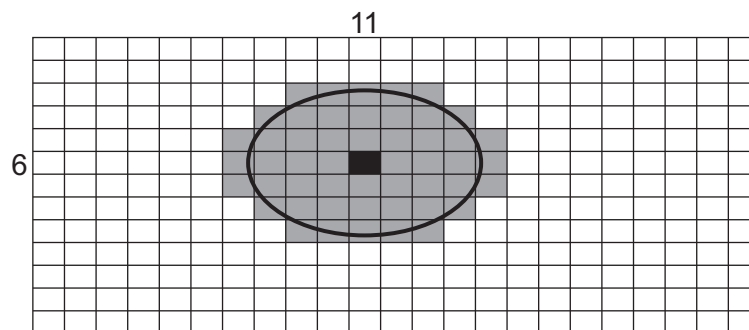
- 6** A mobile phone has a touchscreen. The screen is represented by a grid, divided into 800 rows and 1280 columns.

The grid is represented by a 2D array `Screen` of type `INTEGER`. An array element will be set to 0 unless the user touches that part of the screen.

Many array elements will set to 1 by a single touch of a finger or a stylus.

The following diagram shows a simplified touchscreen. The dark line represents a touch on the screen. All grid elements that are wholly or partly inside the outline will be set to 1. These elements are shaded.

The element shaded in black represents the centre point of the touch.



A program is needed to find the coordinates (the row and column) of the centre point. The centre point on the diagram shown is row 6, column 11.

Assume:

- the user may only touch one area at a time
- screen rotation does not affect the touchscreen.

The programmer has decided to use global values `CentreRow` and `CentreCol` as coordinate values for the centre point.

The programmer has started to define program modules as follows:

Module	Description
<code>FirstRowSet()</code>	- Searches for the first row that has an array element set to 1 - Returns the index of that row (1 is the first row) - Returns -1 if there are no elements set to 1
<code>LastRowSet()</code>	- Searches for the last row that has an array element set to 1 - Returns the index of that row - Returns -1 if there are no elements set to 1
<code>FirstColSet()</code>	- Searches for the first column that has an array element set to 1 - Returns the index of that column (1 is the first column) - Returns -1 if there are no elements set to 1
<code>LastColSet()</code>	- Searches for the last column that has an array element set to 1 - Returns the index of that column - Returns -1 if there are no elements set to 1

(a) Write efficient pseudocode for the module `FirstRowSet()`.

[7]

96. O/N 21/P23/Q1/b

- 1 Sylvia is testing a program that has been written by her colleague. Her colleague tells her that the program does not contain any syntax errors.

(b) Complete the following table by giving the appropriate data type in each case.

Use of variable	Data type
The average mark in a class of 40 students	
An email address	
The number of students in the class	
To indicate whether an email has been read	

[4]

97. O/N 21/P23/Q3/a,b,c

- 3 (a) The diagram below represents a queue Abstract Data Type (ADT) that can hold a maximum of eight items.

The operation of this queue may be summarised as follows:

- The front of queue pointer points to the next item to be removed.
- The end of queue pointer points to the last item added.
- The queue is circular so that empty storage elements can be reused.

0	Frog	← Front of queue pointer
1	Cat	
2	Fish	
3	Elk	← End of queue pointer
4		
5		
6		
7		

- (i) Describe how “Octopus” is added to the given queue.

[2]

- (ii) Describe how the next item in the given queue is removed and stored in the variable `AnimalName`.

[2]

- (iii) Describe the state of the queue when the **front of queue** and the **end of queue** pointers have the same value.

[1]

- (b) Some operations are carried out on the original queue given in **part (a)**.

- (i) The current state of the queue is:

0	Frog
1	Cat
2	Fish
3	Elk
4	
5	
6	
7	

Complete the diagram to show the state of the queue after the following operations:

Add “Wasp”, “Bee” and “Mouse”, and then remove two data items.

[3]

- (ii) The state of the queue after other operations are carried out is shown:

0	Frog	
1	Cat	
2	Fish	
3	Elk	← Front of queue pointer
4	Wasp	
5	Bee	
6	Mouse	← End of queue pointer
7	Ant	

Complete the following diagram to show the state of the queue after the following operations:

Remove one item, and then add “Dolphin” and “Shark”.

0	
1	
2	
3	
4	
5	
6	
7	

[2]

(c) The queue is implemented using a 1D array.

Describe the algorithm that should be used to modify the **end of queue pointer** when adding an item to the queue.

Your algorithm should detect any potential error conditions.

[3]

98. O/N 21/P23/Q5/a,c

- 5 The following data items will be recorded each time a student successfully logs on to the school network:

Data item	Example data
Student ID	"CJL404"
Host ID	"Lib01"
Time and date	"08:30, June 01, 2021"

The Student ID is six characters long. The other two data items are of variable length.

A single string will be formed by concatenating the three data items. A separator character will need to be inserted between items two and three.

For example:

"CJL404Lib01<separator>08:30, June 01, 2021"

Each string represents one log entry.

A programmer decides to store the concatenated strings in a 1D array `LogArray` that contains 2000 elements. Unused array elements will contain an empty string.

- (a) Suggest a suitable separator character **and** give a reason for your choice.

Character

Reason

[2]

- (c) A function `LogEvents()` will:

- take a Student ID as a parameter
- for each element in the array that matches the Student ID parameter:
 - add the value of the array element to the existing text file `LogFile`
 - assign an empty string to the array element
- count the number of lines added to the file
- return this count.

Write pseudocode for the function `LogEvents()`.

[7]

99. O/N 21/P23/Q6/a,b

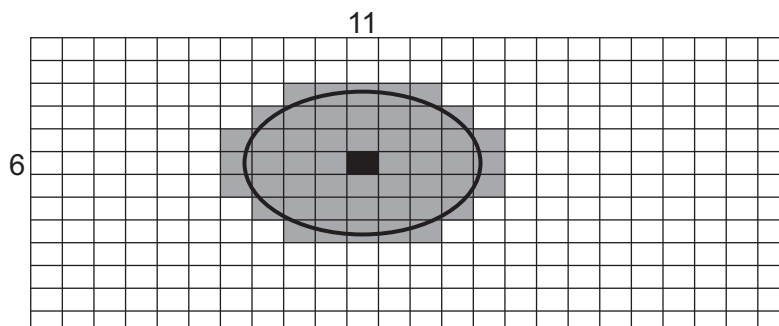
- 6 A mobile phone has a touchscreen. The screen is represented by a grid, divided into 800 rows and 1280 columns.

The grid is represented by a 2D array `Screen` of type `INTEGER`. An array element will be set to 0 unless the user touches that part of the screen.

Many array elements are set to 1 by a single touch of a finger or a stylus.

The following diagram shows a simplified touchscreen. The dark line represents a touch to the screen. All grid elements that are wholly or partly inside the outline will be set to 1. These elements are shaded.

The element shaded in black represents the centre point.



A program is needed to find the coordinates (the row and column) of the centre point. The centre point on the diagram above is row 6, column 11.

Assume:

- the user may only touch one area at a time
- screen rotation does not affect the touchscreen.

The programmer has started to define program modules as follows:

Module	Description
<code>SetRow()</code> (generates test data)	• Called with three parameters of type <code>INTEGER</code>: ◦ a row number ◦ the number of pixels to be skipped starting from column 1 ◦ the number of pixels that should be set to 1 • Sets the required number of pixels to 1 For example, <code>SetRow(3, 8, 5)</code> will give row 3 as in the diagram shown.
<code>SearchInRow()</code>	• Takes two parameters of type <code>INTEGER</code>: ◦ a row number ◦ a start column (1 or 1280) • Searches the given row from the start column (either left to right or right to left) for the first column that contains an element set to 1 • Returns the column number of the first element in the given row that is set to 1 • Returns -1 if no element is set to 1
<code>SearchInCol()</code>	• Takes two parameters of type <code>INTEGER</code>: ◦ a column number ◦ a start row (1 or 800) • Searches the given column from the start row (either up or down) for the first row that contains an element set to 1 • Returns the row number of the first element in the given column that is set to 1 • Returns -1 if no element is set to 1

(a) Write pseudocode to implement the module `SetRow()`.

(b) The module description of `SearchInRow()` is provided here for reference.

Module	Description
<code>SearchInRow()</code>	• Takes two parameters of type <code>INTEGER</code>: ◦ a row number ◦ a start column (1 or 1280) • Searches the given row from the start column (either left to right or right to left) for the first column that contains an element set to 1 • Returns the column number of the first element in the given row that is set to 1 • Returns -1 if no element is set to 1

Write pseudocode to implement the module `SearchInRow()`.

[8]

100. M/J 21/P21/Q1/c

(c) The following data items need to be stored for each student in a group:

- student name (a string)
- test score (an integer).

State a suitable data structure and justify your answer.

Structure

Justification

[3]

101. M/J 21/P21/Q3/a,b

- 3 (a) A concert venue uses a program to calculate admission prices and store information about ticket sales.

A number of arrays are used to store data. The computer is switched off overnight and data has to be input again at the start of each day before any tickets can be sold. This process is very time consuming.

- (i) Explain how the program could use text files to speed up the process.

[2]

- (ii) State the characteristic of text files that allow them to be used as explained in **part (a)(i)**.

[1]

- (iii) Information about ticket sales will be stored as a booking. The booking requires the following data:

- name of person booking
- number of people in the group (for example a family ticket or a school party)
- event type.

Suggest how data relating to each booking may be stored in a text file.

[2]

- (b) A procedure `Preview()` will:

- take the name of a text file as a parameter
- output a warning message if the file is empty
- otherwise output the first five lines from the file (or as many lines as there are in the file if this number is less than five).

Write pseudocode for the procedure `Preview()`.

[5]

102. M/J 21/P21/Q5

- 5 A global 2D array `Result` of type `INTEGER` is used to store a list of exam candidate numbers together with their marks. The array contains 2000 elements, organised as 1000 rows and 2 columns.

Column 1 contains the candidate number and column 2 contains the mark for the corresponding candidate. All elements contain valid exam result data.

A procedure `Sort()` is needed to sort `Result` into ascending order of mark using an efficient bubble sort algorithm.

Write pseudocode for the procedure `Sort()`.

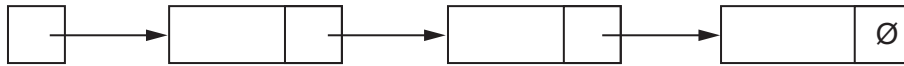
[8]

103. M/J 21/P21/Q6/a,b

6 The following diagram represents an Abstract Data Type (ADT) for a linked list.



The free list is as follows:



(a) Explain how a node containing data value B is added to the list in alphabetic sequence.

[4]

(b) Describe how the linked list in **part (a)** may be implemented using variables and arrays.

[2]

104. M/J 21/P22/Q1/a

also Unit 11 (4 marks); this unit 4 marks

1 (a) (i) Complete the following table by giving the appropriate data type in each case.

Variable	Example data value	Data type
Name	"Catherine"	
Index	100	
Modified	FALSE	
Holiday	25/12/2020	

[4]

(ii) Evaluate each expression in the following table by using the initial data values shown in **part (a)(i)**.

Expression	Evaluates to
Modified OR Index > 100	
LENGTH("Student: " & Name)	
INT(Index + 2.9)	
MID(Name, 1, 3)	

[4]

105. M/J 21/P22/Q3/a,b,c,d

3 The following diagram represents an Abstract Data Type (ADT).



(a) Identify this type of ADT.

[1]

(b) Give the technical term for the item labelled **A** in the diagram.

[1]

(c) Give the technical term for the item labelled **B** in the diagram.

Explain the meaning of the value given to this item.

Term

Meaning

[2]

(d) Complete the diagram to show the ADT after the data has been sorted in alphabetical order.



[2]

106. M/J 21/P22/Q5/a

also Unit 12 (3 marks); this unit 6 marks

5 (a) A student is learning about arrays.

She wants to write a program to:

- declare a 1D array `RNum` of 100 elements of type `INTEGER`
- assign each element a random value in the range 1 to 200 inclusive
- count and output how many numbers generated were between 66 and 173 inclusive.

(i) Write pseudocode to represent the algorithm.

[6]

(ii) The student decides to modify the algorithm so that each element of the array will contain a unique value.

Describe the changes that the student needs to make to the algorithm.

[3]

107. M/J 21/P22/Q6

6 A procedure `CountVowels()` will:

- be called with a string containing alphanumeric characters as its parameter
- count and output the number of occurrences of each vowel (a, e, i, o, u) in the string
- count and output the number of occurrences of the other alphabetic characters (as a single total).

The string may contain both upper and lower case characters.

Each count value will be stored in a unique element of a global 1D array `CharCount` of type `INTEGER`. The array will contain six elements.

Write pseudocode for the procedure `CountVowels()`.

[8]

108. M/J 21/P22/Q8/a

- 8 A program is needed to take a string containing a full name and to produce a new string of initials.

Some words in the full name will be ignored. For example, “the”, “and”, “of”, “for” and “to” may all be ignored.

Each letter of the new string must be upper case.

For example:

Full name	Initials
Integrated Development Environment	IDE
The American Standard Code for Information Interchange	ASCII

The programmer has decided to use the following global variables:

- a ten element 1D array `IgnoreList` of type `STRING` to store the ignored words
- a string `FNString` to store the full name string.

Assume that:

- each alphabetic character in the full name string may be either upper or lower case
- the full name string contains at least one word.

The programmer has started to define program modules as follows:

Module	Description
<code>GetStart()</code>	• Called with an <code>INTEGER</code> as its parameter, representing the number of a word in <code>FNString</code> • Returns the character start position of that word in <code>FNString</code> or returns <code>-1</code> if that word does not exist • For example: <code>GetStart(3)</code> applied to "hot and cold" returns 9
<code>GetWord()</code>	• Called with the position of the first character of a word in <code>FNString</code> as its parameter • Returns the word from <code>FNString</code> • For example: if <code>FNString</code> contains the string "hot and cold", <code>GetWord(9)</code> returns "cold"
<code>IgnoreWord()</code>	• Called with a <code>STRING</code> parameter representing a word • Searches for the word in the <code>IgnoreList</code> array • Returns <code>TRUE</code> if the word is found, otherwise returns <code>FALSE</code>
<code>GetInitials()</code>	• Processes the sequence of words in the full name one word at a time • Calls <code>GetStart()</code>, <code>GetWord()</code> and <code>IgnoreWord()</code> to process each word to form the new string • Outputs the new string

- (a) Write pseudocode for the module `IgnoreWord()`.

109. M/J 21/P23/Q1/c

(c) The following data items need to be stored for each student in a group:

- student name (a string)
- test score (an integer).

State a suitable data structure and justify your answer.

Structure

Justification

[3]

110. M/J 21/P23/Q3/a,b

- 3 (a) A concert venue uses a program to calculate admission prices and store information about ticket sales.

A number of arrays are used to store data. The computer is switched off overnight and data has to be input again at the start of each day before any tickets can be sold. This process is very time consuming.

(i) Explain how the program could use text files to speed up the process.

[2]

(ii) State the characteristic of text files that allow them to be used as explained in **part (a)(i)**.

[1]

(iii) Information about ticket sales will be stored as a booking. The booking requires the following data:

- name of person booking
- number of people in the group (for example a family ticket or a school party)
- event type.

Suggest how data relating to each booking may be stored in a text file.

[2]

(b) A procedure `Preview()` will:

- take the name of a text file as a parameter
- output a warning message if the file is empty
- otherwise output the first five lines from the file (or as many lines as there are in the file if this number is less than five).

Write pseudocode for the procedure `Preview()`.

[5]

111. M/J 21/P23/Q5

- 5 A global 2D array `Result` of type `INTEGER` is used to store a list of exam candidate numbers together with their marks. The array contains 2000 elements, organised as 1000 rows and 2 columns.

Column 1 contains the candidate number and column 2 contains the mark for the corresponding candidate. All elements contain valid exam result data.

A procedure `Sort()` is needed to sort `Result` into ascending order of mark using an efficient bubble sort algorithm.

Write pseudocode for the procedure `Sort()`.

[8]

112. M/J 21/P23/Q6/a,b

- 6 The following diagram represents an Abstract Data Type (ADT) for a linked list.



The free list is as follows:



- (a) Explain how a node containing data value B is added to the list in alphabetic sequence.

[4]

- (b) Describe how the linked list in **part (a)** may be implemented using variables and arrays.

[2]

113. 9608 M/J 21/P21/Q1/b

- (b) Individual elements in a 1D array are referenced using an integer value.

In the pseudocode expression `StockID[n]`, the integer value is represented by the variable `n`.

- (i) Give the technical terms for the minimum and maximum values for the variable `n`.

Minimum value

Maximum value

[1]

- (ii) Give the correct term for the variable `n` in the pseudocode expression `StockID[n]`.

[1]

114. 9608 M/J 21/P21/Q4/a

4 The following is part of a program written in pseudocode:

```
DECLARE ThisArray : ARRAY[1:1000] OF STRING
DECLARE ArrayResult : INTEGER
```



```
PROCEDURE ScanArray(SearchString : STRING)

  DECLARE Index, Total : INTEGER
  DECLARE Error : BOOLEAN

  Index ← 1
  Total ← 0
  Error ← FALSE

  WHILE Index <= 1000 AND Error <> TRUE
    IF LENGTH(ThisArray[Index]) > 5
      THEN
        IF ThisArray[Index] = SearchString
          THEN
            Total ← Total + LENGTH(ThisArray[Index])
          ENDIF
        Index ← Index + 1
      ELSE
        Error ← TRUE
      ENDIF
    ENDWHILE

    ArrayResult ← INT(Total / (Index - 1))

  ENDPROCEDURE
```

The procedure `ScanArray()` is amended as follows:

- `SearchString` is compared with just the first **four** characters of each array element.
- The total ignores the first **five** characters of each array element.
- When calculating `ArrayResult`, prevent any possible division by zero.

Refer to the **Appendix** on pages 22–23 for the list of built-in pseudocode functions and operators.

(a) Write **program code** to implement this **amended** procedure `ScanArray()`.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

`MID(ThisString : STRING, x : INTEGER, y : INTEGER)` RETURNS STRING
returns a string of length `y` starting at position `x` from `ThisString`

Example: `MID("ABCDEFGH", 2, 3)` returns "BCD"

`LENGTH(ThisString : STRING)` RETURNS INTEGER
returns the integer value representing the length of `ThisString`

Example: `LENGTH("Happy Days")` returns 10

`LEFT(ThisString : STRING, x : INTEGER)` RETURNS STRING
returns leftmost `x` characters from `ThisString`

Example: `LEFT("ABCDEFGH", 3)` returns "ABC"

`RIGHT(ThisString : STRING, x : INTEGER)` RETURNS STRING
returns rightmost `x` characters from `ThisString`

Example: `RIGHT("ABCDEFGH", 3)` returns "FGH"

`INT(x : REAL)` RETURNS INTEGER
returns the integer part of `x`

Example: `INT(27.5415)` returns 27

`LCASE(ThisChar : CHAR)` RETURNS CHAR
returns the character value representing the lower case equivalent of `ThisChar`
If `ThisChar` is not an upper case alphabetic character, it is returned unchanged.

Example: `LCASE('W')` returns 'w'

`ASC(ThisChar : CHAR)` RETURNS INTEGER
returns the ASCII value of character `ThisChar`

Example: `ASC('A')` returns 65

`RAND(x : INTEGER)` RETURNS REAL
returns a real number in the range 0 to `x` (not inclusive of `x`).

Example: `RAND(87)` could return 35.43

`NUM_TO_STRING(x : REAL)` RETURNS STRING
returns a string representation of a numeric value.
Note: This function will also work if `x` is of type INTEGER

Example: `NUM_TO_STRING(87.5)` returns "87.5"

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

115. 9608 M/J 21/P21/Q6/a,b,c

6 A program stores stock data in four global arrays as follows:

Array	Data type	Description	Example data value	Initial data value
StockID	STRING	The stock item ID (Eight alpha-numeric characters)	"HWD00001"	" "
Description	STRING	A description of the item (Alphabetic characters only)	"Candle"	" "
Quantity	INTEGER	The number in stock	4	0
Cost	REAL	The cost of the item	2.75	0

- Each array contains 10 000 elements.
- Elements with the same index relate to the same stock item. For example, `StockID[4]` contains the ID for the product whose description is in `Description[4]`.
- The `StockID` array is not sorted and unused elements may occur at any index position.
- Unused elements are assigned the initial data value shown in the table above.

The program is to be modified so that the data from the arrays can be stored in a text file for backup.

The programmer has started to define program modules as follows:

Module	Description
<code>GetValidFilename()</code>	prompts, inputs and returns a valid filename
<code>Check()</code>	- called with an array index as a parameter - checks that the data values for the stock item with the given index are valid - returns <code>TRUE</code> if the values are valid, otherwise returns <code>FALSE</code>
<code>Backup()</code>	- calls <code>GetValidFilename()</code> to get the name of the backup file - combines the data values for each stock item to form a single string. Inserts an asterisk character '*' as a separator between data values - writes the string to the backup file - calls <code>Check()</code> to validate the data values. If there is an error then also writes the string to the file "ERRORLOG.TXT" - repeats for all stock items - returns <code>TRUE</code> if nothing was written to "ERRORLOG.TXT", otherwise returns <code>FALSE</code>
<code>Unpack()</code>	- called with two parameters: - an array index - a string value read from one line of the backup file - extracts the four data values from the string and assigns each to the appropriate array

(a) Write **program code** for the module `Check()`.

Data values relating to an individual stock item are valid if:

- the `StockID` element is of length 8
- the `Description` is not an empty string
- the `Quantity` is zero or above.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

[4]

(b) Write **pseudocode** for the module `Backup()`.

[8]

(c) The module description of `Unpack()` is repeated here for reference.

Module	Description
<code>Unpack()</code>	- called with two parameters: - an array index - a string value read from one line of the backup file - extracts the four data values from the string and assigns each to the appropriate array

Write **program code** for the module `Unpack()`. You may assume that the backup file contains only valid stock information.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

[8]

116. 9608 M/J 21/P22/Q1/c

(c) Each line of a text file contains several data items. A special character is inserted between data items before the line is written to the file.

Explain why a special character is used in this way.

[2]

117. 9608 M/J 21/P22/Q4/b

- (b) A program contains a global 2D array `XRef`. The array consists of 100 rows and 3 columns. The array is of data type `STRING`.

A function, `Search()`, takes two parameters `Par1` and `Par2` as string values and returns an integer value.

The function returns:

- the index number of the first row where:
 - the element in column 1 of that row matches `Par1` and
 - either of the other two elements in that row match `Par2`
- `-1` if no match found in any row.

Write **program code** for the function `Search()`.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

[7]

118. 9608 M/J 21/P22/Q6/a,b

- 6 A program stores data about stock items in four global 1D arrays as follows:

Array	Data type	Description	Example data value	Initial data value
StockID	STRING	the stock item ID (eight alpha-numeric characters)	"JB CD0002"	""
Description	STRING	a description of the item (alphabetic characters only)	"soap"	""
Quantity	INTEGER	the number in stock	9	0
Cost	REAL	the cost of the item	1.45	0.0

- Each array contains 10 000 elements.
- Elements with the same index relate to the same stock item. For example, `StockID[3]` contains the ID for the product whose description is in `Description[3]`.
- The `StockID` array is not sorted.

The program will be modified so that the data from the arrays can be stored in a text file for backup. You may assume that a backup file contains only valid stock data.

The programmer has started to define program modules as follows:

Module	Description
<code>Unpack()</code>	- called with two parameters: - an array index - a string value read from one line of the backup file - extracts the four data values from the string and assigns each to the appropriate array
<code>Restore()</code>	- called with a string representing the name of a backup file - returns <code>FALSE</code> if the file is empty - sets all elements of each array to the initial data value as given in the table - reads the backup file line by line - calls <code>Unpack()</code> to extract data from each line and assign values to the corresponding arrays - returns <code>FALSE</code> if the arrays are full but there are still lines in the file, otherwise returns <code>TRUE</code>
<code>StockSummary()</code>	For all items where <code>StockID</code> does not contain the initial value: - counts the number of stock entries in the <code>StockID</code> array - outputs the overall value of all items in stock (cost multiplied by the quantity) - outputs the number of stock entries

(a) Write **program code** for the module `StockSummary()`.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

[5]

(b) Write **pseudocode** for the module `Restore()`.

[8]

119. 9608 M/J 21/P23/Q6/a,b,c

6 A program stores data about stock items in four global 1D arrays as follows:

Array	Data type	Description	Example data value	Initial data value
StockID	STRING	the stock item ID (eight alpha-numeric characters)	"ABLK0001"	""
Description	STRING	a description of the item (alphabetic characters only)	"torch"	""
Quantity	INTEGER	the number in stock	6	0
Cost	REAL	the cost of the item	4.80	0.0

- Each array contains 10 000 elements.
- Elements with the same index relate to the same stock item. For example, `StockID[5]` contains the ID for the product whose description is in `Description[5]`.
- The `StockID` array is not sorted and unused elements may occur at any index position.
- Unused elements are assigned the initial data value shown in the table above.
- The first four characters of the `StockID` represent a product group. The last four characters represent the item within the group.

The program is to be modified so that:

- data from the arrays are stored in a text file for backup purposes. Data from unused elements are not stored in the file.
- a `Summary` array is added. This will be a global 1D array of 500 elements of type `STRING`. Each product group will occur **once** in the array, for example "ABLK" for the item in the table above.

The programmer has started to define program modules as follows:

Module	Description
<code>GetValidFilename()</code>	• prompts and inputs a filename • returns a valid filename as a <code>STRING</code>
<code>CheckBackupFile()</code>	• calls <code>GetValidFilename()</code> for a filename • checks if the file is empty • If the file is not empty ask the user to confirm that overwrite is intended. If not intended allow the user to re-input a different filename. • returns the filename.

(a) Write **pseudocode** for the module `CheckBackupFile()`.

[8]

(b) Write **program code** for a module `GroupReport()`, which will summarise the stock data for a given product group.

The product group will be passed to the module as a string. The total value of items is calculated by multiplying the cost by the quantity.

An example of the output for group ABLK is as follows:

```
Group: ABLK
Number of items in Group: 11
Total value of items in Group: 387.89
```

If no items are found for group ABLK, the output is as follows:

```
There are no items in Group: ABLK
```

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

[6]

(c) Two additional modules are required:

Module	Description
Lookup()	- called with a <code>STRING</code> representing a product group (for example, "ABLK") - searches the <code>Summary</code> array for the group - returns the index position or returns <code>-1</code> if not found
GroupSummary()	- stores each product group name (found in the <code>StockID</code> array) into the <code>Summary</code> array, if not there already - calls <code>Lookup()</code> to check whether the name is already in the <code>Summary</code> array - returns the number of product groups added to the <code>Summary</code> array

You can assume:

- all elements of the `Summary` array have been initialised to the value `""` before `GroupSummary()` is called
- there will be no more than 500 product groups.

Write **program code** for the module `GroupSummary()`.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

[7]

120. M/J 20/P21/Q3/b

- (b) The following pseudocode searches for a string "Chris" in a 1D array and outputs the index positions where the string is found.

```

DECLARE NameList : ARRAY [1:100] OF STRING
DECLARE n : INTEGER

FOR n ← 1 TO 100
    IF NameList[n] = "Chris"
        THEN
            OUTPUT "Found at: " & NUM_TO_STRING(n)
        ENDIF
    ENDFOR

```

The pseudocode needs to be modified as follows:

- Write the search as a procedure, `Search()`, that takes the search string as a parameter.
- Change the array to a 2D array. The first dimension contains names and the second dimension contains the corresponding status.

For example:

```

NameList[23, 1] ← "Chris"           // name
NameList[23, 2] ← "On Holiday"      // status

```

- Detect a match only when the name contains the search string **and** the status contains "Active".
- If a match has been detected, the procedure will output a **single** message giving all of the index positions where a match occurred. For example, "Found at: 3 6 22".
- If no match has been detected, the procedure will output a suitable message.

Refer to the **Appendix** on page 21 for a list of built-in pseudocode functions and operators.

Write the **pseudocode** for the procedure `Search()`. Assume the array has been declared globally.

[8]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

`MID(ThisString : STRING, x : INTEGER, y : INTEGER)` RETURNS STRING
returns a string of length `y` starting at position `x` from `ThisString`

Example: `MID("ABCDEFGH", 2, 3)` returns "BCD"

`LENGTH(ThisString : STRING)` RETURNS INTEGER
returns the integer value representing the length of `ThisString`

Example: `LENGTH("Happy Days")` returns 10

`LEFT(ThisString : STRING, x : INTEGER)` RETURNS STRING
returns leftmost `x` characters from `ThisString`

Example: `LEFT("ABCDEFGH", 3)` returns "ABC"

`RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING`
 returns rightmost `x` characters from `ThisString`

Example: `RIGHT("ABCDEFGH", 3)` returns "FGH"

`INT(x : REAL) RETURNS INTEGER`
 returns the integer part of `x`

Example: `INT(27.5415)` returns 27

`NUM_TO_STRING(x : REAL) RETURNS STRING`
 returns a string representation of a numeric value
 Note: This function will also work if `x` is of type `INTEGER`

Example: `NUM_TO_STRING(87.5)` returns "87.5"

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: <code>TRUE AND FALSE</code> produces <code>FALSE</code>
OR	Performs a logical OR on two Boolean values Example: <code>TRUE OR FALSE</code> produces <code>TRUE</code>

121. M/J 20/P21/Q5/a,b

- 5 (a) A 1D array, `Directory`, of type `STRING` is used to store a list of school internal telephone numbers. There are 1000 elements in the array. Each element stores a single data item. The format of each data item is as follows:

`<Number><Name>`

`Number` is a four-digit numeric string.
`Name` is a variable-length string.

For example:

`"1024Collins Robbie"`

The following pseudocode is an initial attempt at defining a procedure `SortContacts()` that will perform a bubble sort on `Directory`. The array is to be sorted in ascending order of `Name`.

Fill in the gaps to complete the pseudocode.

Refer to the **Appendix** on page 21 for a list of built-in pseudocode functions and operators.

```

PROCEDURE SortContacts ()
DECLARE Temp : STRING
DECLARE FirstName, SecondName : STRING
DECLARE NoSwaps : .....
DECLARE Boundary, J : INTEGER
Boundary ← .....
REPEAT
    NoSwaps ← TRUE
    FOR J ← 1 TO Boundary
        FirstName ← .....(Directory[J], LENGTH(Directory[J]) - ..... )
        SecondName ← RIGHT(Directory[J + 1], LENGTH(Directory[J + 1]) - 4)
        IF FirstName .....
            THEN
                Temp ← Directory[J]
                Directory[J] ← Directory .....
                Directory[J + 1] ← Temp
                NoSwaps ← .....
            ENDIF
        ENDFOR
        Boundary ← .....
    UNTIL NoSwaps = TRUE
ENDPROCEDURE

```

[8]

(b) The pseudocode contains a mechanism designed to make this an efficient bubble sort.

Describe the mechanism **and** explain why it may be considered to be efficient.

[4]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING
returns a string of length y starting at position x from ThisString

Example: MID("ABCDEFGH", 2, 3) returns "BCD"

LENGTH(ThisString : STRING) RETURNS INTEGER
returns the integer value representing the length of ThisString

Example: LENGTH("Happy Days") returns 10

`LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING`
 returns leftmost x characters from ThisString

Example: `LEFT("ABCDEFGH", 3)` returns "ABC"

`RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING`
 returns rightmost x characters from ThisString

Example: `RIGHT("ABCDEFGH", 3)` returns "FGH"

`INT(x : REAL) RETURNS INTEGER`
 returns the integer part of x

Example: `INT(27.5415)` returns 27

`NUM_TO_STRING(x : REAL) RETURNS STRING`
 returns a string representation of a numeric value
 Note: This function will also work if x is of type INTEGER

Example: `NUM_TO_STRING(87.5)` returns "87.5"

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

122. M/J 20/P21/Q6/a,b

- 6 A company hires out rowing boats on a lake. The company has ten boats, numbered from 1 to 10.

The company is developing a program to help manage and record the hiring out process.

Hire information is stored in three global 1D arrays when a boat is hired out. Each array contains 10 elements representing each of the ten boats.

The three 1D arrays are summarised as follows:

Array	Data type	Description	Example data value
HireTime	STRING	The time the boat was hired out	"10:15"
Duration	INTEGER	The number of minutes of the hire	30
Cost	REAL	The cost of the hire	5.75

If an individual boat is not currently on hire, the corresponding element of the `HireTime` array will be set to "Available".

The programmer has started to define program modules as follows:

Module	Description
<code>AddTime()</code>	- Called with two parameters: - a <code>STRING</code> value representing a time - an <code>INTEGER</code> value representing a duration in minutes - Adds the duration to the time to give a new time - Returns the new time as a <code>STRING</code>
<code>ListAvailable()</code>	- Called with a <code>STRING</code> value representing the time the hire will start - Outputs the boat numbers that will be available for hire at the given start time. A boat will be available for hire if it is either: - currently not on hire, or - due back before the given hire start time - Outputs the number of each boat available - Outputs the total number of boats available or a suitable message if there are none
<code>RecordHire()</code>	- Called with four parameters: - an <code>INTEGER</code> value representing the boat number - a <code>STRING</code> value representing the hire start time - an <code>INTEGER</code> value representing the hire duration in minutes - a <code>REAL</code> value representing the cost of hire - Updates the appropriate element in each array - Adds the cost of hire to the global variable <code>DailyTakings</code> - Converts the four input parameters to strings, concatenated using commas as separators, and writes the resulting string to the end of the existing text file <code>HireLog.txt</code>

(a) Write **pseudocode** for the module `ListAvailable()`.

[8]

(b) Write **program code** for the module `RecordHire()`.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

[7]

123. M/J 20/P22/Q4/a

- 4 (a) Mustafa has developed the following pseudocode to generate ten random integers in the range 1 to 100.

```

DECLARE Random : ARRAY [1:10] OF INTEGER
DECLARE Count, RNum : INTEGER

FOR Count ← 1 TO 10
    Rnum ← INT(RAND(100)) + 1
    Random[Count] ← RNum
ENDFOR

```

Refer to the **Appendix** on page 19 for a list of built-in pseudocode functions and operators.

Rewrite the pseudocode so that there are no duplicated numbers in the list of random numbers.

[6]

Appendix of this paper:

Appendix**Built-in functions (pseudocode)**

Each function returns an error if the function call is not properly formed.

LENGTH(ThisString : STRING) RETURNS INTEGER
returns the integer value representing the length of ThisString

Example: LENGTH("Happy Days") returns 10

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns leftmost x characters from ThisString

Example: LEFT("ABCDEFGH", 3) returns "ABC"

RIGHT(ThisString: STRING, x : INTEGER) RETURNS STRING
returns rightmost x characters from ThisString

Example: RIGHT("ABCDEFGH", 3) returns "FGH"

INT(x : REAL) RETURNS INTEGER
returns the integer part of x

Example: INT(27.5415) returns 27

RAND(x : INTEGER) RETURNS REAL
returns a real number in the range 0 to x (not inclusive of x)

Example: RAND(87) could return 35.43

MOD(ThisNum : INTEGER, ThisDiv : INTEGER) RETURNS INTEGER
returns the integer value representing the remainder when ThisNum is divided by ThisDiv

Example: MOD(10, 3) returns 1

DIV(ThisNum : INTEGER, ThisDiv : INTEGER) RETURNS INTEGER
returns the integer value representing the whole number part of the result when ThisNum is divided by ThisDiv

Example: DIV(10, 3) returns 3

`NUM_TO_STRING(x : REAL)` RETURNS STRING
returns a string representation of a numeric value.

Example: If `x` has the value `87.5` then `NUM_TO_STRING(x)` returns `"87.5"`

Note: This function will also work if `x` is of type `INTEGER`

`STRING_TO_NUM(x : STRING)` RETURNS REAL
returns a numeric representation of a string.

Example: If `x` has the value `"23.45"` then `STRING_TO_NUM(x)` returns `23.45`

Note: This function will also work if `x` is of type `CHAR`

Operators (pseudocode)

Operator	Description
<code>&</code>	Concatenates (joins) two strings Example: <code>"Summer" & " " & "Pudding"</code> produces <code>"Summer Pudding"</code>
<code>AND</code>	Performs a logical <code>AND</code> on two Boolean values Example: <code>TRUE AND FALSE</code> produces <code>FALSE</code>
<code>OR</code>	Performs a logical <code>OR</code> on two Boolean values Example: <code>TRUE OR FALSE</code> produces <code>TRUE</code>

124. M/J 20/P22/Q5/b

- 5** A global 1D array, `Contact`, of type `STRING` is used to store a list of names and email addresses.

There are 1000 elements in the array. Each element stores one data item. The format of each data item is as follows:

`<Name>': '<EmailAddress>`

`Name` and `EmailAddress` are both variable-length strings.

For example:

`"Sharma Himal:hsharma99@stlmail.com"`

A function, `GetName()`, is part of the program that processes the array. A data item string will be passed to the function as a parameter. The function will return the `Name` part. Validation is **not** necessary.

- (b) (i)** The array is to be sorted using an efficient bubble sort algorithm. An efficient bubble sort reduces the number of unnecessary comparisons between elements.

Describe how this could be achieved.

[4]

- (ii)** A procedure, `BubbleSort()`, is needed to sort the 1D array `Contact` into ascending order of `Name` using an efficient bubble sort algorithm.

Write **program code** for the procedure `BubbleSort()`.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

[8]

125. M/J 20/P22/Q6/c

- 6** A company hires out rowing boats on a lake. The company has 17 boats numbered from 1 to 17.

Boats may be hired between 9:00 and 18:00, with a maximum hire duration of 90 minutes.

The company is developing a program to help manage and record the boat hire process.

The programmer has decided to store all values relating to hire time as strings. The program will use a 24-hour clock format. For example:

Time (in words)	String value
Nine o'clock in the morning	"09:00"
Five minutes past ten o'clock in the morning	"10:05"
Ten minutes before three o'clock in the afternoon	"14:50"

The programmer has defined the first module as follows:

Module	Description
AddTime()	- Takes two parameters: - StartTime: a STRING value representing a time as described - Duration: an INTEGER value representing a duration in minutes - Adds the duration to the time to give a new time - Returns the new time as a STRING

- (c) Each time a boat is hired out, details of the hire are added to a text file, `Hirelog.txt`. Each line of the text file corresponds to information about one hire session.

The format of each line is as follows:

<BoatNumber><Date><AmountPaid>

- BoatNumber is a two-digit numeric string
- Date is a six-digit numeric string in the format DDMMYY
- AmountPaid is a variable-length string representing a numeric value, for example "12.75"

The total hire amount from each boat is to be stored in a global array, `Total`. This array is declared in pseudocode as follows:

```
DECLARE Total : ARRAY [1:17] OF REAL
```

The programmer has defined module `GetTotals()` as follows:

Module	Description
GetTotals()	- Search through the file <code>Hirelog.txt</code> - Extract the <code>AmountPaid</code> each time a boat is hired - Store the total of <code>AmountPaid</code> for each boat in the array

Write **program code** for the module `GetTotals()`.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

126. M/J 20/P23/Q5/a,b(i)

- 5 A company hires out rowing boats on a lake. The company has 20 boats, numbered from 1 to 20.

For safety reasons, the boats have to be serviced (checked and any damage repaired) regularly.

The company is developing a program to help manage the servicing of the boats.

Every time a boat is serviced, details are added at the end of the text file, `ServiceLog.txt`, as a single line of information. Each boat is serviced before it is hired out for the first time.

The format of each line is as follows:

`<BoatNumber><Date>`

`BoatNumber` and `Date` are as follows:

- `BoatNumber` is a two-digit numeric string in the range "01" to "20"
- `Date` is an 8-digit numeric string in the format `YYYYMMDD`

The programmer has defined the first module as follows:

Module	Description
<code>GetLastService()</code>	• Called with a string parameter representing the <code>BoatNumber</code> • Searches through the file <code>ServiceLog.txt</code> • Returns the date of the last service in the form "YYYYMMDD"

- (a) Write **pseudocode** for the module `GetLastService()`.

Refer to the **Appendix** on page 19 for a list of built-in pseudocode functions and operators.

[8]

- (b) (i) Every time a boat is hired out, details of the hire are added at the end of a text file, `Hirelog.txt`. Each line of the text file corresponds to information about the hire of one boat.

The format of each line of information is as follows:

`<Date><BoatNumber><HireDuration>`

- `Date` is an 8-digit numeric string in the format `YYYYMMDD`
- `BoatNumber` is a two-digit numeric string in the range "01" to "20"
- `HireDuration` is a variable-length string representing a numeric value in hours. For example, the string "1.5" would represent a hire duration of 1½ hours.

A module `GetHours()` is defined as follows:

Module	Description
<code>GetHours()</code>	- Takes two parameters: - the <code>BoatNumber</code> as a string - the date of the last service for that boat ("YYYYMMDD") as a string - Searches through file <code>Hirelog.txt</code> and calculates the sum of the hire durations for the given boat after the given date (hire durations on or before the given date are ignored) - Returns the total of the hire durations as a real

Note:

Standard comparison operators may be used with dates in this format.
For example:

"20200813" > "20200812" would evaluate to TRUE

Parameter validation is **not** required.

Write **pseudocode** for the module `GetHours()`.

Refer to the **Appendix** on page 19 for a list of built-in pseudocode functions and operators.

[8]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

`LENGTH(ThisString : STRING) RETURNS INTEGER`
returns the integer value representing the length of `ThisString`

Example: `LENGTH("Happy Days")` returns 10

`LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING`
returns leftmost `x` characters from `ThisString`

Example: `LEFT("ABCDEFGH", 3)` returns "ABC"

`RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING`
returns rightmost `x` characters from `ThisString`

Example: `RIGHT("ABCDEFGH", 3)` returns "FGH"

`MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING`
returns a string of length `y` starting at position `x` from `ThisString`

Example: `MID("ABCDEFGH", 2, 3)` returns "BCD"

`INT(x : REAL)` RETURNS INTEGER
returns the integer part of `x`

Example: `INT(27.5415)` returns 27

`NUM_TO_STRING(x : REAL)` RETURNS STRING
returns a string representation of a numeric value.
Note: This function will also work if `x` is of type INTEGER

Example: `NUM_TO_STRING(87.5)` returns "87.5"

`STRING_TO_NUM(x : STRING)` RETURNS REAL
returns a numeric representation of a string.
Note: This function will also work if `x` is of type CHAR

Example: `STRING_TO_NUM("23.45")` returns 23.45

Operators (pseudocode)

Operator	Description
<code>&</code>	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
<code>AND</code>	Performs a logical AND on two Boolean values Example: <code>TRUE AND FALSE</code> produces <code>FALSE</code>
<code>OR</code>	Performs a logical OR on two Boolean values Example: <code>TRUE OR FALSE</code> produces <code>TRUE</code>

127. O/N 19/P21/Q1/b(i)

- 1 Study the following pseudocode.

```

FUNCTION Search() RETURNS INTEGER
    DECLARE N, C : INTEGER
    DECLARE V, L : REAL
    V ← GetLevel()
    L ← V * 1.34
    C ← 0
    FOR N ← 1 TO 10
        V ← GetLevel()
        IF V > L
            THEN
                C ← C + 1
            ENDIF
    ENDFOR
    OUTPUT "Process complete"
    RETURN C
ENDFUNCTION

```

- (b) (i) Programming languages support different data types.

Complete the table by giving a suitable data type for each example value.

Example value	Data type
"NOT TRUE"	
- 4.5	
NOT FALSE	
132	

[4]

128. O/N 19/P21/Q4/b

- 4 A student is developing a program to count how many times each character of the alphabet (A to Z) occurs in a given string. Upper case and lower case characters will be counted as the same. The string may contain non-alphabetic characters, which should be ignored.

The program will:

- check each character in the string to count how many times each alphabetic character occurs
- store the count for each alphabetic character in a 1D array
- output each count together with the corresponding character.

- (b) Write **pseudocode** to implement the program.

You should note the following:

- `InString` contains the string to be checked. It has been assigned a value.
- The elements of the array `Result` have all been initialised to zero.
- The ASCII value of letter 'A' is 65.

You should assume the following lines of pseudocode have been written:

```
DECLARE InString : STRING
DECLARE Result : ARRAY [1:26] OF INTEGER
```

Declare any further variables you use. Do **not** implement the code as a subroutine.

Refer to the **Appendix** on pages 16–17 for the list of built-in functions and operators.

[7]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING
returns a string of length y starting at position x from ThisString

Example: MID("ABCDEFGH", 2, 3) returns "BCD"

LENGTH(ThisString : STRING) RETURNS INTEGER
returns the integer value representing the length of ThisString

Example: LENGTH("Happy Days") returns 10

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns leftmost x characters from ThisString

Example: LEFT("ABCDEFGH", 3) returns "ABC"

RIGHT(ThisString: STRING, x : INTEGER) RETURNS STRING
returns rightmost x characters from ThisString

Example: RIGHT("ABCDEFGH", 3) returns "FGH"

INT(x : REAL) RETURNS INTEGER
returns the integer part of x

Example: INT(27.5415) returns 27

NUM_TO_STRING(x : REAL) RETURNS STRING
returns a string representation of a numeric value.
Note: This function will also work if x is of type INTEGER

Example: NUM_TO_STRING(87.5) returns "87.5"

STRING_TO_NUM(x : STRING) RETURNS REAL
returns a numeric representation of a string.
Note: This function will also work if x is of type CHAR

Example: STRING_TO_NUM("23.45") returns 23.45

`ASC(ThisChar : CHAR)` RETURNS INTEGER
returns the ASCII value of `ThisChar`

Example: `ASC('A')` returns 65

`CHR(x : INTEGER)` RETURNS CHAR
returns the character whose ASCII value is `x`

Example: `CHR(87)` returns 'W'

`UCASE(ThisChar : CHAR)` RETURNS CHAR
returns the character value representing the upper case equivalent of `ThisChar`
If `ThisChar` is not a lower case alphabetic character, it is returned unchanged.

Example: `UCASE('a')` returns 'A'

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

129. O/N 19/P21/Q6/a,b,c,d(i)

- 6 Account information for users of a library is held in one of two text files; `UserListAtoM.txt` and `UserListNtoZ.txt`

The format of the data held in the two files is identical. Each line of the file is stored as a string that contains an account number, name and telephone number separated by the asterisk character ('*') as follows:

`<Account Number>*<Name>*<Telephone Number>`

An example of one line from the file is:

`"GB1234*Kevin Mapunga*07789123456"`

The account number string may be **six** or **nine** characters in length and is **unique for each person**. It is made up of alphabetic and numeric characters only.

An error has occurred and the same account number has been given to different users in the two files. There is **no** duplication of account numbers **within each individual file**.

A program is to be written to search the two files and to identify duplicate entries. The account number of any duplicate found is to be written to an array, `Duplicates`, which is a 1D array of 100 elements of data type `STRING`.

The program is to be implemented as several modules. The outline description of three of these is as follows:

Module	Outline description
<code>ClearArray()</code>	Initialise the global array <code>Duplicates</code>. Set all elements to the empty string.
<code>FindDuplicates()</code>	- Read each line from the file <code>UserListAtoM.txt</code> - Check whether the account number appears in file <code>UserListNtoZ.txt</code> using <code>SearchFileNtoZ()</code> - If the account number does appear then add the account number to the array. - Output an error message and exit the module if there are more duplicates than can be written to the array.
<code>SearchFileNtoZ()</code>	- Search for a given account number in file <code>UserListNtoZ.txt</code> - If found, return <code>TRUE</code>, otherwise return <code>FALSE</code>

- (a) State **one** reason for storing data in a file rather than in an array.

[1]

- (b) Write **program code** for the module `SearchFileNtoZ()`.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

[7]

- (c) Write **pseudocode** for the module `FindDuplicates()`.

The module description is given in the table on page 12.

[8]

- (d) `ClearArray()` is to be modified to make it general purpose. It will be used to initialise any 1D array of data type `STRING` to any value.

It will now be called with three parameters as follows:

1. The array
2. The number of elements
3. The initialisation string

You should assume that the lower bound is 1.

- (i) Write **pseudocode** for the modified `ClearArray()` procedure.

[3]

130. O/N 19/P22/Q1/b(i)

- 1 Study the following pseudocode.

```

PROCEDURE FillTank()

    DECLARE Tries : INTEGER
    DECLARE Full : BOOLEAN

    Tries ← 1

    Full ← ReadSensor("F1")

    IF NOT Full
        THEN
            WHILE NOT Full AND Tries < 4
                CALL TopUp()
                Full ← ReadSensor("F1")
                Tries ← Tries + 1
            ENDWHILE
            IF Tries > 3
                THEN
                    OUTPUT "Too many attempts"
                ELSE
                    OUTPUT "Tank now full"
                ENDIF
            ELSE
                OUTPUT "Already full"
            ENDIF
        ENDIF
    ENDPROCEDURE

```

- (b) (i) Programming languages support different data types.

Complete the table by giving a suitable data type for each example value.

Example value	Data type
43	
TRUE	
-273.16	
"-273.16"	

[4]

131. O/N 19/P22/Q3/a,b

- 3 A student is developing a program to search through a string of numeric digits to count how many times each digit occurs. The variable `InString` will store the string and the 1D array `Result` will store the count values.

The program will:

- check each character in the string to count how many times each digit occurs
- record the count for each digit using the array
- output the count for each element of the array together with the corresponding digit.

(a) The array `Result` is a 1D array of type `INTEGER`.

Write **pseudocode** to declare the array and to initialise all elements to zero.

[3]

(b) Write the **pseudocode** for the program.

Declare any variables you use. Do not implement the code as a subroutine.

Refer to the **Appendix** on page 18 for the list of built-in functions and operators.

[8]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

`MID(ThisString : STRING, x : INTEGER, y : INTEGER)` RETURNS STRING
returns a string of length `y` starting at position `x` from `ThisString`

Example: `MID("ABCDEFGH", 2, 3)` returns "BCD"

`LENGTH(ThisString : STRING)` RETURNS INTEGER
returns the integer value representing the length of `ThisString`

Example: `LENGTH("Happy Days")` returns 10

`LEFT(ThisString : STRING, x : INTEGER)` RETURNS STRING
returns leftmost `x` characters from `ThisString`

Example: `LEFT("ABCDEFGH", 3)` returns "ABC"

`RIGHT(ThisString: STRING, x : INTEGER)` RETURNS STRING
returns rightmost `x` characters from `ThisString`

Example: `RIGHT("ABCDEFGH", 3)` returns "FGH"

`INT(x : REAL)` RETURNS INTEGER
returns the integer part of `x`

Example: `INT(27.5415)` returns 27

`NUM_TO_STRING(x : REAL)` RETURNS STRING
returns a string representation of a numeric value.
Note: This function will also work if `x` is of type `INTEGER`

Example: `NUM_TO_STRING(87.5)` returns "87.5"

`STRING_TO_NUM(x : STRING)` RETURNS REAL
returns a numeric representation of a string.

Note: This function will also work if `x` is of type `CHAR`

Example: `STRING_TO_NUM("23.45")` returns `23.45`

Operators (pseudocode)

Operator	Description
<code>&</code>	Concatenates (joins) two strings Example: <code>"Summer" & " " & "Pudding"</code> produces <code>"Summer Pudding"</code>
<code>AND</code>	Performs a logical AND on two Boolean values Example: <code>TRUE AND FALSE</code> produces <code>FALSE</code>
<code>OR</code>	Performs a logical OR on two Boolean values Example: <code>TRUE OR FALSE</code> produces <code>TRUE</code>

132. O/N 19/P22/Q6/a,b

- 6 A text file, `StudentList.txt`, contains a list of information about students in a school.

Each line of the file contains a reference, name and date of birth for one student. All the information is held as strings and separated by the asterisk character (`'*'`) as follows:

`<Reference>'* '<Name>'* '<Date Of Birth>`

An example of one line from the file is:

`"G1234*Aleza Hilton*05062001"`

The reference string may be five or eight characters in length and is unique for each student. It is made up of alphabetic and numeric characters only.

A global 1D array, `Leavers`, contains the references of all students who have recently left the school. The array consists of 500 elements of data type `STRING`. Unused elements contain the empty string `""`.

A program is to be written to produce a new text file, `UpdatedList.txt`, containing information only for students who are still attending the school.

The program is to be implemented as several modules. The outline description of three of these is as follows:

Module	Outline description
<code>ProcessStudentList()</code>	- Read each line from the file <code>StudentList.txt</code> - Check whether the <code>Reference</code> appears in the array using <code>SearchLeavers()</code> - If the <code>Reference</code> does not appear then write the line to the file <code>UpdatedList.txt</code> - Return the number of lines not copied.
<code>SearchLeavers()</code>	- Search for a given <code>Reference</code> in the array <code>Leavers</code> - If the <code>Reference</code> is found, return <code>TRUE</code>, otherwise return <code>FALSE</code>
<code>CountTimes()</code>	- Take two parameters: the name of an array and a string. - Count the number of elements that are the same as the string. Return the count value.

- (a) Write **program code** for the module `SearchLeavers()`. Declare any additional variables you use.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

[6]

- (b) Write **pseudocode** for the module `ProcessStudentList()`.

The module description is repeated here for reference.

Module	Outline description
<code>ProcessStudentList()</code>	- Read each line from the file <code>StudentList.txt</code> - Check whether the <code>Reference</code> appears in the array using <code>SearchLeavers()</code> - If the <code>Reference</code> does not appear then write the line to the file <code>UpdatedList.txt</code> - Return the number of lines not copied.

[9]

133. O/N 19/P23/Q1/a(i)

- 1 (a) (i) Programming languages can support different data types.

Complete the table by naming **three** different data types together with an example data value for each.

Data type	Example data value

[6]

134. O/N 19/P23/Q6/a,b

- 6 A text file, `Library.txt`, stores information relating to a book collection. The file stores four pieces of information about each book on separate lines of the file, as follows:

```

Line n:      <Book Title>
Line n + 1:  <Author Name>
Line n + 2:  <ISBN>
Line n + 3:  <Location>

```

Information is stored as data strings.

Information relating to two books is shown:

File line	Data
100	"Learning Python"
101	"Brian Smith"
102	"978-14-56543-21-8"
103	"BD345"
104	"Surviving in the mountains"
105	"C T Snow"
106	"978-35-17635-43-9"
107	"ZX001"

- (a) (i) A function, `FindBooksBy()`, will search `Library.txt` for all books by a given author.

The function will store the Book Title and Location in the array `Result`, and will return a count of the number of books found.

Array `Result` is a global 2D array of type `STRING`. It has 100 rows and 2 columns.

Write **pseudocode** to declare the array `Result`.

[3]

(ii) Function `FindBooksBy()` will:

- receive the `Author Name` as a parameter
- search `Library.txt` for matching entries
- store the `Book Title` and `Location` of matching entries in the `Result` array
- return an integer value giving the number of books by the author that were found.

Write **program code** for the function `FindBooksBy()`.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

[8]

(b) The function `FindBooksBy()` has already been called and has stored values in the array `Result`.

The procedure, `DisplayResults()`, will output the information from the array.

The procedure receives the following two parameters:

- a string containing the author name
- an integer value representing the number of books found

The output should be formatted as in the following example:

```
Books written by: Brian Smith

Title                               Location
Learning Python                     BD345
Arrays are not lists                 CZ562
Learning Java                        CZ589

Number of titles found: 3
```

If no books by the author are found, the following should be output:

```
Search found no books by: Brian Smith
```

Write **pseudocode** for the procedure `DisplayResults()`.

Refer to the **Appendix** on page 16 for the list of built-in functions and operators.

[7]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING
returns a string of length y starting at position x from ThisString

Example: MID("ABCDEFGH", 2, 3) returns "BCD"

LENGTH(ThisString : STRING) RETURNS INTEGER
returns the integer value representing the length of ThisString

Example: LENGTH("Happy Days") returns 10

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns leftmost x characters from ThisString

Example: LEFT("ABCDEFGH", 3) returns "ABC"

RIGHT(ThisString: STRING, x : INTEGER) RETURNS STRING
returns rightmost x characters from ThisString

Example: RIGHT("ABCDEFGH", 3) returns "FGH"

INT(x : REAL) RETURNS INTEGER
returns the integer part of x

Example: INT(27.5415) returns 27

ASC(ThisChar : CHAR) RETURNS INTEGER
returns the ASCII value of ThisChar

Example: ASC('A') returns 65

MOD(ThisNum : INTEGER, ThisDiv : INTEGER) RETURNS INTEGER
returns the integer value representing the remainder when ThisNum is divided by ThisDiv

Example: MOD(10, 3) returns 1

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

135. M/J 19/P21/Q1/b(ii)

(b) Program variables have values as follows:

Variable	Value
Title	"101 tricks with spaghetti"
Version	'C'
Author	"Eric Peapod"
PackSize	4
WeightEach	6.2
Paperback	TRUE

(ii) Programming languages support different data types.

Give an appropriate data type for the following variables from **part (b)**.

Variable	Data type
Title	
Version	
PackSize	
WeightEach	
Paperback	

[5]

136. M/J 19/P21/Q4/b

4 The following pseudocode is a string handling function.

For the built-in functions list, refer to the **Appendix** on page 16.

```

FUNCTION Clean(InString : STRING) RETURNS STRING

    DECLARE NewString : STRING
    DECLARE Index : INTEGER
    DECLARE AfterSpace : BOOLEAN
    DECLARE NextChar : CHAR
    CONSTANT Space = ' '

    AfterSpace ← FALSE
    NewString ← ""

    FOR Index ← 1 TO LENGTH(InString)
        NextChar ← MID(InString, Index, 1)
        IF AfterSpace = TRUE
            THEN
                IF NextChar <> Space
                    THEN
                        NewString ← NewString & NextChar
                        AfterSpace ← FALSE
                    ENDIF
            ELSE
                NewString ← NewString & NextChar
                IF NextChar = Space
                    THEN
                        AfterSpace ← TRUE
                    ENDIF
            ENDIF
        ENDIF
    ENDFOR

    RETURN NewString

ENDFUNCTION

```

(b) The following pseudocode declares and initialises an array.

```

DECLARE Code : ARRAY[1:100] OF STRING
DECLARE Index : INTEGER

FOR Index ← 1 TO 100
    Code[Index] ← ""
ENDFOR

```

The design of the program is changed as follows:

- the array needs to be two dimensional, with 500 rows and 4 columns
- the elements of the array need to be initialised to the string "Empty"

Re-write the **pseudocode** to implement the new design.

[4]

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING
returns a string of length y starting at position x from ThisString

Example: MID("ABCDEFGH", 2, 3) returns "BCD"

LENGTH(ThisString : STRING) RETURNS INTEGER
returns the integer value representing the length of ThisString

Example: LENGTH("Happy Days") returns 10

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns leftmost x characters from ThisString

Example: LEFT("ABCDEFGH", 3) returns "ABC"

RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns rightmost x characters from ThisString

Example: RIGHT("ABCDEFGH", 3) returns "FGH"

INT(x : REAL) RETURNS INTEGER
returns the integer part of x

Example: INT(27.5415) returns 27

ASC(ThisChar : CHAR) RETURNS INTEGER
returns the ASCII value of character ThisChar

Example: ASC('A') returns 65

RAND(x : INTEGER) RETURNS REAL
returns a real number in the range 0 to x (x not inclusive).

Example: RAND(87) could return 35.43

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

137. M/J 19/P21/Q5/b

(b) A student is learning about random number generation.

She is investigating how many times the random function needs to be called before every number in a given series is generated.

She is using **pseudocode** to develop a procedure, `TestRand()`, which will:

- use the random number function to generate an integer value in the range 1 to 50 inclusive
- count how many times the random function needs to be called before all 50 values have been generated
- output a message giving the number of times the random function was called.

Write **pseudocode** for the procedure `TestRand()`.

For the built-in functions list, refer to the **Appendix** on page 16.

[8]

Appendix of this paper:

Appendix**Built-in functions (pseudocode)**

Each function returns an error if the function call is not properly formed.

`MID(ThisString : STRING, x : INTEGER, y : INTEGER)` RETURNS STRING
returns a string of length `y` starting at position `x` from `ThisString`

Example: `MID("ABCDEFGH", 2, 3)` returns "BCD"

`LENGTH(ThisString : STRING)` RETURNS INTEGER
returns the integer value representing the length of `ThisString`

Example: `LENGTH("Happy Days")` returns 10

`LEFT(ThisString : STRING, x : INTEGER)` RETURNS STRING
returns leftmost `x` characters from `ThisString`

Example: `LEFT("ABCDEFGH", 3)` returns "ABC"

`RIGHT(ThisString : STRING, x : INTEGER)` RETURNS STRING
returns rightmost `x` characters from `ThisString`

Example: `RIGHT("ABCDEFGH", 3)` returns "FGH"

`INT(x : REAL)` RETURNS INTEGER
returns the integer part of `x`

Example: `INT(27.5415)` returns 27

`ASC(ThisChar : CHAR) RETURNS INTEGER`
 returns the ASCII value of character `ThisChar`

Example: `ASC('A')` returns 65

`RAND(x : INTEGER) RETURNS REAL`
 returns a real number in the range 0 to `x` (`x` not inclusive).

Example: `RAND(87)` could return 35.43

Operators (pseudocode)

Operator	Description
<code>&</code>	Concatenates (joins) two strings Example: <code>"Summer" & " " & "Pudding"</code> produces <code>"Summer Pudding"</code>
<code>AND</code>	Performs a logical AND on two Boolean values Example: <code>TRUE AND FALSE</code> produces <code>FALSE</code>
<code>OR</code>	Performs a logical OR on two Boolean values Example: <code>TRUE OR FALSE</code> produces <code>TRUE</code>

138. M/J 19/P21/Q6/b

- 6 A text file, `MyCDs.txt`, stores information relating to a Compact Disc (CD) collection. Information about each CD is stored on three separate lines in the file as follows:

Line 1: `<Artist Name>`
 Line 2: `<CD Title>`
 Line 3: `<Storage Location>`

Information is stored as data strings.

A section of the file is shown:

File line	Data
100	<code>"Green Floyd"</code>
101	<code>"Bowlful of Cereal"</code>
102	<code>"Shelf 4"</code>
103	<code>"Strolling Bones"</code>
104	<code>"Exile on Station Road"</code>
105	<code>"Box 12"</code>

- (a) A program, `CDOrganiser`, will be written to manage the stored information. The program will consist of three modules: `AddCD`, `FindCD` and `RemoveCD`.

Give **three** reasons why it is good practice to construct the program using modules.

1

2

3

[3]

- (b) The module, `FindCD()`, will check whether a given CD exists in the collection. The module will be implemented as a function.

The function will:

- be called with two strings as parameters, representing the artist name and CD title
- return a string that gives the storage location, or an empty string if the given CD has not been found.

Write **program code** for the function `FindCD()`.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

[8]

139. M/J 19/P22/Q1/b(ii)

- (b) Program variables have values as follows:

Variable	Value
Married	03/04/1982
ID	"M1234"
MiddleInitial	'J'
Height	5.6
IsMarried	TRUE
Children	2

- (ii) Programming languages support different data types.

Give an appropriate data type for the following variables from **part (b)**.

Variable	Data type
Married	
ID	
MiddleInitial	
Height	
IsMarried	

[5]

140. M/J 19/P22/Q5/b

5 A student is learning about text files. She wants to write a program to count the number of lines in a file.

(b) A procedure, `CountLines()`, is being written to count the number of lines in a text file. The procedure will:

- take a filename as a string parameter
- count the number of lines in the file
- output a single suitable message that includes the total number of lines.

Write **pseudocode** for the procedure `CountLines()`.

[6]

141. M/J 19/P23/Q2/c

(c) The student is learning about file handling.

She has been told that there are different file modes that can be used when opening a text file. She wants to make sure that the existing contents are not deleted when the file is opened.

Identify **two** file modes she could use **and** describe their use.

Mode

Description

Mode

Description

[4]

142. M/J 19/P23/Q4/a,b

- 4 A program is being written to process student information. One task involves inputting the names of all students in a class.

A first attempt at the pseudocode for this task is as follows:

```

DECLARE Name1 : STRING
DECLARE Name2 : STRING
DECLARE Name3 : STRING
    ↪
DECLARE Name40 : STRING

OUTPUT "Input the name for student 1"
INPUT Name1
OUTPUT "Input the name for student 2"
INPUT Name2
OUTPUT "Input the name for student 3"
INPUT Name3
    ↪
OUTPUT "Input the name for student 40"
INPUT Name40

```

- (a) Re-write the **pseudocode** to perform this task in a more efficient way, to allow for the class of 40 students.

[4]

- (b) Give **one** advantage of your solution.

[1]

143. M/J 19/P23/Q6/a,b

- 6 A text file, `StudentContact.txt`, contains a list of names and telephone numbers of students in a school. Not all students in the school have provided a contact telephone number. In this case, their name will not be in the file.

Each line of the file is stored as a string that contains a name and telephone number, separated by the asterisk character ('*') as follows:

<Name>'*<TelNumber>, for example:

"Bill Smith*081234567"

A 1D array, `ClassList`, contains the names of students in a particular class. The array consists of 40 elements of string data type. You can assume that student names are unique. Unused elements contain the empty string "".

A program is to be written to produce a **new** text file, `ClassContact.txt`, containing student names and numbers for all students in a particular class.

For each name contained in the `ClassList` array, the program will:

- search the `StudentContact.txt` file
- copy the matching string into `ClassContact.txt` if the name is found
- write the name together with “*No number” into `ClassContact.txt` if the name is not found.

The program will be implemented as three modules. The description of these is as follows:

Module	Description
<code>ProcessArray()</code>	• Check each element of the array: ○ Read the student name from the array ○ Ignore unused elements ○ Call <code>SearchFile()</code> with the student name ○ If the student name is found, call <code>AddToFile()</code> to write the student details to the class file ○ If the student name is not found, call <code>AddToFile()</code> to write a new string to the class file, formed as follows: <Name>“*No number” • Return the number of students who have not provided a telephone number
<code>SearchFile()</code>	• Search for a given student name at the start of each line in the file <code>StudentContact.txt</code>: ○ If the search string is found, return the text line from <code>StudentContact.txt</code> ○ If the search string is not found, return an empty string
<code>AddToFile()</code>	• Append the given string to a specified file, for example, <code>AddToFile(StringName, FileName)</code>

(a) Write **program code** for the module `SearchFile()`.

Visual Basic and Pascal: You should include the declaration statements for variables.
Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

[8]

(b) Write **pseudocode** for the module `ProcessArray()`.

[9]

144. O/N 18/P22/Q1/b

also Unit 11 (5 marks); this unit 5 marks

(b) (i) The following table contains statements written in pseudocode.

Give the most appropriate data type for the variable used in each statement.

Statement	Data type
Revision ← 'B'	
MaxValue ← 13.3	
ArrayFull ← TRUE	
Activity ← "Design"	
NumberOfEdits ← 270	

[5]

(ii) The following table contains statements written in pseudocode.

Complete the table by evaluating each expression using the values from **part (b)(i)**. If any expression is invalid, write "ERROR" in the **Evaluates to** column.

For the built-in functions list, refer to the **Appendix** on page 16.

Expression	Evaluates to
MID(Activity, 3, 4) & "ature"	
INT(MaxValue * 2)	
ArrayFull AND NumberOfEdits < 300	
ASC(Revision + 1)	
Activity = "Testing" OR Revision = 'A'	

[5]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING
returns a string of length y starting at position x from ThisString

Example: MID("ABCDEFGH", 2, 3) returns string "BCD"

LENGTH(ThisString : STRING) RETURNS INTEGER
returns the integer value representing the length of ThisString

Example: LENGTH("Happy Days") returns 10

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns leftmost x characters from ThisString

Example: LEFT("ABCDEFGH", 3) returns string "ABC"

RIGHT(ThisString: STRING, x : INTEGER) RETURNS STRING
returns rightmost x characters from ThisString

Example: RIGHT("ABCDEFGH", 3) returns string "FGH"

NUM_TO_STRING(x : REAL) RETURNS STRING
returns a string representation of a numeric value.

Example: If x has the value 87.5 then NUM_TO_STRING(x) will return "87.5"

Note: This function will also work if x is of type integer.

INT(x : REAL) RETURNS INTEGER
returns the integer part of x

Example: INT(27.5415) returns 27

ASC(ThisChar : CHAR) RETURNS INTEGER
returns the ASCII value of character ThisChar

Example: ASC('A') returns 65

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

145. O/N 18/P22/Q4/a(ii),b,d

4 Part of a program written in pseudocode is shown.

```
010 DECLARE NextArrayElement : INTEGER
```

```
...
```

```

100 FUNCTION ScanFile(SearchString: STRING) RETURNS INTEGER
101
102     DECLARE FileData : STRING
103     DECLARE FileLine : INTEGER
104
105     NextArrayElement ← 1
106     FileLine ← 1
107     FileData ← ReadFileLine("DataFile.txt", FileLine)
108
109     WHILE FileData <> ""
110         IF LEFT(FileData, 7) = SearchString
111             THEN
112                 ResultArray[NextArrayElement] ← FileData
113                 NextArrayElement ← NextArrayElement + 1
114             ENDIF
115             FileLine ← FileLine + 1
116             FileData ← ReadFileLine("DataFile.txt", FileLine)
117     ENDWHILE
118
119     CALL ScanCompleted()
120     RETURN FileLine
121
122 ENDFUNCTION

```

- (ii) Describe in detail the purpose of lines 109 to 117 in the `ScanFile()` function. Do **not** use pseudocode in your answer.

[4]

- (b) The function `ScanFile()` is to be amended so that the first 7 characters of `FileData` are **not** written to the array.

Write **program code** to implement this **amended** `ScanFile()` function.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

[8]

- (d) `ResultArray` is a 1D array of type `STRING`. It contains 100 elements.

Write **program code** to declare `ResultArray` and set all elements to the value "NO DATA".

Programming language

Program code

[3]

146. O/N 18/P22/Q5

5 The procedure `LineNumber()` will:

- read data from a text file
- modify each line by adding a line number, and the string (" : ")
- output each modified line.

For example, when the procedure reads `MyFile.txt`, the output is:

```
10: <First line of MyFile.txt>
15: <Second line of MyFile.txt>
20: <Third line of MyFile.txt>
```

...

```
350: <Last line of MyFile.txt>
```

The procedure takes three parameters:

Identifier	Data type	Description
FileName	STRING	The name of the text file
StartNumber	INTEGER	The first line number to be added
StepNumber	INTEGER	The line number increment

In this example, the procedure call would be:

```
CALL LineNumber("MyFile.txt", 10, 5)
```

After every 20 lines, the program outputs a message asking whether the user wants to continue.

The program ends when the user enters an 'N' or the end of file is reached.

Write **pseudocode** for the `LineNumber()` procedure.

[11]

147. O/N 18/P23/Q1/b

also Unit 11 (5 marks); this unit 5 marks

(b) (i) The following table contains statements written in pseudocode.

Give the most appropriate data type for the variable used in each statement.

Statement	Data type
Revision ← 500	
FuelType ← 'P'	
MinValue ← -6.3	
ServiceDue ← FALSE	
ModelRef ← "W212DEC15"	

[5]

(ii) The following table contains statements written in pseudocode.

Complete the table by evaluating each expression using the values from part (b)(i).

If any expression is invalid, write "ERROR" in the **Evaluates to** column.

For the built-in functions list, refer to the **Appendix** on page 16.

Expression	Evaluates to
"Month: " & MID(ModelRef, 5, 3)	
INT(MinValue * 2)	
ASC(Revision)	
Revision > 500	
ServiceDue = TRUE OR FuelType = 'P'	

[5]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING
returns a string of length y starting at position x from ThisString

Example: MID("ABCDEFGH", 2, 3) returns string "BCD"

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns leftmost x characters from ThisString

Example: LEFT("ABCDEFGH", 3) returns string "ABC"

RIGHT(ThisString: STRING, x : INTEGER) RETURNS STRING
returns rightmost x characters from ThisString

Example: RIGHT("ABCDEFGH", 3) returns string "FGH"

LENGTH(ThisString : STRING) RETURNS INTEGER
returns the integer value representing the length of ThisString

Example: LENGTH("Happy Days") returns 10

INT(x : REAL) RETURNS INTEGER
returns the integer part of x

Example: INT(27.5415) returns 27

`ASC(ThisChar : CHAR)` RETURNS INTEGER
returns the ASCII value of `ThisChar`

Example: `ASC('A')` returns 65

`IS_NUM(ThisString : STRING)` RETURNS BOOLEAN
returns the value `TRUE` if `ThisString` contains only numeric characters ('0' to '9').

Example: `IS_NUM("1234X67")` returns `FALSE`

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: <code>TRUE AND FALSE</code> produces <code>FALSE</code>
OR	Performs a logical OR on two Boolean values Example: <code>TRUE OR FALSE</code> produces <code>TRUE</code>

148. O/N 18/P23/Q4/a(ii),d(i)

4 Part of a program written in pseudocode is shown.

```

010 DECLARE ThisArray : ARRAY [1:100, 1:10] OF STRING
011 DECLARE LastElement : INTEGER
...
100 FUNCTION Update(NewData : STRING) RETURNS INTEGER
101     DECLARE ArrayIndex : INTEGER
102     DECLARE Found : BOOLEAN
103
104     ArrayIndex ← 1
105     Found ← FALSE
106
107     WHILE ArrayIndex ≤ LastElement AND Found = FALSE
108         IF ThisArray[ArrayIndex, 1] > NewData
109             THEN
110                 Found ← TRUE
111             ELSE
112                 ArrayIndex ← ArrayIndex + 1
113             ENDIF
114     ENDWHILE
115
116     IF Found = TRUE
117         THEN
118             CALL Insert(ArrayIndex, NewData)
119         ELSE
120             ArrayIndex ← 0
121         ENDIF
122
123     RETURN ArrayIndex
124 ENDFUNCTION

```

- (ii)** Describe in detail, the purpose of lines 107 to 114 in the `Update()` function. Do **not** use pseudocode in your answer.

[4]

- (d) (i)** `CharArray` is a 1D array of type `CHAR`. It contains 200 elements.

Write **program code** to change all the numeric characters ('0' to '9') in `CharArray` to '*'.
 Programming language

Program code

[3]

149. O/N 18/P23/Q5

- 5 The function `ReadFileLine()` returns a specific line from a text file.

The function takes two parameters:

Identifier	Data type	Description
FileName	STRING	The name of the text file
FileLine	INTEGER	The line number that is required

The following pseudocode gives an example of the use of the function.

```
FileData ← ReadFileLine(FileName, FileLine)
```

The function `ReadFileLine()` will:

- open the file, `FileName`
- read each line from the file until line `FileLine` is found or the end of the file is reached
- if the line exists, return the string from this line; otherwise return the string "****"

Write **pseudocode** for the `ReadFileLine()` function.

[10]

150. M/J 18/P21/Q5/a,c

- 5 A golf club holds information about its members. When a member completes a round of golf, their score is stored along with their membership number and the date of the round.

(a) Explain why the club stores these data in a file rather than an array.

[1]

- (c) The information is stored in a text file, `ScoreDetails.txt`. The format of each line of the text file is as follows:

`<MembershipNumber><Date><Score>`

- `MembershipNumber` is a four-digit numeric string.
- `Date` is a six-digit numeric string in the format `DDMMYY`
- `Score` is a two-digit numeric string in the range "50" to "99".

A procedure, `AddNewScores()`, is being developed. This will allow the user to enter scores for several members on a particular date.

The procedure, `AddNewScores()`, will perform the following actions:

1. Prompt for the date of the scores.
2. Input the date of the scores.
3. Prompt for the membership number.
4. Input the membership number.
5. If the membership number is an empty string then end the procedure.
6. Prompt for the score.
7. Input the score.
8. Validate the score.
9. If the validation fails then repeat from step 6.
10. Form a text string from the data and write this to the `ScoreDetails.txt` file.
11. Repeat from step 3.

Write **program code** for the `AddNewScores()` procedure.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

[9]

151. M/J 18/P21/Q6/a,b

- 6 (a)** The following pseudocode includes references to a 1D array.

```
DECLARE StudentGrade : ARRAY[1:5] OF CHAR
DECLARE n : INTEGER
DECLARE x : CHAR
```

```
n ← 3
x ← StudentGrade[n]
```

- (i)** Use the correct technical terms to explain the meaning of [1:5] in this pseudocode.

[2]

- (ii)** Use the correct technical term to complete the following statement.

Integer *n* is used as the to *StudentGrade*.

[1]

- (b)** A 2D array, *Picture*, contains data representing a bitmap image. Each element of the array represents one pixel of the image. The image is grey-scale encoded where the value of each pixel ranges from 0 (representing black) to 255 (representing white) with intermediate values representing different levels of grey.

The following is an example of an image and the corresponding data values for the *Picture* array.

Bitmap image								Values							
								240	10	10	10	10	10	10	240
								80	80	240	80	80	240	80	80
								80	80	240	80	80	240	80	80
								80	80	150	150	150	150	80	80
								80	80	240	240	240	240	80	80
								80	80	150	150	150	150	80	80
								240	240	150	150	150	150	240	240
								240	240	150	150	150	150	240	240

In pseudocode, the array is declared as follows:

```
DECLARE Picture : ARRAY[1:8, 1:8] OF INTEGER
```

A function, *Lighten()*, is required to lighten the image. Lightening an image may cause it to 'burn out'. An image is said to be 'burnt out' if any pixel is set to the maximum value of 255.

The function *Lighten()* will:

1. increase the value of each pixel by 10%
2. return TRUE if the resultant image is 'burnt out'.

Write **pseudocode** to implement the `Lighten()` function.

Assume that the array `Picture` is a global variable.

[8]

152. M/J 18/P22/Q5/a,d

- 5 A golf club holds information about its members. When a member completes a round of golf, their score is stored along with their membership number and the date of the round.

A program is to be written to store and process the score information. The information to be stored is formed into a string as follows:

```
<MembershipNumber><Date><Score>
```

- (a) The program designer considers storing the strings in either a 1D array, `RoundScore` or as a separate variable for each round, for example, `RoundScore01`, `RoundScore02`, `RoundScore03` and so on.

Describe **two** advantages of storing the strings in a 1D array rather than as separate variables.

Advantage 1

Advantage 2

[2]

- (c) The program needs a function, `GetNumber()`, to return a valid membership number. A valid membership number is a four-digit numeric string between “1111” and “9999”.

The structured English representing the algorithm for this function is as follows:

1. Prompt and input a membership number.
2. Validate the membership number.
3. Repeat from step 1 if the membership number is invalid.
4. Return the valid membership number as a string.

An example of the function call in pseudocode is:

```
MembershipNumber ← GetNumber()
```

Write **program code** for the `GetNumber()` function.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

[4]

- (d) The program designer decides to store the strings from **part (a)** in a text file, `ScoreDetails.txt`

Each string is formatted as follows:

<MembershipNumber><Date><Score>

- `MembershipNumber` is a four-digit numeric string between "1111" and "9999".
- `Date` is a six-digit numeric string in the format DDMMYY, for example "040716".
- `Score` is a two-digit numeric string in the range "50" to "99".

A procedure `OutputLowestScore()` is required to output the lowest score for an individual member.

Assume that there is at least one string stored for each member.

The program needs a procedure, `OutputLowestScore()`, to process the `ScoreDetails.txt` file and output the lowest score for an individual member.

1. Use the `GetNumber()` function to obtain a valid membership number.
2. Search the `ScoreDetails.txt` file for the lowest score for that member.
3. Output a message giving the lowest score for that member and the date of the round. For example: "The lowest score was 66 on 300917"

Write **program code** for the `OutputLowestScore()` procedure.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

[10]

153. M/J 18/P22/Q6/a,b

- 6 (a) Individual elements in a 1D array are referenced using an integer value that is used as the subscript to the array.

Give the technical terms for the minimum and maximum values the subscript may take.

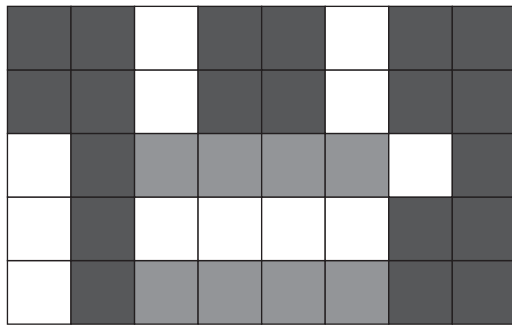
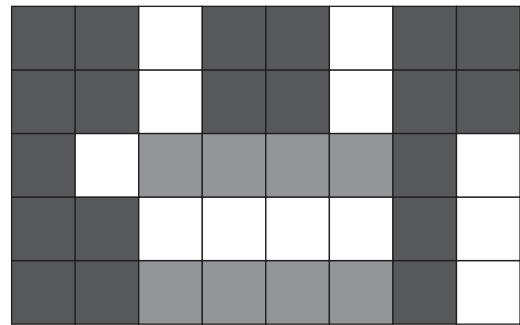
Minimum value

Maximum value

[2]

- (b) A 2D array, `Picture`, contains data representing a bitmap image. Each element of the array represents one pixel of the image. The image is grey-scale encoded where the value of each pixel ranges from 0 (representing black) to 255 (representing white) with intermediate values representing different levels of grey.

A graphics program needs a procedure, `Flip()`, to flip (reflect) the image. An example of an image before and after the function is:

Before flip**After flip**

The values contained in the 2D array before the flip are as follows:

Data values

80	80	255	80	80	255	80	80
80	80	255	80	80	255	80	80
255	80	120	120	120	120	255	80
255	80	255	255	255	255	80	80
255	80	120	120	120	120	80	80

In pseudocode, the array is declared as follows:

```
DECLARE Picture : ARRAY[1:5, 1:8] OF INTEGER
```

Write **pseudocode** to implement the `Flip()` procedure.

Assume that `Picture` is a global variable.

154. M/J 18/P23/Q5/a,c

- 5 A golf club holds information about its members. When a member completes a round of golf, their score is stored along with their membership number and the date of the round.

A program is to be written to store and process the score information.

The information to be stored is as follows:

- `MembershipNumber` is a four-digit numeric string.
- `Date` is a six-digit numeric string in the format DDMMYY
- `Score` is a two-digit numeric string in the range “50” to “99”.

This information is stored in a single string in the format:

`<MembershipNumber><Date><Score>`

- (a) (i) The program designer considers storing the strings in an array.

State how many dimensions the array would require.

[1]

- (ii) The designer has decided to store the strings in a file.

Give a reason for this choice.

[1]

- (c) The strings are stored in a text file, `ScoreDetails.txt`.

The program needs a function, `GetAverageScore()`.

The structured English representing the algorithm for this function is as follows:

- 1 Receive a membership number as its argument.
- 2 Search the `ScoreDetails.txt` file for all scores for that member.
- 3 Calculate the average score.
- 4 Return the average score.

You can assume that there will be at least one score relating to each member.

Write **program code** for the `GetAverageScore()` function.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

Program Code

[10]

155. M/J 18/P23/Q6/a,b

- 6 (a)** Individual elements in a 1D array are referenced using an integer variable.

For example, using the variable *n*, an individual element would be referenced in pseudocode as:

```
StudentGrade[n]
```

Give the correct technical term for the variable *n*.

[1]

- (b)** A 2D array, *Picture*, contains data representing a bitmap image. Each element of the array represents one pixel of the image. The image is grey-scale encoded where the value of each pixel ranges from 0 (representing black) to 255 (representing white) with intermediate values representing different levels of grey.

The following is an example of an image and the corresponding data values for the *Picture* array.

Bitmap image								Values							
								240	10	10	10	10	10	10	240
								80	80	240	80	80	240	80	80
								80	80	240	80	80	240	80	80
								80	80	150	150	150	150	80	80
								80	80	240	240	240	240	80	80
								80	80	150	150	150	150	80	80
								240	240	150	150	150	150	240	240
								240	240	150	150	150	150	240	240

In pseudocode, the array is declared as follows:

```
DECLARE Picture : ARRAY[1:8, 1:8] OF INTEGER
```

A function, *Clip()*, is required that will:

- take an integer parameter, *MaxVal*, to represent the maximum allowed value for each pixel
- limit each pixel value to the maximum allowed value
- return *TRUE* if any of the pixel values have been changed, otherwise return *FALSE*

An example use of the function is:

```
IsClipped ← Clip(234)
```

This will limit the maximum pixel value within the array to 234. Pixels with a value not greater than 234 will remain unchanged.

Write **pseudocode** to implement the `Clip()` function.

Assume that `Picture` is a global variable.

[9]

156. O/N 17/P21/Q3/a,b,c

also Unit 12 (4 marks), Unit 5 (3 marks); this unit 16 marks

- 3 (a)** A multi-user computer system stores information about users. It uses a 1D array, `UserNameArray`, of type `STRING`. There are 100 elements in the array.

The format of the string in each element of the array is as follows:

`<UserID><UserName>`

- `UserID` is a six-character string of numerals.
- `UserName` is a variable-length string.

Write **pseudocode** for a procedure, `BubbleSort`, to perform an efficient bubble sort on `UserNameArray`. The array is to be sorted in ascending order of `UserID`.

You should assume that `UserNameArray` has been declared as a global variable.

[8]

- (b)** The value of `UserID` should be unique for each user but a problem has occurred and repeated `UserID` values may have been issued.

The array is sorted by `UserID`, so any repeated `UserID` values will appear in consecutive array elements.

A procedure, `FindRepeats` is required.

This will:

- compare each element with the previous element and output the `UserID` and `UserName` if the `UserID` is repeated
- output the total number of `UserID`s that are repeated.

For example, the `UserNameArray` contains the following entries.

Array element	Comment
	
122222Jim Moriarty	
	
123456Fred Smith	
123456Eric Sykes	Repeated User ID
123456Kevin Turvey	Repeated User ID
	
222244Alice Chan	
222244Myra Singh	Repeated User ID
	
333333Yasmin Halim	
	

For this example, the output is:

```
123456Eric Sykes
123456Kevin Turvey
222244Myra Singh
There are 3 repeated UserIDs
```

If no repeated `UserIDs` are found, the output is:

```
The array contains no repeated UserIDs
```

Write **program code** for the procedure, `FindRepeats`.

You should assume that `UserNameArray` has been declared as a global variable.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

[8]

- (c) (i) The `FindRepeats` procedure forms part of a program.

Name **three** stages in a program development cycle.

1

2

3

[3]

- (ii) The program containing `FindRepeats` will be created using an IDE.

State what is meant by IDE.

[1]

- (iii) Name **two** features provided by an IDE that assist in the program development cycle.

1

2

[2]

- (iv) The procedure, `FindRepeats`, is written assuming there are 100 elements in `UserNameArray`.

In the main program, the global array, `UserNameArray`, has been declared with only 50 elements.

State the type of error this will cause.

[1]

157. O/N 17/P21/Q5/a

- 5 A sports club maintains a record of the email address of each of its members. The details are stored in a text file, `EmailDetails.txt`. The format of each line of the text file is as follows:

`<MembershipNumber><EmailAddress>`

- `MembershipNumber` is a four-character string of numerals.
- `EmailAddress` is a variable-length string.

Membership of the club has increased and a four-character membership number is no longer adequate.

A procedure, `MakeNewFile`, is required to perform the following actions:

1. Create a new file, `NewEmailDetails.txt`
2. Read a line from file `EmailDetails.txt`
3. Extend `MembershipNumber` by adding two leading zero digits (for example, "1234" becomes "001234")
4. Write the new line to file `NewEmailDetails.txt`
5. Repeat steps 2 to 4 for all lines in the original file.

- (a) Write **pseudocode** for the procedure `MakeNewFile`.

For the built-in functions list, refer to the **Appendix** on page 14.

[8]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

In each function, if the function call is not properly formed, the function returns an error.

`MODULUS(x : INTEGER, y : INTEGER) RETURNS INTEGER`

returns the remainder when `x` is divided by `y` using integer arithmetic.
Example: `MODULUS(5, 2)` returns 1

`INT(x : REAL) RETURNS INTEGER`

returns the integer part of `x`.
Example: `INT(27.5415)` returns 27

`LENGTH(ThisString : STRING) RETURNS INTEGER`

returns the integer value representing the length of string `ThisString`.
Example: `LENGTH("Happy Days")` returns 10

`LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING`

returns leftmost `x` characters from `ThisString`.
Example: `LEFT("ABCDEFGH", 3)` returns string "ABC"

`RIGHT(ThisString: STRING, x : INTEGER) RETURNS STRING`

returns rightmost `x` characters from `ThisString`.
Example: `RIGHT("ABCDEFGH", 3)` returns string "FGH"

Operators (pseudocode)

Operator	Description
<code>&</code>	Concatenates (joins) two strings Example: <code>"Summer" & " " & "Pudding"</code> produces <code>"Summer Pudding"</code>
<code>AND</code>	Performs a logical AND on two Boolean values Example: <code>TRUE AND FALSE</code> produces <code>FALSE</code>
<code>OR</code>	Performs a logical OR on two Boolean values Example: <code>TRUE OR FALSE</code> produces <code>TRUE</code>

158. O/N 17/P22/Q1/a(i,ii)

- 1 (a) (i) Procedural high-level languages usually support different data types.

Give an appropriate data type for each data value in the following table:

Data value	Data type
FALSE	
03/03/2013	
35	
"INTEGER"	
3.5	
"35"	

[6]

- (ii) The following is a declaration in a high-level language:

```
DEFINE MyGrade[1 to 100]
```

State the data structure of variable `MyGrade`.

[1]

159. O/N 17/P22/Q2/a

- 2** A 1D array, `ClassName`, of type `STRING` contains 100 elements.

The following pseudocode represents a simple algorithm to process the array.

```
DECLARE SearchValue : STRING
DECLARE FoundFlag : BOOLEAN
DECLARE Index : INTEGER

INPUT SearchValue
FoundFlag ← FALSE
Index ← 1

WHILE Index < 101 AND FoundFlag = False
    IF ClassName[Index] = SearchValue
        THEN
            OUTPUT Index
            FoundFlag ← TRUE
        ENDIF
    Index ← Index + 1
ENDWHILE

IF FoundFlag = FALSE
    THEN
        OUTPUT "Not found"
    ENDIF
```

- (a)** Describe the purpose of the algorithm.

[2]

160. O/N 17/P22/Q4/a

- 4 Programming languages provide built-in functions to generate random numbers. To be truly random, the frequency of each number generated should be the same.

You are required to write program code to test the random number generator of your chosen language.

The test should:

- generate a given number of random numbers between 1 and 10 inclusive
- keep a count of the number of times each number is generated
- calculate the expected frequency of each number 1 to 10
- output the actual frequency of each number 1 to 10
- output the difference between the actual frequency and the expected frequency.

The program code should be written as a procedure. In pseudocode, the procedure heading will be:

```
PROCEDURE TestRandom(Repetitions AS INTEGER)
```

The parameter, `Repetitions`, contains a value representing the total number of random numbers that should be generated.

The following example shows the expected output for the procedure call, `TestRandom(200)`.

The expected frequency is 20.

Number	Frequency	Difference
1	17	-3
2	21	1
3	12	-8
4	28	8
5	20	0
6	19	-1
7	21	1
8	16	-4
9	24	4
10	22	2

- (a) Write **program code** for the procedure, `TestRandom`.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

[16]

161. O/N 17/P22/Q5

- 5 A sports club maintains a record of the email address of each of its members. The details are stored in a text file, `EmailDetails.txt`. The format of each line of the text file is as follows:

```
<MembershipNumber><EmailAddress>
```


- `MembershipNumber` is a four-character string of numerals
- `EmailAddress` is a variable-length string

When members leave the club their details need to be removed from the file.

A procedure, `RemoveDetails` is required. This will perform the following actions:

1. Input the `MembershipNumber` of the club member to be removed
2. Create a new file, `NewEmailDetails.txt`
3. Copy all the lines from `EmailDetails.txt` to `NewEmailDetails.txt`, except the line with the matching `MembershipNumber`

Write **pseudocode** for the procedure, `RemoveDetails`.

For the built-in functions list, refer to the **Appendix** on page 15.

[9]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

In each function, if the function call is not properly formed, the function returns an error.

`MODULUS(x : INTEGER, y : INTEGER) RETURNS INTEGER`

returns the remainder when `x` is divided by `y` using integer arithmetic.

Example: `MODULUS(5, 2)` will return 1

`INT(x : REAL) RETURNS INTEGER`

returns the integer part of `x`.

Example: `INT(27.5415)` returns 27

`LENGTH(ThisString : STRING) RETURNS INTEGER`

returns the integer value representing the length of string `ThisString`.

Example: `LENGTH("Happy Days")` returns 10

`LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING`

returns leftmost `x` characters from `ThisString`.

Example: `LEFT("ABCDEFGH", 3)` returns string "ABC"

`RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING`

returns rightmost `x` characters from `ThisString`.

Example: `RIGHT("ABCDEFGH", 3)` returns string "FGH"

`TONUM(ThisString : STRING) RETURNS INTEGER`

returns a numeric value equivalent to `ThisString`.

Example: `TONUM("1201")` returns integer value 1201

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings. Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values. Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values. Example: TRUE OR FALSE produces TRUE

162. O/N 17/P23/Q3/a,b,c

also Unit 12 (4 marks), Unit 5 (3 marks); this unit 16 marks

- 3 (a) A multi-user computer system stores information about users. It uses a 1D array, `UserNameArray`, of type `STRING`. There are 100 elements in the array.

The format of the string in each element of the array is as follows:

`<UserID><UserName>`

- `UserID` is a six-character string of numerals.
- `UserName` is a variable-length string.

Write **pseudocode** for a procedure, `BubbleSort`, to perform an efficient bubble sort on `UserNameArray`. The array is to be sorted in ascending order of `UserID`.

You should assume that `UserNameArray` has been declared as a global variable.

[8]

- (b) The value of `UserID` should be unique for each user but a problem has occurred and repeated `UserID` values may have been issued.

The array is sorted by `UserID`, so any repeated `UserID` values will appear in consecutive array elements.

A procedure, `FindRepeats` is required.

This will:

- compare each element with the previous element and output the `UserID` and `UserName` if the `UserID` is repeated
- output the total number of `UserID`s that are repeated.

For example, the `UserNameArray` contains the following entries.

Array element	Comment
	
122222Jim Moriarty	
	
123456Fred Smith	
123456Eric Sykes	Repeated User ID
123456Kevin Turvey	Repeated User ID
	
222244Alice Chan	
222244Myra Singh	Repeated User ID
	
333333Yasmin Halim	
	

For this example, the output is:

```
123456Eric Sykes
123456Kevin Turvey
222244Myra Singh
There are 3 repeated UserIDs
```

If no repeated `UserIDs` are found, the output is:

```
The array contains no repeated UserIDs
```

Write **program code** for the procedure, `FindRepeats`.

You should assume that `UserNameArray` has been declared as a global variable.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

[8]

- (c) (i) The `FindRepeats` procedure forms part of a program.

Name **three** stages in a program development cycle.

1

2

3

[3]

- (ii) The program containing `FindRepeats` will be created using an IDE.

State what is meant by IDE.

[1]

- (iii) Name **two** features provided by an IDE that assist in the program development cycle.

1

2

[2]

- (iv) The procedure, `FindRepeats`, is written assuming there are 100 elements in `UserNameArray`.

In the main program, the global array, `UserNameArray`, has been declared with only 50 elements.

State the type of error this will cause.

[1]

163. O/N 17/P23/Q5/a

- 5 A sports club maintains a record of the email address of each of its members. The details are stored in a text file, `EmailDetails.txt`. The format of each line of the text file is as follows:

`<MembershipNumber><EmailAddress>`

- `MembershipNumber` is a four-character string of numerals.
- `EmailAddress` is a variable-length string.

Membership of the club has increased and a four-character membership number is no longer adequate.

A procedure, `MakeNewFile`, is required to perform the following actions:

1. Create a new file, `NewEmailDetails.txt`
2. Read a line from file `EmailDetails.txt`
3. Extend `MembershipNumber` by adding two leading zero digits (for example, "1234" becomes "001234")
4. Write the new line to file `NewEmailDetails.txt`
5. Repeat steps 2 to 4 for all lines in the original file.

- (a) Write **pseudocode** for the procedure `MakeNewFile`.

For the built-in functions list, refer to the **Appendix** on page 14.

[8]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

In each function, if the function call is not properly formed, the function returns an error.

MODULUS(*x* : INTEGER, *y* : INTEGER) RETURNS INTEGER

returns the remainder when *x* is divided by *y* using integer arithmetic.
Example: MODULUS(5, 2) returns 1

INT(*x* : REAL) RETURNS INTEGER

returns the integer part of *x*.
Example: INT(27.5415) returns 27

LENGTH(ThisString : STRING) RETURNS INTEGER

returns the integer value representing the length of string ThisString.
Example: LENGTH("Happy Days") returns 10

LEFT(ThisString : STRING, *x* : INTEGER) RETURNS STRING

returns leftmost *x* characters from ThisString.
Example: LEFT("ABCDEFGH", 3) returns string "ABC"

RIGHT(ThisString: STRING, *x* : INTEGER) RETURNS STRING

returns rightmost *x* characters from ThisString.
Example: RIGHT("ABCDEFGH", 3) returns string "FGH"

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

164. M/J 17/P21/Q1/b(i)

- (b) (i)** AND and OR are two operators that may be used when implementing an algorithm. An example of their use is given in the following pseudocode statement:

```
MyFlag ← VarA OR VarB
```

State the data type of variable `MyFlag`.

[1]

165. M/J 17/P21/Q5

- 5 A multi-user computer system records user login information in a text file, `LoginFile.txt`. Each time a user successfully logs into the system, the following information is recorded:

Item	Information	Example data
1	A five character user ID	"JimAA"
2	A four character port ID	"3456"
3	A fourteen character time and date	"08:30Jun012015"

The data items are concatenated to form a single string. Each string is saved as a separate line in the text file.

The example data in the preceding table would result in the following text line in the file:

"JimAA345608:30Jun012015"

The computer system can produce a list of the successful login attempts by a given user.

The file `LoginFile.txt` is searched for a given user ID and the corresponding data are copied into a 2D array, `LoginEvents`.

`LoginEvents` has been declared in pseudocode as:

```
DECLARE LoginEvents[1 : 1000, 1 : 2] OF STRING
```

A procedure, `SearchFile`, is needed to search the file and copy selected data to the array.

The main steps of the procedure are as follows:

- Input a user ID.
- Search `LoginFile.txt` for entries with matching user ID.
- For matching entries, copy items 2 and 3 above into the `LoginEvents` array.

You can assume that:

- the system initialises all elements of `LoginEvents` to an empty string "", before it calls `SearchFile`
- there will be no more than 1000 successful logins for a single user.

Write **program code** for the procedure `SearchFile`.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

166. M/J 17/P22/Q5/a,b

- 5 A multi-user computer system records user login data. Each time a user successfully logs into the system, it records the following data.

Data item	Example data
User ID	"Jim27"
Port ID	"3456"
Time and date	"08:30 Jun 01 2015"

The data items are concatenated (joined) using a separator character to form a single string. Each string represents one log entry.

- (a) (i) Suggest a suitable separator character. Give the reason for your choice.

Character

Reason

[2]

- (ii) The concatenated strings are stored in an array, `LogArray`, which may contain up to 20 log entries.

Use **pseudocode** to declare `LogArray`.

[2]

- (b) A procedure, `LogEvents`, is required to add the log entry data from `LogArray` to the existing text file, `LoginFile.txt`.

Unused array elements are assigned the value `"*****"`. These can occur anywhere in the array and should not be written to the file.

Write **program code** for the procedure `LogEvents`.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

[8]

167. M/J 17/P23/Q1/b(i)

- (b) (i) `AND` and `OR` are two operators that may be used when implementing an algorithm. An example of their use is given in the following pseudocode statement:

```
MyFlag ← VarA OR VarB
```

State the data type of variable `MyFlag`.

[1]

168. M/J 17/P23/Q5

- 5 A multi-user computer system records user login information in a text file, `LoginFile.txt`. Each time a user successfully logs into the system, the following information is recorded:

Item	Information	Example data
1	A five character user ID	"JimAA"
2	A four character port ID	"3456"
3	A fourteen character time and date	"08:30Jun012015"

The data items are concatenated to form a single string. Each string is saved as a separate line in the text file.

The example data in the preceding table would result in the following text line in the file:

"JimAA345608:30Jun012015"

The computer system can produce a list of the successful login attempts by a given user.

The file `LoginFile.txt` is searched for a given user ID and the corresponding data are copied into a 2D array, `LoginEvents`.

`LoginEvents` has been declared in pseudocode as:

```
DECLARE LoginEvents[1 : 1000, 1 : 2] OF STRING
```

A procedure, `SearchFile`, is needed to search the file and copy selected data to the array.

The main steps of the procedure are as follows:

- Input a user ID.
- Search `LoginFile.txt` for entries with matching user ID.
- For matching entries, copy items 2 and 3 above into the `LoginEvents` array.

You can assume that:

- the system initialises all elements of `LoginEvents` to an empty string "", before it calls `SearchFile`
- there will be no more than 1000 successful logins for a single user.

Write **program code** for the procedure `SearchFile`.

Visual Basic and Pascal: You should include the declaration statements for variables.

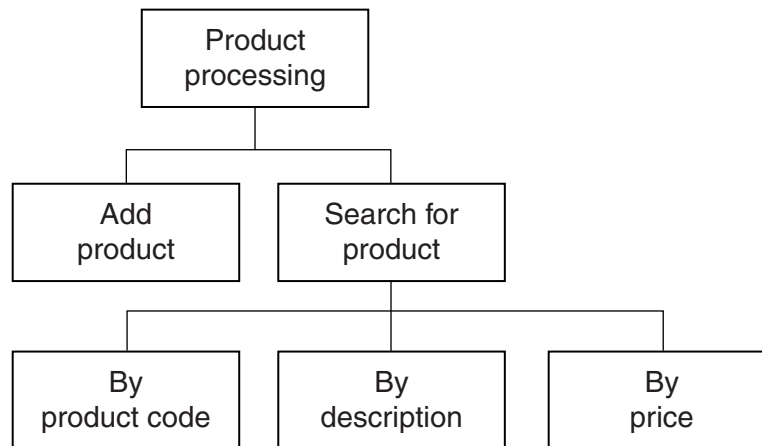
Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

169. O/N 16/P21/Q4/c,d,e

also Unit 12 (4 marks); this unit 13 marks

4 A company employs Ahmed as a programmer.**(c)** The company maintains a file of product data. Ahmed is to write a program to add a new product and search for a product based on the structure diagram shown:

The program records the following data for each product:

- product code
- product description
- product retail price

The text file `PRODUCTS` stores each data item on a separate line, as shown below:**File `PRODUCTS`**

0198
Plums (10kg)
11.50
0202
Onions (20kg)
10.00
0376
Mango chutney (1kg)
02.99
0014
Mango (10kg)
12.75

The program uses the variables shown in the identifier table.

Identifier	Data type	Description
PRODUCTS	TEXT FILE	Storing the code, description and retail price for all current products
PCode	ARRAY[1:1000] OF STRING	Array storing the product codes
PDescription	ARRAY[1:1000] OF STRING	Array storing the product descriptions
PRetailPrice	ARRAY[1:1000] OF REAL	Array storing the product retail prices
i	INTEGER	Array index used by all three arrays

- (i) The first operation of the program is to read all the product data held in file `PRODUCTS` and write them into the three 1D arrays.

Complete the pseudocode below.

```

OPEN .....
i ← 1
WHILE .....
    READFILE ("PRODUCTS", ..... )
    READFILE ("PRODUCTS", ..... )
    READFILE ("PRODUCTS", ..... )
ENDWHILE
CLOSE "PRODUCTS"
OUTPUT "Product file contents written to arrays"

```

[5]

When Ahmed designed the `PRODUCTS` file, he considered the alternative file structure shown opposite.

It stores one product per line in the text file.

File `PRODUCTS`

0198 Plums (10kg)	11.50
0202 Onions (20kg)	10.00
⋈	
0376 Mango chutney (1kg)	02.99
⋈	
0014 Mango (10kg)	12.75

- (ii) State **one** benefit and **one** drawback of this file design.

Benefit

Drawback

[2]

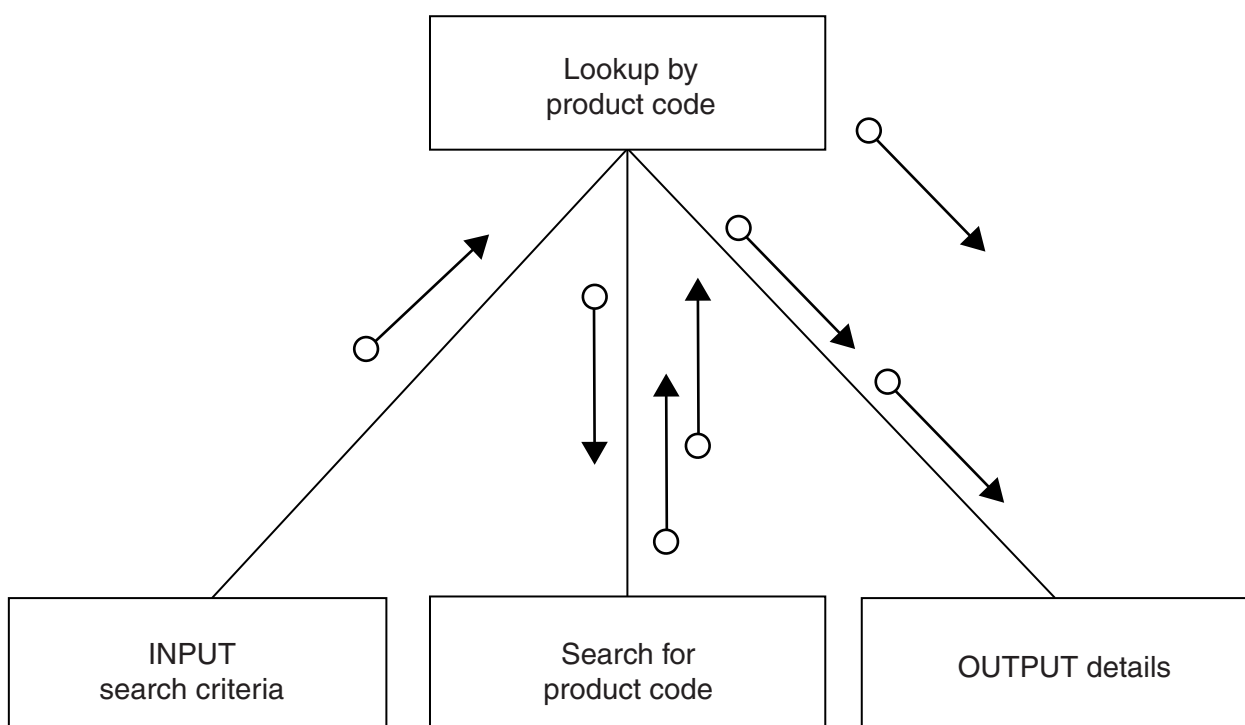
- (d) To code the 'Search by product code' procedure, Ahmed draws a structure chart showing the different stages.

The procedure uses the variables shown in the identifier table.

Identifier	Data type	Description
SearchCode	STRING	Product code input by the user
ThisIndex	INTEGER	Array index position for the corresponding product
ThisDescription	STRING	Product description found
ThisRetailPrice	REAL	Product retail price found

You can assume that before the procedure is run, all the product data is read from file `PRODUCTS` and then stored in three 1D arrays as described in **part (c)(i)**.

Label the structure chart to show the input(s) and output(s).



[4]

- (e) A first attempt was made at writing the 'Search for product code' module. Ahmed designs this as a function `ProductCodeSearch`.

The function returns an integer value as follows:

- if the product code is found, it returns the index position of the 1D array `PCode` being searched
- if the product code is not found, the function returns `-1`

Write **program code** for function `ProductCodeSearch`.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

[6]

170. O/N 16/P22/Q5/a,b(ii,iii)

- 5** A team keeps a record of the scores made by each of their eight players in a number of games.

The data in the two tables below shows:

- the scores of the eight players after twenty games
- the eight player names.

	1	2	3		8
1	12	17	67		31
2	35	82	44		29
3	61	39	80		17
4	81	103	21		11
5	56	0	98		4
...					
19	45	6	81		77
20	12	11	3		6

1	Vorma
2	Ravi
3	Chada
4	Nigam
5	Bahri
6	Smith
7	Goyal
8	Lata

The team wants a computer program to input and record the player data.

- (a)** A programmer designs the following pseudocode for the input of a player's score from one game.

```

01 INPUT GameNumber
02 INPUT PlayerNumber
03 INPUT PlayerGameScore
04 PlayerScore[GameNumber, PlayerNumber] ← PlayerGameScore

```

Describe the data structure the programmer has used for the storage of all player scores.

[2]

- (b) The player names are permanently stored in a text file `NAMES.TXT`, with one name on each line. The player names will be read by the program and stored in a 1D array.

The design given in **part (a)** will be expanded so that the user is prompted for the player name instead of the player number. Step 02 now becomes:

```
02.1  Read the player names from file NAMES.TXT into the array PlayerName
02.2  INPUT ThisPlayerName
02.3  Search the PlayerName array for ThisPlayerName to find the PlayerNumber
```

- (ii) Write the **program code** for step 02.1

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

[4]

- (iii) Program code is to be designed and written for step 02.3

The program will use these identifiers:

Identifier	Data type	Description
PlayerName	ARRAY[1 : 8] OF STRING	Stores the player names (read from the file)
ThisPlayerName	STRING	Input by the user (step 02.2)
Found	BOOLEAN	Flags when ThisPlayerName is found when searching the PlayerName array
i	INTEGER	Array index

Write **program code** to carry out the linear search for step 02.3

There is no requirement to declare or comment on variables used.

Programming language

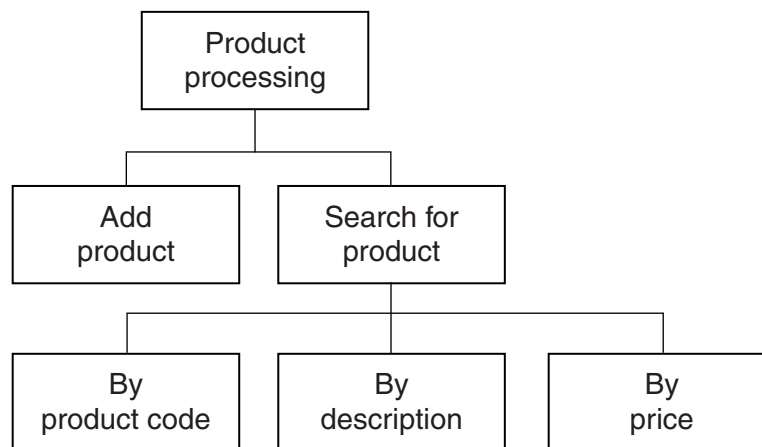
[4]

171. O/N 16/P23/Q4/c,d,e

also Unit 12 (4 marks); this unit 13 marks

- 4 A company employs Ahmed as a programmer.

- (c) The company maintains a file of product data. Ahmed is to write a program to add a new product and search for a product based on the structure diagram shown:



The program records the following data for each product:

- product code
- product description
- product retail price

The text file `PRODUCTS` stores each data item on a separate line, as shown below:

File **PRODUCTS**

0198
Plums (10kg)
11.50
0202
Onions (20kg)
10.00
0376
Mango chutney (1kg)
02.99
0014
Mango (10kg)
12.75

The program uses the variables shown in the identifier table.

Identifier	Data type	Description
PRODUCTS	TEXT FILE	Storing the code, description and retail price for all current products
PCode	ARRAY[1:1000] OF STRING	Array storing the product codes
PDescription	ARRAY[1:1000] OF STRING	Array storing the product descriptions
PRetailPrice	ARRAY[1:1000] OF REAL	Array storing the product retail prices
i	INTEGER	Array index used by all three arrays

- (i) The first operation of the program is to read all the product data held in file `PRODUCTS` and write them into the three 1D arrays.

Complete the pseudocode below.

```

OPEN .....
i ← 1
WHILE .....
    READFILE ("PRODUCTS", ..... )
    READFILE ("PRODUCTS", ..... )
    READFILE ("PRODUCTS", ..... )
ENDWHILE
CLOSE "PRODUCTS"
OUTPUT "Product file contents written to arrays"

```

[5]

When Ahmed designed the `PRODUCTS` file, he considered the alternative file structure shown opposite.

It stores one product per line in the text file.

File `PRODUCTS`

0198 Plums (10kg)	11.50
0202 Onions (20kg)	10.00
⋈	
0376 Mango chutney (1kg)	02.99
⋈	
0014 Mango (10kg)	12.75

- (ii) State **one** benefit and **one** drawback of this file design.

Benefit

Drawback

[2]

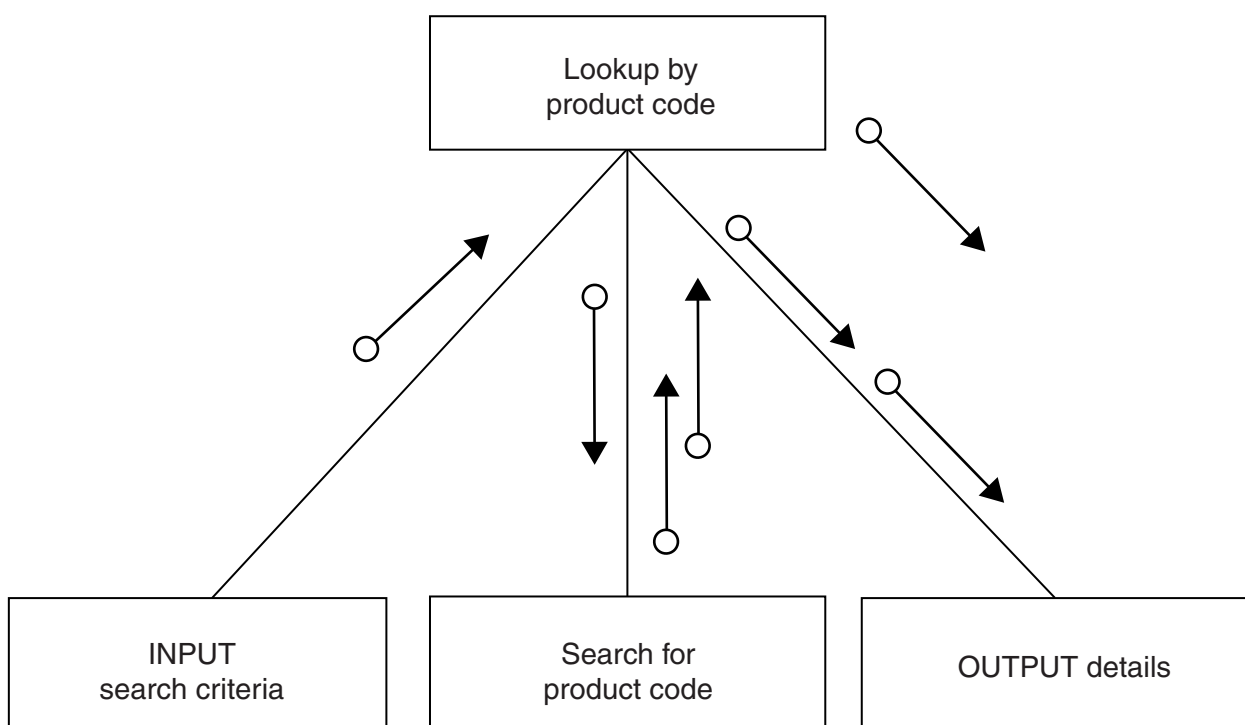
- (d) To code the 'Search by product code' procedure, Ahmed draws a structure chart showing the different stages.

The procedure uses the variables shown in the identifier table.

Identifier	Data type	Description
SearchCode	STRING	Product code input by the user
ThisIndex	INTEGER	Array index position for the corresponding product
ThisDescription	STRING	Product description found
ThisRetailPrice	REAL	Product retail price found

You can assume that before the procedure is run, all the product data is read from file `PRODUCTS` and then stored in three 1D arrays as described in **part (c)(i)**.

Label the structure chart to show the input(s) and output(s).



[4]

- (e) A first attempt was made at writing the 'Search for product code' module. Ahmed designs this as a function `ProductCodeSearch`.

The function returns an integer value as follows:

- if the product code is found, it returns the index position of the 1D array `PCode` being searched
- if the product code is not found, the function returns `-1`

Write **program code** for function `ProductCodeSearch`.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

[6]

172. M/J 16/P21/Q3/a,b(i)

- 3** A string encryption function is needed. The encryption uses a simple character-substitution method.

In this method, a new character substitutes for each character in the original string. This will create the encrypted string.

The substitution uses the 7-bit ASCII value for each character. This value is used as an index for a 1D array, `Lookup`, which contains the substitute characters.

`Lookup` contains an entry for each of the ASCII characters. It may be assumed that the original string and the substitute characters are all printable.

For example:

- 'A' has ASCII value 65
- Array element with index 65 contains the character 'Y' (the substitute character)
- Therefore, 'Y' substitutes for 'A'
- There is a different substitute character for every ASCII value

The programmer writes a function, `EncryptString`, to return the encrypted string. This function will receive two parameters, the original, `PlainText` string and the 1D array.

- (a)** The first attempt at writing the pseudocode for this function is shown below.

Complete the pseudocode.

For the built-in functions list, refer to the **Appendix** on page 18.

```

FUNCTION EncryptString(.....) RETURNS STRING
    DECLARE ..... , ..... : CHAR
    DECLARE OldCharValue : .....
    DECLARE n : INTEGER
    DECLARE OutString : STRING
    ..... //initialise the return string
    //loop through PlainText to produce OutString
    FOR n ← 1 TO ..... //from first to last character
        OldChar ← .....//get next character
        OldCharValue ← .....//find the ASCII value
        NewChar ← .....//look up substitute character
        .....//concatenate to OutString
    ENDFOR
ENDFUNCTION

```

[10]

- (b)** Additional code needs to be written to allow the user to change some of the characters in the array `Lookup`.

The user will input:

- the array start position
- the number of elements to change
- each new substitute character

At the end, the program will finally output a confirmation message.

The first version of the algorithm is represented by the flowchart on the following page.

(i) Write **program code** to declare the array `Lookup`.

Programming language

[2]

Appendix of this paper:

Appendix

Built-in functions

In each function below, if the function call is not properly formed, the function returns an error.

`MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING`

returns the string of length `y` starting at position `x` from `ThisString`

Example: `MID ("ABCDEFGH", 2, 3)` will return string `"BCD"`

`LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING`

returns the leftmost `x` characters from `ThisString`

Example: `LEFT ("ABCDEFGH", 3)` will return string `"ABC"`

`RIGHT(ThisString: STRING, x : INTEGER) RETURNS STRING`

returns the rightmost `x` characters from `ThisString`

Example: `RIGHT ("ABCDEFGH", 3)` will return string `"FGH"`

`ASC(ThisChar : CHAR) RETURNS INTEGER`

returns the ASCII value of character `ThisChar`

Example: `ASC ('W')` will return `87`

`LENGTH(ThisString : STRING) RETURNS INTEGER`

returns the integer value representing the length of string `ThisString`

Example: `LENGTH ("Happy Days")` will return `10`

String operator

& operator

concatenates (joins) two strings

Example: `"Summer" & " " & "Pudding"` produces `"Summer Pudding"`

173. M/J 16/P21/Q5/a,b

- 5 Toni has a large collection of jazz CDs that are stored in different places. She wants to record where the CDs are stored. She decides to write a program to do this.

The program must store the data in a file, `MyMusic`.

- (a) (i) Why is a file needed?

[1]

- (ii) `MyMusic` is a text file with the data for each CD as one line of text.

Data for a typical CD are:

Title: Kind of Green
Artist: Miles Coltrane
Location: Rack1-5

The line will be formed by concatenating the three data items.

For the example above, the line stored will be:

Kind of GreenMiles ColtraneRack1-5

Describe a problem that might occur when organising the data in this way.

Describe a possible solution.

[4]

- (b) Toni must input the data into the file for all of her CDs.

A procedure, `InputData`, is needed to do this.

Toni designs the procedure and chooses the following identifiers:

Identifier	Data type
<code>CDTitle</code>	STRING
<code>CDArtist</code>	STRING
<code>CDLocation</code>	STRING

The procedure repeatedly performs the following steps:

- input a CD title (A rogue value of “##” is to be used to end the input)
- input the artist
- input the location
- create the text line
- write the text line to the file

When the rogue value is encountered the file is closed.

Write **program code** for the procedure `InputData`.

Visual Basic and Pascal: You should include declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

[8]

174. M/J 16/P22/Q3/a,b(i)

- 3** A string encryption function is needed. The encryption uses a simple character-substitution method.

In this method, a new character substitutes for each character in the original string. This will create the encrypted string.

The substitution uses the 7-bit ASCII value for each character. This value is used as an index for a 1D array, `Lookup`, which contains the substitute characters.

`Lookup` contains an entry for each of the ASCII characters. It may be assumed that the original string and the substitute characters are all printable.

For example:

- 'A' has ASCII value 65
- Array element with index 65 contains the character 'Y' (the substitute character)
- Therefore, 'Y' substitutes for 'A'
- There is a different substitute character for every ASCII value

The programmer writes a function, `EncryptString`, to return the encrypted string. This function will receive two parameters, the original, `PlainText` string and the 1D array.

- (a)** The first attempt at writing the pseudocode for this function is shown below.

Complete the pseudocode.

For the built-in functions list, refer to the **Appendix** on page 18.

```

FUNCTION EncryptString(.....) RETURNS STRING
    DECLARE ..... : CHAR
    DECLARE OldCharValue : .....
    DECLARE n : INTEGER
    DECLARE OutString : STRING
    ..... //initialise the return string
    //loop through PlainText to produce OutString
    FOR n ← 1 TO ..... //from first to last character
        OldChar ← .....//get next character
        OldCharValue ← .....//find the ASCII value
        NewChar ← .....//look up substitute character
        .....//concatenate to OutString
    ENDFOR
ENDFUNCTION

```

[10]

- (b)** Additional code needs to be written to allow the user to change some of the characters in the array `Lookup`.

The user will input:

- the array start position
- the number of elements to change
- each new substitute character

At the end, the program will finally output a confirmation message.

The first version of the algorithm is represented by the flowchart on the following page.

(i) Write **program code** to declare the array `Lookup`.

Programming language

[2]

Appendix of this paper:

Appendix

Built-in functions

In each function below, if the function call is not properly formed, the function returns an error.

`MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING`

returns the string of length `y` starting at position `x` from `ThisString`

Example: `MID ("ABCDEFGH", 2, 3)` will return string `"BCD"`

`LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING`

returns the leftmost `x` characters from `ThisString`

Example: `LEFT ("ABCDEFGH", 3)` will return string `"ABC"`

`RIGHT(ThisString: STRING, x : INTEGER) RETURNS STRING`

returns the rightmost `x` characters from `ThisString`

Example: `RIGHT ("ABCDEFGH", 3)` will return string `"FGH"`

`ASC(ThisChar : CHAR) RETURNS INTEGER`

returns the ASCII value of character `ThisChar`

Example: `ASC ('W')` will return `87`

`LENGTH(ThisString : STRING) RETURNS INTEGER`

returns the integer value representing the length of string `ThisString`

Example: `LENGTH ("Happy Days")` will return `10`

String operator

& operator

concatenates (joins) two strings

Example: `"Summer" & " " & "Pudding"` produces `"Summer Pudding"`

175. M/J 16/P22/Q5/a,b

- 5 Toni has a large collection of jazz CDs that are stored in different places. She wants to record where the CDs are stored. She decides to write a program to do this.

The program must store the data in a file, `MyMusic`.

- (a) (i) Why is a file needed?

[1]

- (ii) `MyMusic` is a text file with the data for each CD as one line of text.

Data for a typical CD are:

Title: Kind of Green
Artist: Miles Coltrane
Location: Rack1-5

The line will be formed by concatenating the three data items.

For the example above, the line stored will be:

`Kind of GreenMiles ColtraneRack1-5`

Describe a problem that might occur when organising the data in this way.

Describe a possible solution.

[4]

- (b) Toni must input the data into the file for all of her CDs.

A procedure, `InputData`, is needed to do this.

Toni designs the procedure and chooses the following identifiers:

Identifier	Data type
<code>CDTitle</code>	STRING
<code>CDArtist</code>	STRING
<code>CDLocation</code>	STRING

The procedure repeatedly performs the following steps:

- input a CD title (A rogue value of “##” is to be used to end the input)
- input the artist
- input the location
- create the text line
- write the text line to the file

When the rogue value is encountered the file is closed.

Write **program code** for the procedure `InputData`.

Visual Basic and Pascal: You should include declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

[8]

176. M/J 16/P23/Q3/a

- 3** String encryption was implemented using a simple character-substitution method. A function, `Decrypt`, is needed to reverse the encryption process and return the original character.

The encryption uses the 7-bit ASCII value for each character. This value is used as an index for a 1D array, `Lookup`, which contains the substitute characters.

`Lookup` contains an entry for each of the ASCII characters.

This function, `Decrypt`, will accept two parameters, a single character, `CipherChar`, and the 1D array, `Lookup`.

The steps involved in `Decrypt` are follows:

- Search for the character in the array
- Note the index value where the character is found (the index value is the ASCII value of the original character)
- Use the index value to obtain the original character

- (a)** The first attempt at writing the pseudocode for this function is shown below.

Complete the pseudocode.

For the built-in functions list, refer to the **Appendix** on the last page.

```

FUNCTION Decrypt(..... , ..... ) RETURNS CHAR
    DECLARE Found      : .....
    DECLARE .....      : INTEGER
    DECLARE OriginalChar : CHAR
    Index ← 1          // .....
    Found ← FALSE
    //search for CipherChar in Lookup:
    WHILE .....
        //compare CipherChar with this array element:
        IF .....
            THEN
                .....//Set the flag
            ELSE
                Index .....//Move to next array element
            ENDIF
        ENDWHILE
    //dropped out of loop so must have found CipherChar:
    .....//convert Index to original character
    RETURN .....
ENDFUNCTION

```

[11]

Appendix of this paper:

Appendix

Built-in functions (Pseudocode)

In each function below, if the function call is not properly formed, the function returns an error.

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING

returns the string of length y starting at position x from ThisString

Example: **MID**("ABCDEFGH", 2, 3) will return string "BCD"

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING

returns the leftmost x characters from ThisString

Example: **LEFT**("ABCDEFGH", 3) will return string "ABC"

RIGHT(ThisString: STRING, x : INTEGER) RETURNS STRING

returns the rightmost x characters from ThisString

Example: **RIGHT**("ABCDEFGH", 3) will return string "FGH"

CHR(x : INTEGER) RETURNS CHAR

returns the character whose ASCII value is x

Example: **CHR**(87) will return 'w'

ASC(x : CHAR) RETURNS INTEGER

returns the ASCII value of character x

Example: **ASC**('w') will return 87

LCASE(x : CHAR) RETURNS CHAR

returns the lower case equivalent character of x

Example: **LCASE**('W') will return 'w'

UCASE(x : CHAR) RETURNS CHAR

returns the upper case equivalent character of x

Example: **UCASE**('h') will return 'H'

INT(x : REAL) RETURNS INTEGER

returns the integer part of x

Example: **INT**(27.5415) will return 27

177. M/J 16/P23/Q5/a,b

- 5 Claudia stores her large collection of music CDs in different places. Claudia wants to record where she stores each CD. She decides to write a program to do this.

Data items for a typical CD are:

Title: Kind of Green
Artist: Miles Coltrane
Location: Rack3-23

The data is to be stored in a text file, MyMusic. Each line of the text file will be a string, formed by concatenating the three data items.

Before concatenation, the title and artist will each be made into a fixed-length string of 40 characters. Space characters may need to be added to each data item.

The location is always 8 characters long.

- (a) (i) Explain the benefit of making the stored data into fixed-length strings.

State a drawback of this file design.

[3]

- (ii) When Claudia buys a new CD, the CD data must be added to the existing file, `MyMusic`. She has written a procedure in pseudocode. This has the following file-handling statements:

```
OPENFILE "MyMusic" FOR WRITE
WRITEFILE "MyMusic", OutputString
CLOSEFILE "MyMusic"
```

There is a problem with the logic of this pseudocode.

State the problem.

Identify the effect it will have if the final code is implemented in this way.

Give a possible solution.

[3]

- (b) Claudia needs to output a list of all the CDs in a particular location.

She designs a procedure, `OutputLocationList`, to do this. She also chooses the following identifiers:

Identifier	Data type
<code>CDTitle</code>	STRING
<code>CDArtist</code>	STRING
<code>CDLocation</code>	STRING

The procedure will:

- prompt for the name of the location
- input the location (such as "Rack3-23")
- search the file for all CDs at this location
- output the title and artist of each CD found
- output the total number of CDs found at that location (such as "17 CDs found")

Write **program code** for the procedure `OutputLocationList`.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

[10]

178. O/N 15/P22/Q5/a(ii)

5 A program is to process a set of integers.

The integers are stored in an array, Num. The first N elements are to be processed.

The pseudocode for this program is shown below:

```
FOR i ← 1 TO (N - 1)
    j ← 1
    REPEAT
        IF Num[j] > Num[j + 1]
            THEN
                Temp ← Num[j]
                Num[j] ← Num[j + 1]
                Num[j + 1] ← Temp
            ENDIF
        j ← j + 1
    UNTIL j = (N - i + 1)
ENDFOR
```

(ii) State the purpose of the algorithm.

[1]

179. M/J 15/P21/Q5/a,d,e

also Unit 9 (3 marks); this unit 13 marks

5 A company creates two new websites, Site X and Site Y, for selling bicycles.

Various programs are to be written to process the sales data.

These programs will use data about daily sales made from Site X (using variable SalesX) and Site Y (using variable SalesY).

Data for the first 28 days is shown below.

	SalesDate	SalesX	SalesY
1	03/06/2015	0	1
2	04/06/2015	1	2
3	05/06/2015	3	8
4	06/06/2015	0	0
5	07/06/2015	4	6
6	08/06/2015	4	4
7	09/06/2015	5	9
8	10/06/2015	11	9
9	11/06/2015	4	1
...			
28	01/07/2015	14	8

(a) Name the data structure to be used in a program for SalesX.

[2]

(d) The company decides to offer a discount on selected dates. A program is written to indicate the dates on which a discount is offered.

The program creates a text file, DISCOUNT_DATES (with data as shown), for a number of consecutive dates.

03/06/2015	TRUE
04/06/2015	FALSE
05/06/2015	FALSE
06/06/2015	FALSE
07/06/2015	FALSE
08/06/2015	FALSE
09/06/2015	FALSE
10/06/2015	TRUE
11/06/2015	FALSE
	
01/07/2015	FALSE

Each date and discount indicator is separated by a single <Space> character.

The discount indicators are:

- FALSE – indicates a date on which no discount is offered
- TRUE – indicates a date on which a discount is offered

A programming language has the built-in function CONCAT defined as follows:

```
CONCAT(String1 : STRING, String2 : STRING [, String3 : STRING] )
                                                    RETURNS STRING
For example:      CONCAT("San", "Francisco") returns "SanFrancisco"
                  CONCAT("New", "York", "City") returns "NewYorkCity"
```

The use of the square brackets indicates that the parameter is optional.

The following incomplete pseudocode creates the text file DISCOUNT_DATES.

Complete the pseudocode.

```

OPENFILE "DISCOUNT_DATES" FOR .....
INPUT .....
WHILE NextDate <>"XXX"
    INPUT Discount
    ..... = CONCAT(NextDate, " ", Discount)
    WRITEFILE "DISCOUNT_DATES", NextLine
    INPUT NextDate
OUTPUT "File now created"
CLOSEFILE

```

[4]

(e) The DISCOUNT_DATES text file is successfully created.

The company now wants a program to:

- key in a date entered by the user
- search the text file for this date
- if found, output one of the following messages:
 - "No discount on this date"
 - "This is a discount date"
- if not found, output "Date not found"

(i) Add to the identifier table to show the variables you need for this new program.

Identifier	Data type	Description
DISCOUNT_DATES	FILE	Text file to be used

[3]

(ii) Write the **program code**.

Do not include any declaration or comment statements for the variables used.

Programming language

[7]

180. M/J 15/P22/Q5/a,d,e

also Unit 9 (3 marks); this unit 13 marks

- 5** A company creates two new websites, Site X and Site Y, for selling bicycles.

Various programs are to be written to process the sales data.

These programs will use data about daily sales made from Site X (using variable `SalesX`) and Site Y (using variable `SalesY`).

Data for the first 28 days is shown below.

	SalesDate	SalesX	SalesY
1	03/06/2015	0	1
2	04/06/2015	1	2
3	05/06/2015	3	8
4	06/06/2015	0	0
5	07/06/2015	4	6
6	08/06/2015	4	4
7	09/06/2015	5	9
8	10/06/2015	11	9
9	11/06/2015	4	1
...			
28	01/07/2015	14	8

- (a)** Name the data structure to be used in a program for `SalesX`.

[2]

- (d)** The company decides to offer a discount on selected dates. A program is written to indicate the dates on which a discount is offered.

The program creates a text file, DISCOUNT_DATES (with data as shown), for a number of consecutive dates.

03/06/2015	TRUE
04/06/2015	FALSE
05/06/2015	FALSE
06/06/2015	FALSE
07/06/2015	FALSE
08/06/2015	FALSE
09/06/2015	FALSE
10/06/2015	TRUE
11/06/2015	FALSE
	
01/07/2015	FALSE

Each date and discount indicator is separated by a single <Space> character.

The discount indicators are:

- FALSE – indicates a date on which no discount is offered
- TRUE – indicates a date on which a discount is offered

A programming language has the built-in function CONCAT defined as follows:

```
CONCAT(String1 : STRING, String2 : STRING [, String3 : STRING] )
                                                    RETURNS STRING
For example:      CONCAT("San", "Francisco") returns "SanFrancisco"
                  CONCAT("New", "York", "City") returns "NewYorkCity"
```

The use of the square brackets indicates that the parameter is optional.

The following incomplete pseudocode creates the text file DISCOUNT_DATES.

Complete the pseudocode.

```
OPENFILE "DISCOUNT_DATES" FOR .....
INPUT .....
WHILE NextDate <>"XXX"
    INPUT Discount
    ..... = CONCAT(NextDate, " ", Discount)
    WRITEFILE "DISCOUNT_DATES", NextLine
    INPUT NextDate
```

OUTPUT "File now created"

CLOSEFILE

[4]

- (e) The DISCOUNT_DATES text file is successfully created.

The company now wants a program to:

- key in a date entered by the user
- search the text file for this date
- if found, output one of the following messages:
 - "No discount on this date"
 - "This is a discount date"
- if not found, output "Date not found"

- (i) Add to the identifier table to show the variables you need for this new program.

Identifier	Data type	Description
DISCOUNT_DATES	FILE	Text file to be used

[3]

- (ii) Write the **program code**.

Do not include any declaration or comment statements for the variables used.

Programming language

[7]

181. M/J 15/P23/Q4/b(i,iii,iv),c

- 4 A programming language has the built-in function CONCAT defined as follows:

```
CONCAT(String1 : STRING, String2 : STRING [, String3 : STRING] ) RETURNS STRING
For example: CONCAT("San", "Francisco") returns "SanFrancisco"
              CONCAT("New", "York", "City") returns "NewYorkCity"
```

The use of the square brackets indicates that the parameter is optional.

- (b) A country has a number of banks. There are cash dispensers all over the country. Each bank is responsible for a number of dispensers.

- banks have a three digit code in the range 001 – 999

- each dispenser has a five digit code in the range 00001 – 99999

A text file, DISPENSERS, is to be created.

It has one line of text for each dispenser. For example: 00342▼007.

This line in the file is the data for dispenser 00342 which belongs to bank 007.

Incomplete pseudocode follows for the creation of the file DISPENSERS.

For the creation of the file, data is entered by the user at the keyboard.

- (i) Complete the **pseudocode**.

```

OPENFILE ..... FOR WRITE
    OUTPUT "Enter dispenser code (XXXXX to end)"
    INPUT DispenserCode
    IF DispenserCode <> "XXXXX"
        THEN
            OUTPUT "Enter bank code"
            INPUT BankCode
            LineString ← CONCAT(....., "▼", BankCode)
            // now write the new line to the file
        ENDIF
    UNTIL .....
    OUTPUT "DISPENSERS file now created" [6]

```

- (iii) The programmer coded this algorithm above and the user successfully entered 15 dispenser records into the text file.

There is data for another 546 dispensers which needs to be added.

State the error that will occur if the user runs the program a second time for further data entry.

[1]

- (iv) Give the 'file mode' available in the programming language which will be used to address this issue.

[1]

(c) The complete data file is created with the structure shown.

A new program is to be written to search the file.

The program will:

- input a bank code
- output a list of all the dispensers which belong to this bank
- output the total number of dispensers for this bank

An example of a run of the program is shown:

Enter bank code 007

00001

00011

00022

00026

00027

There are 5 dispensers for this bank

```

00001▼007
00002▼001
00003▼002
00004▼003
00005▼101
00006▼004
00007▼004

/

00024▼002
00025▼003
00026▼007
00027▼007
00028▼102

/

99867▼013

```

Write the **program code**. Do not attempt to include any validation checks.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

182. M/J 15/P23/Q5/a

- 5** A firm employs workers who assemble amplifiers. Each member of staff works an agreed number of hours each day.

The firm records the number of completed amplifiers made by each employee each day.

Management monitor the performance of all its workers.

Production data was collected for 3 workers over 4 days.

Daily hours worked		Production data			
Worker 1	5	Worker 1	Worker 2	Worker 3	
Worker 2	10	Day 1	10	20	9
Worker 3	10	Day 2	11	16	11
		Day 3	10	24	13
		Day 4	14	20	17

A program is to be written to process the production data.

- (a)** The production data is to be stored in a 2-dimensional array `ProductionData`, declared as follows:

```
DECLARE ProductionData ARRAY[1:4, 1:3] : INTEGER
```

- (i)** Describe **two** features of an array.

1

2

[2]

- (ii)** Give the value of `ProductionData[3, 2]`.

[1]

- (iii)** Describe the information produced by the expression:

```
ProductionData[2, 1] + ProductionData[2, 2] + ProductionData[2, 3]
```

[2]

Answers Section

1. M/J 26/P21/Q2/a(i)

2(a)(i)	BOOLEAN	1
---------	---------	----------

2. M/J 26/P21/Q6/c

6(c)	<i>Example solution:</i> <pre> DECLARE ThisPacket : STRING DECLARE ThisLength : STRING DECLARE ThisMachine : CHAR DECLARE ThisDescription : STRING OPENFILE "Rejects.txt" FOR READ WHILE NOT EOF("Rejects.txt") READFILE "Rejects.txt", ThisPacket ThisLength ← LEFT(ThisPacket, 3) ThisMachine ← MID(ThisPacket, 5, 1) ThisDescription ← RejectDescription(ThisPacket) IF STR_TO_NUM(ThisLength) > 560 AND (ThisMachine = 'B' OR ThisMachine = 'C') THEN OUTPUT ThisDescription ENDIF ENDWHILE CLOSEFILE "Rejects.txt" </pre> <i>Mark as follows:</i> MP1 OPENFILE in READ mode and CLOSEFILE MP2 Conditional loop - terminates when EOF("Rejects.txt") MP3 READFILE – with correct syntax MP4 'Attempted' isolation of length and machine data MP5 IF STR_TO_NUM(ThisLength) > 560 AND (ThisMachine = 'B' OR ThisMachine = 'C') MP6 OUTPUT of return value from RejectDescription() inside the loop	6
6(a)	MP1 LEFT(DataPacket, 3) // MID(DataPacket, 1, 3) MP2 MID(DataPacket, 5, 1)	2

3. M/J 26/P21/Q7/a,b,c

7(a)(i)	Printer queue on a computer network // modelling/simulation of a real-world problem using a queue //accept anything reasonable	1
7(a)(ii)	The queue operates on the principle the first item to join the queue will be the next item to leave MP1 First in – first out 'feel' MP2 Must refer to their application stated in part (i)	2

7(b)(i)	MP1 DECLARE MyQueue : ARRAY [1:8] OF STRING MP2 DECLARE Front : INTEGER DECLARE Rear : INTEGER	2																								
7(b)(ii)	Queue <table style="margin-left: auto; margin-right: auto;"> <tr><td>1</td><td>{ "PIG" }</td><td></td></tr> <tr><td>2</td><td>"DOG"</td><td>← Front</td></tr> <tr><td>3</td><td>"RAT"</td><td></td></tr> <tr><td>4</td><td>"SPIDER"</td><td></td></tr> <tr><td>5</td><td>"HORSE"</td><td></td></tr> <tr><td>6</td><td>"CAT"</td><td>← Rear</td></tr> <tr><td>7</td><td></td><td></td></tr> <tr><td>8</td><td></td><td></td></tr> </table> Mark as follows: MP1 Correct sequence of five values in locations <u>2–6</u> MP2 Front points to DOG in location 2 and Rear points to CAT in location 6	1	{ "PIG" }		2	"DOG"	← Front	3	"RAT"		4	"SPIDER"		5	"HORSE"		6	"CAT"	← Rear	7			8			2
1	{ "PIG" }																									
2	"DOG"	← Front																								
3	"RAT"																									
4	"SPIDER"																									
5	"HORSE"																									
6	"CAT"	← Rear																								
7																										
8																										
7(b)(iii)	MyQueue <table style="margin-left: auto; margin-right: auto;"> <tr><td>1</td><td>"MONKEY"</td><td></td></tr> <tr><td>2</td><td>"SNAKE"</td><td>← Rear</td></tr> <tr><td>3</td><td></td><td></td></tr> <tr><td>4</td><td>("LION")</td><td></td></tr> <tr><td>5</td><td>"GIRAFFE"</td><td>← Front</td></tr> <tr><td>6</td><td>"TIGER"</td><td></td></tr> <tr><td>7</td><td>"ANT"</td><td></td></tr> <tr><td>8</td><td>"GERBIL"</td><td></td></tr> </table> Mark as follows: MP1 MONKEY and SNAKE in locations 1, 2 with correct Rear pointer MP2 GIRAFFE in 5 with Front pointer and TIGER, ANT, GERBIL in locations 6, 7 and 8	1	"MONKEY"		2	"SNAKE"	← Rear	3			4	("LION")		5	"GIRAFFE"	← Front	6	"TIGER"		7	"ANT"		8	"GERBIL"		2
1	"MONKEY"																									
2	"SNAKE"	← Rear																								
3																										
4	("LION")																									
5	"GIRAFFE"	← Front																								
6	"TIGER"																									
7	"ANT"																									
8	"GERBIL"																									

7(c)	Example solution: <pre> PROCEDURE InitialiseQueue() DECLARE i : INTEGER Front ← 0 /...7/8 Rear ← 0 / ... 7/8 FOR i ← 1 TO 8 MyQueue[i] ← "" NEXT i ENDPROCEDURE </pre> Mark as follows: MP1 Front and Rear are each assigned a value in range MP2 Correct FOR-NEXT loop // other loop structure MP3 MyQueue[i] assigned an empty string in a loop	3
------	---	----------

4. M/J 26/P21/Q9/a,b,c

9(a)	DECLARE Version1 : ARRAY [0:2, 1:15] OF CHAR MP1 MP2	2
9(b)	Example solution: <pre> DECLARE RoomNumber, FloorNumber, Count : INTEGER Count ← 0 OUTPUT "Floor number: " INPUT FloorNumber OUTPUT "Floor number: " & NUM_TO_STR(FloorNumber) FOR RoomNumber ← 1 TO 15 IF Version1[FloorNumber, RoomNumber] = 'A' THEN OUTPUT RoomNumber Count ← Count + 1 ENDIF NEXT RoomNumber OUTPUT NUM_TO_STR(Count) & " room(s) available" </pre> Mark as follows: MP1 Declaration of all integer variables MP2 Input of the floor number (with prompt) MP3 Loop for 15 iterations MP4 Test array element to check if room is 'available'? MP5 Initialisation and increment of count and Output room number MP6 Output of the final count line including the message outside the loop	6

9(c)(i)	Example solution: <pre>TYPE RoomData (DECLARE Available : <u>CHAR</u>) DECLARE RoomType : CHAR DECLARE View : BOOLEAN DECLARE Grade : STRING ENDTYPE</pre> <i>Mark as follows:</i> MP1 and MP2 Two variables correct 1 mark Three variables correct 2 marks MP3 ENDTYPE	3
---------	---	----------

9(c)(ii)	Example solution: <pre> DECLARE BoolView : BOOLEAN DECLARE MyRoomType : CHAR DECLARE InputView, MyRoomGrade : STRING DECLARE Floor, Room : INTEGER OUTPUT "Room type - S/D/F ?" INPUT MyRoomType OUTPUT "View - True/False ?" INPUT InputView IF TO_UPPER(InputView) = "TRUE" THEN BoolView ← TRUE ELSE BoolView ← FALSE ENDIF OUTPUT "Room grade - ST/DL ?" INPUT MyRoomGrade OUTPUT "Matching floor number and room number:" FOR Floor ← 0 TO 2 FOR Room ← 1 TO 15 IF Version2[Floor, Room].RoomType = MyRoomType AND Version2[Floor, Room].View = BoolView AND Version2[Floor, Room].RoomGrade = MyRoomGrade AND Version2[Floor, Room].Available = 'A' THEN OUTPUT Floor, " ", Room ENDIF NEXT Room NEXT Floor </pre> Marks as follows: MP1 Three correctly formed INPUT statements each with a prompt MP2 OUTPUT of the “Matching floor etc” heading MP3 Outer FOR loop / or other (for the floors) MP4 Inner FOR loop / or other (for the rooms) MP5 Correct use of array with dot notation with at least one data item inside loop MP6 ‘Attempt’ to combine all three tests (room type, view, grade) inside loop MP7 Output of floor and room inside loop	7
----------	---	---

5. M/J 26/P22/Q1/a(i)

1(a)(i)	STRING	1	
---------	--------	---	--

6. M/J 26/P22/Q4/a,b

4(a)	After group 1 back stack www.aaa.home www.ccc.my ← TOS forward stack www.bbb.home www.333.corp www.111.home www.999.corp www.222.me ← TOS Web address currently displayed in the browser: www.zzz.corp After group 2 back stack www.zzz.corp www.aaa.home www.ccc.my ← TOS forward stack www.333.corp www.111.home www.999.corp www.222.me ← TOS Website displayed in the browser: www.bbb.home After group 3 back stack www.111.home www.333.corp www.bbb.home www.zzz.corp www.aaa.home www.ccc.my ← TOS forward stack www.222.me ← TOS	6
------	---	---

4(a)	Website displayed in the browser: www.999.corp Mark as follows: Group 1 1 1 mark for back stack as shown and TOS position as shown on both stacks 2 1 mark for forward stack as shown and current web address Group 2 3 1 mark for stacks and correct TOS pointers as shown and current web address Group 3 4 1 mark for back stack as shown and TOS as shown 5 1 mark for forward stack as shown and TOS as shown 6 1 mark current web address		
4(b)	One mark for identifying an ADT. Queue / Linked List One mark per feature Features of a Queue: 1 1 Each queue element contains one data item 2 A front/start pointer and a rear/end pointer 3 Data is added at back / end and removed from front / start // works on a FIFO basis 4 Can be circular or linear MAX 2 marks for features Features of a Linked List: 1 1 Each node / item contains data and a pointer (to the next node/item) 2 A pointer to the start of the list 3 Last node in the list has a null pointer 4 Data may be added / removed by manipulating <u>pointers</u> 5 Pointers are followed to move through nodes in the list (in a given sequence) 6 Unused nodes are stored on a free list // a free-list pointer to the Free List 7 Is a dynamic (data structure) MAX 2 marks for features	3	Need both Reject use of 2 arrays

7. M/J 26/P22/Q5/a,b

5(a)	The greater than logic operator / > in the selection statement would need to be changed to a less than operator / < Or by example <pre>IF Numbers[Index] > Numbers[Index + 1] THEN</pre> would be changed to <pre>IF Numbers[Index] < Numbers[Index + 1] THEN OR IF Numbers[Index] <= Numbers[Index + 1] THEN OR IF Numbers[Index + 1] > Numbers[Index] THEN OR IF Numbers[Index + 1] >= Numbers[Index] THEN</pre>	1	
5(b)(i)	2 marks for inner loop MP 1: The inner loop iterates one less time (in algorithm B) each time the outer / repeat loop iterates. MP 2: this is because for each iteration of the outer / repeat loop (in algorithm B) the last / one more element in the <code>Numbers</code> array will be in the correct place. 2 marks for outer loop MP 3: The outer / repeat loop (in algorithm B) would stop iterating if no swaps are made / all numbers in array are sorted MP 4: so not as many iterations would be needed (if the <code>Numbers</code> array is not initially sorted)	4	Reject answers that just repeat the algorithm Reject program stops rather than iteration ends or algorithm ends
5(b)(ii)	1 mark: When <code>Numbers</code> (array) / array is already in (ascending) order / sorted	1	

8. M/J 26/P22/Q6/b

6(b)(i)	Solutions: • Grid : ARRAY[1 : 5, 1 : 5] OF <u>CHAR</u> • Grid : ARRAY[0 : 4, 0 : 4] OF <u>CHAR</u> 1 mark for boundaries of both array dimensions: • [1 : 5, 1 : 5] • [0 : 4, 0 : 4] 1 mark for all other parts of declaration	2	Reject use of different brackets eg () Reject CHARACTER instead of CHAR Reject STRING instead of CHAR Must be as exactly as above but accept use of lowercase
---------	---	----------	--

6(b)(ii)	Solution One : Using nested loop and output of each row in a loop following nested loop Example Solution <pre> PROCEDURE PopulateGrid(Plaintext : STRING) DECLARE Row, Column, Index : INTEGER Index ← 1 FOR Row ← 1 TO 5 FOR Column ← 1 TO 5 Grid[Row, Column] ← MID(Plaintext, Index, 1) Index ← Index + 1 NEXT Column Next Row FOR Row ← 1 TO 5 OUTPUT Grid[Row, 1], Grid[Row, 2], Grid[Row, 3], _ Grid[Row, 4], Grid[Row, 5] NEXT Row ENDPROCEDURE </pre> Mark Points 1 Generate Index, 1 to 25, used to extract all characters from Plaintext 2 Generate both Row and Col in a loop 3 Extract an individual character from Plaintext in a loop 4 Attempt to assign the extracted character to Grid in a loop 5 Extraction and assignment of all 25 characters in Plaintext to Grid in a loop 6 Attempt to identify / isolate 5 characters comprising at least one row 7 Identify / isolate all 5 characters for all 5 rows 8 Output all five rows of GRID, each on a separate line	8 Syntax for MID() function must use parameters of correct type – allow other string handling function combinations if they work. Max 7 if Procedure heading/parameter type incorrect Penalise incorrect syntax for indexes i.e. [Row] [Col] instead once.
----------	--	--

6(b)(ii)	Solution Two: Using nested loop and output of each string in a loop following nested loop - Example Solution <pre> PROCEDURE PopulateGrid(Plaintext : STRING) DECLARE Row, Column, Index : INTEGER DECLARE RowString : STRING FOR Row ← 1 TO 5 FOR Column ← 1 TO 5 Index ← (Row - 1) * 5 + Column Grid[Row, Column] ← MID(Plaintext, Index, 1) NEXT Column NEXT Row FOR Row ← 1 TO 5 RowString ← "" For Col ← 1 TO 5 RowString ← RowString & Grid[Row, Column] Next Col OUTPUT RowString NEXT Row ENDPROCEDURE </pre> Solution Three: Using nested loop where output string is formed in the inner loop and output in string in the outer loop – complex use of Index. No additional loop after nested loop Example Solution <pre> PROCEDURE PopulateGrid(Plaintext : STRING) DECLARE Row, Column, Index : INTEGER DECLARE RowString : STRING FOR Row ← 1 TO 5 RowString ← "" FOR Column ← 1 TO 5 Index ← (Row - 1) * 5 + Column Grid[Row, Column] ← MID(Plaintext, Index, 1) RowString ← RowString & MID(Plaintext, Index, 1) NEXT Column OUTPUT RowString NEXT Row ENDPROCEDURE </pre>		
----------	---	--	--

6(b)(ii)	Solution Four: No nested loop Example Soution <pre> PROCEDURE PopulateGrid(Plaintext : STRING) DECLARE Row, Column, Index : INTEGER Row ← 1 Col ← 0 FOR Index ← 1 TO 25 IF Index MOD 5 = 0 THEN Row ← Row + 1 Col ← 0 ENDIF Col ← Col + 1 Grid[Row, Column] ← MID(Plaintext, Index, 1) NEXT Index FOR Row ← 1 TO 5 OUTPUT Grid[Row, 1], Grid[Row, 2], _ Grid[Row, 3], Grid[Row, 4], Grid[Row, 5] NEXT Row ENDPROCEDURE </pre>		
----------	--	--	--

9. M/J 26/P23/Q2/a

2(a)	BOOLEAN	1	
------	---------	----------	--

10. M/J 26/P23/Q5/b

5(b)	<pre> PROCEDURE Search(Initial : CHAR, ShortSurname : STRING) DECLARE ThisApplicant, ThisSurname : STRING DECLARE ThisInitial : CHAR DECLARE Found : BOOLEAN OPENFILE "JobApps.txt" (FOR) READ Found ← FALSE REPEAT READFILE "JobApps.txt", ThisApplicant ThisInitial ← LEFT(ThisApplicant, 1) ThisSurname ← MID(ThisApplicant, 2, 5) IF ThisSurname = ShortSurname AND ThisInitial = Initial THEN Found ← TRUE OUTPUT "Name found" ENDIF UNTIL Found = TRUE OR EOF("JobApps.txt") IF Found = FALSE THEN OUTPUT "Name not Found" ENDIF CLOSE(FILE) "JobApps.txt" ENDPROCEDURE </pre> Mark as follows: 1 mark per gap	7	
------	---	---	--

11. M/J 26/P23/Q6/a,b,c,d

6(a)	MP1 5 1 "LEOPARD" 2 "DOG" 3 "BISON" 4 "ZEBRA" 5 "ANT" 6
------	---

6(c)(i)	Start pointer: 5 Mark as follows: Start is 5 and other pointer values are correct and BISON pointer is changed to 1 and arrows are as shown and DOG is 'isolated'	1	
6(c)(ii)	MP1 The item is not in the list MP2 The data item is the only item in the list MP3 The item is the first in the list	2	
6(d)	MP1 Store 'BAT' at the next available location /with index 6 / in the Data array MP2 Visit the start pointer node MP3 Compare this data value with 'BAT' //compare 'BAT' with 'ANT' MP4 When 'BAT' is less than the current value - 'position is found' MP5 IF not, move to next node // move to 'BISON' MP6 and store the previous position MP7 Repeat until 'BAT' less that current value MP8 Link pointer of the previous node // 'ANT' points/connects to 'BAT' MP9 Link pointer of 'BAT' points to the current node / 3 // 'BAT' points/connects to 'BISON' Max 6	6	

12. M/J 26/P23/Q8/a,b

8(a)(i)	DECLARE Bins : ARRAY [0:2, 1:120] OF BOOLEAN <u>MP1</u> <u>MP2</u>	2	Do not accept variations of syntax.
---------	---	---	-------------------------------------

8(a)(ii)	Example solution: <pre> PROCEDURE InactiveReport() DECLARE Level, BinNo : INTEGER DECLARE BinCount : INTEGER BinCount ← 0 FOR Level ← 0 TO 2 IF Level = 0 THEN OUTPUT "Level: ground floor" ELSE OUTPUT "Level: " & NUM_TO_STR(Level) ENDIF FOR BinNo ← 1 TO 5 IF Bins[Level, BinNo] = FALSE THEN OUTPUT "Bin " & NUM_TO_STR(BinNo) BinCount ← BinCount + 1 ENDIF NEXT BinNo NEXT Level OUTPUT NUM_TO_STR(BinCount) & " bins are not in use" ENDPROCEDURE </pre> Mark as follows: MP1 Bin count declared and initialised before the loop and correctly incremented MP2 (Outer) level loop – iterates 3 times MP3 (Inner) bin loop – iterates 5 times MP4 Correct 2D array syntax and test for not in use with a Boolean comparison MP5 Correct header OUTPUT statement(s) - includes the 'Ground floor' text and 'Bin' text and bin number MP6 Correct final OUTPUT statement with 'total not in use' text and the count Max 5	5	
8(a)(iii)	<pre> Bins[0, 3] ← FALSE Bins[1, 2] ← TRUE Bins[2, 1] ← FALSE Bins[2, 2] ← FALSE Bins[2, 3] ← TRUE Bins[2, 4] ← TRUE Bins[2, 5] ← FALSE </pre> Mark as follows: MP1 Bins(0, 3) ← FALSE MP2 Bins(1, 2) ← TRUE MP3 Level 2 assignment all complete	3	

8(b)(i)	Example solution: <pre> TYPE BinData DECLARE Size : STRING DECLARE Active : BOOLEAN DECLARE StockCode : STRING DECLARE StockLevel : INTEGER ENDTYPE </pre> Mark as follows: Two variables correct – 1 Mark 4 variables correct – 2 marks	2	
8(b)(ii)	Example solution: <pre> FUNCTION FindBin(MySize : STRING) RETURNS STRING DECLARE Level, BinNo : INTEGER DECLARE Text : STRING Text ← "No bins available" Level ← 0 REPEAT BinNo ← 1 REPEAT IF Bins[Level, BinNo].Size = MySize AND Bins[Level, BinNo].Active = FALSE THEN Text ← "Level: " & NUM_TO_STR(Level) & " Bin: " & NUM_TO_STR(BinNo) OUTPUT Text RETURN Text ELSE BinNo ← BinNo + 1 ENDIF UNTIL BinNo = 121 Level ← Level + 1 UNTIL Level = 3 OUTPUT Text RETURN Text ENDFUNCTION </pre> Mark as follows: MP1 Function header and string parameter and return string and ending MP2 Use of any (outer level) loop for 3 iterations MP3 Use of any (inner) bin loop for 5 / 120 iterations MP4 Correct 2D array syntax with either data field – .Size or .Active MP5 Correct IF statement condition including AND MP6 Formation of the text string with correct syntax – MP7 Correct return value (and output) – in both cases and in the correct place	7	Do not accept comma syntax.

13. O/N 25/P21/Q3/a,b

3(a)(i)	Pseudocode: <pre> TYPE RentalRecord DECLARE RentalID : STRING DECLARE CarID : INTEGER DECLARE DisCode : CHAR DECLARE Start : DATE DECLARE Duration : INTEGER DECLARE Completed : BOOLEAN ENDTYPE </pre> Mark as follows: 1 One mark for TYPE RentalRecord and ENDTYPE statements 2 One mark for RentalID and CarID declared correctly 3 One mark for DisCode and Start declared correctly 4 One mark for Duration and Completed declared correctly	4
3(a)(ii)	<u>DECLARE Rental : ARRAY[1:500] OF RentalRecord</u> One mark per underlined phrase	2
3(b)	One mark per point: 1 Different data types held (for each rental) under a single identifier 2 Allows for iteration through <u>rentals/records</u> // Can access rentals/records in a loop // Can (directly) access a specific <u>rental/record</u> <u>using an</u> (array) <u>index</u> 3 Simplifies the code/program / less code needed / reduces code duplication // Makes code/program easier to understand // Makes testing / debugging easier // Program maintenance easier	3

14. O/N 25/P21/Q4/a,b

4(a)	Example solution - conditional: <pre> PROCEDURE Store() DECLARE ThisNum, LastNum : REAL DECLARE Index : INTEGER // index to global array INPUT ThisNum LastNum ← ThisNum Number[1] ← ThisNum INPUT ThisNum Index ← 2 WHILE Index < 21 AND ThisNum > LastNum Number[Index] ← ThisNum LastNum ← ThisNum Index ← Index + 1 IF Index < 21 THEN INPUT ThisNum ENDIF ENDWHILE OUTPUT Index - 1, " values were stored in the array" ENDPROCEDURE </pre> Alternative loop: <pre> // Loop without using LastNum WHILE Index < 21 AND ThisNum > Number[Index - 1] Number[Index] ← ThisNum Index ← Index + 1 IF Index < 21 THEN //skip input if array full INPUT ThisNum ENDIF ENDWHILE </pre> Mark as follows: 1 Declare local variables Index as integer, ThisNum and LastNum as real 2 Input value and assign to first element 3 Attempt at conditional loop including both tests 4 Completely correct conditional loop including both tests 5 Assign input value to current element if greater than previous element and input next value in a loop 6 Final output giving attempted count of elements stored plus message after a loop	6
------	---	----------

4(a)	Alternative FOR loop example solution: <pre> PROCEDURE Store() DECLARE ThisNum, LastNum : REAL DECLARE Index : INTEGER // index to global array DECLARE Count : INTEGER INPUT ThisNum LastNum ← ThisNum Number[1] ← ThisNum Count ← 1 INPUT ThisNum FOR Index ← 2 TO 20 IF ThisNum > LastNum THEN Number[Index] ← ThisNum LastNum ← ThisNum Count ← Count + 1 IF Index < 20 THEN INPUT ThisNum ENDIF ELSE BREAK ENDIF NEXT Index OUTPUT Count, " values were stored in the array" ENDPROCEDURE </pre> Alternative loop: <pre> Count ← 1 INPUT ThisNum FOR Index ← 2 TO 20 IF ThisNum > Number[Index - 1] THEN Number[Index] ← ThisNum Count ← Count + 1 IF Index < 20 THEN INPUT ThisNum ENDIF ELSE BREAK ENDIF NEXT Index </pre> Mark as follows: 1 Declare local variables <code>Index</code> as integer, <code>ThisNum</code> and <code>LastNum</code> as real 2 Input value and assign to first element 3 Count-controlled loop 4 Count-controlled loop with BREAK if current value greater than previous value 5 ... Assign input value to current element following correct comparison in a loop 6 Final output giving count of elements stored plus message following a reasonable attempt after loop	
------	--	--

4(b)(i)	One mark per point: 1 The amount of values stored (in a text file) is not fixed / not limited (other than by secondary storage available) // The amount values stored is not limited to the size of the array 2 The data is stored permanently / non-volatile // Data can be restored / reused without re-entering values	2
4(b)(ii)	Each (real) value would have to be converted to a string	1

15. O/N 25/P21/Q5/a

5(a)(i)	It is a value that is invalid as an (array) index // not in the range of valid (index) values	1
5(a)(ii)	To test whether the current element / value at index matches the given / search string / data / parameter	1
5(a)(iii)	The value of <code>FoundAt</code> is only updated to the (current) index if it currently stores the initial value / -1 // <code>FoundAt</code> is not updated when it currently stores any other value than -1	1

16. O/N 25/P21/Q8/a,b

8(a)	Example solution: <pre> PROCEDURE OKToBorrow(ThisStudentID : STRING) DECLARE Index, Count : INTEGER CONSTANT Max = 5 Count ← 0 Index ← 1 / 0 WHILE Index <= 5000 / Index <= 4999 AND Count < Max IF Loan[Index].StudentID = ThisStudentID AND Loan[Index].OnLoan = TRUE THEN Count ← Count + 1 ENDIF Index ← Index + 1 ENDWHILE IF Count < Max THEN OUTPUT "Student may borrow another book" ELSE OUTPUT "Student may not borrow another book" ENDIF ENDPROCEDURE </pre> Mark as follows: 1 Procedure heading, parameter and ending 2 Loop through all elements in Loan array 3 ... terminating if Max reached 4 Test for correct StudentID field in a loop 5 ... and that OnLoan = TRUE in a loop ... if so, then increment Count in a loop 6 Output an appropriate message in both cases after the loop	7
------	---	----------

8(b)	Example solution: <pre> FUNCTION ReturnBook(ThisStudentID, ThisBookID : STRING) RETURNS BOOLEAN DECLARE Index : INTEGER Index ← 1 WHILE Index < 5001 IF Loan[Index].StudentID = ThisStudentID AND ___ Loan[Index].BookID = ThisBookID THEN IF Loan[Index].OnLoan = TRUE THEN // Not returned Loan[Index].OnLoan ← FALSE // Mark as returned now RETURN TRUE //Found and not already returned ELSE RETURN FALSE // Found and already returned ENDIF ENDIF Index ← Index + 1 ENDWHILE RETURN False // loan not found ENDFUNCTION </pre> Mark as follows: 1 Loop through all elements in Loan array 2 ...and book loan not found 3 Attempt to reference an individual data item in a loop 4 Correct test for StudentID and correct BookID in a loop 5 ... If book not returned yet, then set OnLoan field to FALSE 6 RETURN TRUE if book loan Found and not already returned 7 RETURN FALSE if book loan Found and loan already returned 8 RETURN FALSE if book loan not found Alterantive example solution – not immediate return <pre> FUNCTION ReturnBook(ThisStudentID, ThisBookID : STRING) RETURNS BOOLEAN DECLARE Index : INTEGER DECLARE Found, OK : BOOLEAN Index ← 1 Found ← FALSE OK ← TRUE </pre>	8
------	--	---

8(b)	<pre> WHILE Index < 5001 AND NOT Found IF Loan[Index].StudentID = ThisStudentID AND ___ Loan[Index].BookID = ThisBookID THEN Found ← TRUE IF Loan[Index].OnLoan = FALSE THEN // Already returned OK ← FALSE ELSE Loan[Index].OnLoan ← FALSE // Mark as returned now ENDIF ENDIF Index ← Index + 1 ENDWHILE RETURN Found AND OK ENDFUNCTION </pre>	
------	---	--

17. O/N 25/P22/Q1/b

1(b)	<table border="1"> <thead> <tr> <th>Variable</th><th>Example data value</th><th>Data type</th></tr> </thead> <tbody> <tr> <td>Result</td><td>5.42</td><td>REAL</td></tr> <tr> <td>MonthLetter</td><td>"JFMAMJJASOND"</td><td>STRING</td></tr> <tr> <td>Birthday</td><td>15/11/2009</td><td>DATE</td></tr> </tbody> </table> One mark per row	Variable	Example data value	Data type	Result	5.42	REAL	MonthLetter	"JFMAMJJASOND"	STRING	Birthday	15/11/2009	DATE	3
Variable	Example data value	Data type												
Result	5.42	REAL												
MonthLetter	"JFMAMJJASOND"	STRING												
Birthday	15/11/2009	DATE												

18. O/N 25/P22/Q3/b,c

3(b)	It does not appear in (any of) the data items // it does not appear in the ID and Name	1
3(c)	MP1 Add a (single) new separator at the end of the third (original) item MP2 Calculate the length of both new data items MP3 Store / append the length (of the new each items) MP4 as a fixed length / separated digit string // by example Max 3	3

19. O/N 25/P22/Q4/a,b

4(a)	Example solution: <pre> DECLARE CheckTotal : ARRAY [0:35] OF BOOLEAN DECLARE Index : INTEGER FOR Index ← 0 TO 35 CheckTotal[Index] ← FALSE NEXT Index </pre> Mark as follows: MP1 Declaration of CheckTotal array MP2 Declaration of a loop counter MP3 Loop – with correct syntax and number of iterations MP4 Assignment of array element to FALSE	4
4(b)	<pre> DECLARE EasyQ, HardQ : INTEGER FOR EasyQ ← 0 TO 15 STEP 3 MP1 MP2 </pre> <pre> FOR HardQ ← 0 TO 20 STEP 5 MP3 MP4 </pre> <pre> CheckTotal[HardQ + EasyQ] ← TRUE NEXT HardQ MP5 NEXT EasyQ </pre> Mark as follows: One mark per gap (boldened)	5

20. O/N 25/P22/Q8/a,b,c

8(a)	Example solution: <pre> PROCEDURE CountLoans(ThisStudentID : STRING) DECLARE Index, LCount, RCount : INTEGER LCount ← 0 RCount ← 0 FOR Index ← 1 TO 7000 IF Loan[Index].StudentID = ThisStudentID THEN IF Loan[Index].OnLoan = FALSE THEN RCount ← RCount + 1 ELSE LCount ← LCount + 1 ENDIF ENDIF NEXT Index OUTPUT "Student has ", LCount, " books on loan" OUTPUT "and has returned ", RCount, " books" ENDPROCEDURE </pre> Mark as follows: MP1 Declaration of Index and two count variables MP2 Loop for 7000 iterations MP3 Index references an indexed data item with correct .dot notation MP4 Attempt to compare the StudentID field = parameter in a loop MP5 Test of the OnLoan field and correct logic for both outcomes MP6 Both count variables initialised and increment appropriate count in a loop MP7 Output includes both counts with a <u>suitable</u> message	7
------	---	----------

8(b)	Example solution: <pre> FUNCTION NewLoan(ThisStudentID, ThisBookID : STRING) RETURNS BOOLEAN DECLARE Index : INTEGER DECLARE IsStored : BOOLEAN CONSTANT Blank = "" Index ← 1 IsStored ← FALSE WHILE Index <= 7000 AND NOT IsStored IF Loan[Index].StudentID = Blank THEN Loan[Index].StudentID ← ThisStudentID Loan[Index].BookID ← ThisBookID Loan[Index].OnLoan ← TRUE IsStored ← TRUE ENDIF Index ← Index + 1 ENDWHILE RETURN IsStored ENDFUNCTION </pre> Mark as follows: MP1 Loop through 7000 elements in <code>Loan</code> array – correct loop structure MP2 or until the first unused element is found // Correct <code>RETURN</code> inside a <code>FOR</code> loop MP3 Correct use of the indexed dot notation MP4 Test if record is unused (<code>.StudentID = ""</code>) in a loop MP5 Two item assignment statements correct MP6 <code>Loan[Index].OnLoan</code> assigned <code>TRUE</code> MP7 Correct logic to return <code>BOOLEAN</code> in both cases (position found / no position available) Alternative solution: loop only (no ‘found’ mechanism) <pre> FOR Index ← 1 TO 7000 IF Loan[Index].StudentID = Blank THEN Loan[Index].StudentID ← ThisStudentID Loan[Index].BookID ← ThisBookID Loan[Index].OnLoan ← TRUE RETURN TRUE ENDIF NEXT RETURN FALSE </pre>	7
8(c)	MP1 Save to/Open a (text) file MP2 Single line – form a string with separator characters and write to the file // Multiple lines - write each data item to a separate line in the file	2

21. O/N 25/P23/Q3

3	<pre>FUNCTION Pop() RETURNS PopData DECLARE ThisPop : PopData MP 2 IF SP < 1 // SP = 0 THEN MP 3 PopData.Exists ← FALSE // Stack is empty MP 1 ELSE PopData.Data ← ThisStack[SP] MP 4 PopData.Exists ← TRUE MP 1 SP ← SP - 1 MP 5 ENDIF RETURN ThisPop` ENDFUNCTION</pre> Mark as follows: MP 1 One mark for both assignments to PopData.Exists (FALSE and TRUE) MP 2, 3, 4 ,5 One mark for each of remaining four bold parts	5
---	--	---

22. O/N 25/P23/Q4

4	Example solution: <pre> PROCEDURE Store() DECLARE Index, StartPos, ThisNum : INTEGER DECLARE ThisString : STRING Index ← 1 StartPos ← 1 WHILE StartPos < LENGTH(NumString) AND Index < 21 ThisString ← MID(NumString, StartPos, 3) ThisNum ← STR_TO_NUM(ThisString) Data[Index] ← ThisNum Index ← Index + 1 StartPos ← StartPos + 4 ENDWHILE ENDPROCEDURE </pre> Mark as follows: 1 Declare all local variables used 2 Conditional loop while end of string not reached 3 ... and array not full 4 Attempted use of MID() // Attempted use of Right() and Left() 5 Correct extraction and use next three letter substring from NumString in a loop 6 Convert to integer and assign value to array element in a loop 7 Correct increment of array index by 1 and start position of substring by 4 Max 6 Alternative FOR loop solution: <pre> PROCEDURE Store() DECLARE Index, StartPos, ThisNum : INTEGER DECLARE ThisString : STRING StartPos ← 1 FOR Index ← 1 TO 20 ThisString ← MID(NumString, StartPos, 3) ThisNum ← STR_TO_NUM(ThisString) Data[Index] ← ThisNum StartPos ← StartPos + 4 IF StartPos > LENGTH(NumString) THEN Break ENDIF NEXT Index ENDPROCEDURE </pre>	6
---	---	---

4	Alternative where number of 3-digit substrings calculated <pre> StartPos ← 1 NumOfSubStrings ← ((LENGTH(NumString) + 1) DIV 4)) FOR Index ← 1 TO NumOfSubString ThisString ← MID(NumString, StartPos, 3) ThisNum ← STR_TO_NUM(ThisString) Data[Index] ← ThisNum StartPos ← StartPos + 4 IF Index = 20 THEN Break ENDIF NEXT Index </pre> Mark as follows: 1 Declare all variables used 2 Loop for 20 iterations // number of 3-digit numbers in the NumString 3 Attempted use of MID() // Attempted use of Right() and Left() 4 Correct extraction and use next three letter substring from NumString in a loop 5 Convert to integer and assign value to array element in a loop 6 Increment StartPos by 4 in a loop 7 Test for end of NumString and Break // Test for end of array and Break Max 6	
---	--	--

4	Alternative Solution extracting one character at a time and testing for comma <pre> PROCEDURE Store() DECLARE Index, Position : INTEGER DECLARE SubString : STRING Declare ThisCharacter : CHAR Index ← 1 Position ← 0 SubString ← "" WHILE Index <= 20 AND Position <= LENGTH(NumString) Charcter ← MID(NumString, Position, 1) IF Character = ',' Data[Index] ← STR_TO_NUM(SubString) Index ← Index + 1 SubString ← "" ELSE SubString ← SubString & Character ENDIF Position ← Position + 1 ENDWHILE Data[Index] ← STR_TO_NUM(SubString) //last 3 digit string ENDPROCEDURE </pre> Mark as follows: 1 Declare all variables used 2 Conditional loop while end of string not reached 3 ... and array not full 4 Correct extraction and use of a character from NumString in a loop 5 Test for comma in a loop 6 If true convert substring to number ... and store in Data array and set SubString to "" and increment Index 7 Otherwise concatenate next character from NumString to end of Substring Max 6	
---	--	--

23. O/N 25/P23/Q8/a,b,c

8(a)	Example solution: <pre> PROCEDURE LoanStatus(ThisStudentID, ThisBookID : STRING) DECLARE Index : INTEGER DECLARE Found : BOOLEAN Index ← 1 / 0 Found ← FALSE WHILE Index <= 8000 / 7999 AND NOT Found IF Loan[Index].StudentID = ThisStudentID AND Loan[Index].BookID = ThisBookID THEN IF Loan[Index].OnLoan = FALSE THEN OUTPUT "Loan has been returned." ELSE OUTPUT "Loan has not been returned." ENDIF Found ← TRUE ENDIF Index ← Index + 1 ENDWHILE IF NOT Found THEN OUTPUT "Warning - Loan not found." ENDIF ENDPROCEDURE </pre> Mark as follows: 1 Procedure heading, parameters and ending 2 Loop through all elements in Loan array 3 ... or until record loan is found 4 Test for correct StudentID in a loop 5 ... and correct BookID 6 ... check OnLoan data item 7 ... and output <u>both</u> messages as appropriate regarding book loan status once 8 If no matching loan found, Output 'not found' after the loop	8
------	--	----------

8(b)	Example solution: <pre> FUNCTION LoansPerTutor(TutorID : STRING) RETURNS INTEGER DECLARE Index, Count : INTEGER Count ← 0 FOR Index ← 1 / 0 TO 8000 / 7999 IF LEFT(Loan[Index].StudentID, 3) = TutorID AND Loan[Index].OnLoan = TRUE THEN Count ← Count + 1 ENDIF NEXT Index RETURN Count ENDFUNCTION </pre> Mark as follows: 1 Initialise local integer for Count 2 Loop through all elements in the Loan array 3 Attempt at referencing a data item in a loop 4 Extract TutorID from Loan[Index].StudentID in a loop 5 Test for Matching Tutor AND test if the book has been returned using correct dot notation in a loop 6 ... if true then increment Count in a loop 7 Return count	7
8(c)(i)	One mark per point: 1 Include (a suffix string based on) the date of the archive / the next number in sequence 2 Concatenate with a root filename such as 'Archive'	2
8(c)(ii)	Boolean data cannot be written to a text file // OnLoan is not a string // OnLoan will have to be converted to text	1
8(c)(iii)	One mark per point: Benefit: Algorithm to store / extract a record is easier // unpacking of data is not required // No need for string concatenation Drawback: More file accesses needed (to read / write a complete record)	2

24. M/J 25/P21/Q5

5	MP1 Pop an item off the stack MP2 Update Stack pointer MP3 Add it to the queue MP4 Update Rear pointer MP5 Repeat until the stack is empty MP6 Remove an item from the queue MP7 Update Front pointer MP8 Push it onto the stack MP9 Update the Stack pointer MP10 Repeat until the queue is empty Max 7	7
---	---	----------

25. M/J 25/P21/Q6/b

6(b)	Example solution: <pre> PROCEDURE Sort() DECLARE Temp, J, Boundary : INTEGER DECLARE NoSwaps : BOOLEAN Boundary ← 1999 REPEAT NoSwaps ← TRUE FOR J ← 1 TO Boundary IF Reading[J, 1] > Reading[J+1, 1] THEN //first swap sensor value Temp ← Reading[J, 1] Reading[J, 1] ← Reading[J+1, 1] Reading[J+1, 1] ← Temp //now swap corresponding ID Temp ← Reading[J, 2] Reading[J, 2] ← Reading[J+1, 2] Reading[J+1, 2] ← Temp NoSwaps ← FALSE ENDIF NEXT J Boundary ← Boundary - 1 UNTIL NoSwaps = TRUE ENDPROCEDURE </pre> MP1 Procedure heading and ending MP2 Conditional loop correctly formed including Boolean flag declared and initialised MP3 An inner loop MP4 Correct range 1 to 1999 for inner loop MP5 Comparison of element J with J+1 in a loop MP6 Declare <code>Temp</code> variable and swap elements in a loop MP7 Both SensorID and Speed values were swapped in a loop MP8 'No-Swap' mechanism: • Conditional outer loop including flag reset • Flag set in inner loop to indicate swap MP9 Reducing <code>Boundary</code> in the <u>outer</u> loop	8
------	--	----------

26. M/J 25/P21/Q7/b(i)

7(b)(i)	Example solution: <pre> PROCEDURE AddNewCustomers (NumToAdd : INTEGER) DECLARE CustomerID : INTEGER DECLARE Count : INTEGER DECLARE Line : STRING DECLARE NewLine : STRING OPENFILE "Loyalty.txt" FOR READ WHILE NOT EOF("Loyalty.txt") READFILE "Loyalty.txt", Line ENDWHILE CLOSEFILE "Loyalty.txt" OPENFILE "Loyalty.txt" FOR APPEND CustomerID ← STR_TO_NUM((LEFT(Line, 6)) FOR Count ← 1 TO NumToAdd CustomerID ← CustomerID + 1 OUTPUT CustomerID NewLine ← NUM_TO_STR(CustomerID) & ",0" WRITEFILE "Loyalty.txt", NewLine NEXT Count CLOSEFILE "Loyalty.txt" ENDPROCEDURE </pre> Mark as follows: MP1 All variables used are declared using the correct type including a string and an integer MP2 Open file in read mode and close (before opening in append mode) MP3 Conditional loop with <u>EOF("Loyalty.txt")</u> MP4 Read <u>Line</u> from file // Count number of records in the file MP5 Open the file in append mode and subsequently close MP6 Extract <u>CustomerID</u> from <u>Line</u> and convert to integer // use count from MP4 to generate last <u>CustomerID</u> stored MP7 A (count controlled) loop for the number of customers to add MP8 Increment <u>CustomerID</u> and output the new <u>CustomerID</u> in loop MP9 Create string for new <u>CustomerID</u> and write to file in loop Max 8	8
---------	--	----------

27. M/J 25/P22/Q1/c,d

1(c)	1 mark for each correct row		4
	Expression	Data type	
	RIGHT (MachineCode, 4)	STRING	
	Speed * 2.5	REAL	
	NOT Status	BOOLEAN	
	IS_NUM(Check)	BOOLEAN	
1(d)(i)	Two / 2		1
1(d)(ii)	1000		1
1(d)(iii)	DECLARE Product : ARRAY [0:99, 0:9] OF INTEGER 1 mark for correct upper and lower bound [0:99, 0:9] 1 mark for all other parts of declaration <u>DECLARE Product : ARRAY [0:99, 0:9] OF INTEGER</u>		2

28. M/J 25/P22/Q5/b

5(b)	Conditional Solution Example Solution <pre> DECLARE Count : INTEGER DECLARE Line : STRING DECLARE NextHour, Hour : STRING OPENFILE "TimeTaken.txt" FOR READ Count ← 1 READFILE "TimeTaken.txt", Line Hour ← LEFT(Line, 2) WHILE NOT EOF("TimeTaken.txt") READFILE "TimeTaken.txt", Line NextHour ← LEFT(Line, 2) IF NextHour = Hour THEN Count ← Count + 1 ELSE OUTPUT "Hour : ", Hour, " Total : ", Count Hour ← NextHour Count ← 1 ENDIF ENDWHILE OUTPUT "Hour : ", Hour, " Total : ", Count CLOSEFILE "TimeTaken.txt" </pre> Mark as follows: 1. Any Initialisation of count for number of pictures taken each hour 2. Open "TimeTaken.txt" for read and subsequently close 3. Conditional loop until EOF 4. Read a line from "TimeTaken.txt" in a loop 5. Extract Hour from line read in a loop 6. A mechanism to compare current hour extracted from file with last one read from file in a loop 7. If hours same increment count in a loop 8. If not same output count (with a suitable message) and update Hour ← NextHour and set Count to 1 in a loop 9. Output final Count and Hour (with a suitable message) once only Note: max 8	8
------	--	----------

5(b)	Alternative solution and mark scheme use of an array to hold count for each hour Also, for use of 24 variables and 24 selection conditions <pre> DECLARE HoursArray[0 : 23] OF INTEGER DECLARE Index, Hour : INTEGER DECLARE Line : STRING FOR Index ← 0 TO 23 HoursArray[Index] ← 0 NEXT Index OPENFILE "TimeTaken.txt" FOR READ WHILE NOT EOF("TimeTaken.txt") READFILE "TimeTaken.txt", Line Hour ← STR_TO_NUM(LEFT(Line, 2)) HoursArray[Hour] ← HoursArray[Hour] + 1 ENDWHILE FOR Index ← 0 TO 23 IF HoursArray[Index] <> 0 THEN OUTPUT "Hour : ", Index, " Total : ", HoursArray[Index] ENDIF NEXT Index CLOSEFILE "TimeTaken.txt" </pre> Mark as follows: 1. Initialisation of 24 array elements to 0 // Initialisation of 24 Integer variables to 0 2. Open "TimeTaken.txt" for read and subsequently close 3. Conditional loop until EOF 4. Read a line from "TimeTaken.txt" in a loop 5. Extract Hour from line read in a loop 6. Convert Hour to an integer value in a loop 7. Increment appropriate array element / variable in a loop 8. Output of each hour and corresponding count variable (with a suitable message) for all values where count is not zero	
------	---	--

29. M/J 25/P22/Q6/b

6(b)	Example answer: The ADT is a linked list and the procedure <code>Place()</code> inserts / add a new node / value into it / the linked list Mark as follows: 1. 1 mark for identifying the ADT as a linked list 2. 1 mark for identifying operation as inserting / adding a value / node into a linked list	2
------	---	----------

30. M/J 25/P22/Q7/a,b,c

7(a)	Example solution Conditional Loop <pre> FUNCTION FindCustomer(CustomerID : INTEGER) RETURNS INTEGER DECLARE Index : INTEGER DECLARE Found : BOOLEAN CONSTANT Unused = 99999 CONSTANT Upper = 1000 Found ← FALSE Index ← 1 IF CustomerID < 10001 OR CustomerID > 11000 THEN RETURN -1 // Out of range value for customer ID ENDIF WHILE Found = FALSE AND Loyalty[Index, 1] <> 99999 IF Loyalty[Index,1] = CustomerID THEN Found ← TRUE ELSE Index ← Index + 1 ENDIF IF Index > Upper THEN RETURN -1 ENDIF ENDWHILE IF Found THEN RETURN Loyalty[Index, 2] ELSE RETURN -1 ENDIF ENDFUNCTION </pre> Mark as follows: 1. Create function header and ending with correct parameter and return type 2. Check CustomerID is in range and if not return -1 3. (Conditional) loop iterating through each element in array 4. Check for current element in array contains required customer ID in a loop 5. ... If found set value to terminate loop 6. Terminating loop when customer ID found 7. Terminating loop when current customer ID is 99999 8. Return either loyalty points for the customer found or -1 if not found	8
------	---	----------

7(a)	Alternative solution using FOR Loop Example solution <pre> FUNCTION FindCustomer(CustomerID : INTEGER) RETURNS INTEGER DECLARE Index : INTEGER IF CustomerID < 10001 OR CustomerID > 11000 THEN RETURN -1 // Out of range value for customer ID ENDIF FOR Index ← 1 TO 1000 IF Loyalty[Index, 1] = CustomerID THEN RETURN Loyalty[Index, 2] ENDIF IF Loyalty[Index, 1] = 99999 THEN RETURN -1 ENDIF NEXT Index ENDFUNCTION </pre> Mark as follows: 1. Create function header and ending with correct parameter and return type 2. Check CustomerID is within range and if not return -1 3. FOR Loop 4. Loop for elements 1 to 1000 / 0 to 999 5. Check if current element in array contains required Customer ID in a loop 6. ... If found correctly return loyalty points // store correct loyalty points and return after loop 7. Check if current element in array is 99999 and return -1 / break loop in a loop 8. return -1 if Customer ID not found Direct access solution Alternative solution subtracting 10000 from CustomerID <pre> FUNCTION FindCustomer(CustomerID : INTEGER) RETURNS INTEGER DECLARE Index : INTEGER IF CustomerID < 10001 OR CustomerID > 11000 THEN RETURN -1 // Out of range value for customer ID ENDIF </pre>	
------	--	--

7(a)	<pre> Index = CustomerID - 10000 IF Loyalty[Index, 1] = CustomerID THEN RETURN Loyalty[Index, 2] ENDIF RETURN -1 ENDFUNCTION </pre> Mark as follows: 1. Create function header and ending with correct parameter and return type 2. Declare Index as Integer 3. Check CustomerID is less than 10001 and return -1 if it is 4. Check CustomerID is greater than 11000 and return -1 if it is 5. Calculate Index by subtracting a 10000 from CustomerID 6. Check if array contains the CustomerID 7. Return Loyalty Points if CustomerID found 8. Return -1 if not found Binary Search Mark Scheme Example Solution <pre> FUNCTION FindCustomer(CustomerID : INTEGER) RETURNS INTEGER DECLARE Start : INTEGER DECLARE End : INTEGER DECLARE Mid : INTEGER IF CustomerID < 10001 OR CustomerID > 11000 THEN RETURN -1 // Out of range value for customer ID ENDIF Start ← 1 End ← 1000 WHILE Start <= End Mid ← (Start + End) DIV 2 IF Loyalty[Mid, 1] = CustomerID THEN RETURN Loyalty[Mid, 2] ELSE IF Loyalty[Mid, 1] > CustomerID THEN End ← Mid - 1 ELSE Start ← Mid + 1 ENDIF ENDWHILE RETURN -1 ENDFUNCTION </pre>	
------	--	--

7(a)	Mark points 1. Create function header and ending with correct parameter and return type 2. Check <code>CustomerID</code> is within range and if not return <code>-1</code> 3. Conditional loop that halves array search space with each iteration 4. Check if <code>CustomerID</code> is stored in current array element in loop 5. Mechanism to end loop if <code>CustomerID</code> is found in array in a loop 6. Mechanism to end loop if <code>CustomerID</code> is not in array in a loop 7. If <code>CustomerID</code> found in array then return correct loyalty points 8. If <code>CustomerID</code> not in array then return <code>-1</code>	
7(b)	Example solution <pre> PROCEDURE PointsReport () DECLARE Count : INTEGER DECLARE Index : INTEGER DECLARE Sum : INTEGER DECLARE Average : REAL Index ← 1 Count ← 0 Sum ← 0 WHILE Loyalty[Index, 1] <> 99999 AND Index <= 1000 IF Loyalty[Index, 2] >= 11 THEN OUTPUT Loyalty[Index, 1] ENDIF Count ← Count + 1 Sum ← Sum + Loyalty[Index, 2] Index ← Index + 1 ENDWHILE Average ← Sum / Count OUTPUT "The average points of all the customers is ", Average ENDPROCEDURE </pre> Mark as follows: 1. Create procedure header and ending 2. Declare and initialise <code>Index</code>, <code>Sum</code> and <code>Count</code> 3. Loop through all elements in <code>Loyalty</code> array 4. ... or terminates when column1 of <code>Loyalty</code> array equals 99999 in a loop 5. Check if loyalty points greater than or equal to 11 in a loop 6. If true output customer ID 7. Sum points and Increment <code>Count</code> in a loop 8. Calculate and output average with an appropriate message Max 7	7
7(c)(i)	An array can only store data of the same type // An array cannot store data of different types	1

7(c)(ii)	1 mark for each point 1. a new (composite) data type / record is defined (that consists of both <code>INTEGER</code> and <code>STRING</code>) 2. an array based on this new type is declared Alternative 1 mark for each point 1. Converting loyalty points to string and concatenating / joining / append with Customer ID 2. Storing concatenated string in an array of string	2
----------	--	----------

31. M/J 25/P23/Q1/c

1(c)	<table><tr><th>Variable</th><th>Data type</th></tr><tr><td>MemberCount</td><td>INTEGER</td></tr><tr><td>TotalTakings</td><td>REAL</td></tr><tr><td>BookingConfirmed</td><td>BOOLEAN</td></tr><tr><td>MemberDOB</td><td>DATE</td></tr></table> Mark as follows: 1 mark per row	Variable	Data type	MemberCount	INTEGER	TotalTakings	REAL	BookingConfirmed	BOOLEAN	MemberDOB	DATE	4
Variable	Data type											
MemberCount	INTEGER											
TotalTakings	REAL											
BookingConfirmed	BOOLEAN											
MemberDOB	DATE											

32. M/J 25/P23/Q2/a,b,c

2(a)(i)	MP1 400 MP2 6	2
---------	--------------------------------	----------

2(a)(ii)	Stack <table><thead><tr><th>Memory location</th><th>Value</th></tr></thead><tbody><tr><td>409</td><td></td></tr><tr><td>408</td><td>'Y'</td></tr><tr><td>407</td><td>'Z'</td></tr><tr><td>406</td><td>'C'</td></tr><tr><td>405</td><td>'F'</td></tr><tr><td>404</td><td>'K'</td></tr><tr><td>403</td><td>'B'</td></tr><tr><td>402</td><td>'S'</td></tr><tr><td>401</td><td>'R'</td></tr><tr><td>400</td><td>'D'</td></tr></tbody></table> ← TopOfStack MP1 MP2 MP3 TopOfStack pointing to 'Y' and value 'Y' in location 408 Values 'C' and 'Z' in 406 and 407 Values 'D' to 'F' unchanged in 400 to 405 and 409 is empty	Memory location	Value	409		408	'Y'	407	'Z'	406	'C'	405	'F'	404	'K'	403	'B'	402	'S'	401	'R'	400	'D'	3
Memory location	Value																							
409																								
408	'Y'																							
407	'Z'																							
406	'C'																							
405	'F'																							
404	'K'																							
403	'B'																							
402	'S'																							
401	'R'																							
400	'D'																							
2(b)(i)	<table><thead><tr><th>Index</th><th>Data</th><th>Pointer</th></tr></thead><tbody><tr><td>0</td><td>"Neptune"</td><td>2</td></tr><tr><td>1</td><td>"Jupiter"</td><td>4</td></tr><tr><td>2</td><td>"Saturn"</td><td>5</td></tr><tr><td>3</td><td>"Earth"</td><td>1</td></tr><tr><td>4</td><td>"Mercury"</td><td>0</td></tr><tr><td>5</td><td>"Uranus"</td><td>-1</td></tr></tbody></table> MP1 MP2 MP3 Earth pointer 1 Uranus pointer -1 Other four correct pointers	Index	Data	Pointer	0	"Neptune"	2	1	"Jupiter"	4	2	"Saturn"	5	3	"Earth"	1	4	"Mercury"	0	5	"Uranus"	-1	3	
Index	Data	Pointer																						
0	"Neptune"	2																						
1	"Jupiter"	4																						
2	"Saturn"	5																						
3	"Earth"	1																						
4	"Mercury"	0																						
5	"Uranus"	-1																						
2(b)(ii)	3	1																						

2(c)	MP1 First value added to queue is the first value removed // FIFO // LILO MP2 Two pointers needed to indicate the start and end of the queue MP3 May behave as a circular queue MP4 Manipulated using enqueue () / dequeue () operations // by description Max 2	2
------	--	----------

33. M/J 25/P23/Q7/b(i)

7(b)(i)	Example solution: <pre> FUNCTION (MondayCheck(CustomerID : STRING) RETURNS INTEGER DECLARE LoyaltyPoints : INTEGER DECLARE LoyaltyPointsString : STRING DECLARE Line : STRING Line ← FindCustomer(CustomerID) LoyaltyPointsString ← MID(Line, 8, LENGTH(Line) - 16)) LoyaltyPoints ← STR_TO_NUM(LoyaltyPointsString) IF DAYINDEX(TODAY ()) = 2 THEN LoyaltyPoints ← LoyaltyPoints + 10 ENDIF RETURN LoyaltyPoints ENDFUNCTION </pre> MP1 Value is returned from FindCustomer (CustomerID) and stored MP2 Function MID used MP3 Attempt to extract LoyaltyPoints from the string returned by FindCustomer () MP4 Correct extraction of LoyaltyPoints from the string returned by FindCustomer () MP5 Conversion of LoyaltyPoints extracted to an integer MP6 Check - is today's date a Monday using TODAY () MP7 ... increase LoyaltyPoints by 10 MP8 Return LoyaltyPoints	8
---------	--	----------

7(b)(i)	Alternative example solution: Uses a loop to calculate the number of digits in the <code>LoyaltyPoints</code> string <pre> FUNCTION MondayCheck(CustomerID : STRING) RETURNS INTEGER DECLARE LoyaltyPoints : INTEGER DECLARE LoyaltyPointsString : STRING DECLARE Index : INTEGER DECLARE Line : STRING Line ← FindCustomer(CustomerID) Index ← 9 WHILE MID(Line, Index, 1) <> ',' Index ← Index + 1 ENDWHILE // calculate the number of digits in LoyaltyPoints string LoyaltyPointsString ← MID(Line, 8, Index - 8) LoyaltyPoints ← STR_TO_NUM(LoyaltyPointsString) IF DAYINDEX(TODAY()) = 2 THEN LoyaltyPoints ← LoyaltyPoints + 10 ENDIF RETURN LoyaltyPoints ENDFUNCTION </pre>	
---------	--	--

34. O/N 24/P21/Q1/c

1(c)	One mark per row:	2						
	<table><tr><th>Variable name</th><th>Data type</th></tr><tr><td>HighRate</td><td>Boolean</td></tr><tr><td>TaxPayable</td><td>Real</td></tr></table>	Variable name	Data type	HighRate	Boolean	TaxPayable	Real	
Variable name	Data type							
HighRate	Boolean							
TaxPayable	Real							

35. O/N 24/P21/Q6/a

6(a)	One mark per point: - Fewer lines of code are needed - The program is simpler/ less complex // Program is easier to design / code / maintain / modify / test / debug - Direct access to days in a month / data (using month number as index) // Can use index / month to (directly) access days in month / data Max 2 marks	2
------	--	----------

36. O/N 24/P21/Q8/b,c

8(b)	Example Solution: <pre> PROCEDURE CheckAll(CNum : INTEGER) DECLARE Index, Count, ThisMark: INTEGER Count ← 0 OPENFILE "GRList.txt" FOR WRITE FOR Index ← 1 to CNum ThisMark ← Result[Index, 1] // 2D array: mark + ID IF CheckMark(ThisMark) = TRUE THEN WRITE "GRList.txt", NUM_TO_STR(Result[Index, 2]) Count ← Count + 1 ENDIF NEXT Index CLOSEFILE "GRList.txt" OUTPUT "There are ", Count, " papers to check" ENDPROCEDURE </pre> Mark as follows: 1 Procedure heading, parameter and ending. 2 Open file in write mode and subsequently close 3 Loop through all CNum candidates 4 Extract candidate mark from Result array in a loop 5 Use of CheckMark() with candidate mark as parameter in a loop 6 Test return value and if TRUE then increment Count in a loop 7 ... write ID to file ... 8 ... after conversion to a string in a loop 9 Final output of message giving number of papers to check not in a loop Max 8 marks	8
8(c)	The file will need to be opened in <u>APPEND</u> mode	1

37. O/N 24/P22/Q1/d

1(d)	One mark per row: <table border="1" data-bbox="284 1541 1310 1832"> <thead> <tr> <th>Variable name</th><th>Used to store</th><th>Data type</th></tr> </thead> <tbody> <tr> <td>Name</td><td>a customer name</td><td>STRING</td></tr> <tr> <td>Index</td><td>an array index</td><td>INTEGER</td></tr> <tr> <td>Result</td><td>the result of the division of any two non-zero numbers</td><td>REAL</td></tr> </tbody> </table>	Variable name	Used to store	Data type	Name	a customer name	STRING	Index	an array index	INTEGER	Result	the result of the division of any two non-zero numbers	REAL	3
Variable name	Used to store	Data type												
Name	a customer name	STRING												
Index	an array index	INTEGER												
Result	the result of the division of any two non-zero numbers	REAL												

1(c)	Features include: MP1 Editor MP2 <u>Auto</u>- (syntax) complete / <u>auto</u> correction // identify undeclared variables(s) MP3 Prettyprint / <u>auto</u>-indentation / <u>auto</u> (structure) highlighter MP4 <u>Dynamic</u> syntax checking MP5 Expand / collapse code blocks MP6 Context sensitive prompts Max 2 marks	2
------	--	----------

38. O/N 24/P22/Q2/b(i)

2(b)(i)	DECLARE CardNumber : ARRAY [1:100, 1:2] OF STRING MP1 Correct dimensions MP2 All other parts of the statement correct	2
---------	--	----------

39. O/N 24/P22/Q3/a,b

3(a)(i)	SP: 1 OnStack: 0 One mark for both correct values	1
3(a)(ii)	MP1 Unused values cannot be popped / taken off the stack // initialised values would never be used / unused elements cannot be accessed MP2 ... until a value has first been pushed / written // overwrites previous value	2
3(b)	Example solution: <pre> FUNCTION Push(ThisValue : REAL) RETURNS BOOLEAN DECLARE ReturnValue : BOOLEAN IF OnStack = 60 / >59 // SP = 61 / SP > 60 // SP outside the range 1 to 60 THEN RETURN FALSE // Stack is already full ENDIF ThisStack[SP] ← ThisValue SP ← SP + 1 OnStack ← OnStack + 1 RETURN TRUE ENDFUNCTION1 </pre> Mark as follows: One mark per gap	4

40. O/N 24/P22/Q6/a

6(a)	Example solution: <pre> PROCEDURE Special() DECLARE Index : INTEGER DECLARE Filling1, Filling2 : STRING REPEAT Index ← INT(RAND(35)) + 1 UNTIL Filling[Index] <> "" Filling1 ← Filling[Index] REPEAT Index ← INT(RAND(35)) + 1 UNTIL Filling[Index] <> "" AND Filling1 <> Filling[Index] Filling2 ← Filling[Index] REPEAT Index ← INT(RAND(10) + 1) UNTIL Bread[Index] <> "" OUTPUT "The daily special is ", Filling1, " and ", — Filling2, " on ", Bread[Index], " bread." ENDPROCEDURE </pre> Mark as follows: MP1 Loop for Filling 1, avoiding unused elements MP2 Loop for Filling 2 avoiding unused elements MP3 Check Filling 2 is different from Filling 1 – could correctly compare either the indices or the array contents MP4 Loop for Bread, avoiding unused elements MP5 Using <code>RAND(10) / RAND(35)</code> MP6 Completely correct use of <code>RAND()</code> – including <code>INT()</code> and <code>+1</code> in all cases MP7 Correct output - once only – following a reasonable attempt at selection of fillings and bread	7
------	--	----------

41. O/N 24/P22/Q8/a,b,c,d

8(a)	Example solution: <pre> PROCEDURE Assign(ThisRole : STRING, ThisPlayer : INTEGER) DECLARE Index : INTEGER DECLARE Done : BOOLEAN Done ← FALSE Index ← 1 WHILE Index < 46 AND Done = FALSE IF Character[Index].Player = 0 AND __ Character[Index].Role = ThisRole THEN Character[Index].Player ← ThisPlayer Done ← TRUE ELSE Index ← Index + 1 ENDIF ENDWHILE IF Done = TRUE THEN OUTPUT Character[Index].Name, " the ", __ Character[Index].Role, __ " has been assigned to player ", ThisPlayer ELSE OUTPUT "No characters with this role are available" ENDIF ENDPROCEDURE </pre> Mark as follows: MP1 Loop until 'found' or all 45 elements considered MP2 Test of <code>Player</code> field – i.e. not value in a loop MP3 ... AND <code>Role</code> – i.e. match for ThisRole parameter in a loop MP4 If available character found, assign <code>ThisPlayer</code> to the character in a loop MP5 When character found set termination condition/flag MP6 Both OUTPUT messages logically <u>correctly placed</u> MP7 Both OUTPUT statements <u>correctly formed</u>	7
------	---	----------

8(b)	Example solution: <pre> PROCEDURE Save() DECLARE Index : INTEGER DECLARE Line : STRING CONSTANT SEP = '^' OPENFILE "SaveFile.txt" FOR WRITE FOR Index ← 1 TO 45 Line ← NUM_TO_STR(Character[Index].Player) & SEP Line ← Line & Character[Index].Role & SEP Line ← Line & Character[Index].Name & SEP Line ← Line & NUM_TO_STR(Character[Index].Level) WRITEFILE "SaveFile.txt", Line NEXT Index CLOSEFILE "SaveFile.txt" ENDPROCEDURE </pre> Mark as follows: MP1 Declaration of local integer for <code>Index</code> (and string type for <code>Line</code>) MP2 Open "SaveFile.txt" in write mode and subsequently close MP3 Loop through 45 elements MP4 Attempt to form <code>Line</code> - four fields in a loop MP5 Correct use of <code>NUM_TO_STR()</code>x2 (<code>Player</code> and <code>Level</code>) in a loop MP6 Correct use of <u>three</u> <code>&<separator>&</code> strings in a loop MP7 Line from MP4 written to file in a loop	7
8(c)	MP1 Encode <code>Status</code> as a character / string MP2 Append the '^' separator and the character/string	2
8(d)	MP1 Method: Create a filename suffix which is incremented for each file save MP2 Example: SaveFile01.txt, SaveFile02.txt	2

42. O/N 24/P23/Q1/b

1(b)	<table border="1"> <thead> <tr> <th>Variable</th><th>Example data value</th><th>Data type</th></tr> </thead> <tbody> <tr> <td>Available</td><td>TRUE</td><td>BOOLEAN</td></tr> <tr> <td>Received</td><td>"18/04/2021"</td><td>STRING</td></tr> <tr> <td>Index</td><td>100</td><td>INTEGER</td></tr> </tbody> </table> One mark per row	Variable	Example data value	Data type	Available	TRUE	BOOLEAN	Received	"18/04/2021"	STRING	Index	100	INTEGER	3
Variable	Example data value	Data type												
Available	TRUE	BOOLEAN												
Received	"18/04/2021"	STRING												
Index	100	INTEGER												

43. O/N 24/P23/Q3/a

3(a)(i)	0 is not a valid <u>array / List index</u> value // No 0 th element in the array	1
3(a)(ii)	0 to 200	1

44. O/N 24/P23/Q4/a,b

4(a)	One mark per highlighted part: <pre> FOR Index ← 1 TO <u>5</u> Lower ← GB[Index] - 2 Upper ← <u>GB[Index] + 2</u> // <u>Lower + 4</u> IF Mark <u>>= Lower</u> AND Mark <u><= Upper</u> THEN //IF Mark <u><= Upper</u> AND Mark <u>>= Lower</u> THEN OUTPUT "Check this paper" ENDIF NEXT Index </pre>	4
4(b)(i)	MP1 Use Mark as the <u>index</u> to the Check array // to specify an array element MP2 If value of indexed element is TRUE, then the paper will need to be checked	2

4(b)(ii)	Example:solution <pre> PROCEDURE GBInitialise() DECLARE GBIndex, GBMark, ThisIndex : INTEGER FOR ThisIndex ← 0 TO 75 Check[ThisIndex] ← FALSE // initialise all values NEXT ThisIndex FOR GBIndex ← 1 TO 5 GBMark ← GB[GBIndex] FOR ThisIndex ← GBMark - 2 TO GBMark + 2 Check[ThisIndex] ← TRUE //Set GBMark ± 2 to TRUE NEXT ThisIndex NEXT GBIndex ENDPROCEDURE </pre> Mark as follows: MP1 Procedure heading and ending MP2 Initialise Check array to FALSE MP3 Loop through GB array MP4 Extract the GB mark MP5 Loop for 5 elements across GBMark ± 2 // Check if within GBMark ± 2 MP6 Assign each element of Check array to TRUE ALTERNATIVE – Example using selection Version 1 <pre> FOR ThisIndex ← 0 TO 75 CASE ThisIndex GB[1] - 2 TO GB[1] + 2 : Check[ThisIndex] ← TRUE GB[2] - 2 TO GB[2] + 2 : Check[ThisIndex] ← TRUE GB[3] - 2 TO GB[3] + 2 : Check[ThisIndex] ← TRUE GB[4] - 2 TO GB[4] + 2 : Check[ThisIndex] ← TRUE GB[5] - 2 TO GB[5] + 2 : Check[ThisIndex] ← TRUE OTHERWISE: Check[ThisIndex] ← FALSE ENDCASE NEXT ThisIndex </pre>	6
----------	---	----------

4(b)(ii)	ALTERNATIVE – Example using selection Version 2 <pre> DECLARE ThisIndex, GBIndex : INTEGER DECLARE Lower, Higher : INTEGER FOR ThisIndex ← 0 TO 75 For GBIndex ← 1 TO 5 Lower ← GB[GBIndex] - 2 Higher ← GB[GBIndex] + 2 IF ThisIndex >= Lower AND ThisIndex <= Higher THEN Check[ThisIndex] ← TRUE ELSE Check[ThisIndex] ← FALSE ENDIF NEXT Index NEXT ThisIndex </pre> Mark as follows: MP1 Procedure heading and ending MP2 Loop through <i>Check</i> array// loop from 0 to 75 MP3 Attempt at using CASE / Selection structure with five clauses/ five checks for range MP4 Extract GB grade MP5 Compare <i>ThisIndex</i> with each element in GB array ± 2 MP6 Assign each element of <i>Check</i> array to TRUE if in range or FALSE if not in range	
----------	---	--

45. O/N 24/P23/Q8/a,b,c

8(a)	Example solution <pre> PROCEDURE Count(ThisPlayer : INTEGER, ThisRole : STRING) DECLARE Index, Num, Total : INTEGER Num ← 0 Total ← 0 FOR Index ← 1 TO 45 IF Character[Index].Player = ThisPlayer AND ____ Character[Index].Role = ThisRole THEN Num ← Num + 1 Total ← Total + Character[Index].SkillLevel NEXT Index IF Num > 0 THEN OUTPUT "Player ", ThisPlayer, ____ " has ", Num, " characters with the role of ", ThisRole, " and the total skill level is ", Total ELSE OUTPUT "No characters with that role are assigned to this player" ENDIF ENDPROCEDURE </pre> MP1 Initialisation of local integers for Num and Total MP2 Loop through 45 elements MP3 Attempt to check Player and Role fields in a loop MP4 Correctly compare Player field with parameter in a loop MP5 Correctly compare Role field with parameter in a loop MP6 ... if player and role found, increment Num and sum Skill Total in a loop MP7 Test for any matches after the loop MP8 Both possible OUTPUT statements correctly formed following an attempt at MP6 but outputting one only Max 7 marks	7
------	--	----------

8(b)	Example solution: <pre> PROCEDURE Restore() DECLARE Index : INTEGER DECLARE Line : STRING OPENFILE "SaveFile.txt" FOR READ FOR Index ← 1 TO 45 READFILE "SaveFile.txt", Line Character[Index].Player ← STR_TO_NUM(Extract(Line, 1)) Character[Index].Role ← Extract(Line, 2) Character[Index].Name ← Extract(Line, 3) Character[Index].Skill ← STR_TO_NUM(Extract(Line, 4)) NEXT Index CLOSEFILE "SaveFile.txt" ENDPROCEDURE </pre> Mark as follows: MP1 Open the file in read mode and subsequently close MP2 Loop through 45 elements MP3 Read a line from the file in a loop MP4 Attempt to use <code>Extract()</code> in a loop MP5 Correct use of <code>Extract()</code> for all fields in a loop MP6 Use of <code>STR_TO_NUM()</code> on Player and Skill in a loop MP7 Completely correct extraction and assignment of all fields in a loop	7
8(c)	MP1 The <u>Character</u> array / <u>character data</u> can be saved before the program is closed MP2 Allowing the game to continue using the same data / from the point it was saved	2

46. M/J 24/P21/Q1/a

1(a)	<table><tr><th>Assignment statement</th><th>Data type</th></tr><tr><td>A ← LEFT(MyName, 1)</td><td>CHAR / STRING</td></tr><tr><td>B ← Total * 2</td><td>INTEGER / REAL</td></tr><tr><td>C ← INT(ItemCost) / 3</td><td>REAL</td></tr><tr><td>D ← "Odd OR Even"</td><td>STRING</td></tr></table>		Assignment statement	Data type	A ← LEFT(MyName, 1)	CHAR / STRING	B ← Total * 2	INTEGER / REAL	C ← INT(ItemCost) / 3	REAL	D ← "Odd OR Even"	STRING	4
	Assignment statement	Data type											
	A ← LEFT(MyName, 1)	CHAR / STRING											
	B ← Total * 2	INTEGER / REAL											
	C ← INT(ItemCost) / 3	REAL											
	D ← "Odd OR Even"	STRING											
One mark per row													

47. M/J 24/P21/Q3/a,b

3(a)	One mark per point: 1 The PS contains a null pointer 2 The PF points to the first element on the free list 3 All the nodes are on the free list	3							
3(b)	Max 2 marks for 'Variables': <table><tr><td>The two variables will be of type Integer</td></tr><tr><td>The two variables will be used as pointers / indexes to the arrays.</td></tr><tr><td>The values stored in the two variables will indicate the first element in each list</td></tr><tr><td>The first 1D array will be of type String</td></tr><tr><td>The first 1D array will be used to store the values // data items // User IDs</td></tr><tr><td>The second 1D array will be of type Integer</td></tr><tr><td>The second 1D array will be used to store the pointers // point to next item</td></tr></table> Mark as follows: One mark for each of the first three rows One mark for both Array 1 rows One mark for both Array 2 rows	The two variables will be of type Integer	The two variables will be used as pointers / indexes to the arrays.	The values stored in the two variables will indicate the first element in each list	The first 1D array will be of type String	The first 1D array will be used to store the values // data items // User IDs	The second 1D array will be of type Integer	The second 1D array will be used to store the pointers // point to next item	5
The two variables will be of type Integer									
The two variables will be used as pointers / indexes to the arrays.									
The values stored in the two variables will indicate the first element in each list									
The first 1D array will be of type String									
The first 1D array will be used to store the values // data items // User IDs									
The second 1D array will be of type Integer									
The second 1D array will be used to store the pointers // point to next item									

48. M/J 24/P21/Q4

4	Example: <pre> FUNCTION Check() RETURNS STRING DECLARE Odd, Even, Index : INTEGER Odd ← 0 Even ← 0 FOR Index ← 1 TO 100 IF Index MOD 2 = 0 THEN Even ← Even + Data[Index] ELSE Odd ← Odd + Data[Index] ENDIF NEXT Index ENDFUNCTION </pre> Mark as follows: 1. Function heading, ending and return type 2. Declare local variables <code>Odd</code>, <code>Even</code> and <code>Index</code> as integers 3. Initialise <code>Odd</code> and <code>Even</code> 4. Loop for 100 iterations // through array 5. Sum <code>Odd</code> and <code>Even</code> element values in a loop 6. Compare <code>Odd</code> and <code>Even</code> after the loop and Return appropriate string	6
---	---	---

49. M/J 24/P21/Q8/b

8(b)	Example: <pre> FUNCTION Stage_1(StudentName : STRING) RETURNS INTEGER DECLARE OldFile, NewFile, Line : STRING DECLARE Count : INTEGER OldFile ← StudentName & "_src.txt" NewFile ← StudentName & "_S1.txt" OPENFILE OldFile FOR READ OPENFILE NewFile FOR WRITE Count ← 0 WHILE NOT EOF(OldFile) READFILE OldFile, Line Line ← DeleteComment(Line) IF LENGTH(Line) <> 0 THEN WRITEFILE NewFile, Line Count ← Count + 1 ENDIF ENDWHILE CLOSEFILE OldFile CLOSEFILE NewFile RETURN Count ENDFUNCTION </pre> Mark as follows: 1 Generate filenames condone missing “_” 2 Open both files in correct modes and subsequently close 3 Loop to EOF(OldFile) 4 Read a line from OldFile and execute DeleteComment() in a loop 5 Skip blank lines after DeleteComment() in a loop 6 Write Line to stage 1 file and increment Count 7 Return the number of lines after a reasonable attempt at counting	7
------	---	----------

50. M/J 24/P22/Q3/a,b

3(a)(i)	Pseudocode: <pre> TYPE Component DECLARE Item_Num : INTEGER DECLARE Reject : BOOLEAN DECLARE Stage : CHAR DECLARE Limit_1 : REAL DECLARE Limit_2 : REAL ENDTYPE </pre> Mark as follows: 1 One mark for TYPE and ENDTYPE statements 2 One mark for Item_Num and Reject fields 3 One mark for Stage field 4 One mark for Limit fields as REAL	4
3(a)(ii)	<u>DECLARE Item : ARRAY [1:2000] OF Component //</u> <u>DECLARE Item : ARRAY [2000] OF Component //</u> <u>DECLARE Item : ARRAY [0:1999] OF Component</u> One mark per underlined phrase	2
3(b)	One mark per point: 1 Allows for iteration / can use a loop to access the records / data items 2 Use of index to directly access a record in the array // Example of simplification of code e.g. use of dot notation <code>Item[1].Stage</code> 3 Simplifies the code / algorithm // Reduces duplication of code // Program easier to write / understand / maintain / test / debug // Data items/record easier to search / sort / manipulate	3

51. M/J 24/P22/Q6/b

6(b)(i)	A (very) large file is created // redundant zeroes are stored in the file	1
6(b)(ii)	One mark for: • Values are delimited by a special character / a separator character • First character indicates sample length Max 1 mark	1
6(b)(iii)	The algorithm to store / extract / separate the individual values is more complex / takes longer to execute / run / process NE Algorithm is more complicated	1

52. M/J 24/P22/Q8/c

8(c)	Example: <pre> PROCEDURE Stage_2(F1, F2 : STRING) DECLARE Line : STRING DECLARE Count : INTEGER Count ← 0 OPEN F1 FOR READ OPEN F2 FOR APPEND WHILE NOT EOF(F1) READFILE F1, Line Line ← DeleteSpaces(Line) Line ← DeleteComment(Line) IF Line <> "" THEN WRITEFILE F2, Line // skip blank lines ELSE Count ← Count + 1 ENDIF ENDWHILE CLOSEFILE F1 CLOSEFILE F2 OUTPUT Count, " blank lines were removed" ENDPROCEDURE </pre> Mark as follows: 1 Procedure heading, parameters, ending 2 Open both files in correct modes and subsequently close 3 Loop to EOF(F1) 4 Read a line from F1 in a loop 5 Assign return values from DeleteComment() and DeleteSpaces() in a loop 6 Check return value following both MP5 function calls is not an empty string and if so write to F2 in a loop 7 Count the blank lines in a loop 8 Output number of blank lines removed following a reasonable attempt after the loop	8
------	---	---

53. M/J 24/P23/Q3/a

3(a)(i)	Example solution using AND <pre>IF Batch[ThisIndex].Weight >= Min AND Batch[ThisIndex].Weight <= Max THEN</pre> Alternative solution using OR <pre>IF Batch[ThisIndex].Weight < Min OR Batch[ThisIndex].Weight > Max THEN</pre> Mark as follows: 1. Reference to <code>Batch[ThisIndex].Weight</code> 2. A valid check for one boundary 3. A valid check for other boundary with correct logic operator	3
3(a)(ii)	One mark for either: • Set the <code>Item_ID</code> field to an empty string / NULL / invalid value • Set <code>Weight</code> to <code><= 0</code> / zero	1

54. M/J 24/P23/Q4/b

4(b)(i)	(1D) <u>array</u> of <u>20 reals</u> Marks as follows: 1 mark for array 1 mark for 20 reals	2
4(b)(ii)	One mark per point: 1 (Multiple instances referenced via a single identifier so) fewer identifiers needed 2 Easier to process / search / organise / access the data // Values may be accessed via a loop-controlled variable / an index // An array / data can be iterated through 3 Makes the program/algorithm easier to write / design / understand / maintain / modify // Simplifies the program // Easier to debug / test	3

55. M/J 24/P23/Q8/b

8(b)	Example: <pre> PROCEDURE FindModules(FileName : STRING) DECLARE Line : STRING DECLARE Index, LineNum : INTEGER OPENFILE FileName FOR READ Index ← 1 LineNum ← 0 WHILE NOT EOF(FileName) READFILE FileName, Line LineNum ← LineNum + 1 Line ← Header(Line) IF Line <> "" THEN ModInfo[Index, 1] ← NUM_TO_STR(LineNum) ModInfo[Index, 2] ← LEFT(Line, 1) ModInfo[Index, 3] ← RIGHT(Line, LENGTH(Line) - 1) Index ← Index + 1 ENDIF ENDWHILE CLOSEFILE FileName ENDPROCEDURE </pre> Mark as follows: 1 Open file in READ mode and subsequently close 2 Loop to EOF(FileName) 3 Read a line from the file and maintain LineNum in a loop 4 Call Header() and use return value in a loop 5 Test return value for "" in a loop 6 Attempt at all three-array assignment for all columns in correct row 7 Correct values assigned to all columns of array 8 Maintain correct array row index	8
------	---	----------

56. O/N 23/P21/Q1/d

1(d)	One mark per point: • ThisValue: INTEGER • Check: REAL • Level: STRING	3
------	---	----------

57. O/N 23/P21/Q3/a,b

3(a)	<table><tr><th>Array element</th><th>Data</th></tr><tr><td>8</td><td></td></tr><tr><td>7</td><td></td></tr><tr><td>6</td><td></td></tr><tr><td>5</td><td>D1</td></tr><tr><td>4</td><td>D3</td></tr><tr><td>3</td><td>D4</td></tr><tr><td>2</td><td>D5</td></tr><tr><td>1</td><td>D2</td></tr></table> Variables <table><tr><td>TopOfStack</td><td>5</td></tr><tr><td>BottomOfStack</td><td>1</td></tr></table> MP1 all values in the order and location shown MP2 TopOfStack value is index of element containing D1 MP3 BottomOfStack value is index of element containing D2	Array element	Data	8		7		6		5	D1	4	D3	3	D4	2	D5	1	D2	TopOfStack	5	BottomOfStack	1	3
Array element	Data																							
8																								
7																								
6																								
5	D1																							
4	D3																							
3	D4																							
2	D5																							
1	D2																							
TopOfStack	5																							
BottomOfStack	1																							
3(b)	MP1 If TopOfStack = 8 // (stack) full then return FALSE MP2 Otherwise, increment TopOfStack MP3 Use TopOfStack as an index to the Array MP4 Set the element at this index / location / position to the value / data / item being added MP5 Return TRUE	5																						

58. O/N 23/P21/Q4/a,b

4(a)	<pre> FUNCTION TooMany(Search : STRING, Max : INTEGER) RETURNS BOOLEAN DECLARE Count, Index : INTEGER Count ← 0 FOR Index ← 1 TO 150 IF Data[Index] = Search THEN Count ← Count + 1 ENDIF NEXT Index IF Count > Max THEN RETURN TRUE ELSE RETURN FALSE ENDIF ENDFUNCTION </pre> MP1 Function heading, ending and return type MP2 Declare Count and Index as integers MP3 Initialise Count MP4 Loop (any type) for 150 iterations MP5 Compare Data element with parameter - if equal, increment Count in a loop MP6 Compare Count with Max and return Boolean in both cases outside the loop	6
4(b)	MP1 Test for row being even number MP2 Test for either column value equal to Search <pre> IF <u>Row MOD 2 = 0</u> AND ____ (<u>Data[Row, 1] = Search OR Data[Row, 2] = Search</u>) THEN </pre> ALTERNATIVE using nested IFs: <pre> IF <u>Row MOD 2 = 0</u> THEN IF <u>Data[Row, 1] = Search OR Data[Row, 2] = Search</u> THEN </pre> MP3 Selection structure is either: • Single IF statement using AND, or • Two nested IFs using AND, or • Single IF and the use of a two-iteration loop Either of these structures correctly formed scores the mark ALTERNATIVE SOLUTION: A FOR loop using 'STEP 2' <pre> FOR Row ← 2 TO 150 / NEXT Row <u>STEP 2</u> (<u>Data[Row, 1] = Search OR Data[Row, 2] = Search</u>) THEN </pre>	3

59. O/N 23/P21/Q8/a,b

8(a)	<pre> PROCEDURE SendFile(FileName, DestID : STRING, Port : INTEGER) DECLARE FileData : STRING CONSTANT STX = CHR(02) CONSTANT ETX = CHR(03) OPENFILE FileName FOR READ WHILE NOT EOF(FileName) READFILE FileName, FileData FileData ← STX & DestID & MyID & FileData & ETX CALL Transmit(FileData, Port) ENDWHILE CLOSEFILE FileName CALL Transmit(STX & DestID & MyID & "*****" & ETX, Port) ENDPROCEDURE </pre> Mark as follows: MP1 OPEN file in READ mode – using parameter - and subsequently CLOSE MP2 Conditional loop to EOF() MP3 Use of READFILE to get a line from the file MP4 'Attempt' to form a message (minimum is DestID, MyID, FileData) MP5 Message formed is completely correct MP6 Call Transmit() with correct MP4 string in a loop MP7 Transmit the "*****" message (all parts present) after the loop	7
8(b)	Max 2 marks MP1 Indicates that all the lines of the file have been sent // it is the end of the transmission / file transfer MP2 So that the receiving program can stop waiting for further data MP3 The file can be closed / saved	2

60. O/N 23/P22/Q1/c

1(c)	One mark per point Max 3 marks Declaration 1 Declare a composite / record (type) 2 Declare an array of the given composite / record (type) Expansion of record: 3 ... containing all data items required // containing items of different data types Expansion of array: 4 ... where each array element represents data for one order / customer (order)	3
------	---	----------

1(a)

4

One mark for each row with appropriate variable name and data type

Example value	Explanation	Variable name	Data type
"Mr Khan"	The name of the customer	CustomerName	STRING
3	The number of items in the order	NumItems	INTEGER
TRUE	A value to indicate whether this is a new customer	NewCustomer	BOOLEAN
15.75	The deposit paid by the customer	Deposit	REAL

61. O/N 23/P22/Q3/a,b

3(a)

5

Variable	Value
Start_Pointer	4
Free_List_Pointer	5

Index	Data Array	Pointer Array
1	D32	2
2	D11	3
3	D100	0
4	D40	1
5	F1	6
6	F2	7
7	F3	8
8	F4	0

Mark as follows:

- One mark for Start_Pointer value
- One mark each group of row(s):
- 2
- 3 and 4
- 5
- 7 and 8

For null pointer: accept 0 / ∅ / an out-of-bound index value less than 1, greater than 8

3(b)	One mark per step: 1 Assign the data item D6 to F1 2 Set the pointer of this node to point to D11 3 Set Ptr2 to point to F2 4 Set pointer of D32 to point to D6	4
------	--	----------

62. O/N 23/P22/Q6/a,b(iii)

6(a)	Example Solution <pre> PROCEDURE CreateFiles(NameRoot : STRING, NumFiles : INTEGER) DECLARE FileName, Suffix : STRING DECLARE Count : INTEGER FOR Count ← 1 TO NumFiles Suffix ← NUM_TO_STR(Count) WHILE LENGTH(Suffix) <> 3 Suffix ← '0' & Suffix ENDWHILE FileName ← NameRoot & '.' & Suffix OPENFILE FileName FOR WRITE WRITEFILE FileName, "This is File " & FileName CLOSEFILE FileName NEXT Count ENDPROCEDURE </pre> Mark as follows: 1 Procedure heading, including parameters, and ending 2 Loop for NumFiles iterations 3 Attempt to create filename suffix using NUM_TO_STR() in a loop 4 Completely correct filename 5 OPENFILE in WRITE mode and subsequent CLOSE in a loop 6 WRITE initial first line to the file in a loop	6
6(b)(iii)	Read	1

63. O/N 23/P22/Q8/a,c

8(a)	Solution using a loop Example solution – using a loop <pre> FUNCTION GetPort(ThisDest : STRING) RETURNS INTEGER DECLARE Index, DNum, Port : INTEGER DNum ← STR_TO_NUM(ThisDest) Index ← 1 Port ← -1 REPEAT IF RouteTable[Index, 1] <> -1 THEN IF DNum >= RouteTable[Index, 1] AND ____ DNum <= RouteTable[Index, 2] THEN Port ← RouteTable[Index, 3] ENDIF ENDIF Index ← Index + 1 UNTIL Index = 7 OR Port <> -1 // end of array or range found RETURN Port ENDFUNCTION </pre> Mark as follows Max 7: 1. Function heading and ending including parameter and return type 2 Convert parameter to a number 3 (Conditional) loop through array 4 Skip unused element in a loop 5 Attempt to check one range with destination in a loop 6 Test all ranges correctly with destination in a loop 7 Store port value if destination matched in a loop (and exit loop) 8 Return port value including -1 if no match found Solution using selection statement(s) Example solution <pre> FUNCTION GetPort(ThisDest : STRING) RETURNS INTEGER DECLARE Index, DNum, Port : INTEGER DNum ← STR_TO_NUM(ThisDest) Port ← -1 IF RouteTable[1, 1] <> -1 AND RouteTable[1, 1] >= DNum AND RouteTable[1, 2] <= DNum THEN </pre>	7
------	--	----------

8(a)	<pre> Port ← RouteTable[1, 3] END IF IF RouteTable[2, 1] <> -1 AND RouteTable[2, 1] >= DNum AND __ RouteTable[2, 2] <= DNum THEN Port ← RouteTable[2, 3] END IF IF RouteTable[3, 1] <> -1 AND RouteTable[3, 1] >= DNum AND __ RouteTable[3, 2] <= DNum THEN Port ← RouteTable[3, 3] END IF IF RouteTable[4, 1] <> -1 AND RouteTable[4, 1] >= DNum AND __ RouteTable[4, 2] <= DNum THEN Port ← RouteTable[4, 3] END IF IF RouteTable[5, 1] <> -1 AND RouteTable[5, 1] >= DNum AND __ RouteTable[5, 2] <= DNum THEN Port ← RouteTable[5, 3] END IF IF RouteTable[6, 1] <> -1 AND RouteTable[6, 1] >= DNum AND __ RouteTable[6, 2] <= DNum THEN Port ← RouteTable[6, 3] END IF RETURN Port ENDFUNCTION </pre> Mark as follows Max 7: 1 Function heading and ending including parameter and return type 2 Convert parameter to a number 3 Skip all unused elements 4 Attempt to check one range 5 Check two ranges with destination correctly 6 Check all ranges with destination correctly 7 Store port value if destination matched 8 Return port value including -1 if no match found	
------	--	--

8(c)(i)	One mark per point Max 3 marks: Decide on scenario and mark accordingly. Scenario one: • If more than one line is / all lines are stored on the stack (before line(s) are removed) • The stack operates as a FILO device // Last item added to stack will be in first item out • So lines in the file appear out of sequence Scenario two: • Stack is Full • Not all lines can be stored on the stack • so resulting file will not contain all the original lines Scenario three: • (All) the data in a line read can't be stored on the stack • Stack elements have not been allocated enough memory • so only part of each line is stored in the file Scenario four: • Stack is empty • The stack is being read faster than it is being written to • so blank lines may be inserted into the file	3
8(c)(ii)	Queue	1

8(b)	Example solution <pre> PROCEDURE ProcessMsg(ThisMsg : STRING) DECLARE ThisDest : STRING DECLARE Response : BOOLEAN DECLARE StackNum : INTEGER IF LENGTH(ThisMsg) >= 4 THEN ThisDest ← LEFT(ThisMsg, 3) IF ThisDest = MyID THEN // It's for this computer StackNum ← 1 ELSE StackNum ← 2 ENDIF Response ← StackMsg(ThisMsg, StackNum) IF Response = FALSE THEN OUTPUT "Message discarded - no room on stack" ENDIF ENDIF ENDPROCEDURE </pre> Mark as follows: 1. Ignore message if data field is empty 2. Extract <code>ThisDest</code> from <code>ThisMsg</code> 3. Test if destination is this computer 4. Attempt to use <code>StackMsg()</code> 5. Fully correct use of <code>StackMsg()</code> for both cases / stacks 6. Test <code>StackMsg()</code> return value for both cases 7. Following a reasonable attempt at MP6 output warning if <code>StackMsg()</code> returns <code>FALSE</code>	7
------	---	---

64. O/N 23/P23/Q1/a

1(a)		4										
	<table><tr><th>Assignment statement</th><th>Data type</th></tr><tr><td><u>MyVar1</u> ← Total1 / Total2</td><td>REAL</td></tr><tr><td><u>MyVar2</u> ← 27/10/2023</td><td>DATE</td></tr><tr><td><u>MyVar3</u> ← "Sum1 / Sum2"</td><td>STRING</td></tr><tr><td><u>MyVar4</u> ← Result1 AND Result2</td><td>BOOLEAN</td></tr></table>	Assignment statement	Data type	<u>MyVar1</u> ← Total1 / Total2	REAL	<u>MyVar2</u> ← 27/10/2023	DATE	<u>MyVar3</u> ← "Sum1 / Sum2"	STRING	<u>MyVar4</u> ← Result1 AND Result2	BOOLEAN	
Assignment statement	Data type											
<u>MyVar1</u> ← Total1 / Total2	REAL											
<u>MyVar2</u> ← 27/10/2023	DATE											
<u>MyVar3</u> ← "Sum1 / Sum2"	STRING											
<u>MyVar4</u> ← Result1 AND Result2	BOOLEAN											

65. O/N 23/P23/Q2/b,c

2(b)	MP1 Simplifies the algorithm // easier to write / understand / test / debug MP2 It is possible to iterate through the values // can use a loop // allows the storage of many values using a single identifier	2
------	---	---

2(c)	One mark per underlined section <u>DECLARE</u> <u>Data</u> : <u>ARRAY[1:120]</u> OF <u>REAL</u>	2
2(a)	<pre>graph TD; START([START]) --> Init[Set Min to Data[1] Set MinIndex to 1]; Init --> SetIndex[Set Index to 2]; SetIndex --> LoopStart(()); LoopStart --> IsIndex{Is Index > 30?}; IsIndex -- YES --> Output[/OUTPUT MinIndex/]; Output --> End([END]); IsIndex -- NO --> IsData{Is Data[Index] < Min?}; IsData -- YES --> Update[Set MinIndex to Index Set Min to Data[Index]]; IsData -- NO --> Increment[Set Index to Index + 1]; Update --> Join(()); Increment --> Join; Join --> LoopStart;</pre> MP1 Initialise <code>Min</code> to first value in <code>Data</code> and <code>MinIndex</code> to 1 MP2 Loop through 29 more values (or 30 values in total) MP3 Compare element from <code>Data[]</code> with <code>Min</code> MP4 Set new <code>Min</code> AND save <code>MinIndex</code> when element value < <code>Min</code> in a loop MP5 Output <code>MinIndex</code>	5

66. O/N 23/P23/Q3/a,b

3(a)	Index Array <table><tr><td>1</td><td></td></tr><tr><td>2</td><td>D3</td></tr><tr><td>3</td><td>D4</td></tr><tr><td>4</td><td>D1</td></tr><tr><td>5</td><td>D2</td></tr><tr><td>6</td><td>D5</td></tr><tr><td>7</td><td></td></tr><tr><td>8</td><td></td></tr></table> Variable <table><tr><td>FrontOfQueue</td><td>2</td></tr><tr><td>EndOfQueue</td><td>6</td></tr><tr><td>NumItems</td><td>5</td></tr></table> : MP1 D3, D4, D1, D2 and D5 in question order or reversed - in any five consecutive locations MP2 FoQ value matches index with D3 MP3 EoQ value matches index with D5	1		2	D3	3	D4	4	D1	5	D2	6	D5	7		8		FrontOfQueue	2	EndOfQueue	6	NumItems	5	3
1																								
2	D3																							
3	D4																							
4	D1																							
5	D2																							
6	D5																							
7																								
8																								
FrontOfQueue	2																							
EndOfQueue	6																							
NumItems	5																							
3(b)	MP1 If NumItems <u>is / = 8 // (queue) full</u> then jump to step 6 MP2 Increment EndOfQueue MP3 If EndOfQueue = 9 then set EndOfQueue to 1 MP4 Increment NumItems MP5 Set the Element at the index ... MP6 stored in EndOfQueue to value/data/item being added Mark as follows: Steps 1 to 4: One mark for gaps filled as shown Step 5: One mark for 'element' and one mark for other two terms	6																						

67. O/N 23/P23/Q7/b(i)

7(b)(i)	<pre> TYPE MyType DECLARE RA : INTEGER DECLARE RB : STRING DECLARE RC : BOOLEAN ENDTYPE </pre> MP1 TYPE MyType . . . ENDTYPE MP2 RA as integer and RB as string MP3 RC as Boolean	3
---------	--	----------

68. O/N 23/P23/Q8/a

8(a)	<pre> PROCEDURE ReceiveFile(FileName : STRING) DECLARE FileData : STRING DECLARE CharCount : INTEGER CONSTANT Terminator = "*****" OPENFILE FileName FOR WRITE FileData ← GetData() CharCount ← 0 WHILE FileData <> Terminator WRITEFILE FileName, FileData CharCount ← CharCount + LENGTH(FileData) FileData ← GetData() ENDWHILE CLOSEFILE FileName OUTPUT CharCount (, " characters were written to ", FileName) ENDPROCEDURE </pre> MP1 OPEN file in WRITE mode and subsequently CLOSE MP2 Conditional loop until terminator received MP3 'Attempted' use of GetData() – Ignore CALL ... MP4 Fully correct use GetData() to return the data in a loop MP5 Maintain CharCount in a loop MP6 Write each line to the file - except the terminator in a loop MP7 Final output of message giving number of characters written outside loop	7
------	---	----------

69. M/J 23/P21/Q3/a

3(a)(i)	One mark for structure Structure: Record One mark for each point Advantage: • A set of data / all data related to one customer • of different types • is held under a single identifier/entity	4
3(a)(ii)	A (1D) <u>array</u> of <u>records</u> // An <u>array</u> of the given <u>type</u> could be used One mark per underlined word	2

70. M/J 23/P21/Q6

6	<pre> PROCEDURE Mix() DECLARE Count, Total ThisNum : INTEGER DECLARE ThisUser, ThisSample : INTEGER FOR ThisSample ← 1 TO 128 Count ← 0 Total ← 0 FOR ThisUser ← 1 TO 6 IF Sample[ThisUser, ThisSample] > 10 THEN Count ← Count + 1 Total ← Total + Sample[ThisUser, ThisSample] ENDIF NEXT ThisUser Result[ThisSample] ← INT(Total / Count) NEXT ThisSample ENDPROCEDURE </pre> Mark as follows: 1 Declaration and initialisation before inner loop of Count and Total 2 Outer Loop for 128 iterations 3 Inner loop for six iterations 4 Test for sample > 10 in a loop 5 and if true sum Total and increment Count 6 Calculate average value and assign to Result array after inner loop and within outer loop 7 Use of INT() / DIV to convert average to integer Max 6 Marks	6
---	--	---

71. M/J 23/P21/Q8/b,d,e

8(b)	<pre> FUNCTION CheckNewItem(NewLine : STRING) RETURNS BOOLEAN DECLARE NotFound : BOOLEAN DECLARE NewItemNum, ThisItemNum, ThisLine : STRING NotFound ← TRUE OPENFILE "Stock.txt" FOR READ NewItemNum ← LEFT(NewLine, 4) ThisItemNum ← "0000" //rogue initial value WHILE NOT EOF("Stock.txt") AND NotFound = TRUE AND__ ThisItemNum < NewItemNum READFILE("Stock.txt", ThisLine) //brackets optional ThisItemNum ← LEFT(ThisLine, 4) IF ThisItemNum = NewItemNum THEN NotFound ← FALSE ENDIF ENDWHILE CLOSEFILE "Stock.txt" RETURN NotFound ENDFUNCTION </pre> Mark as follows: 1 Open Stock.txt in READ mode and subsequently close 2 Extract NewItemNum from parameter 3 Conditional loop until EOF("Stock.txt") 4 ... OR NewItemNum found 5 ... OR when ThisItemNum < NewItemNum 6 Read a line from Stock.txt AND extract ThisItemNum in a loop 7 If ThisItemNum = NewItemNum then terminate loop / set flag in a loop 8 Return Boolean after reasonable attempt Max 7 marks Max 6 if function wrapper (heading and ending) missing or incorrect	7
8(d)	Append	1
8(e)	One mark for each part: • The algorithm / search / iteration can stop /only iterates • if the current value read from the file // current line in file • is greater than the value being searched for	3

72. M/J 23/P22/Q1/c

1(c)	One mark per bullet point: • BOOLEAN • REAL • STRING	3
------	---	----------

73. M/J 23/P22/Q3/a

3(a)(i)	One mark per point: 1 Check that the queue is not full 2 EoQ pointer will move to point to location 9 3 Data item Orange will be stored in location referenced by EoQ pointer 4 EoQ pointer will move to point to location 0 5 Data item Yellow will be stored in location referenced by EoQ pointer Note: max 4 marks	4
3(a)(ii)	7	1

74. M/J 23/P22/Q8/b

8(b)	<pre> PROCEDURE AddItem(NewLine : STRING) DECLARE NewItemNum, ThisItemNum : STRING OPENFILE "Stock.txt" FOR READ OPENFILE "NewStock.txt" FOR WRITE NewItemNum ← LEFT(NewLine, 4) WHILE NOT EOF("Stock.txt") READFILE("Stock.txt", ThisLine) ThisItemNum ← LEFT(ThisLine, 4) IF ThisItemNum > NewItemNum THEN WRITEFILE("NewStock.txt", NewLine) // write New Line... NewItemNum ← "9999" // ...once only ENDIF WRITEFILE("NewStock.txt", ThisLine) ENDWHILE IF NewItemNum <> "9999" THEN WRITEFILE("NewStock.txt", NewLine) //New last line in the file ENDIF CLOSEFILE "Stock.txt" CLOSEFILE "NewStock.txt" ENDPROCEDURE </pre> Mark as follows: 1 Open both files, in correct modes, and subsequently close 2 Conditional loop until end of file <code>Stock.txt</code> 3 Read a line from <code>Stock.txt</code> AND extract <code>ThisItemNum</code> in a loop 4 Test <code>ThisItemNum > NewItemNum</code> then write <code>NewLine</code> to <code>NewStock.txt</code> 5 ...including mechanism to only do this once only 6 Write line read from <code>Stock</code> to <code>NewStock</code> 7 Deal with the case where <code>NewLine</code> is the new last line	7
------	---	----------

8(b)	Example of array-based solution: <pre> PROCEDURE AddItem(NewLine : STRING) DECLARE ThisItemNum, ThisLine : STRING DECLARE Temp : ARRAY [1:5999] OF STRING DECLARE Index : INTEGER FOR Index ← 1 TO 5999 Temp[Index] ← "" //Initialise array NEXT Index Index ← STR_TO_NUM(LEFT(NewLine, 4)) Temp[Index] ← NewLine //Add new line to array OPENFILE "Stock.txt" FOR READ WHILE NOT EOF("Stock.txt") READFILE("Stock.txt", ThisLine) Index ← STR_TO_NUM(LEFT(ThisLine, 4)) Temp[Index] ← ThisLine //Add line from file to array ENDWHILE CLOSEFILE "Stock.txt" OPENFILE "NewStock.txt" FOR WRITE FOR Index ← 1 TO 5999 IF Temp[Index] <> "" THEN //Write non-blank element... WRITEFILE("NewStock.txt", Temp[Index]) //...to new file ENDIF NEXT Index CLOSEFILE "NewStock.txt" ENDPROCEDURE </pre> Mark as follows: 1 Open both files, in correct modes, and subsequently close 2 Declare AND initialise Temp array 3 Store NewLine in appropriate array element 4 Loop until end of file Stock.txt 5 Read a line from Stock.txt AND extract Index in a loop 6 Assign line read to appropriate array element in a loop 7 Loop through array, writing non-blank elements to file NewStock.txt	
------	---	--

75. M/J 23/P23/Q1/a,c

1(a)	Answer		3
	The dimension of the array	2	
	The name of the variable used as an array index	PCount	
	The number of elements in the array	100	

1(c)	Integer	1
------	---------	---

76. M/J 23/P23/Q3/a,b(i)

3(a)(i)	6	1																						
3(a)(ii)	Stack <table><thead><tr><th>Memory location</th><th>Value</th></tr></thead><tbody><tr><td>506</td><td></td></tr><tr><td>505</td><td>BBB</td></tr><tr><td>504</td><td>AAA</td></tr><tr><td>503</td><td>XXX</td></tr><tr><td>502</td><td>ZZZ</td></tr><tr><td>501</td><td>NNN</td></tr><tr><td>500</td><td>PPP</td></tr></tbody></table> Pointer ← TopOfStack ← BottomOfStack <table><thead><tr><th>Variable</th><th>Value</th></tr></thead><tbody><tr><td>Data1</td><td>WWW</td></tr><tr><td>Data2</td><td>AAA</td></tr></tbody></table> One mark for: MP1 Values 'BBB' and 'AAA' MP2 Values 'XXX' to 'PPP' (unchanged) MP3 Both pointers and labelled22 MP4 Values of both variables	Memory location	Value	506		505	BBB	504	AAA	503	XXX	502	ZZZ	501	NNN	500	PPP	Variable	Value	Data1	WWW	Data2	AAA	4
Memory location	Value																							
506																								
505	BBB																							
504	AAA																							
503	XXX																							
502	ZZZ																							
501	NNN																							
500	PPP																							
Variable	Value																							
Data1	WWW																							
Data2	AAA																							
3(b)(i)	So that the data may be recovered / restored (the next time the program is run) // the data is permanently saved / data is not lost when the <u>program terminates</u>	1																						

77. M/J 23/P23/Q8/a,b

8(a)	<pre> FUNCTION ChangeSupp(Code1, Code2 : STRING) RETURNS INTEGER DECLARE Count : INTEGER DECLARE ThisLine, ThisCode : STRING OPENFILE "Stock.txt" FOR READ OPENFILE "NewStock.txt" FOR WRITE Count ← 0 WHILE NOT EOF("Stock.txt") READFILE("Stock.txt ", ThisLine) // brackets optional ThisCode ← MID(ThisLine, 5, 3) IF ThisCode = Code1 THEN ThisLine ← LEFT(ThisLine, 4) & Code2 & RIGHT(ThisLine, LENGTH(ThisLine) - 7) Count ← Count + 1 ENDIF WRITEFILE("NewStock.txt", ThisLine) // brackets optional ENDWHILE CLOSEFILE "NewStock.txt" CLOSEFILE "Stock.txt" RETURN Count ENDFUNCTION </pre> MP1 Open both files, in correct modes, and subsequently close MP2 Conditional loop until EOF("Stock.txt") MP3 Read a line from Stock.txt AND extract ThisCode in a loop MP4 Test ThisCode = Code1 AND if true, increment Count (must have been Initialised) in a loop MP5 Update ThisLine using substring functions and '&' in a loop MP6 completely correct update of ThisLine in a loop MP7 Write ThisLine to NewStock.txt in a loop MP8 Return count after loop	8
------	---	----------

8(b)	<pre> PROCEDURE Report_1(Supp : STRING) DECLARE Count : INTEGER DECLARE ThisItemNum, ThisDesc, ThisLine, ThisCode : STRING Count ← 0 OPENFILE "Stock.txt" FOR READ OUTPUT "Report for Supplier:" & Supp OUTPUT "" //Blank line as per example OUTPUT "Item Description" OUTPUT "" //Blank line as per example WHILE NOT EOF("Stock.txt") READFILE("Stock.txt", ThisLine) ThisCode ← Mid(ThisLine, 5, 3) IF ThisCode = Supp THEN ThisItemNum ← LEFT(ThisLine, 4) ThisDesc ← RIGHT(ThisLine, LENGTH(ThisLine) - 7) OUTPUT ThisItem & " " & ThisDesc Count ← Count + 1 ENDIF ENDWHILE CLOSEFILE "Stock.txt" OUTPUT "" //Blank line as per example OUTPUT "Number of items listed: ", Count ENDPROCEDURE </pre> MP1 Output report header (blank lines optional) – Must contain the parameter code MP2 Conditional loop until EOF("Stock.txt") MP3 Read a line from Stock.txt AND extract SupplierCode in a loop MP4 Test if SupplierCode = Supp then increment count (must have been Initialised) MP5 Extract AND output item and description in a loop MP6 Output the final line with count	6
------	--	---

78. O/N 22/P21/Q1/b

1(b)(i)	One mark per point: 1 Algorithm to process / search / organise the data is easier to implement // Values may be accessed via a loop-controlled variable / iterated through using index 2 Makes the program easier to design / code / test / understand 3 Multiple instances referenced via a single identifier / so fewer identifiers needed // Easier to amend the program when the number of students increases	3
---------	--	---

1(b)(ii)	One mark per point: Purpose: It identifies / references an individual array element // provides the <u>index</u> to the array Data type: Integer	2
----------	---	---

79. O/N 22/P21/Q4/a,b

4(a)(i)	One mark for each: 1 Data A and K stored in new / existing nodes 2 Start pointer points to Node A 3 Node A points to Node C and Node C points to Node K 4 Node K contains Null Pointer Start pointer <pre> graph LR Start[Start pointer] --> A["'A' "] A --> C["'C' "] C --> K["'K' Ø"] J["'J' "] L["'L' "] </pre>	4
4(a)(ii)	One mark per point: 1 Pointers determine the ordering of data // only the pointers need to be changed when data changed 2 Easier to add / delete data (to maintain correct sequence) in a linked list // description of moving data to maintain correct sequence when array used	2
4(a)(iii)	One mark per point: 1 Need to store pointers as well as data 2 More complex (to setup / implement)	2
4(b)	One mark per point (Max 4): 1 Declare two (1D) arrays 2 One for data, one for pointers 3 Elements from same index represent one node 4 Declare an integer / variable for <code>StartPointer</code> // explain its use 5 Define appropriate value for null pointer // explain its use 6 Declare an integer / variable for <code>FreeList</code> pointer // explain its use 7 Routines are needed to add / delete / search Alternative MP1, 2 and 3 for record-based implementation: 1 Define a record type with fields for data and pointer 2 Declare one (1D) array 3 ...of the defined record type	4

80. O/N 22/P21/Q5

5	Mark as follows (Max 5): 1 Procedure heading and ending and declaration of both indexes 2 Loop to process all elements from <code>Array1</code> 3 Sum (any) three consecutive values from <code>Array1</code> and divide by 3 in a loop 4 Convert result to Integer 5 Assign value to correct element of <code>Array2</code> in a loop 6 Increment <code>Array2</code> index in a loop <pre>PROCEDURE Summarise() DECLARE Value : REAL DECLARE IxA, IxB : INTEGER // Index variables IxB ← 1 FOR IxA ← 1 TO 598 STEP 3 Value ← Array1[IXA] + Array1[IXA + 1] + Array1[IXA + 2] Value ← Value / 3 Array2[IxB] ← INT(Value) IxB ← IxB + 1 NEXT IxA ENDPROCEDURE</pre>	5
---	---	---

81. O/N 22/P21/Q8/a,c

8(a)	One mark for each point (Max 7) as follows: 1 Function heading and ending including parameter and return type 2 Declaration and initialisation of local Integer for <code>Count</code> 3 OPEN in READ mode and CLOSE 4 Conditional loop until <code>EOF()</code> 5 Read a line in a loop 6 If non-blank, increment count in a loop 7 Terminate loop when 10 non-blank lines have been read 8 Return Boolean in both cases <pre> FUNCTION CheckFile(Thisfile : STRING) RETURNS BOOLEAN DECLARE Valid : BOOLEAN DECLARE ThisLine : STRING DECLARE Count : INTEGER Count ← 0 Valid ← FALSE OPEN ThisFile FOR READ WHILE NOT EOF(ThisFile) AND Valid = FALSE READFILE ThisFile, ThisLine IF ThisLine <> "" THEN Count ← Count + 1 IF Count > 9 THEN Valid ← TRUE ENDIF ENDIF ENDWHILE CLOSEFILE ThisFile RETURN Valid ENDFUNCTION </pre>	7
------	--	----------

8(c)	Mark as follows: 1 Procedure heading and ending including parameters 2 Declaration and initialisation of local Integer value for <code>ErrCount</code> 3 Use of <code>CheckFile()</code>, output message and terminate if it returns <code>FALSE</code> 4 Conditional loop until <code>EOF()</code> 5 ...or <code>ErrCount > MaxErrors</code> 6 Read line and use as parameter to <code>CheckLine()</code> in a loop 7 Test return value and increment <code>ErrCount</code> if non-zero in a loop 8 Output either message once only as appropriate <pre> PROCEDURE CountErrors(ThisFile : STRING, MaxErrors : INTEGER) DECLARE ErrCount, ThisError : INTEGER DECLARE ThisLine : STRING ErrCount ← 0 IF CheckFile(ThisFile) = FALSE THEN OUTPUT "That program file is not valid" ELSE OPEN ThisFile FOR READ REPEAT READFILE, ThisFile, ThisLine ThisError ← CheckLine(ThisLine) IF ThisError <> 0 THEN ErrCount ← ErrCount + 1 ENDIF UNTIL ErrCount > MaxErrors OR EOF(ThisFile) IF EOF(ThisFile) = FALSE THEN OUTPUT "Check terminated - too many errors" ELSE OUTPUT "There were ", ErrCount, " errors." ENDIF CLOSEFILE ThisFile ENDIF ENDPROCEDURE </pre>	8
------	--	---

82. O/N 22/P22/Q3/a,b,c

3(a)	One mark for name, Max two for features (Max 3 in total) Name: Queue Features: 1 Each queue element contains one data item 2 A Pointer to the front / start of the queue 3 A Pointer to the back / end of the queue 4 Data is added at back / end and removed from front / start // works on a FIFO basis 5 May be circular ALTERNATIVE: Name: Linked List Features: 1 Each node contains data and a pointer to the next node 2 A Pointer to the start of the list 3 Last node in the list has a null pointer 4 Data may be added / removed by manipulating pointers (not moving data) 5 Nodes are traversed in a specific sequence 6 Unused nodes are stored on a free list // a free-list pointer to the Free List	3
3(b)	One mark per point (Max 5): 1 Declare a (1D) array of data type <code>STRING</code> 2 The number of elements in that array corresponds to the size of the required stack 3 Declare an integer / variable for <code>StackPointer</code> 4. Declare an integer / variable for the size of the stack // for the max value of <code>StackPointer</code> 5 Use the <code>StackPointer</code> as an index to the array 6 Pointers and variables initialised to indicate empty stack 7 Store each item on the stack as one array element / Each stack item maps to one array element 8 Attempt to describe Push and Pop operations 9 Push and Pop routines need to check for full or empty conditions	5

3(c)

One mark for each:

5

- 1 Data 'After Group 1' (as shown, including blank cells)
- 2 Data 'After Group 2' (as shown, including blank cells)
- 3 Data 'After Group 3' (as shown, including blank cells)
- 4 SP 'After Group 1' pointing to location 955
- 5 Final two SPs pointing to locations 952 and 954

Memory location	Initial state	After Group 1	After Group 2	After Group 3
957				
956				
955		E	E	E
954		D	D	C
953	X	X	X	A
952	Y	Y	Y	Y
951	Z	Z	Z	Z
950	P	P	P	P

[5]

83. O/N 22/P22/Q7/a,b,c

7(a)	One mark per point (Max 6): 1 Procedure heading and ending including parameters 2 Conditional loop containing incrementing Index... 3 ...terminating when ErrNum found 4 ...terminating when ErrCode[Index] > ErrNum (i.e. ErrNum not found) 5 ... OR after element 500 tested 6 Test if found and OUTPUT 'Found' message 7 ...otherwise OUTPUT 'Not Found' message <pre> PROCEDURE OutputError(LineNum, ErrNum : INTEGER) DECLARE Index : INTEGER Index ← 0 // Search until ErrNum found OR not present OR end of array REPEAT Index ← Index + 1 UNTIL ErrCode[Index] >= ErrNum OR Index = 500 IF ErrCode[Index] = ErrNum THEN OUTPUT ErrText[Index], " on line ", LineNum //Found ELSE OUTPUT "Unknown error on line ", LineNum //Not found ENDIF ENDPROCEDURE </pre>	6
------	--	----------

7(b)	One mark per point (Max 8): 1 Procedure heading and ending as shown 2 Conditional loop correctly terminated 3 An inner loop 4 Correct range for inner loop 5 Comparison (element J with J+1) in a loop 6 Swap elements in both arrays in a loop 7 'No-Swap' mechanism: • Conditional outer loop including flag reset • Flag set in inner loop to indicate swap 8 Efficiency (this scenario): terminate inner loop when ErrCode = 999 9 Reducing Boundary in the <u>outer</u> loop <pre> PROCEDURE SortArrays() DECLARE TempInt, J, Boundary : INTEGER DECLARE TempStr : STRING DECLARE NoSwaps : BOOLEAN Boundary ← 499 REPEAT NoSwaps ← TRUE FOR J ← 1 TO Boundary IF ErrCode[J] > ErrCode[J+1] THEN //first swap ErrCode elements TempInt ← ErrCode[J] ErrCode[J] ← ErrCode[J+1] ErrCode[J+1] ← TempInt //now swap corresponding ErrText elements TempStr ← ErrText[J] ErrText[J] ← ErrText[J+1] ErrText[J+1] ← TempStr NoSwaps ← FALSE ENDIF NEXT J Boundary ← Boundary - 1 UNTIL NoSwaps = TRUE ENDPROCEDURE </pre>	8
7(c)(i)	ErrCode should be an INTEGER // ErrCode should not be a STRING	1

7(c)(ii)	Benefits include: 1 Array of records can store mixed data types / multiple data types under a single identifier 2 Tighter / closer association between <code>ErrCode</code> and <code>ErrText</code> // simpler code as fields may be referenced together // values cannot get out of step as with two arrays 3 Program easier to design / write / debug / test / maintain / understand One mark per point Note: Max 2 marks	2
7(c)(iii)	<code>DECLARE Error : ARRAY[1:500] OF ErrorRec</code>	1

84. O/N 22/P23/Q1/a

1(a)	One mark per row	4										
<table><tr><th>Variable use</th><th>Data type</th></tr><tr><td>Store the number of days in the current month</td><td>INTEGER</td></tr><tr><td>Store the first letter of the customer's first name</td><td>CHAR</td></tr><tr><td>Store an indication of whether a year is a leap year</td><td>BOOLEAN</td></tr><tr><td>Store the average amount spent per customer visit</td><td>REAL</td></tr></table>			Variable use	Data type	Store the number of days in the current month	INTEGER	Store the first letter of the customer's first name	CHAR	Store an indication of whether a year is a leap year	BOOLEAN	Store the average amount spent per customer visit	REAL
Variable use	Data type											
Store the number of days in the current month	INTEGER											
Store the first letter of the customer's first name	CHAR											
Store an indication of whether a year is a leap year	BOOLEAN											
Store the average amount spent per customer visit	REAL											

85. O/N 22/P23/Q4/a,b

4(a)	The data type (of the item to be stored)	1
4(b)(i)	Operation: Add an item / Enqueue Check: There are unused elements in the array // The queue is not full Operation: Remove an item / Dequeue Check: There are items in the array // The queue is not empty One mark for reason and one mark for reason it could not be completed.	4

4(b)(ii)	One mark per point (Max 5): 1 Declare a (1D) array of size ≥ 10 2 ...of data type <code>CHAR</code> 3 Declare integer variable for <code>FrontOfQueuePointer</code> 4 Declare integer variable for <code>EndOfQueuePointer</code> 5 Initialise <code>FrontOfQueuePointer</code> and <code>EndOfQueuePointer</code> to represent an empty queue 6 Declare integer variable or <code>NumberInQueue</code> 7 Declare integer variable for <code>SizeOfQueue</code> to count / limit the max number of items allowed // Reference to mechanism for defining 'wrap' of circular queue 8 Initialise <code>SizeOfQueue</code> // Initialise <code>NumberInQueue</code>	5
----------	--	----------

86. O/N 22/P23/Q5/b(i)

5(b)(i)	<code>LineData.Cost ← 12.99</code>	1
---------	------------------------------------	----------

87. O/N 22/P23/Q7/a,b(i)

7(a)	One mark per point (Max 8): 1 Declaration and initialisation of local integer for <code>Count</code> 2 Appropriate prompt and two inputs 3 (Conditional) loop while error number input is in range // error code 999 reached 4 ...and not end of array 5 Check if this <code>ErrCode</code> needs to be output in a loop 6 if so check for blank error text in a loop 7 Output in both cases 8and increment count in a loop 9 OUTPUT of header and summary including count <pre> PROCEDURE OutputRange() DECLARE First, Last, Count, Index, ThisErr : INTEGER DECLARE ThisMess : STRING DECLARE PastLast: BOOLEAN Count ← 0 Index ← 1 PastLast ← FALSE OUTPUT "Please input first error number: " INPUT First OUTPUT "Please input last error number: " INPUT Last OUTPUT "List of error numbers from ", First, " to ", Last WHILE Index < 501 AND NOT PastLast ThisErr ← ErrCode[Index] IF ThisErr > Last THEN PastLast ← TRUE ELSE IF ThisErr >= First THEN ThisMess ← ErrText[Index] IF ThisMess = "" THEN ThisMess ← "Error Text Missing" ENDIF OUTPUT ThisErr, " : ", ThisMess Count ← Count + 1 ENDIF ENDIF Index ← Index + 1 ENDWHILE OUTPUT Count, " error numbers output" ENDPROCEDURE </pre>	8
------	---	----------

7(b)(i)	One mark per point: 1 (Conditional) loop terminating when item added OR end of array reached 2 Test for unused element in a loop 3 Assignment of values to arrays // save index of first blank location and assign after loop 4 Set loop termination if empty element found in a loop 5 Call SortArrays() once 6 Calculation of remaining unused elements and return Integer value (for both cases) <pre> FUNCTION AddError(ErrNum : INTEGER, ErrMess : STRING) RETURNS INTEGER DECLARE Index, Remaining : INTEGER CONSTANT Unused = 999 Index ← 1 Remaining ← -1 REPEAT IF ErrCode[Index] = Unused THEN ErrCode[Index] ← ErrNum ErrText[Index] ← ErrMess CALL SortArrays() Remaining ← 500 - Index ENDIF Index ← Index + 1 UNTIL Remaining <> -1 OR Index > 500 RETURN Remaining ENDFUNCTION </pre>	6
---------	---	----------

88. O/N 21/P21/Q1/b

1(b)																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																												
------	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

89. O/N 21/P21/Q3/a,b,c

3(a)(i)	One mark per point: • E_{OQ} pointer will move to point to location 4 // incremented E_{OQ} (by 1) • Data value "Octopus" will be stored in location pointed to be E_{OQ} / location 4	2																		
3(a)(ii)	One mark for each bullet • Value "Frog" // value pointed to by F_{OQ} / location 0 is assigned to variable <code>AnimalName</code> • F_{OQ} pointer will move to point to location 1 / point to "Cat" // incremented F_{OQ} (by 1) <table border="1"> <tr> <td>0</td><td>Frog</td><td rowspan="2">← Front of queue pointer</td></tr> <tr> <td>1</td><td>Cat</td></tr> <tr> <td>2</td><td>Fish</td><td rowspan="2">← End of queue pointer</td></tr> <tr> <td>3</td><td>Elk</td></tr> </table>	0	Frog	← Front of queue pointer	1	Cat	2	Fish	← End of queue pointer	3	Elk	2								
0	Frog	← Front of queue pointer																		
1	Cat																			
2	Fish	← End of queue pointer																		
3	Elk																			
3(a)(iii)	There is only one data item in the queue	1																		
3(b)(i)	One mark for data values plus one mark for pointers <table border="1"> <tr> <td>0</td><td>Frog</td><td rowspan="4">← Front of queue pointer</td></tr> <tr> <td>1</td><td>Cat</td></tr> <tr> <td>2</td><td>Fish</td></tr> <tr> <td>3</td><td>Elk</td></tr> <tr> <td>4</td><td>Wasp</td><td rowspan="4">← End of queue pointer</td></tr> <tr> <td>5</td><td>Bee</td></tr> <tr> <td>6</td><td>Mouse</td></tr> <tr> <td>7</td><td></td></tr> </table> One mark for each pointer One mark for three new data values	0	Frog	← Front of queue pointer	1	Cat	2	Fish	3	Elk	4	Wasp	← End of queue pointer	5	Bee	6	Mouse	7		3
0	Frog	← Front of queue pointer																		
1	Cat																			
2	Fish																			
3	Elk																			
4	Wasp	← End of queue pointer																		
5	Bee																			
6	Mouse																			
7																				

3(b)(ii)	<table><tr><td>0</td><td>Shark</td><td rowspan="4">← End of queue pointer</td></tr><tr><td>1</td><td>(Cat)</td></tr><tr><td>2</td><td>(Fish)</td></tr><tr><td>3</td><td>(Elk)</td></tr><tr><td>4</td><td>Wasp</td><td rowspan="4">← Front of queue pointer</td></tr><tr><td>5</td><td>Bee</td></tr><tr><td>6</td><td>Mouse</td></tr><tr><td>7</td><td>Dolphin</td></tr></table> One mark for BOTH pointers One mark for all data values as shown	0	Shark	← End of queue pointer	1	(Cat)	2	(Fish)	3	(Elk)	4	Wasp	← Front of queue pointer	5	Bee	6	Mouse	7	Dolphin	2
0	Shark	← End of queue pointer																		
1	(Cat)																			
2	(Fish)																			
3	(Elk)																			
4	Wasp	← Front of queue pointer																		
5	Bee																			
6	Mouse																			
7	Dolphin																			
3(c)	One mark per point: 1 If incremented $E_{OQ} = F_{OQ}$ then error condition: queue is full 2 Increment the E_{OQ} 3 Manage wrap-around	3																		

90. O/N 21/P21/Q5/a,c

5(a)	One mark for the character and one for the corresponding reason. • Character: Any except alphabetic, numeric, ',' ':' or space • Reason: character doesn't occur in data to be recorded	2
------	---	----------

5(c)	<pre> FUNCTION LogEvents(StudentID : STRING) RETURNS INTEGER DECLARE FileData : STRING DECLARE Index, Count : INTEGER CONSTANT LogFile = "LogFile" Count ← 0 OPENFILE LogFile FOR APPEND FOR Index ← 1 TO 2000 FileData ← LogArray[Index] IF LEFT(FileData, 6) = StudentID THEN WRITEFILE (LogFile, FileData) //brackets optional Count ← Count + 1 LogArray[Index] ← "" // clear the element ENDIF NEXT Index CLOSEFILE LogFile RETURN Count ENDFUNCTION </pre> 1 mark for each of the following: 1 Function heading and ending including parameter and return type 2 OPEN file LogFile for APPEND and subsequent CLOSE 3 Loop for 2000 iterations 4 Extract first 6 characters from array element in a loop 5 Compare first 6 characters with parameter in a loop 6 If equal: • write whole array element string to file and • increment Count and • clear array element in a loop 7 Return Count (must have been declared and initialised)	7
------	--	---

91. O/N 21/P21/Q6/a,b

6(a)	<pre> PROCEDURE SetRow(Row, SkipNum, SetNum : INTEGER) DECLARE Col : INTEGER // array is 1280 x 800 FOR Col ← SkipNum + 1 TO SkipNum + SetNum Screen[Row, Col] ← 1 NEXT Index ENDPROCEDURE ALTERNATIVE 1: FOR Col ← 1 TO SetNum Screen[Row, SkipNum + Col] ← 1 NEXT Col ALTERNATIVE 2: WHILE SetNum > 0 Screen[Row, SkipNum + SetNum] ← 1 SetNum ← SetNum - 1 ENDWHILE Mark as follows: 1 Procedure heading and ending including parameters 2 Declaration of local Integer for Col 3 Count-controlled loop with meaningful start number 4 correct stop number 5 Reference Screen Array element and set to 1 in a loop </pre>	5
------	--	----------

6(b)	<pre> FUNCTION SearchInRow(ThisRow, StartCol : INTEGER) RETURNS INTEGER DECLARE ThisCol, Step : INTEGER DECLARE Found: BOOLEAN // array is 1280 x 800 Found ← FALSE ThisCol ← StartCol // first decide which way to search IF StartCol = 1 THEN Step ← 1 EndCol ← 1281 ELSE Step ← -1 EndCol ← 0 ENDIF WHILE ThisCol <> EndCol AND Found = FALSE IF Screen[ThisRow, ThisCol] <> 1 THEN ThisCol ← ThisCol + Step ELSE Found ← TRUE ENDIF ENDWHILE IF Found = FALSE THEN ThisCol ← -1 ENDIF RETURN ThisCol ENDFUNCTION </pre> Mark as follows: 1 Interpreting <code>StartCol</code> parameter to determine direction of search 2 An attempt at searching both up and down 3 Conditional Loop / Count-controlled loop with use of <code>ThisCol</code> index 4 Using correct values for <code>StartCol</code>, <code>EndCol</code> and <code>Step</code> 5 Reference a <code>Screen</code> element and compare with 1 in a loop 6 If equal save column or immediately Return column in a loop 7 Return column number or -1 Loop(s) terminate when element with value = 1 found Max 7 marks if function heading, including return type, and ending is incorrect or incomplete	8
------	--	---

92. O/N 21/P22/Q1/b

1(b)	Answer		3
	The number of dimensions of <code>ThisArray</code>	1	
	The technical terms for minimum and maximum values that variable <code>n</code> may take	Lower bound, upper bound	
	The technical term for the variable <code>n</code> in the pseudocode expression.	Index / Subscript	
One mark per row			

93. O/N 21/P22/Q3/a,b

3(a)(i)	Pseudocode: <pre> TYPE Student DECLARE StudentID : STRING DECLARE Email : STRING DECLARE Club_1 : INTEGER DECLARE Club_2 : INTEGER DECLARE Club_3 : INTEGER ENDTYPE </pre> Mark as follows: - One mark for <code>TYPE</code> and <code>ENDTYPE</code> - One mark for <code>StudentID</code> and <code>Email</code> fields as <code>STRING</code> - One mark for all <code>Club</code> fields as <code>INTEGER</code>	3
3(a)(ii)	<u>DECLARE Membership : ARRAY [1:3000] OF Student</u> One mark per underlined phrase	2
3(a)(iii)	One mark for any one of: - Assign a value (of the correct data type) outside the normal range to one of the fields - Assign an empty string to the <code>StudentID</code> field / <code>Email</code> field - or value out of range to any club field	1
3(a)(iv)	A number outside the range 1 to 99	1

3(b)	<pre> PROCEDURE GetIDs() DECLARE Index : INTEGER DECLARE ThisClub, Count : INTEGER OUTPUT "Please Input Club Number: " INPUT ThisClub Count ← 0 FOR Index ← 1 TO 3000 IF Membership[Index].Club_1 = ThisClub OR ____ Membership[Index].Club_2 = ThisClub OR ____ Membership[Index].Club_3 = ThisClub THEN Count ← Count + 1 OUTPUT Membership[Index].StudentID ENDIF NEXT Index OUTPUT "There are ", Count, " Students in the club" ENDPROCEDURE </pre> Mark as follows: 1 Declare and initialise Count 2 Prompt and Input club number before the loop 3 Loop through 3000 elements 4 Compare one club field with number input 5 Compare all Club fields with number input 6 If number found, OUTPUT of StudentID field and increment Count 7 Final OUTPUT of Count outside the loop Note: Max 6 if procedure heading and ending missing or incorrect (but allow array as parameter)	7
------	--	---

94. O/N 21/P22/Q5/a,b

5(a)	<pre> PROCEDURE MakeNewFile(OldFile, NewFile, Status : STRING) DECLARE Line1, Line2, Line3 : STRING DECLARE NumCopied, NumRecs : INTEGER NumRecs ← 0 NumCopied ← 0 OPENFILE OldFile FOR READ OPENFILE NewFile FOR WRITE WHILE NOT EOF(OldFile) READFILE OldFile, Line1 READFILE OldFile, Line2 READFILE OldFile, Line3 NumRecs ← NumRecs + 1 IF Line3 <> Status THEN WRITEFILE NewFile, Line1 WRITEFILE NewFile, Line2 WRITEFILE NewFile, Line3 NumCopied ← NumCopied + 1 ENDIF ENDWHILE OUTPUT "File " , OldFile , " contained " , NumRecs , __ " employee details" OUTPUT Numcopied , " employee sets of details were __ written to file", NewFile CLOSEFILE OldFile CLOSEFILE NewFile ENDPROCEDURE </pre> Mark as follows: 1 Procedure heading and ending, including parameters 2 OPEN OldFile for READ and NewFile for WRITE and subsequently CLOSE both files 3 Conditional loop until EOF(OldFile) 4 Read three lines from OldFile in a loop 5 Compare 3rd line read with Status parameter and if not equal write 3 lines to NewFile in a loop 6 Count number of sets read and those written in a loop 7 Final output including both counts and file names with suitable text after a loop Note: MP6: Both counts must have been declared and initialised	7
5(b)(i)	Store all three items on one line	1
5(b)(ii)	One mark per point: Advantage: Fewer file operations required Disadvantage: Algorithm to combine / extract individual data items is more complex	2

95. O/N 21/P22/Q6/a

6(a)	<pre> FUNCTION FirstRowSet() RETURNS INTEGER DECLARE Row, Col : INTEGER DECLARE Found : BOOLEAN // array is 1280 × 800 Row ← 1 Found ← FALSE WHILE Row <= 800 AND Found = FALSE // top to bottom Col ← 1 WHILE Col <= 1280 AND Found = FALSE // left to right IF Screen[Row,Col] = 1 THEN Found ← TRUE // end function as soon as first // found ENDIF Col ← Col + 1 ENDWHILE Row ← Row + 1 ENDWHILE IF Found = FALSE THEN // nothing found Row ← 0 ENDIF RETURN Row - 1 ENDFUNCTION </pre> Mark as follows: 1 Function heading and ending and return type 2 (Conditional) outer loop 1 to 800 (row) 3 (Conditional) inner loop 1 to 1280 // 1280 to 1 (column) 4 Reference <code>Screen</code> element and test for = 1 // <> 0 5 and if true save row number and exit loops 6 Increment index variables in both inner and outer loop 7 Return Row number or -1, following a reasonable attempt	7
------	---	---

96. O/N 21/P23/Q1/b

1(b)			4

97. O/N 21/P23/Q3/a,b,c

3(a)(i)	One mark per point: • E_{OQ} pointer will move to point to location 4 // incremented E_{OQ} (by 1) • Data value "Octopus" will be stored in location pointed to be E_{OQ} / location 4	2																		
3(a)(ii)	One mark for each bullet • Value "Frog" // value pointed to by F_{OQ} / location 0 is assigned to variable <code>AnimalName</code> • F_{OQ} pointer will move to point to location 1 / point to "Cat" // incremented F_{OQ} (by 1) <table border="1"> <tr> <td>0</td><td>Frog</td><td rowspan="2">← Front of queue pointer</td></tr> <tr> <td>1</td><td>Cat</td></tr> <tr> <td>2</td><td>Fish</td><td rowspan="2">← End of queue pointer</td></tr> <tr> <td>3</td><td>Elk</td></tr> </table>	0	Frog	← Front of queue pointer	1	Cat	2	Fish	← End of queue pointer	3	Elk	2								
0	Frog	← Front of queue pointer																		
1	Cat																			
2	Fish	← End of queue pointer																		
3	Elk																			
3(a)(iii)	There is only one data item in the queue	1																		
3(b)(i)	One mark for data values plus one mark for pointers <table border="1"> <tr> <td>0</td><td>Frog</td><td rowspan="4">← Front of queue pointer</td></tr> <tr> <td>1</td><td>Cat</td></tr> <tr> <td>2</td><td>Fish</td></tr> <tr> <td>3</td><td>Elk</td></tr> <tr> <td>4</td><td>Wasp</td><td rowspan="4">← End of queue pointer</td></tr> <tr> <td>5</td><td>Bee</td></tr> <tr> <td>6</td><td>Mouse</td></tr> <tr> <td>7</td><td></td></tr> </table> One mark for each pointer One mark for three new data values	0	Frog	← Front of queue pointer	1	Cat	2	Fish	3	Elk	4	Wasp	← End of queue pointer	5	Bee	6	Mouse	7		3
0	Frog	← Front of queue pointer																		
1	Cat																			
2	Fish																			
3	Elk																			
4	Wasp	← End of queue pointer																		
5	Bee																			
6	Mouse																			
7																				

3(b)(ii)	<table><tr><td>0</td><td>Shark</td><td rowspan="4">← End of queue pointer</td></tr><tr><td>1</td><td>(Cat)</td></tr><tr><td>2</td><td>(Fish)</td></tr><tr><td>3</td><td>(Elk)</td></tr><tr><td>4</td><td>Wasp</td><td rowspan="4">← Front of queue pointer</td></tr><tr><td>5</td><td>Bee</td></tr><tr><td>6</td><td>Mouse</td></tr><tr><td>7</td><td>Dolphin</td></tr></table> One mark for BOTH pointers One mark for all data values as shown	0	Shark	← End of queue pointer	1	(Cat)	2	(Fish)	3	(Elk)	4	Wasp	← Front of queue pointer	5	Bee	6	Mouse	7	Dolphin	2
0	Shark	← End of queue pointer																		
1	(Cat)																			
2	(Fish)																			
3	(Elk)																			
4	Wasp	← Front of queue pointer																		
5	Bee																			
6	Mouse																			
7	Dolphin																			
3(c)	One mark per point: 1 If incremented $E_{OQ} = F_{OQ}$ then error condition: queue is full 2 Increment the E_{OQ} 3 Manage wrap-around	3																		

98. O/N 21/P23/Q5/a,c

5(a)	One mark for the character and one for the corresponding reason. • Character: Any except alphabetic, numeric, ',' ':' or space • Reason: character doesn't occur in data to be recorded	2
------	---	----------

5(c)	<pre> FUNCTION LogEvents(StudentID : STRING) RETURNS INTEGER DECLARE FileData : STRING DECLARE Index, Count : INTEGER CONSTANT LogFile = "LogFile" Count ← 0 OPENFILE LogFile FOR APPEND FOR Index ← 1 TO 2000 FileData ← LogArray[Index] IF LEFT(FileData, 6) = StudentID THEN WRITEFILE (LogFile, FileData) //brackets optional Count ← Count + 1 LogArray[Index] ← "" // clear the element ENDIF NEXT Index CLOSEFILE LogFile RETURN Count ENDFUNCTION </pre> 1 mark for each of the following: 1 Function heading and ending including parameter and return type 2 OPEN file LogFile for APPEND and subsequent CLOSE 3 Loop for 2000 iterations 4 Extract first 6 characters from array element in a loop 5 Compare first 6 characters with parameter in a loop 6 If equal: • write whole array element string to file and • increment Count and • clear array element in a loop 7 Return Count (must have been declared and initialised)	7
------	--	---

99. O/N 21/P23/Q6/a,b

6(a)	<pre> PROCEDURE SetRow(Row, SkipNum, SetNum : INTEGER) DECLARE Col : INTEGER // array is 1280 x 800 FOR Col ← SkipNum + 1 TO SkipNum + SetNum Screen[Row, Col] ← 1 NEXT Index ENDPROCEDURE ALTERNATIVE 1: FOR Col ← 1 TO SetNum Screen[Row, SkipNum + Col] ← 1 NEXT Col ALTERNATIVE 2: WHILE SetNum > 0 Screen[Row, SkipNum + SetNum] ← 1 SetNum ← SetNum - 1 ENDWHILE Mark as follows: 1 Procedure heading and ending including parameters 2 Declaration of local Integer for Col 3 Count-controlled loop with meaningful start number 4 correct stop number 5 Reference Screen Array element and set to 1 in a loop </pre>	5
------	--	----------

6(b)	<pre> FUNCTION SearchInRow(ThisRow, StartCol : INTEGER) RETURNS INTEGER DECLARE ThisCol, Step : INTEGER DECLARE Found: BOOLEAN // array is 1280 x 800 Found ← FALSE ThisCol ← StartCol // first decide which way to search IF StartCol = 1 THEN Step ← 1 EndCol ← 1281 ELSE Step ← -1 EndCol ← 0 ENDIF WHILE ThisCol <> EndCol AND Found = FALSE IF Screen[ThisRow, ThisCol] <> 1 THEN ThisCol ← ThisCol + Step ELSE Found ← TRUE ENDIF ENDWHILE IF Found = FALSE THEN ThisCol ← -1 ENDIF RETURN ThisCol ENDFUNCTION </pre> Mark as follows: 1 Interpreting <code>StartCol</code> parameter to determine direction of search 2 An attempt at searching both up and down 3 Conditional Loop / Count-controlled loop with use of <code>ThisCol</code> index 4 Using correct values for <code>StartCol</code>, <code>EndCol</code> and <code>Step</code> 5 Reference a <code>Screen</code> element and compare with 1 in a loop 6 If equal save column or immediately Return column in a loop 7 Return column number or -1 Loop(s) terminate when element with value = 1 found Max 7 marks if function heading, including return type, and ending is incorrect or incomplete	8
------	--	---

100. M/J 21/P21/Q1/c

1(c)	Structure: - Record Justification: - Allows different data <u>types</u> - to be stored under one <u>identifier</u> Mark as follows: One mark for Structure Two marks for justification. Alternative - Two (1D) arrays - One of string, one of integer - Where same index links the name with the score	3
------	--	----------

101. M/J 21/P21/Q3/a,b

3(a)(i)	One mark per bullet point: - Data from the arrays is written to the <u>files</u> at the end of the day / before the program is terminated / computer is switched off - Data can then be read from the <u>files</u> at the start of the next day and written to / stored in the <u>arrays</u> - No need to (re-)enter the data manually // only need to enter data once Note: Max 2 marks	2
3(a)(ii)	The data is retained when the program is terminated / after the computer is switched off // data is stored permanently // non-volatile storage	1
3(a)(iii)	One mark per bullet point: - Data items are combined to form a single string / saved as a single line in the file - Data items are separated by a special character // make each data item a fixed length ALTERNATIVE: - Convert all data items / 'number of people' to strings - Consecutive / each line stores a separate data item	2

3(b)	<pre>PROCEDURE Preview (ThisFile : STRING) DECLARE LineNum : INTEGER DECLARE ThisLine : STRING OPENFILE ThisFile FOR READ IF EOF(ThisFile) THEN OUTPUT "Warning Message" ELSE LineNum ← 1 WHILE LineNum < 6 AND NOT EOF(ThisFile) READFILE Thisfile, ThisLine OUTPUT ThisLine LineNum ← LineNum + 1 ENDWHILE ENDIF CLOSEFILE ThisFile ENDPROCEDURE</pre> Marks as follows: 1 Procedure heading (including parameter) and ending 2 File OPEN and subsequently CLOSE 3 Check if file is empty and output a warning message if it is 4 Conditional Loop 5 Output line (including blank lines) and read next line in a loop	5
------	--	----------

102. M/J 21/P21/Q5

5	<pre> PROCEDURE Sort() DECLARE Temp : INTEGER DECLARE NoSwaps : BOOLEAN DECLARE Boundary, Row, Col : INTEGER Boundary ← 999 REPEAT NoSwaps ← TRUE FOR Row ← 1 TO Boundary IF Result[Row, 2] > Result[Row + 1, 2] THEN FOR Col ← 1 TO 2 Temp ← Result [Row, Col] Result [Row, Col] ← Result [Row + 1, Col] Result [Row + 1, Col] ← Temp NEXT Col NoSwaps ← FALSE ENDIF NEXT J Boundary ← Boundary - 1 UNTIL NoSwaps = TRUE ENDPROCEDURE </pre> Mark as follows: 1 Outer loop 2 Inner loop 3 Correct comparison in a loop 4 Correct swap of col1 array elements in a loop 5 Correct swap of col2 array elements in a loop (via loop or separate statements) 6 'NoSwap' mechanism: Conditional outer loop including flag reset 7 'NoSwap' mechanism: Set flag in inner loop to indicate swap 8 Reducing Boundary in the <u>outer</u> loop	8
---	---	---

103. M/J 21/P21/Q6/a,b

6(a)	One mark per point: 1 Check for a free node 2 Search for correct insertion point 3 Assign data value B to first node in free list / node pointed to by start pointer of free list 4 Pointer from A will be changed to point to node containing B (instead of C) 5 Pointer from B will be changed to point to node containing C 6 Start pointer in free list moved to point to next free node Note: max 4 marks	4
------	--	---

6(b)	One mark per point: - An array (1D) to store the data and a second array (1D) to store the pointers - An (integer) variable to hold the start pointer and an (integer) variable to store the next free pointer ALTERNATIVE: - Define a record type comprising a data element and a pointer and declare an array (1D) of this type - An integer variable to hold the start pointer and an integer variable to store the next free pointer	2
------	--	----------

104. M/J 21/P22/Q1/a

1(a)(i)	<table><tr><th>Variable</th><th>Example data value</th><th>Data type</th></tr><tr><td>Name</td><td>"Catherine"</td><td>STRING</td></tr><tr><td>Index</td><td>100</td><td>INTEGER</td></tr><tr><td>Modified</td><td>FALSE</td><td>BOOLEAN</td></tr><tr><td>Holiday</td><td>25/12/2020</td><td>DATE</td></tr></table>	Variable	Example data value	Data type	Name	"Catherine"	STRING	Index	100	INTEGER	Modified	FALSE	BOOLEAN	Holiday	25/12/2020	DATE	4
	Variable	Example data value	Data type														
	Name	"Catherine"	STRING														
	Index	100	INTEGER														
	Modified	FALSE	BOOLEAN														
	Holiday	25/12/2020	DATE														
One mark per data type																	

1(a)(ii)	<table><tr><th>Expression</th><th>Evaluates to</th></tr><tr><td>Modified OR Index > 100</td><td>FALSE</td></tr><tr><td>LENGTH("Student: " & Name)</td><td>18</td></tr><tr><td>INT(Index + 2.9)</td><td>102</td></tr><tr><td>MID(Name, 1, 3)</td><td>"Cat"</td></tr></table>	Expression	Evaluates to	Modified OR Index > 100	FALSE	LENGTH("Student: " & Name)	18	INT(Index + 2.9)	102	MID(Name, 1, 3)	"Cat"	4
	Expression	Evaluates to										
	Modified OR Index > 100	FALSE										
	LENGTH("Student: " & Name)	18										
	INT(Index + 2.9)	102										
	MID(Name, 1, 3)	"Cat"										
One mark per value												
Quotation marks must be present for final row and must be capital C												

105. M/J 21/P22/Q3/a,b,c,d

3(a)	Linked list	1
3(b)	Start pointer	1
3(c)	One mark for each: Name: Null pointer Meaning: There are no further nodes in the list	2

3(d)	One mark for: - Start Pointer pointing to 'Cat' node - Remaining arrows: Cat → Dolphin → Elk → Fish	2
------	---	---

106. M/J 21/P22/Q5/a

5(a)(i)	<pre> DECLARE RNum : ARRAY[1:100] OF INTEGER DECLARE Index, Count : INTEGER Count ← 0 FOR Index ← 1 TO 100 RNum[Index] ← INT(RAND(200)) + 1 IF RNum[Index] >= 66 AND RNum[Index] <= 173 THEN Count ← Count + 1 ENDIF NEXT Index OUTPUT Count </pre> Mark as follows: - Array declaration - Loop for 100 iterations - Array element index 'syntax' (left-hand side of assignment expression) in a loop - Use of <code>RAND()</code> to generate value in range (and assign to array element) in a loop - Check if random number within range and if so, increment count in a loop - Output of count (following a reasonable attempt) after the loop	6
5(a)(ii)	One mark per bullet / sub-bullet point - Initialise the array to a rogue value (to indicate 'unassigned' element) - Add a conditional loop to: - Generate and store a random number (in the correct range) - Check the stored number against values already in the array - If the stored number is found then generate another random value - Otherwise add it to the array (and exit loop) Note: Max 3 marks	3

107. M/J 21/P22/Q6

6	<pre> PROCEDURE CountVowels(ThisString : STRING) DECLARE Index : INTEGER DECLARE ThisChar : CHAR FOR Index ← 1 to 6 CharCount[Index] ← 0 //initialise elements NEXT Index Index ← 1 FOR Index ← 1 TO LENGTH(ThisString) ThisChar ← LCASE(MID(ThisString, Index, 1)) CASE OF ThisChar 'a' : CharCount[1] ← CharCount[1] + 1 'e' : CharCount[2] ← CharCount[2] + 1 'i' : CharCount[3] ← CharCount[3] + 1 'o' : CharCount[4] ← CharCount[4] + 1 'u' : CharCount[5] ← CharCount[5] + 1 'a' TO 'z': CharCount[6] ← CharCount[6] + 1 ENDCASE NEXT Index FOR Index ← 1 to 6 OUTPUT CharCount[Index] //output results NEXT Index ENDPROCEDURE </pre> 1 mark for each of the following: 1 Procedure heading (with parameter) and ending 2 Declare local variable for Index as loop counter but not CharCount array 3 Initialise elements of CharCount array to zero 4 Loop through all characters in ThisString 5 Use of MID() to extract single character 6 Test for each vowel and increment associated count 7 Test for consonants and increment associated count 8 Output the results (supporting text not necessary) after the loop Note: Max 7 if CharCount not used to store count	8
---	---	---

108. M/J 21/P22/Q8/a

8(a)	<pre> FUNCTION IgnoreWord (ThisWord : STRING) RETURNS BOOLEAN DECLARE Found : BOOLEAN DECLARE Index : INTEGER Found ← False Index ← 1 ThisWord ← TO_LOWER(ThisWord) REPEAT IF TO_LOWER(IgnoreList[Index]) = ThisWord THEN Found ← TRUE ENDIF Index ← Index + 1 UNTIL Found = TRUE OR Index > 10 RETURN Found ENDFUNCTION </pre> 1 mark for each of the following: 1 Loop through array elements 2 Convert both strings to same case 3 Compare array element with parameter in a loop 4 Set a flag (or similar) if match found (after reasonable attempt at MP3) in a loop 5 Return TRUE or FALSE in all cases Note: Max 4 if function declaration incorrect	5
------	--	----------

109. M/J 21/P23/Q1/c

1(c)	Structure: • Record Justification: • Allows different data <u>types</u> • to be stored under one <u>identifier</u> Mark as follows: One mark for Structure Two marks for justification. Alternative • Two (1D) arrays • One of string, one of integer • Where same index links the name with the score	3
------	---	----------

110. M/J 21/P23/Q3/a,b

3(a)(i)	One mark per bullet point: - Data from the arrays is written to the <u>files</u> at the end of the day / before the program is terminated / computer is switched off - Data can then be read from the <u>files</u> at the start of the next day and written to / stored in the <u>arrays</u> - No need to (re-)enter the data manually // only need to enter data once Note: Max 2 marks	2
3(a)(ii)	The data is retained when the program is terminated / after the computer is switched off // data is stored permanently // non-volatile storage	1
3(a)(iii)	One mark per bullet point: - Data items are combined to form a single string / saved as a single line in the file - Data items are separated by a special character // make each data item a fixed length ALTERNATIVE: - Convert all data items / 'number of people' to strings - Consecutive / each line stores a separate data item	2
3(b)	<pre> PROCEDURE Preview (ThisFile : STRING) DECLARE LineNum : INTEGER DECLARE ThisLine : STRING OPENFILE ThisFile FOR READ IF EOF(ThisFile) THEN OUTPUT "Warning Message" ELSE LineNum ← 1 WHILE LineNum < 6 AND NOT EOF(ThisFile) READFILE Thisfile, ThisLine OUTPUT ThisLine LineNum ← LineNum + 1 ENDWHILE ENDIF CLOSEFILE ThisFile ENDPROCEDURE </pre> Marks as follows: - Procedure heading (including parameter) and ending - File OPEN and subsequently CLOSE - Check if file is empty and output a warning message if it is - Conditional Loop - Output line (including blank lines) and read next line in a loop	5

111. M/J 21/P23/Q5

5	<pre> PROCEDURE Sort () DECLARE Temp : INTEGER DECLARE NoSwaps : BOOLEAN DECLARE Boundary, Row, Col : INTEGER Boundary ← 999 REPEAT NoSwaps ← TRUE FOR Row ← 1 TO Boundary IF Result[Row, 2] > Result[Row + 1, 2] THEN FOR Col ← 1 TO 2 Temp ← Result [Row, Col] Result [Row, Col] ← Result [Row + 1, Col] Result [Row + 1, Col] ← Temp NEXT Col NoSwaps ← FALSE ENDIF NEXT J Boundary ← Boundary - 1 UNTIL NoSwaps = TRUE ENDPROCEDURE </pre> Mark as follows: 1 Outer loop 2 Inner loop 3 Correct comparison in a loop 4 Correct swap of col1 array elements in a loop 5 Correct swap of col2 array elements in a loop (via loop or separate statements) 6 'NoSwap' mechanism: Conditional outer loop including flag reset 7 'NoSwap' mechanism: Set flag in inner loop to indicate swap 8 Reducing Boundary in the <u>outer</u> loop	8
---	--	---

112. M/J 21/P23/Q6/a,b

6(a)	One mark per point: 1 Check for a free node 2 Search for correct insertion point 3 Assign data value B to first node in free list / node pointed to by start pointer of free list 4 Pointer from A will be changed to point to node containing B (instead of C) 5 Pointer from B will be changed to point to node containing C 6 Start pointer in free list moved to point to next free node Note: max 4 marks	4
------	--	---

6(b)	One mark per point: • An array (1D) to store the data and a second array (1D) to store the pointers • An (integer) variable to hold the start pointer and an (integer) variable to store the next free pointer ALTERNATIVE: • Define a record type comprising a data element and a pointer and declare an array (1D) of this type • An integer variable to hold the start pointer and an integer variable to store the next free pointer	2
------	--	----------

113. 9608 M/J 21/P21/Q1/b

1(b)(i)	One mark for both answers correct. Exact terms only. • Lower bound • Upper bound	1
1(b)(ii)	index / subscript	1

114. 9608 M/J 21/P21/Q4/a

4(a)	'Pseudocode' solution included here for development and clarification of mark scheme. Programming language solutions appear in the Appendix. <pre> PROCEDURE ScanArray(SearchString STRING) DECLARE Index, Total : INTEGER DECLARE Error : BOOLEAN Index ← 1 Total ← 0 Error ← FALSE WHILE Index <= 1000 AND Error <> TRUE IF LENGTH(ThisArray[Index]) > 5 THEN IF LEFT(ThisArray[Index], 4) = SearchString THEN Total ← Total + LENGTH(ThisArray[Index]) - 5 ENDIF Index ← Index + 1 ELSE Error ← TRUE ENDIF ENDWHILE IF Index > 1 THEN ArrayResult ← INT(Total / (Index - 1)) ENDIF </pre> ENDPROCEDURE Mark as follows: 1 Procedure header including parameter and end (where required) 2 Local variable declarations and initialisation of Index, Total and Error but no local declaration of ArrayResult 3 WHILE / ENDWHILE loop 4 Nested IF statement comparing first four character of array element with SearchString 5 Summation of Total using appropriate LENGTH function and subtracting 5 6 Assignment to ArrayResult using appropriate INT function AND check for division by zero	6
------	--	----------

Q4 (a): Visual Basic

```
Sub ScanArray(SearchString As String)

    Dim Index, Total As Integer
    Dim Error As Boolean

    Index = 1
    Total = 0
    Error = FALSE

    While Index <= 1000 And Error <> TRUE
        If Len(ThisArray(Index)) > 5 Then
            If Left(ThisArray(Index), 4) = SearchString Then
                Total = Total + Len(ThisArray(Index)) - 5
            End If
            Index = Index + 1
        Else
            Error = TRUE
        End If
    End While

    If Index > 1 Then
        ArrayResult = Int(Total / (Index - 1))
    End If

End Sub
```

Q4 (a): Pascal

```
procedure ScanArray(SearchString : string);

var
  Index, Total : integer;
  Error : boolean;

begin
  Index := 1;
  Total := 0;
  Error := FALSE;

  while Index <= 1000 And Error <> TRUE do
    begin
      if Length(ThisArray[Index]) > 5 then
        begin
          if LeftStr(ThisArray[Index], 4) = SearchString then
            Total := Total + Length(ThisArray[Index]) - 5;

            Index := Index + 1;
          else
            Error := TRUE;
          end;
        end;

      if Index > 1 then
        ArrayResult := int(Total / (Index - 1));
      end;
    end;
  end;
```

Q4(a): Python

```
def ScanArray(SearchString):

    ## Index, Total As Integer
    ## Error As Boolean

    Index = 1
    Total = 0
    Error = FALSE

    while Index <= 1000 and Error <> TRUE:
        if len(ThisArray[Index]) > 5:
            ThisElement = ThisArray[Index]
            if ThisElement[:4] == SearchString:
                Total = Total + len(ThisArray[Index]) - 5
                Index = Index + 1
            else:
                Error = TRUE

    if Index > 1:
        ArrayResult = int(Total / (Index - 1))
```


115. 9608 M/J 21/P21/Q6/a,b,c

6(a)	'Pseudocode' solution included here for development and clarification of the mark scheme. Programming language example solutions appear in the Appendix. <pre>FUNCTION Check(Index : INTEGER) RETURNS BOOLEAN IF LENGTH(StockID[Index]) <> 8 OR ____ Description[Index] = "" OR ____ Quantity[Index] < 0 THEN RETURN FALSE ELSE RETURN TRUE ENDIF ENDFUNCTION</pre> One mark for each of the following: 1 Function heading and ending (where appropriate) 2 Three comparisons... 3 ... connected by logical OR // AND / correct nested IF 4 RETURN value in both cases	4
------	---	----------

Q6 (a): Visual Basic

```
Function Check(Index As Integer) As Boolean

    If Len(StockID(Index)) <> 8 Or _
       Description(Index) = "" Or _
       Quantity(Index) < 0 Then
        Return FALSE
    Else
        Return TRUE
    End If

End Function
```

Q6(a): Pascal

```
function Check(Index : Integer) : boolean;

begin

    if Length(StockID[Index]) <> 8 Or
       Description[Index] = "" Or
       Quantity[Index] < 0 then
        Check := FALSE // result := FALSE
    else
        Check := TRUE // result := TRUE
    end;

end
```

Q6(a): Python

```
def Check(Index):

    if len(StockID[Index]) <> 8 or \
       Description[Index] == "" or \
       Quantity[Index] < 0:
        return FALSE
    else:
        return TRUE
```

6(b)	<pre> FUNCTION Backup() RETURNS BOOLEAN DECLARE Index : INTEGER DECLARE FileName, FileLine : STRING DECLARE ALLOK : BOOLEAN CONSTANT ASTERISK = '*' ALLOK ← TRUE FileName ← GetValidFileName() OPENFILE Filename FOR WRITE OPENFILE "ERRORLOG.TXT" FOR WRITE FOR Index ← 1 TO 10000 IF StockID[Index] <> "" THEN FileLine ← StockID[Index] & ASTERISK FileLine ← FileLine & Description[Index] & ASTERISK FileLine ← FileLine & NUM_TO_STRING(Quantity[Index]) & ASTERISK FileLine ← FileLine & NUM_TO_STRING(Cost[Index]) WRITEFILE FileName, FileLine //now check for sensible data IF Check(Index) <> TRUE THEN WRITEFILE, "ERRORLOG.TXT", FileLine ALLOK ← FALSE ENDIF ENDIF ENDFOR CLOSEFILE FileName CLOSEFILE "ERRORLOG.TXT" RETURN ALLOK ENDFUNCTION </pre> 1 mark for each of the following: 1 Declare local variable for backup filename and index 2 Call to function GetValidFileName() 3 OPEN and CLOSE both files 4 Loop all 10 000 elements 5 Form FileLine using at least one correct array index expression and asterisk 6 Use of NUM_TO_STRING() to convert at least one of QUANTITY or COST 7 Write line to backup file 8 Call Check() to determine whether values are valid and if not, write to ERRORLOG.TXT 9 Return ALLOK Max 8 marks from possible 10 mark points	8
------	--	---

6(c)	'Pseudocode' solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix. <pre> PROCEDURE Unpack(Index : INTEGER, FileLine : STRING) DECLARE Pointer : INTEGER DECLARE NextChar : CHAR DECLARE TempString : STRING CONSTANT ASTERISK = '*' StockID[Index] ← LEFT(FileLine, 8) // the only fixed length field Pointer ← 10 // Point to start of Description (skip the '*') NextChar ← MID(FileLine, Pointer, 1) TempString ← "" WHILE NextChar <> ASTERISK TempString ← TempString & NextChar Pointer ← Pointer + 1 NextChar ← MID(FileLine, Pointer, 1) ENDWHILE Description[Index] ← TempString Pointer ← Pointer + 1 NextChar ← MID(FileLine, Pointer, 1) TempString ← "" WHILE NextChar <> ASTERISK TempString ← TempString & NextChar Pointer ← Pointer + 1 NextChar ← MID(FileLine, Pointer, 1) ENDWHILE Quantity[Index] ← STRING_TO_NUM(TempString) TempString ← RIGHT(FileLine, LENGTH(FileLine) - Pointer) Cost[Index] ← STRING_TO_NUM(TempString) ENDFUNCTION </pre> 1 mark for each of the following: 1 Procedure heading with parameters 2 Extract first 8 chars of FileLine 3 Assign to StockID 4 Search for asterisk for place separator 5 Extract Description string and assign to Description array 6 Extract Quantity string, and assign to Quantity array... 7 Extract Cost string and assign to Cost array 8 Type conversion for Cost and Quantity	8
------	--	---

Q6(c): Visual Basic

```

Sub Unpack(Index As Integer, FileLine As String)

    Dim Pointer As Integer
    Dim NextChar As Char
    Dim TempString As String
    Const ASTERISK = '*'

    StockID(Index) = LEFT(FileLine, 8)

    Pointer = 10           'point to start of Description (skip the '*')
    NextChar = Mid(FileLine, Pointer, 1)
    TempString = ""
    While NextChar <> ASTERISK
        TempString = TempString & NextChar
        Pointer = Pointer + 1
        NextChar = Mid(FileLine, Pointer, 1)
    End While

    Description(Index) = TempString
    Pointer = Pointer + 1
    NextChar = Mid(FileLine, Pointer, 1)

    TempString = ""
    While NextChar <> ASTERISK
        TempString = TempString & NextChar
        Pointer = Pointer + 1
        NextChar = Mid(FileLine, Pointer, 1)
    End While

    Quantity(Index) = CInt(TempString)
    TempString = Right(FileLine, Len(FileLine) - Pointer)
    Cost(Index) = CDec(TempString)

End Sub

```

Q6(c): Pascal

```

procedure Unpack(Index : Integer, FileLine : String);

var
    Pointer : integer;
    NextChar : char;
    TempString : string;

const
    ASTERISK = '*';

begin
    StockID[Index] := LeftStr(FileLine, 8);

    Pointer := 10;           //point to start of Description (skip the '*')
    NextChar := MidStr(FileLine, Pointer, 1);
    TempString := "";
    while NextChar <> ASTERISK do

```

```

begin
    TempString := TempString & NextChar;
    Pointer := Pointer + 1;
    NextChar := MidStr(FileLine, Pointer, 1);
end;

Description[Index] := TempString;
Pointer := Pointer + 1;
NextChar := MidStr(FileLine, Pointer, 1);

TempString := "";
while NextChar <> ASTERISK do
begin
    TempString := TempString & NextChar;
    Pointer := Pointer + 1;
    NextChar := MidStr(FileLine, Pointer, 1);
end;

Quantity[Index] := StrToInt(TempString);
TempString := RightStr(FileLine, Length(FileLine) - Pointer);
Cost[Index] := StrToFloat(TempString);

end;

```

Q6(c): Python

```

def Unpack(Index, FileLine):

    ## Pointer As Integer
    ## NextChar As Char
    ## TempString As String
    ASTERISK = '*'

    StockID[Index] = FileLine[:8] #characters 0 to 7

    Pointer = 9          #point to start of Description (skip the '*')
    NextChar = FileLine[Pointer]
    TempString = ""

    while NextChar <> ASTERISK:
        TempString = TempString + NextChar
        Pointer = Pointer + 1
        NextChar = FileLine[Pointer]

    Description[Index] = TempString
    Pointer = Pointer + 1
    NextChar = FileLine[Pointer]

    TempString = ""

    while NextChar <> ASTERISK:
        TempString = TempString + NextChar
        Pointer = Pointer + 1
        NextChar = FileLine[Pointer]

    Quantity[Index] = int(TempString)
    TempString = FileLine[Len(FileLine) - Pointer - 1:]

```

```
Cost[Index] = float(TempString)
```

Alternative

```
def Unpack(Index, FileLine):  
    ## TempString As String  
  
    StockID[Index] = FileLine[:8]  
    TempString = FileLine[8:] // remove first 8 characters  
    Description[Index], Quantity[Index], Cost[Index] =  
    (TempString.split('*'))
```

116. 9608 M/J 21/P22/Q1/c

1(c)	One mark per bullet point: • To provide a <u>separator</u> (or implied) between the data items • Algorithm to extract / locate individual items from a line of text is simplified • The special character does not appear in the data Note: Max 2 marks	2
------	--	----------

117. 9608 M/J 21/P22/Q4/b

4(b)	'Pseudocode' solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix. <pre> FUNCTION Search(Par1, Par2 : STRING) RETURNS INTEGER DECLARE Index, RetVal : INTEGER Index ← 1 RetVal ← -1 WHILE Index <= 100 AND RetVal = -1 IF XRef[Index, 1] = Par1 _ AND (XRef[Index, 2] = Par2 OR XRef[Index, 3] = Par2) THEN RetVal ← Index ENDIF Index ← Index + 1 ENDWHILE RETURN RetVal ENDFUNCTION </pre> Mark as follows: 1 Function heading and ending including parameters 2 Declaration of local variable for array index (Index) but not of global XRef array 3 Any loop for 100 elements 4 Use of correct XRef 2D array syntax in a loop 5 Logical test of array elements in a loop 6 Exit loop if match found (following attempt at MP5) Return index number or -1 if not found	7
------	--	----------

Question 4(b): Visual Basic

```

Function Search(Par1, Par2 As STRING) As INTEGER

Dim Index, RetVal As INTEGER

Index = 1
RetVal = -1

While Index <= 100 And RetVal = -1
    If XRef(Index, 1) = Par1 _
        AND (XRef(Index, 2) = Par2 OR XRef(Index, 3) = Par2) Then
        RetVal = Index
    End If
    Index = Index + 1
End While

Return RetVal
End Function

```

Question 4(b): Pascal

```

function Search(Par1, Par2 : string) : integer;

var
    Index : integer;
    RetVal : integer;

begin

    Index := 1;
    RetVal := -1;

    While Index <= 100 And RetVal = -1 do
        begin
            if XRef[Index, 1] = Par1
                and (XRef[Index, 2] = Par2 or XRef[Index, 3] = Par2) then
                RetVal := Index;

                Index := Index + 1;
            end;

            Search := RetVal // result := RetVal
        end;
    end;

```

Question 4(b): Python

```

def Search(Par1, Par2):

    ## Index, RetVal As INTEGER

    Index = 1
    RetVal = -1

    while Index <= 100 and RetVal = -1:
        if XRef[Index][1] == Par1 \
            and XRef[Index][2] == Par2 or XRef[Index][3] == Par2:
            RetVal = Index
            Index = Index + 1

    return RetVal

```

118. 9608 M/J 21/P22/Q6/a,b

6(a)	'Pseudocode' solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix. <pre> PROCEDURE StockSummary() DECLARE Index, Count : INTEGER DECLARE Total : REAL Count ← 0 Total ← 0.0 FOR Index ← 1 TO 10000 IF StockID[Index] <> "" THEN Count ← Count + 1 Total ← Total + (Quantity[Index] * Cost[Index]) ENDIF ENDFOR OUTPUT "Total Value is ", Total OUTPUT "Number of Stock Entries is ", Count ENDPROCEDURE </pre> One mark for each of the following: 1 Declaration and initialisation of Count and Total (by comment in Python) 2 Loop for 10000 elements 3 Skip row when StockID element = "" in a loop 4 Increment Count and sum Total in a loop 5 OUTPUT the total and the count after the loop (following a reasonable attempt at MP4)	5
------	---	----------

Question 6(a): Visual Basic

```

Sub StockSummary()
    Dim Index, Count As Integer
    Dim Total As Real

    Count = 0
    Total = 0.0

    For Index = 1 To 10000
        If StockID(Index) <> "" then
            Count = Count + 1
            Total = Total + (Quantity(Index)) * Cost(Index)
        End If
    Next Index

    Console.WriteLine("Total Value is " & Total.ToString)
    Console.WriteLine("Number of Stock Entries is " & Count.ToString)

End Sub

```

Question 6(a): Pascal

```

procedure StockSummary();

var
    Index, Count : Integer;
    Total : Real;

begin
    Count := 0;
    Total := 0.0;

    for Index := 1 TO 10000 do
        begin
            if StockID[Index] <> "" then
                begin
                    Count := Count + 1;
                    Total := Total + (Quantity[Index]) * Cost[Index];
                end;
            end;
        end;

    writeln('Total Value is ', Total);
    writeln('Number of Stock Entries is ', Count);

end;

```

Question 6(a): Python

```

def StockSummary():

    ## Index, Count : Integer
    ## Total : Real

    Count = 0
    Total = 0.0

    for Index in range(1, 10001):
        if StockID[Index] <> "":
            Count = Count + 1
            Total = Total + (Quantity[Index]) * Cost[Index]

    print("Total Value is ", Total)
    print("Number of Stock Entries is ", Count)

```

6(b)	<pre> FUNCTION Restore(Filename : STRING) RETURNS BOOLEAN DECLARE Index : INTEGER DECLARE FileLine : STRING DECLARE Success : BOOLEAN Success ← TRUE OPENFILE Filename FOR READ READFILE Filename, FileLine IF FileLine = "" THEN //alt: IF EOF(filename) CLOSEFILE Filename RETURN FALSE //file is empty ENDIF FOR Index ← 1 TO 10000 //first initialise arrays StockID[Index] ← "" Description[Index] ← "" Quantity[Index] ← 0 Cost[Index] ← 0.0 // 0 ENDFOR Index ← 1 //starts loop with first FileLine WHILE NOT EOF(Filename) AND Index <= 10000 CALL Unpack(Index, FileLine) Index ← Index + 1 READFILE Filename, FileLine ENDWHILE IF Index = 10001 AND NOT EOF(Filename) THEN Success ← FALSE ENDIF CLOSEFILE Filename RETURN Success ENDFUNCTION </pre> 1 mark for each of the following: 1 Function heading including input parameter and function End 2 Declare local variable for line read from file 3 OPEN file in READ mode and CLOSE 4 Check whether file is empty and return FALSE if it is (no restore 5 Loop through all 10 000 elements initialising arrays 6 Loop until EOF(Filename) OR Index > 10000 7 Call Unpack() correctly for each line from the file in a loop 8 Return FALSE if more than 10000 lines in file, otherwise returns TRUE	8
------	--	---

119. 9608 M/J 21/P23/Q6/a,b,c

6(a)	<pre> FUNCTION CheckBackupFile() RETURNS STRING DECLARE Filename, FileLine, Response : STRING Filename ← "" WHILE Filename = "" Filename ← GetValidFilename() OPENFILE Filename FOR READ READFILE Filename, FileLine CLOSEFILE Filename IF FileLine <> "" //check if data in file THEN OUTPUT "File already exists - do you want to overwrite? " INPUT Response IF Response <> "Yes" THEN Filename ← "" OUTPUT "Please input a different filename " ENDIF ENDIF ENDWHILE RETURN Filename ENDFUNCTION </pre> One mark for each of the following: 1 Conditional loop 2 Use of GetValidFilename() in a loop 3 OPEN file in READ mode and CLOSE in a loop 4 Test if file not empty (using EOF() or READ empty string) 5 If not empty, prompt and input (in case of a non-empty file) in a loop 6 and process response 7 Set loop termination condition by checking for new file or overwrite confirmed in a loop 8 Return Filename	8
------	--	----------

6(b)	'Pseudocode' solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix <pre> PROCEDURE GroupReport (Group : STRING) DECLARE Total : REAL DECLARE Count, Index : INTEGER Total ← 0 Count ← 0 FOR Index ← 1 TO 10000 IF LEFT(StockID[Index], 4) = Group THEN Count ← Count + 1 Total ← Total + (Quantity[Index] * Cost[Index]) ENDIF ENDFOR IF Count = 0 THEN OUTPUT "There are no items in Group: ", Group ELSE OUTPUT "Group: ", Group OUTPUT "Number of items in Group: ", Count OUTPUT "Total value of items in Group: ", Total ENDIF ENDPROCEDURE </pre> 1 mark for each of the following: 1 Procedure heading and ending including input parameter 2 Declare and initialise local variables for Total and Count 3 Loop through all 10 000 elements (allow LEN(StockID)) 4 Check for required group using substring function in a loop 5 ...Increment Count and sum Total using correct array notation 6 Generate both sets of output as appropriate after loop	6
------	---	---

Question 6(b): Visual Basic

```

Sub GroupReport(Group As String)

    Dim Total As Double
    Dim Count, Index As Integer

    Total = 0
    Count = 0

    For Index = 0 To 3 \1 to 10000
        If Left(StockID(Index), 4) = Group Then
            Count = Count + 1
            Total = Total + (Quantity(Index) * Cost(Index))
        End If
    Next Index

    If Count = 0 Then
        Console.WriteLine("There are no items in Group: " & Group)
    Else
        Console.WriteLine("Group: " & Group)
        Console.WriteLine("Number of items in Group: " & Count)
        Console.WriteLine("Total value of items in Group: " & Total)
    End If

End Sub

```

Question 6(b): Pascal

```

procedure GroupReport(Group : string);

var
    Total : real;
    Count, Index : integer;

begin
    Total := 0;
    Count := 0;

    for Index := 1 TO 10000 do
        begin
            if LeftStr(StockID[Index], 4) = Group then
                begin
                    Count := Count + 1;
                    Total := Total + (Quantity[Index] * Cost[Index]);
                end;
            end;
        end;

    if Count = 0 then
        writeln('There are no items in Group: ', Group)
    else
        begin
            writeln('Group: ', Group);
            writeln('Number of items in Group: ', Count);
            writeln('Total value of items in Group: ', Total);
        end;
    end;

end;

```

Question 6(b): Python

```
def GroupReport(Group):  
  
    ## Total As Real  
    ## Count, Index As Integer  
    ## ThisID As String  
  
    Total = 0  
    Count = 0  
  
    for Index in range(1, 10001):  
        ThisID = StockID[Index]  
        if ThisID[:4] == Group:  
            Count = Count + 1  
            Total = Total + (Quantity[Index] * Cost[Index])  
  
    if Count == 0:  
        print("There are no items in Group: ", Group)  
    else:  
        print("Group: ", Group)  
        print("Number of items in Group: ", Count)  
        print("Total value of items in Group: ", Total)
```


6(c)	'Pseudocode' solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix <pre> FUNCTION GroupSummary() RETURNS INTEGER DECLARE Index, GroupIndex Total : INTEGER DECLARE ThisGroup : STRING Total ← 0 FOR Index ← 1 TO 10000 IF StockID[Index] <> "" THEN ThisGroup ← LEFT(StockID[Index], 4) GroupIndex ← Lookup(ThisGroup) IF GroupIndex = -1 //ThisGroup not found THEN //add new group Summary[Total + 1] ← ThisGroup Total ← Total + 1 ENDIF ENDIF ENDFOR RETURN Total ENDFUNCTION </pre> 1 mark for each of the following: 1 Function heading and ending and final return of Total 2 Declare and initialise Total 3 Loop through all 10 000 elements 4 Skip empty StockID in a loop 5 Extract ThisGroup from StockID and pass to Lookup() 6 ... if Lookup() returns -1 (following reasonable attempt at MP5) 7Store ThisGroup to Summary[Total] and increment Total	7
------	---	---

Question 6(c): Visual Basic

```
Function GroupSummary() As Integer

    Dim Index, GroupIndex, Total As Integer
    Dim ThisGroup As String

    Total = 0

    For Index = 1 TO 10000
        If StockID(Index) <> "" Then
            ThisGroup = Left(StockID(Index), 4)
            GroupIndex = Lookup(ThisGroup)
            If GroupIndex = -1 Then                // ThisGroup not found
                Summary(Total + 1) = ThisGroup    // Add new Group
                Total = Total + 1
            End If
        End If
    Next Index

    Return Total

End Function
```

Question 6(c): Pascal

```

function GroupSummary() : Integer;

var
  Index, GroupIndex, Total : Integer;
  ThisGroup : String;

begin
  Total := 0;

  for Index := 1 TO 10000 do
    begin
      if StockID[Index] <> "" then
        begin
          ThisGroup := LeftStr(StockID[Index], 4);
          GroupIndex := Lookup(ThisGroup);
          If GroupIndex = -1 then           //ThisGroup not found
            begin
              Summary[Total + 1] := ThisGroup; //Add new Group
              Total := Total + 1;
            end;
          end;
        end;
      end;

    GroupSummary := Total; // result := Total;

  end;

```

Question 6(c): Python

```

def GroupSummary():

    ## Index, GroupIndex, Total As Integer
    ## ThisGroup, ThisID As String

    Total = 0

    for Index in range(1, 10001):
        ThisID = StockID[Index]
        if ThisID != "":
            ThisGroup = ThisID[:4]
            GroupIndex = Lookup(ThisGroup)
            if GroupIndex == -1:           #ThisGroup not found
                Summary[Total + 1] = ThisGroup    #Add new Group
                Total = Total + 1

    return Total

```

120. M/J 20/P21/Q3/b

3(b)	<pre> PROCEDURE Search(SearchString : STRING) DECLARE Index, Msg : STRING Msg ← "Found at:" //initial value FOR Index ← 1 TO 100 IF NameList[Index, 1] = SearchString__ AND NameList[Index, 2] = "Active" THEN Msg ← Msg & " " & NUM_TO_STRING(Index) ENDIF ENDFOR IF Msg = "Found at:" // no change to initial value THEN OUTPUT "Search String not found" ELSE OUTPUT Msg ENDIF ENDPROCEDURE </pre> 1 mark for each of the following: 1 PROCEDURE heading and ending including parameter 2 Declare local variables for Index and Msg and initialise Msg to appropriate string 3 Loop structure 4 Compare SearchString to name (column 1)... 5 ... AND Compare status to "Active" (column 2) in a loop 6 Add Index to Msg when a match is encountered (using type conversion) 7 Condition to determine which string is output after loop 8 Correct output of single message Note: Credit alternative solutions for forming and checking a single output string	8
------	---	----------

121. M/J 20/P21/Q5/a,b

5(a)(i)	<pre> PROCEDURE SortContacts() DECLARE Temp : STRING DECLARE FirstName, SecondName : STRING DECLARE NoSwaps : BOOLEAN DECLARE Boundary, J : INTEGER Boundary ← 999 REPEAT NoSwaps ← TRUE FOR J ← 1 TO Boundary FirstName ← RIGHT(Directory[J], __ LENGTH(Directory[J]) - 4) SecondName ← RIGHT(Directory[J + 1], __ LENGTH(Directory[J + 1]) - 4) IF FirstName > SecondName THEN Temp ← Directory[J] Directory[J] ← Directory[J + 1] Directory[J + 1] ← Temp NoSwaps ← FALSE ENDIF ENDFOR Boundary ← Boundary - 1 UNTIL NoSwaps = TRUE ENDPROCEDURE </pre> One mark per highlighted phrase	8
5(b)	Description: • uses a flag variable to stop the outer loop • after no more swaps made during one pass of the inner loop • the flag is reset before the inner loop starts, and set whenever a swap is made • decreases the loop size at end of inner loop (Boundary decremented) Max 3 for description Effective because: • It prevents unnecessary iterations / passes through the array (i.e. when the array is already sorted) // terminates the algorithm when all elements are in order // reduces the number of unnecessary comparisons	4

122. M/J 20/P21/Q6/a,b

6(a)	<pre> PROCEDURE ListAvailable(StartTime : STRING) DECLARE NumAvailable, Index : INTEGER DECLARE TimeBack : STRING DECLARE Available : BOOLEAN NumAvailable ← 0 FOR Index ← 1 TO 10 Available ← FALSE // initialise IF HireTime[Index] = "Available" // not on hire THEN Available ← TRUE // available now ELSE TimeBack ← AddTime(HireTime[Index], __ Duration[Index]) IF TimeBack < StartTime // < or <= THEN Available ← TRUE // will be available ENDIF ENDIF ENDIF IF Available = TRUE THEN OUTPUT "Boat " , Index , " is available" NumAvailable ← NumAvailable + 1 ENDIF ENDIF ENDFOR IF NumAvailable > 0 THEN OUTPUT "There are " , NumAvailable , __ " boats available." ELSE OUTPUT "Sorry, there are no boats available" ENDIF ENDIF ENDPROCEDURE </pre> 1 mark for each of the following: 1 Procedure heading and ending including input parameter 2 Declare local variable for the count of available boats and initialise to 0 3 Loop through all 10 boats 4 Use of AddTime() to calculate TimeBack 5 Check for boats that are not on hire OR those due back in time in a loop 6 Increment count for number of available boats in a loop 7 Output a message for each available boat in a loop 8 Output both messages as appropriate outside a loop	8
------	---	----------

6(b)	'Pseudocode' solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix. <pre> PROCEDURE RecordHire(HBoatNumber, HDuration : INTEGER, __ HTime : STRING, HCost : REAL) DECLARE FileLine : STRING CONSTANT Comma = ',' HireTime[HBoatNumber] ← HTime Duration[HBoatNumber] ← HDuration Cost[HBoatNumber] ← HCost DailyTakings ← DailyTakings + HCost OPENFILE "HireLog.txt" FOR APPEND FileLine ← NUM_TO_STRING(HBoatNumber) & Comma FileLine ← FileLine & HTime & Comma FileLine ← FileLine & NUM_TO_STRING(HDuration) FileLine ← FileLine & Comma & NUM_TO_STRING(HCost) WRITEFILE "HireLog.txt", FileLine CLOSEFILE "HireLog.txt" ENDFUNCTION </pre> One mark for each of the following: 1 Procedure heading and ending (where appropriate), including input parameters (order not essential) 2 Updating the three arrays from parameter values 3 Totalling DailyTakings 4 OPEN "HireLog.txt" in append mode 5 Creating file text line including separators 6making use of type conversion as required 7 Writing the line to the file 8 Closing the file Solutions may combine mark points 5 and 6 (and 7) Max 7	7
------	--	---

Q6(b)(i): Visual Basic

```
Sub RecordHire(HBoatNumber, HDuration As Integer, HTime As String, HCost As Real)
```

```
    Dim FileLine As String
```

```
    Const Comma = ','
```

```
    HireTime(HBoatNumber) = HTime
```

```
    Duration(HBoatNumber) = HDuration
```

```
    Cost(HBoatNumber) = HCost
```

```
    DailyTakings = DailyTakings + HCost
```

```
    FileOpen(1, "HireLog.txt", OpenMode.Append)
```

```
    FileLine = CStr(HBoatNumber) & Comma
```

```
    FileLine = FileLine & HTime & Comma
```

```
    FileLine = FileLine & CStr(HDuration) & Comma
```

```
    FileLine = FileLine & CStr(HCost)
```

```
    Print(1, FileLine)
```

```
    PrintLine(1)
```

```
    Fileclose(1)
```

```
End Sub
```


Q6(b)(i): Pascal

```
procedure RecordHire(HBoatNumber, HDuration : integer; HTime : string;
HCost : Real);

var
  Fileline : string;
  ThisFile: TextFile;

const Comma = ',';

begin

  HireTime[HBoatNumber] := HTime;
  Duration[HBoatNumber] := HDuration;
  Cost[HBoatNumber] := HCost;

  DailyTakings := DailyTakings + HCost;

  AssignFile(Thisfile, "HireLog.txt");

  FileLine := IntToStr(HBoatNumber) + Comma;
  FileLine := FileLine + HTime + Comma;
  FileLine := FileLine + IntToStr (HDuration) + Comma;
  FileLine := FileLine + IntToStr (HCost);

  writeln(ThisFile, FileLine);

  CloseFile(ThisFile);

end;
```

Q6(b)(i): Python

```
def RecordHire(HBoatNumber, HDuration, HTime, HCost)

    # FileLine : String
    # File : File handle

    Comma = ','

    HireTime[HBoatNumber] = HTime
    Duration[HBoatNumber] = HDuration
    Cost[HBoatNumber] = HCost

    DailyTakings = DailyTakings + HCost

    File = Open("HireLog.txt", "a")

    FileLine = Str(HBoatNumber) + Comma
    FileLine = FileLine + HTime + Comma
    FileLine = FileLine + Str(HDuration) + Comma
    FileLine = FileLine + Str(HCost)

    File.write(FileLine)

    File.close
```

123. M/J 20/P22/Q4/a

4(a)	<pre> DECLARE Random : ARRAY [1:10] OF INTEGER DECLARE NextNum, Index, Rnum : INTEGER DECLARE Exists : BOOLEAN NextNum ← 1 // index position for the next random number REPEAT Rnum ← INT(RAND(100)) + 1 // from original question Exists ← FALSE FOR Index ← 1 to NextNum - 1 // search for Rnum IF Random[Index] = Rnum THEN Exists ← TRUE ENDIF ENDFOR IF Exists = FALSE THEN Random[NextNum] ← Rnum // store Rnum NextNum ← NextNum + 1 // increment index ENDIF UNTIL NextNum > 10 1 mark for each of the following: 1 Conditional (outer) loop to generate 10 values 2 Inner loop to search array for duplicate number 3 Check for duplicate by comparing number generated with array element in a loop 4 Avoid checking uninitialised elements // array initialisation to rogue value at start of algorithm 5 If Rnum is a duplicate then repeat outer loop 6 If Rnum not a duplicate then assign to array element and Increment index Notes: Max 5 if statement to generate random number (as given in Q) not present or incorrectly placed.</pre>	6
------	--	----------

124. M/J 20/P22/Q5/b

5(b)(i)	Description: • Reduce the number of items to be checked by one after each pass • Use a flag variable to stop the outer loop • ... after no more swaps made on a single pass of the inner loop • ... resetting before the inner loop starts, and setting it whenever a swap is made	4
---------	--	----------

5(b)(ii)	'Pseudocode' solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix. <pre> PROCEDURE BubbleSort () DECLARE Temp : STRING DECLARE NoSwaps : BOOLEAN DECLARE Boundary, J : INTEGER Boundary ← 999 REPEAT NoSwaps ← TRUE FOR J ← 1 TO Boundary IF Contact [J] > Contact [J+1] THEN Temp ← Contact [J] Contact [J] ← Contact [J+1] Contact [J+1] ← Temp NoSwaps ← FALSE ENDIF ENDFOR Boundary ← Boundary - 1 UNTIL NoSwaps = TRUE ENDPROCEDURE </pre> Mark as follows: 1 Procedure heading and ending 2 Outer loop 3 Inner loop 4 Correct comparison in a loop 5 Correct swap of array elements in a loop 6 'NoSwap' mechanism: Post-conditional outer loop including flag reset 7 'NoSwap' mechanism: Set flag in inner loop to indicate swap 8 Reducing Boundary in the <u>outer</u> loop	8
----------	---	----------

Q5(b)(i): Visual Basic

```
Sub BubbleSort()  
    Dim Temp As String  
    Dim NoSwaps As Boolean  
    Dim Boundary, J As Integer  
  
    Boundary = 999  
    Do  
        NoSwaps = TRUE  
        For J = 1 To Boundary  
            If Contact(J) > Contact(J+1) Then  
                Temp = Contact(J)  
                Contact(J) = Contact(J+1)  
                Contact(J+1) = Temp  
                NoSwaps = FALSE  
            End If  
        Next  
        Boundary = Boundary - 1  
    Loop Until NoSwaps = TRUE  
  
End Sub
```

Q5(b)(i): Pascal

```
procEDURE BubbleSort()  
var  
    Temp : String;  
    NoSwaps : Boolean;  
    Boundary, J : Integer;  
  
    Boundary := 999  
    repeat  
        begin  
            NoSwaps := TRUE  
            for J := 1 to Boundary do  
                begin  
                    if Contact[J] > Contact[J+1] then  
                        begin  
                            Temp := Contact[J];  
                            Contact[J] := Contact[J+1];  
                            Contact[J+1] := Temp;  
                            NoSwaps := FALSE;  
                        end;  
                    end;  
                end;  
            Boundary := Boundary - 1  
        end;  
    until NoSwaps = TRUE;  
  
End Sub
```

Q5(b)(i): Python

```
def BubbleSort()  
    # Temp : String  
    # NoSwaps : Boolean  
    # Boundary, J : Integer  
  
    Boundary = 999  
    NoSwaps = TRUE  
  
    while NoSwaps == TRUE:  
        NoSwaps = TRUE  
        For J in range(Boundary + 1)  
            If Contact[J] > Contact[J+1]:  
                Temp = Contact[J]  
                Contact[J] = Contact[J+1]  
                Contact[J+1] = Temp  
                NoSwaps = FALSE  
  
        Boundary = Boundary - 1  
  
End Sub
```

125. M/J 20/P22/Q6/c

6(c)	'Pseudocode' solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix. <pre> PROCEDURE GetTotals() DECLARE BoatNum : INTEGER DECLARE Paid : REAL DECLARE FileLine : STRING FOR BoatNum ← 1 TO 17 Total[BoatNum] ← 0 ENDFOR OPENFILE "Hirelog.txt" FOR READ WHILE NOT EOF("Hirelog.txt") READFILE "Hirelog.txt", FileLine BoatNum ← STRING_TO_NUM(LEFT(FileLine, 2)) Paid ← STRING_TO_NUM (RIGHT(FileLine, __ LENGTH(Fileline) - 8)) Total[BoatNum] ← Total[BoatNum] + Paid ENDWHILE CLOSEFILE "Hirelog.txt" ENDPROCEDURE </pre> One mark for each of the following: 1 Procedure heading and ending (where appropriate) with no parameters 2 Initialisation of elements in Total array 3 OPEN "Hirelog.txt" in read mode and CLOSE after use 4 Loop until EOF() 5 Read line from file in a loop 6 Extract and convert BoatNum 7 Extract and convert Paid 8 Update appropriate array total in a loop	8
------	--	----------

Q6(c): Visual Basic

```

Sub GetTotals()

    Dim BoatNum As Integer
    Dim Paid As Real
    Dim File As StreamReader("Hirelog.txt")

    For BoatNum = 1 To 17
        Total(BoatNum) = 0
    Next

    Do While File.Peek >= 0
        FileLine = File.ReadLine()
        BoatNum = CInt(Left(FileLine, 2))
        Paid = CSng(Right(FileLine, Len(FileLine) - 8))
        Total(boatnumber) = Total(boatbnumber) + Paid
    Loop

    File.Close()

End Sub

```

Q6(c): Pascal

```

procedure GetTotals()

var
    BoatNum : Integer;
    Paid : Real;
    MyFile : testfile;

    for BoatNum := 1 to 17 do
        Total[BoatNum] := 0;

    assignFile(MyFile, "Hirelog.txt");
    reset(MyFile);

    while not eof(MyFile) do
        begin
            readln(MyFile, FileLine);
            BoatNum = StrToInt(copy(FileLine, 1, 2));
            Paid = StrToFloat(copy(FileLine, 9, length(FileLine) - 8));
            Total(boatnumber) = Total(boatbnumber) + Paid;
        end;

        close(MyFile)

    end;

```

Alternative FreePascal string functions) :

```

    BoatNum := LeftStr(FileLine, 2);

    Paid := StrToFloat(RightStr(FileLine, length(FileLine) - 8));

```

Q6(c): Python

```
def GetTotals()

    # BoatNum : Integer
    # Paid : Real
    # File : File Handle
    # FileData : String

    For BoatNum in range (1, 18)
        Total[BoatNum] = 0
    Next

    File = open("Hirelog.txt", "r")
    FileData = File.readline()
    while FileData != "":
        FileLine = File.ReadLine()
        BoatNum = int(FileLine[1, 3])
        Paid = float(FileLine[8, len(Fileline) - 7))
        Total[boatnumber] = Total[boatbnumber] + Paid
        FileData = File.readline()

    File.Close()
```


126. M/J 20/P23/Q5/a,b(i)

5(a)	<pre> FUNCTION GetLastService(BoatNum : STRING) RETURNS STRING DECLARE LastService, FileData : STRING LastService ← "" OPEN "ServiceLog.txt" FOR READ WHILE NOT EOF("ServiceLog.txt") READFILE "ServiceLog.txt", FileData IF LEFT(FileData, 2) = BoatNum THEN LastService ← RIGHT(FileData, 8) ENDIF ENDWHILE Close "ServiceLog.txt" RETURN LastService // Most recent service will be the last one read ENDFUNCTION </pre> 1 mark for each of the following: 1 Function heading and ending including parameters 2 Declaration of variables for LastService and FileData 3 Open file in read mode and close 4 Conditional loop - while not EOF() 5 Read line from file in a loop 6 Compare BoatNum with data from file in a loop 7 If matched, assign value to LastService in a loop 8 Return LastService	8
------	--	----------

5(b)(i)	<pre> FUNCTION GetHours(BNum, SDate, : STRING) RETURNS REAL DECLARE FileData, HString : STRING DECLARE TotHours : REAL DECLARE HLength : INTEGER TotHours ← 0 OPEN "HireLog.txt" FOR READ WHILE NOT EOF("HireLog.txt") READFILE "HireLog.txt" FileData IF MID(FileData, 9, 2) = BNum THEN IF LEFT(FileData, 8) > SDate THEN HLength ← LENGTH(Filedata) - 10 HString ← RIGHT(Filedata, HLength) TotHours ← TotHours + STRING_TO_NUM(HString) ENDIF ENDIF ENDWHILE Close "HireLog.txt" RETURN TotHours ENDFUNCTION </pre> 1 mark for each of the following: 1 Function heading and ending including parameters 2 Declaration of TotHours as integer 3 ...and initialisation to zero 4 Extract and compare BoatNum ... 5 ... extract date and compare to ServData and if later then... 6 ... extract HireDuration and ... 7 ... convert HireDuration to REAL and sum TotHours 8 Return TotHours	8
---------	--	---

127. O/N 19/P21/Q1/b(i)

1(b)(i)	One mark per row		4
	Example value	Data type	
	"NOT TRUE"	STRING	
	−4.5	REAL	
	NOT FALSE	BOOLEAN	
	132	INTEGER	

128. O/N 19/P21/Q4/b

4(b)	<pre> DECLARE Index : INTEGER DECLARE Count : INTEGER FOR Count ← 1 TO LENGTH(InString) NextChar ← UCASE(MID(InString, Count, 1)) IF NextChar >= 'A' AND NextChar <= 'Z' THEN Index ← ASC(NextChar) - 64 Result[Index] ← Result[Index] + 1 ENDIF ENDFOR FOR Index ← 1 TO 26 OUTPUT "Letter " & CHR(Index + 64) & " : " & NUM_TO_STRING(Result[Index]) ENDFOR </pre> One mark for each of the following (max 7): 1 First loop from 1 to length of InString: 2 Extract each character in turn in a loop 3 Check that character is alphabetic (must cater for lower & upper case) in a loop 4 Obtain array index using ASC() - 64 in a loop 5 Increment element of Result array in a loop 6 Second loop from 1 to 26: 7 Attempt to OUTPUT character A to Z and corresponding count in a loop 8 Fully complete OUTPUT including any necessary type conversion in a loop	7
------	---	----------

129. O/N 19/P21/Q6/a,b,c,d(i)

6(a)	To retain data when the computer is shut down / turned off // after the program ends Accept equivalent answer.	1
------	--	----------

6(b)	'Pseudocode' solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix. <pre> FUNCTION SearchFileNtoZ(AccNum : STRING) RETURNS BOOLEAN DECLARE FileData : STRING DECLARE Found : BOOLEAN CONSTANT SearchFile = "UserListNtoZ.txt" Found ← FALSE OPENFILE SearchFile FOR READ WHILE NOT EOF(SearchFile) AND NOT Found READFILE SearchFile, FileData IF AccNum & '*' = LEFT(FileData, LENGTH(AccNum) + 1) THEN Found ← TRUE ENDIF ENDWHILE CLOSEFILE SearchFile RETURN Found ENDFUNCTION </pre> One mark for each of the following: 1. Function heading and ending, (ignore parameter) and returned BOOLEAN 2. File OPEN UserListNtoZ.txt in READ mode and CLOSE 3. Conditional loop repeating until EOF() or 'Found' 4. Read a line from the file in a loop 5. Compare the correct number of characters with AccNum in a loop 6. Set termination logic if found in a loop 7. Return Boolean value	7
------	--	---

Question 6(b): Visual Basic

```
Function SearchFileNtoZ(ByVal SearchString As String) As Boolean
    Dim FileData As String
    Dim Found As Boolean

    Found = FALSE

    FileOpen(1, "UserListNtoZ.txt", OpenMode.Input)

    While Not EOF(1) And Not Found

        Filedata = LineInput(1)
        If SearchString & '*' = Left(FileData, Len(SearchString)+1) Then
            Found = TRUE
        End If

    End While

    FileClose(1)

    Return Found

End Function
```

Question 6(b): Pascal

```
function SearchFileNtoZ (SearchString : string): boolean;
var
    FileData : string;
    Found : boolean;
    MyFile : text;

begin

    Found := FALSE;

    assign(MyFile, "UserListNtoZ.txt");
    reset (Myfile);

    while Not EOF(MyFile) And Not Found do
        begin
            readLn(MyFile, FileData);
            if SearchString + '*' = LeftStr(FileData, length(SearchString)+1)
then
                Found := TRUE;

            end;

        close(MyFile);

        result := Found; // SearchFileB := Found;

    end;
```

Question 6(b): Python

```
def SearchFileNtoZ(SearchString):
    ## FileData : String
    ## Found : Boolean
    ## MyFile : Text

    Found = False

    MyFile = open("UserListNtoZ.txt", 'r')
    FileData = MyFile.readline()
    while FileData != "" and not Found :
        if SearchString + '*' == FileData[0: len(SearchString)+1]:
            Found = True

        FileData = MyFile.readline()

    MyFile.close

    return(Found)
```

6(c)	<pre> PROCEDURE FindDuplicates() DECLARE Index : INTEGER DECLARE FileData : STRING DECLARE Continue : BOOLEAN DECLARE AccNum : STRING Index ← 1 // assuming array is [1:100] Continue ← TRUE OPENFILE "UserListAtoM.txt" FOR READ WHILE NOT EOF("UserListAtoM.txt") AND Continue = TRUE READFILE "UserListAtoM.txt", FileData IF MID(FileData, 7, 1) = '*' // six character reference THEN AccNum ← LEFT(FileData, 6) ELSE AccNum ← LEFT(FileData, 9) ENDIF IF SearchFileNtoZ(AccNum) = TRUE THEN IF Index = 101 // is the array already full? THEN OUTPUT "Error - Array Full" Continue ← FALSE ELSE Duplicates[Index] ← AccNum Index ← Index + 1 ENDIF ENDIF ENDWHILE CLOSEFILE "UserListAtoM.txt" ENDPROCEDURE </pre> One mark for each of the following (max 8): 1. Declaration and Initialisation of Index and used to index array Duplicates 2. OPEN file UserListAtoM.txt in READ mode and CLOSE 3. Pre-Condition loop to go through the file until EOF() and early termination if array full 4. Read line from file and extract account number (AccNum) in a loop 5. Call SearchFileNtoZ (with AccNum) following an attempt at MP4 in a loop 6. Check if return value is TRUE and if so: in a loop 7. store AccNum in correct array element 8. increment array index following an attempt at MP7 9. If array overflow OUTPUT error message	8
------	---	---

6(d)(i)	<pre>PROCEDURE ClearArray(BYREF ThisArray : ARRAY, NumElements : INTEGER, InitVal : STRING) DECLARE Index : INTEGER FOR Index ← 1 TO NumElements ThisArray[Index] ← InitVal ENDFOR ENDPROCEDURE</pre> Mark as follows: • Procedure header • Loop • Assignment within loop	3
---------	---	----------

130. O/N 19/P22/Q1/b(i)

1(b)(i)	One mark per row	4										
<table><tr><th>Example value</th><th>Data type</th></tr><tr><td>43</td><td>INTEGER</td></tr><tr><td>TRUE</td><td>BOOLEAN</td></tr><tr><td>-273.16</td><td>REAL</td></tr><tr><td>"-273.16"</td><td>STRING</td></tr></table>			Example value	Data type	43	INTEGER	TRUE	BOOLEAN	-273.16	REAL	"-273.16"	STRING
Example value	Data type											
43	INTEGER											
TRUE	BOOLEAN											
-273.16	REAL											
"-273.16"	STRING											

131. O/N 19/P22/Q3/a,b

3(a)	<pre>DECLARE Result : ARRAY [0:9] OF INTEGER DECLARE Index : INTEGER FOR Index ← 0 TO 9 Result[Index] ← 0 ENDFOR</pre> One mark for each of the following: 1. Declaration of RESULT array (10 elements of type INTEGER) 2. Loop 3. Assignment within a loop	3
------	--	----------

3(b)	<pre> DECLARE Index : INTEGER DECLARE NextChar : CHAR DECLARE NextCharValue : INTEGER FOR Index ← 1 TO LENGTH(InString) NextChar ← MID(InString, Index, 1) NextCharValue ← STRING_TO_NUM(NextChar) Result[NextCharValue] ← Result[NextCharValue] + 1 ENDFOR FOR Index ← 0 TO 9 OUTPUT "Count of digit " & NUM_TO_STRING(Index) & " : " & NUM_TO_STRING(Result[Index]) ENDFOR </pre> One mark for each of the following: 1. Declaration of <code>INTEGER</code> variable for <code>Index</code> (or equivalent) to index <code>Result</code> array 2. First loop from 1 to length of <code>InString</code>: 3. Select each character (e.g <code>MID</code>) in first loop 4. Apply type conversion to obtain integer value for <code>index</code> in first loop 5. Increment element of <code>Result</code> array in a loop 6. Separate second loop to repeat 10 times: 7. Attempt to <code>OUTPUT</code> two items (digit 0 to 9 plus corresponding count) in any loop 8. <code>OUTPUT</code> statement including index and count in any loop including type conversion of element from array if required in a loop	8
------	--	---

132. O/N 19/P22/Q6/a,b

6(a)	'Pseudocode' solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix. <pre> FUNCTION SearchLeavers(Reference : STRING) RETURNS BOOLEAN DECLARE Index : INTEGER DECLARE Found : BOOLEAN Found ← FALSE Index ← 0 WHILE Index < 500 AND NOT Found IF Reference = Leavers[Index] THEN Found ← TRUE ENDIF Index ← Index + 1 ENDWHILE RETURN Found ENDFUNCTION </pre> One mark for each of the following: 1. Function heading (and ending) as above 2. Initialisation and increment of <code>Index</code> used to index <code>Leavers</code> array in a loop 3. Conditional loop repeating while <code>Index < 500</code> and exit loop if <code>Reference</code> is found: 4. Compare indexed array element value with <code>Reference</code> in a loop 5. Set termination logic if found in a loop 6. Return Boolean value	6
------	---	----------

Q6 (a): Visual Basic

```
Function SearchLeavers(ByVal Reference As String) As Boolean

    Dim Index As Integer
    Dim Found As Boolean

    Found = FALSE
    Index = 0

    Do While Index < 500 And Not Found

        If Reference = Leavers(Index) Then
            Found = TRUE
        End If

        Index = Index + 1

    Loop

    Return Found

End Function
```

Q6 (a): Pascal

```
function SearchLeavers(Reference : String) : boolean;

    var Index : integer;
    var Found : boolean;

    begin
        Found := FALSE;
        Index := 0;

        While Index < 500 And Not Found
            begin
                if Reference = Leavers[Index] Then
                    Found := TRUE;

                Index := Index + 1;

            end;

        result := Found; // SearchLeavers := Found
    end;
```

Q6 (a): Python

```
def SearchLeavers(Reference):

    ## Index : Integer
    ## Found : Boolean

    Found = False
    Index = 0

    while Index < 500 and not Found:
        if Reference == Leavers[Index]:
            Found = True

        Index = Index + 1

    return Found
```

6(b)	FUNCTION ProcessStudentList() RETURNS INTEGER DECLARE NotCopied : INTEGER DECLARE FileData : STRING DECLARE Reference : STRING NotCopied ← 0 OPENFILE "StudentList.txt" FOR READ OPENFILE "UpdatedList.txt" FOR WRITE WHILE NOT EOF("StudentList.txt") READFILE "StudentList.txt", FileData IF MID(FileData, 6, 1) = '*' THEN Reference ← MID(FileData, 1, 5) // five char reference ELSE Reference ← MID(FileData, 1, 8) // eight char reference ENDIF IF SearchLeavers(Reference) = FALSE THEN WriteFile "UpdatedList.txt", FileData ELSE NotCopied ← NotCopied + 1 ENDIF ENDWHILE CLOSEFILE "StudentList.txt" CLOSEFILE "UpdatedList.txt" RETURN NotCopied ENDFUNCTION One mark for each of the following: 1. Function heading and ending as above 2. Declaration and use of three local variables and initialisation of count to 0 3. OPEN both files in correct mode and CLOSE 4. Pre-Condition loop to go through the file StudentList.txt until EOF() 5. Read line from StudentList.txt and extract correct Reference (either 5 or 8 characters) in a loop 6. Call SearchLeavers() with Reference (after attempted extraction) in a loop 7. If result is FALSE then write FileData to UpdatedList.txt in a loop 8. Otherwise increment NotCopied in a loop 9. Return NotCopied count	9
------	--	---

133. O/N 19/P23/Q1/a(i)

1(a)(i)	One mark for each (different) data type and one mark for a corresponding example value Acceptable types: Integer, Real, String, Char, Boolean, Date,	6
---------	--	----------

134. O/N 19/P23/Q6/a,b

6(a)(i)	One mark per underlined section: <u>DECLARE Result : ARRAY</u> <u>[0:99, 0:1]</u> <u>OF STRING</u>	3
---------	---	----------

6(a)(ii)	'Pseudocode' solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix. <pre> FUNCTION FindBooksBy(SearchAuthor : STRING) RETURNS INTEGER DECLARE Title : STRING DECLARE Author : STRING DECLARE Isbn : STRING DECLARE Location : STRING DECLARE Count : INTEGER Count ← 0 OPENFILE "Library.txt" FOR READ WHILE NOT EOF ("Library.txt") READFILE "Library.txt", Title READFILE "Library.txt", ThisAuthor READFILE "Library.txt", ISBN READFILE "Library.txt", Location IF SearchAuthor = ThisAuthor THEN Result[Count, 0] ← Title Result[Count, 1] ← Location Count ← Count + 1 ENDIF ENDWHILE CLOSEFILE("Library.txt") RETURN Count ENDFUNCTION </pre> One mark for each of the following: 1 Function heading (and ending) including parameters 2 Declaration of variables used 3 Open file for reading (Allow Library or Library.txt) 4 WHILE loop checking for EOF(): 5 Read all information 'fields', in the correct order, in a loop 6 If the author matches write Title and Location to Result array in a loop 7 And increment array index in a loop // number found 8 Close file and RETURN Count	8
----------	--	---

Q6 (a) (ii): Visual Basic

```
FUNCTION FindBooksBy(ByVal SearchAuthor As String) As Integer

    Dim Title As String
    Dim Author As String
    Dim Isbn As String
    Dim Location As String
    Dim Count As Integer

    Count = 0

    FileOpen(1, "Library.txt", OpenMode.Input)

    While Not EOF(1)
        Title = LineInput(1)
        ThisAuthor = LineInput(1)
        Isbn = LineInput(1)
        Location = LineInput(1)

        If SearchAuthor = ThisAuthor Then
            Result(Count, 0) = Title
            Result(Count, 1) = Location
            Count = Count + 1
        End If
    End While

    FileClose(1)

    Return Count
END FUNCTION
```


Q6 (a) (ii): Pascal

```
function FindBooksBy(SearchAuthor : string) : integer;

var
  Title : string;
  Author : string;
  Isbn : string;
  Location : string;
  Count : integer;
  MyFile : text;

begin
  Count := 0;
  assign(MyFile, 'Library.txt');
  reset(MyFile);

  while not EOF(MyFile) do
  begin
    readln(MyFile, Title);
    readln(MyFile, ThisAuthor);
    readln(MyFile, Isbn);
    readln(MyFile, Location);

    if SearchAuthor = ThisAuthor then
    begin
      Result[Count, 0] := Title;
      Result[Count, 1] := Location;
      Count := Count + 1;
    end;

  end;

  close(MyFile);
  FindBooksBy := Count;

end;
```

Q6 (a) (ii): Python

```
def FindBooksBy(SearchAuthor):

    ## Title : STRING
    ## Author : STRING
    ## Isbn : STRING
    ## Location : STRING
    ## Count : INTEGER

    Count = 0
    MyFile = open("Library.txt", 'r')
    Title = MyFile.readline()

    while Title != "":
        ThisAuthor = MyFile.readline()
        Isbn = MyFile.readline()
        Location = MyFile.readline()
        if SearchAuthor == ThisAuthor.strip():
            Result[Count][0] = Title
            Result[Count][1] = Location
            Count = Count + 1
        Title = MyFile.readline()

    MyFile.close()
    return(Count)
```

6(b)	<pre> PROCEDURE DisplayResults (Author:STRING, Count:INTEGER) DECLARE Index, GLen : INTEGER DECLARE Gap : STRING Gap ← " " // 25 spaces IF Count = 0 THEN OUTPUT "Search found no books by: " & Author ELSE OUTPUT "Books written by: " & Author OUTPUT "Title" & LEFT(Gap, 20) & "Location" FOR Index ← 1 TO Count GLen ← 25 - LENGTH(Result[Index, 0]) OUTPUT Result[Index, 0] & LEFT(Gap, GLen) & Result[Index, 1] ENDFOR OUTPUT "Number of titles found: " & NUM_TO_STRING(Count) ENDIF ENDPROCEDURE </pre> One mark for each of the following (max 7): 1 Procedure heading and ending including parameters 2 Declaration of local INTEGER variable for use as index 3 Test if count = 0 and if so output suitable message including Author for no books found otherwise output the two header strings (exact format not important) 4 A FOR loop for Count times 5 ... output two array elements from Result array in a loop 6 Final output statement 7 A reasonable attempt at calculating the number of spaces required to align 'Location' column: 8 Alignment correct	7
------	---	---

135. M/J 19/P21/Q1/b(ii)

1(b)(ii)	<table><tr><th>Variable</th><th>Data type</th></tr><tr><td>Tile</td><td>STRING</td></tr><tr><td>Version</td><td>CHAR</td></tr><tr><td>PackSize</td><td>INTEGER</td></tr><tr><td>WeightEach</td><td>REAL</td></tr><tr><td>Paperback</td><td>BOOLEAN</td></tr></table>	Variable	Data type	Tile	STRING	Version	CHAR	PackSize	INTEGER	WeightEach	REAL	Paperback	BOOLEAN	5
	Variable	Data type												
	Tile	STRING												
	Version	CHAR												
	PackSize	INTEGER												
	WeightEach	REAL												
	Paperback	BOOLEAN												
One mark per data type														

136. M/J 19/P21/Q4/b

4(b)	<pre>DECLARE Code : ARRAY[1:500, 1:4] OF STRING DECLARE RowIndex : INTEGER DECLARE ColIndex : INTEGER FOR RowIndex ← 1 TO 500 FOR ColIndex ← 1 TO 4 Code[RowIndex, ColIndex] ← "Empty" ENDFOR ENDFOR</pre> One mark for each of the following: 1 Array declaration 2 Additional local variable 3 Nested loops 4 Array element assignment within the inner loop RowIndex and ColIndex can be interchangeable	4
------	---	----------

137. M/J 19/P21/Q5/b

5(b)	<pre> PROCEDURE TestRand() DECLARE MyArray : ARRAY [1:50] OF BOOLEAN DECLARE Attempts : INTEGER DECLARE NumFound : INTEGER DECLARE ThisRndNumber : INTEGER DECLARE Index : INTEGER FOR Index ← 1 TO 50 Myarray[Index] ← FALSE ENDFOR NumFound ← 0 Attempts ← 0 WHILE NumFound < 50 ThisRndNumber ← 1 + INT(RAND(50)) Attempts ← Attempts + 1 IF MyArray[ThisRndNumber] = FALSE THEN MyArray[ThisRndNumber] ← TRUE NumFound ← NumFound + 1 ENDIF ENDWHILE OUTPUT "Number of calls to RAND() was ", Attempts ENDPROCEDURE </pre> 1 mark for each of the following: 1 Declaration of array of 50 elements 2 Loop to initialise array 3 Conditional loop stopping when all numbers generated 4 Generate a random integer in the range 1 to 50 in a loop 5 Count each call to <code>RND()</code> in a loop 6 check if the number has already been generated in a loop 7 if true, record as generated in a loop 8 Output a message plus the <code>Attempts</code> outside a loop	8
------	---	---

138. M/J 19/P21/Q6/b

6(b)	'Pseudocode' solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix. <pre> FUNCTION FindCD(SearchCDArtist : STRING, SearchCDTitle : STRING) RETURNS STRING DECLARE CDTitle : STRING DECLARE CDArtist : STRING DECLARE CDLocation : STRING DECLARE Location : STRING Location ← "" OPENFILE "MyCDs.txt" FOR READ WHILE NOT EOF ("MyCDs.txt") AND Location = "" READFILE "MyCDs.txt", CDArtist READFILE "MyCDs.txt", CDTitle READFILE "MyCDs.txt", CDLocation IF SearchCDArtist = CDArtist AND SearchCDTitle = CDTitle THEN Location ← CDLocation ENDIF ENDWHILE CLOSEFILE("MyCDs.txt") RETURN Location ENDFUNCTION </pre> One mark for each of the following: 1 Function header and close (where appropriate), including parameters 2 Declaration of local STRING variables for CDArtist and CDTitle 3 OPEN and CLOSE file for reading (Allow MyCDs or MyCDs.txt) 4 (WHILE) loop checking for EOF 5 read three lines from file in a loop 6 compare search values with file values in a loop... 7 ...If true, set Location and exit loop in a loop 8 Return Location	8
------	---	----------

Q6 (b) (i): Visual Basic

```
Function FindCD(SearchCDArtist As String, SearchCDTitle As String) As String

    Dim CDTitle As String
    Dim CDArtist As String
    Dim CDLocation As String
    Dim Location As String

    Location = ""
    FileOpen(1,"MyCDs.txt", OpenMode.Input)

    Do While Not EOF(1) And Location = ""
        CDArtist = LineInput(1)
        CDTitle = LineInput(1)
        CDLocation = LineInput(1)

        If SearchCDArtist = CDArtist And SearchCDTitle = CDTitle Then
            Location = CDLocation
        End If
    Loop
    FileClose(1)

EndFunction
```

Q6 (b) (i): Python

```
def FindCD(SearchCDArtist, SearchCDTitle):

    # CDTitle, CDArtist, CDLocation, Location : string

    Location = ""
    myFile = open("MyCDs.txt", 'r')
    while True:
        # or Location == "":
        CDArtist = myFile.readline()

        if CDArtist == "":
            break
        else:
            CDTitle = myFile.readline()
            CDLocation = myFile.readline()

            if SearchCDArtist == CDArtist.strip() and SearchCDTitle == CDTitle.strip():

                Location = CDLocation

    myFile.close
    return (Location)
```

Q6 (b) (i): Pascal

```

function FindCD(SearchCDArtist, SearchCDTitle:string): string;

var
    CDTitle, CDArtist, CDLocation, Location : string;
    FileHandle : TextFile;

begin
    Location := '';
    AssignFile(FileHandle, 'MyCDs.txt');
    Reset (FileHandle);

    while not eof(FileHandle) and (Location = '') do
    begin
        readln(FileHandle, CDArtist);
        readln(FileHandle, CDTitle);
        readln(FileHandle, CDLocation);
        if (SearchCDArtist = CDArtist) and (SearchCDTitle = CDTitle)
then
            Location := CDLocation;
        end;

        Close (FileHandle);
        FindCD := Location;

    end;
end;

```

139. M/J 19/P22/Q1/b(ii)

1(b)(ii)	Variable	Data type	5
	Married	DATE	
	ID	STRING	
	MiddleInitial	CHAR	
	Height	REAL	
	IsMarried	BOOLEAN	
	One mark per data type		

140. M/J 19/P22/Q5/b

5(b)	<pre> PROCEDURE CountLines(FileName : STRING) DECLARE NumLines : INTEGER DECLARE Dummy : STRING NumLines ← 0 OPENFILE FileName FOR READ WHILE NOT EOF(FileName) READFILE FileName, Dummy NumLines ← NumLines + 1 ENDWHILE CLOSEFILE FileName OUTPUT "Number of lines in the file : ", NumLines ENDPROCEDURE </pre> One mark for each of the following: 1. Procedure header and end, including parameter 2. Declaration and initialisation of a local <code>INTEGER</code> to count lines (e.g. <code>NumLines</code>) 3. <code>OPEN</code> file in read mode and <code>CLOSE</code> file 4. <code>WHILE</code> loop stopping when <code>EOF(FileName)</code> 5. Read a line from the file and increment <code>NumLines</code> in a loop 6. Output a message plus the <code>NumLines</code> outside a loop	6
------	---	----------

141. M/J 19/P23/Q2/c

2(c)	Mode: READ Description: Used to read data from a file // input data from a file to the program // only allows you to read data from the file / can't change the data Mode: APPEND Description: Used to add data // write to the end of the text file // output data from the program to the end of a file (without changing / deleting anything) One mark for mode; one mark for corresponding description	4
------	--	----------

142. M/J 19/P23/Q4/a,b

4(a)	<pre>DECLARE Name : ARRAY [1:40] OF STRING DECLARE Index : INTEGER FOR Index ← 1 TO 40 OUTPUT "Input the name for student ", Index INPUT Name[Index] ENDFOR</pre> One mark for each of the following: 1. Declaration of array and index 2. Loop for 40 elements 3. Prompt (as above, including student number) and input for name in a loop 4. Assign the name to an array element in a loop	4
4(b)	• Program code easier to read / modify / debug • Easier to access individual elements of / search for a value in the 'data set' // single identifier used	1

143. M/J 19/P23/Q6/a,b

6(a)	'Pseudocode' solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix. <pre> FUNCTION SearchFile (SearchString : STRING) RETURNS STRING DECLARE FileData : STRING DECLARE Found : BOOLEAN DECLARE SearchLength : INTEGER Found ← FALSE SearchLength ← LENGTH(SearchString) OPENFILE "StudentContact.txt" FOR READ WHILE NOT EOF("StudentContact.txt") AND NOT Found READFILE "StudentContact.txt", FileData IF SearchString = LEFT(FileData, SearchLength) THEN Found ← TRUE ENDIF ENDWHILE CLOSEFILE "StudentContact.txt" IF NOT FOUND THEN RETURN "" ELSE RETURN FileData ENDIF ENDFUNCTION </pre> One mark for each of the following: 1. Function header and end (where appropriate). Parameter optional but if present must be of type <code>STRING</code> 2. Calculate length of string from parameter // extract substring from file line 3. File <code>OPEN()</code> in <code>READ</code> mode and subsequent <code>CLOSE()</code> 4. <code>WHILE</code> loop repeating until <code>EOF()</code> 5. read a line from the file in a loop 6. compare name from file with <code>SearchString</code> in a loop 7. exit loop if <code>SearchString</code> found 8. Return the line from the file if <code>SearchString</code> found or an empty string if not found	8
------	--	----------

Q6 (a): Visual Basic

```
Function SearchFile(SearchString As String) As String
    Dim FileData As String
    Dim Found As Boolean
    Dim SearchLength As Integer
    Dim FileName As String

    Found = False
    SearchLength = Len(SearchString)

    FileName = "StudentContact.txt"
    FileOpen(1, FileName, OpenMode.Input)
    While Not EOF(1) And NOT Found
        FileData = LineInput(1)
        If SearchString = Left(FileData, SearchLength) Then
            Found = True
        End If
    End While

    FileClose(1)

    If Not Found Then
        Return ""
    Else
        Return FileData
    End If

End Function
```

Alternative:

```
Function SearchFile (SearchString As String) As String
    Dim FileData As String
    Dim Found As Boolean
    Dim SearchLength As Integer
    Dim MyFile As System.IO.StreamReader

    MyFile = My.Computer.FileSystem.OpenTextFileReader("StudentContact.txt")
    Found = False
    SearchLength = Len(SearchString)

    Do While MyFile.Peek <> -1
        FileData = MyFile.Readline()
        If SearchString = LEFT(FileData, SearchLength) Then
            Found = True
            return(FileData)
        End If
    Loop
    MyFile.Close
    If NOT Found then
        return ( "")
    End If

End Function
```

Q6 (a): Pascal

```
function SearchFile(var SearchString: string):string;
    var Found : Boolean;
        SearchLength : integer;
        FileData : string;
        MyFile : text;

begin
    Found := False;
    SearchLength := Length(SearchString);

    Assign(MyFile, 'StudentContact.txt');
    Reset(MyFile);

    While NOT EOF(MyFile) AND Found = False do
        Begin
            Readln(MyFile, FileData);
            If SearchString = LeftStr(FileData,SearchLength) then
                Found := True;
        End;
        Close(MyFile);

    If NOT Found then
        SearchFile := ''
    else
        SearchFile := FileData;

End;
```

Q6 (a): Python

```
def searchFile(searchString):
    ##Declare filedata : string, found : boolean, searchLength : integer
    ##returns a string value

    found = False
    searchLength = len(searchString)
    myFile = open("StudentContact.txt", 'r')
    fileData = myFile.readline()
    while found == False:
        fileData = myFile.readline()
        if not fileData.strip(): #check if no data/end of file
            break
        else:
            if searchString == fileData[0:searchLength]:
                found = True
                print(searchString)

    myFile.close
    if found == False:
        return("")
    else:
        return(fileData)
```

6(b)	FUNCTION ProcessArray() RETURNS INTEGER <pre> DECLARE NoTelNumber : INTEGER DECLARE Index : INTEGER DECLARE ThisName : STRING DECLARE StudentData : STRING NoTelNumber ← 0 FOR Index ← 1 to 40 ThisName ← ClassList[Index] IF ThisName <> "" //Skip blanks THEN StudentData ← SearchFile(ThisName) IF StudentData = "" //Student not found THEN StudentData ← ThisName & "*No number" NoTelNumber ← NoTelNumber + 1 ENDIF CALL AddToFile(StudentData, "ClassContact.txt") ENDIF ENDIF ENDFOR RETURN NoTelNumber ENDFUNCTION </pre> One mark for each of the following: 1. Function header and end, including return parameter 2. Declaration and initialisation of local count variable (NoTelNumber) 3. FOR loop for 40 array elements ... 4. skip empty elements in a loop 5. use SearchFile(ThisName) and save return value in a loop 6. if Searchfile() returns an empty string, add "*No number" to SearchString ... 7. ... and increment count 8. call AddToFile with both parameters as above in a loop 9. Return count outside the loop	9
------	---	---

144. O/N 18/P22/Q1/b

1(b)(i)	<table><tr><th>Statement</th><th>Data type</th></tr><tr><td>Revision ← 'B'</td><td>CHAR</td></tr><tr><td>MaxValue ← 13.3</td><td>REAL</td></tr><tr><td>ArrayFull ← TRUE</td><td>BOOLEAN</td></tr><tr><td>Activity ← "Design"</td><td>STRING</td></tr><tr><td>NumberOfEdits ← 270</td><td>INTEGER</td></tr></table>	Statement	Data type	Revision ← 'B'	CHAR	MaxValue ← 13.3	REAL	ArrayFull ← TRUE	BOOLEAN	Activity ← "Design"	STRING	NumberOfEdits ← 270	INTEGER	5
	Statement	Data type												
	Revision ← 'B'	CHAR												
	MaxValue ← 13.3	REAL												
	ArrayFull ← TRUE	BOOLEAN												
	Activity ← "Design"	STRING												
	NumberOfEdits ← 270	INTEGER												
1 mark per correct row														

1(b)(ii)	<table><tr><th>Expression</th><th>Evaluates to</th></tr><tr><td>MID(Activity, 3, 4) & "ature"</td><td>"signature"</td></tr><tr><td>INT(MaxValue * 2)</td><td>26</td></tr><tr><td>ArrayFull AND NumberOfEdits < 300</td><td>TRUE</td></tr><tr><td>ASC(Revision + 1)</td><td>ERROR</td></tr><tr><td>Activity = "Testing" OR Revision = 'A'</td><td>FALSE</td></tr></table>	Expression	Evaluates to	MID(Activity, 3, 4) & "ature"	"signature"	INT(MaxValue * 2)	26	ArrayFull AND NumberOfEdits < 300	TRUE	ASC(Revision + 1)	ERROR	Activity = "Testing" OR Revision = 'A'	FALSE	5
	Expression	Evaluates to												
	MID(Activity, 3, 4) & "ature"	"signature"												
	INT(MaxValue * 2)	26												
	ArrayFull AND NumberOfEdits < 300	TRUE												
	ASC(Revision + 1)	ERROR												
	Activity = "Testing" OR Revision = 'A'	FALSE												
1 mark per correct row														

145. O/N 18/P22/Q4/a(ii),b,d

4(a)(ii)	Example mark points, max 4 marks: • If <code>FileData</code> is not empty, start and continue a loop // while <code>FileData</code> is not empty... • Compare the Left 7 characters of <code>FileData</code> with <code>SearchString</code> • If they match, add <code>FileData</code> to the array / <code>ResultArray[]</code> • If they match, increment the array index variable • Increment <code>FileLine</code> • Read the next line from the file	4
----------	---	----------

4(b)	Pseudocode solution included here for development and clarification of mark scheme. Programming language solutions appear at the end of this mark scheme. <pre> FUNCTION ScanFile(SearchString STRING) RETURNS INTEGER DECLARE FileData : STRING DECLARE FileLine : INTEGER DECLARE NewData : STRING DECLARE Size : INTEGER NextArrayElement ← 1 FileLine ← 1 FileData ← ReadFileLine("DataFile.txt", FileLine) WHILE FileData <> "" IF LEFT(FileData, 7) = SearchString THEN Size ← LENGTH(FileData) NewData ← RIGHT(FileData, Size-7) ResultArray[NextArrayElement] ← NewData NextArrayElement ← NextArrayElement + 1 ENDIF FileLine ← FileLine + 1 FileData ← ReadFileLine("DataFile.txt", FileLine) ENDWHILE CALL ScanCompleted() RETURN FileLine ENDFUNCTION </pre> 1 mark for each of the following: 1 Function heading and ending including parameters and return statement 2 Local variable declarations for FileData and FileLine but NOT declaration of NextArrayElement 3 initialisation of FileLine and NextArrayElement (allow 0 or 1) 4 WHILE ... ENDWHILE loop 5 Comparing SearchString with first seven characters of FileData 6 Use of substring function and subtraction of 7 from length 7 Assign NewData to array ResultArray following attempt at MP6 8 Increment NextArrayElement and FileLine (as above)	8
------	--	---

4(d)	Pseudocode solution included here for development and clarification of mark scheme. Programming language solutions appear at the end of this mark scheme. <pre>DECLARE ResultArray : ARRAY [1:100] OF STRING DECLARE Index: INTEGER FOR Index ← 1 TO 100 ResultArray[Index] ← "NO DATA" ENDFOR</pre> 1 mark for each of the following: • ResultArray declaration / commented in Python • Loop for 100 elements • ...assignment of string "NO DATA" to indexed array element	3
------	--	----------

146. O/N 18/P22/Q5

5	<pre> PROCEDURE LineNumber(FileName: STRING, StartNumber: INTEGER, StepNumber: INTEGER) DECLARE FileData : STRING DECLARE Count : INTEGER DECLARE Reply : CHAR DECLARE Continue : BOOLEAN Count ← 0 Continue ← TRUE OPENFILE FileName FOR READ WHILE NOT EOF(FileName) AND Continue = TRUE READFILE FileName, FileData FileData ← NUM_TO_STRING(StartNumber) & ": " & FileData OUTPUT FileData Count ← Count + 1 IF Count = 20 THEN OUTPUT "Do you wish to continue?" INPUT Reply IF Reply = 'N' THEN Continue ← FALSE ELSE Count ← 0 ENDIF ENDIF StartNumber ← StartNumber + StepNumber ENDWHILE CLOSEFILE FileName ENDPROCEDURE </pre>	11
5	1 mark for each of the following to max 11: 1 Procedure heading including parameters 2 Declare an integer variable for the count 3 Open file in READ mode 4 Initialise Count variable before loop and increment in the loop (or other mechanism) 5 Loop including until EOF(FileName) (see note) 6 Call READFILE() in a loop 7 Convert StartNumber to string in a loop 8 Output concatenated string in a loop 9 Check if count = 20 10 Prompt and Input (inside an IF) 11 If user input = 'N' terminate loop 12 Add StepNumber to StartNumber 13 Close file	

147. O/N 18/P23/Q1/b

1(b)(i)	<table><tr><th>Statement</th><th>Data type</th></tr><tr><td>Revision ← 500</td><td>INTEGER</td></tr><tr><td>FuelType ← 'P'</td><td>CHAR</td></tr><tr><td>MinValue ← -6.3</td><td>REAL</td></tr><tr><td>ServiceDue ← FALSE</td><td>BOOLEAN</td></tr><tr><td>ModelRef ← "W212DEC15"</td><td>STRING</td></tr></table>		Statement	Data type	Revision ← 500	INTEGER	FuelType ← 'P'	CHAR	MinValue ← -6.3	REAL	ServiceDue ← FALSE	BOOLEAN	ModelRef ← "W212DEC15"	STRING	5
	Statement	Data type													
	Revision ← 500	INTEGER													
	FuelType ← 'P'	CHAR													
	MinValue ← -6.3	REAL													
	ServiceDue ← FALSE	BOOLEAN													
	ModelRef ← "W212DEC15"	STRING													
One mark per row															

1(b)(ii)			5

148. O/N 18/P23/Q4/a(ii),d(i)

4(a)(ii)	Example mark points: - Conditional loop through array <code>ThisArray</code> one element at a time until found - Compare the element from row / column 1 of the array with <code>NewData</code> - If the current element is greater than <code>NewData</code> set <code>Found</code> to TRUE (to exit the loop) - If the current element is not greater than <code>NewData</code> increment <code>ArrayIndex</code>	4
----------	---	---

4(d)(i)	Pseudocode solution included here for development and clarification of mark scheme. Programming language solutions appear at the end of this mark scheme. <pre>DECLARE i : INTEGER FOR i ← 1 to 200 IF CharArray[i] >= '0' AND CharArray[i]<= '9' THEN CharArray[i] ← '*' ENDIF ENDFOR</pre> 1 mark for each of the following: • looping through 200 elements • selection statement • assignment of '*'	3
---------	--	----------

149. O/N 18/P23/Q5

5	<pre> FUNCTION ReadFileLine(FileName: STRING, FileLine: INTEGER) RETURNS STRING DECLARE FileData : STRING DECLARE LineNumber : INTEGER OPENFILE FileName FOR READ LineNumber ← 0 // no line read yet WHILE (NOT EOF(FileName)) AND FileLine <> LineNumber READFILE FileName, FileData LineNumber ← LineNumber + 1 ENDWHILE IF FileLine <> LineNumber THEN FileData ← "*****" ENDIF CLOSEFILE FileName RETURN FileData ENDFUNCTION </pre> 1 mark for each of the following: 1 Function heading including parameters. 2 Declare local variables FileData and LineNumber 3 Open FileName in READ mode 4 WHILE loop 5 Call to READFILE() (in a loop) 6 Incrementing LineNumber (in a loop) 7 IF FileLine <> LineNumber (after a loop) 8 ...Set FileData to "*****" 9 Close FileName 10 Return FileData	10
---	---	----

150. M/J 18/P21/Q5/a,c

5(a)	Details are saved after the program ends // after the computer is switched off	1
------	--	---

5(c)	'Pseudocode' solution included here for development and clarification of mark scheme. Programming language solutions appear in the Appendix. <pre> PROCEDURE AddNewScores() DECLARE FileData : STRING DECLARE ScoreDate : STRING DECLARE MembershipNumber : STRING DECLARE Score : STRING OUTPUT "Input the date for the scores" INPUT ScoreDate OPENFILE "ScoreDetails.txt" FOR APPEND OUTPUT "Input the Membership number" INPUT MembershipNumber WHILE NOT MembershipNumber = "" OUPUT "Input the score" INPUT Score WHILE (INT(SCORE) < 50) OR (INT(SCORE) > 99) OUTPUT "Input a valid score from 50 to 99" INPUT Score ENDWHILE FileData = MembershipNumber & ScoreDate & Score WRITEFILE "ScoreDetails.txt", FileData OUTPUT "Input the Membership number" INPUT MembershipNumber ENDWHILE CLOSEFILE("ScoreDetails.txt") ENDPROCEDURE </pre> 1 mark for each of the following: 1 Declare MembershipNumber as STRING and Score as INTEGER / STRING (commented in Python) 2 Prompt and Input of ScoreDate 3 Open ScoreDetails.txt in APPEND mode 4 Prompt and Input of MembershipNumber in a loop 5 (Outer) loop terminated when MembershipNumber = "" 6 Input Score and loop until valid 7 Form the text string from the three variables 8 Write the text string to the file 9 Close the file	9
------	---	---

Q5(c)**Visual Basic**

```
Sub AddNewScores()  
  
    Dim FileData As String  
    Dim MembershipNumber As String  
    Dim Score As Integer  
    Dim ScoreDate As String  
    Dim FileHandle As IO.StreamWriter  
  
    FileHandle = New IO.StreamWriter("ScoreDetails.txt")  
  
    Console.WriteLine("Input the date for the scores")  
    ScoreDate = Console.ReadLine()  
  
    Console.WriteLine("Input the Membership number")  
    MembershipNumber = Console.ReadLine()  
  
    Do While MembershipNumber <> ""  
        Console.WriteLine("Input the score")  
        Score = Console.ReadLine()  
        Do While Score < 50 Or Score > 99  
            Console.WriteLine("Input a valid score from 50 to 99")  
            Score = Console.ReadLine()  
        Loop  
        FileData = MembershipNumber & ScoreDate & Str(Score)  
        FileHandle.WriteLine(FileData)  
        Console.WriteLine("Input the Membership number")  
        MembershipNumber = Console.ReadLine()  
    Loop  
    FileHandle.Close()  
  
End Sub
```

Python

```
def AddNewScores():
    #ScoreDate as string
    #MembershipNumber as string
    #Score as integer
    #FileHandle as text file

    FileHandle = open("ScoreDetails.txt", "a")
    ScoreDate = str(input("Input the date for the scores"))
    MembershipNumber = str(input("Input the Membership number"))
    while MembershipNumber != "":
        Score = int(input("Input the score"))
        while Score < 50 or Score > 99:
            Score = int(input("Input a valid score from 50 to 99"))
            FileData = MembershipNumber + ScoreDate + str(Score)
            FileHandle.write(FileData)
            MembershipNumber = str(input("Input the Membership number"))
    FileHandle.close
```

Pascal

```
procedure AddNewScores;
var FileData, ScoreDate, MembershipNumber: String;
    Score: Integer;
    MyFile: text;

begin
    assign(MyFile, 'ScoreDetails.txt');
    append(MyFile);
    writeln('Input the date for the scores');
    readln(ScoreDate);
    writeln('Input the Membership number');
    readln(MembershipNumber);

    while MembershipNumber <> '' do
        begin
            writeln('Input the score');
            readln(Score);
            while (Score < 50) or (Score > 99) do
                begin
                    writeln('Input the score');
                    readln(Score);
                end;
            FileData := MembershipNumber + ScoreDate + IntToStr(Score);
            write(MyFile, FileData);
            writeln('Input the Membership number');
            readln(MembershipNumber);
        end;
    close (MyFile);
end;
```

151. M/J 18/P21/Q6/a,b

6(a)(i)	- The array is 1D 1 is the lower bound 5 is the upper bound - size of array / number of elements = 5	Max2
6(a)(ii)	subscript / index	1
6(b)	<pre> FUNCTION Lighten() RETURNS BOOLEAN DECLARE OldPixelValue : INTEGER DECLARE NewPixelValue : INTEGER DECLARE PixelTemp : REAL DECLARE BurnFlag : BOOLEAN DECLARE i : INTEGER DECLARE j : INTEGER BurnFlag ← FALSE FOR i ← 1 TO 8 FOR j ← 1 TO 8 OldPixelValue ← Picture[i, j] PixelTemp ← OldPixelValue * 1.1 NewPixelValue ← INT(PixelTemp) IF NewPixelValue >= 255 THEN NewPixelValue ← 255 BurnFlag ← TRUE ENDIF Picture[i, j] ← NewPixelValue ENDFOR ENDFOR RETURN BurnFlag ENDFUNCTION </pre> 1 mark for each of the following: - Function heading as above and ending - Declare and initialise local variable for return <code>BOOLEAN</code> / other mechanism to record 'burnt out' - Declare local variables for loop counters - Correct nested loops - Accessing element from array - Calculating new value and convert to an <code>INTEGER</code> - Comparing new value with 255 and if greater: ...limit to 255 and assign to original element ...Set flag / other mechanism if limit applied (Only change once) - Return a <code>BOOLEAN</code> (following conversion if other mechanism used) MUST WORK	MAX8

152. M/J 18/P22/Q5/a,d

5(a)	One mark for each of: • Same data type using a single identifier / more efficient coding / less declaration statements needed • Access of individual elements (using subscript / index) • Ability to iterate through the data // easier to search / sort the data • Code easier to understand / maintain / modify	Max 2
------	--	--------------

5(d)	'Pseudocode' solution included here for development and clarification of mark scheme. Programming language solutions appear in the Appendix. <pre> PROCEDURE OutputLowestScore() DECLARE FileData, FileMembershipNumber : STRING DECLARE FileScore, LowestScore : INTEGER DECLARE LowestScoreDate, MembershipNumber : STRING MembershipNumber ← GetNumber() OPENFILE "ScoreDetails.txt" FOR READ LowestScore ← 100 WHILE NOT EOF("ScoreDetails.txt") READFILE "ScoreDetails.txt", FileData FileMembershipNumber ← LEFT(FileData, 4) IF FileMembershipNumber = MembershipNumber THEN FileScore ← INT(RIGHT(FileData,2)) IF FileScore < LowestScore THEN LowestScore ← FileScore LowestScoreDate ← MID(FileData(5,6)) ENDIF ENDIF ENDWHILE OUTPUT ("The lowest score was " & LowestScore & " on " & _ LowestScoreDate) CLOSEFILE("ScoreDetails.txt ") ENDPROCEDURE </pre> 1 mark for each of the following: 1 Declare variables to store <code>LowestScore</code> as <code>INTEGER</code> and <code>FileData</code> as <code>STRING</code> (commented in Python) (variable names may be different) 2 Function call <code>GetNumber()</code> for membership number 3 Initialisation of <code>LowestScore</code> to 99 or above 4 Open file in <code>READ</code> mode 5 Loop until <code>EOF()</code> 6 Read a line from the file in a loop 7 Use of substring operations to extract at least one data item 8 Compare the membership numbers 9 Convert score to <code>INTEGER</code> 10 Compare and assign (if appropriate) new value to <code>LowestScore</code> 11 Output the lowest score message including lowest score and date (outside the loop) 12 Close the file	Max 10
-------------	--	----------------------

Q5 (d): Visual Basic

```

Sub OutputLowestScore()
    Dim FileData As String
    Dim FileMembershipNumber As String
    Dim FileScore As Integer
    Dim LowestScore As Integer
    Dim LowestScoreDate As String
    Dim MembershipNumber As String
    MembershipNumber = GetNumber()
    FileOpen(1, "ScoreDetails.txt", OpenMode.Input)
    LowestScore = 100
    While Not EOF(1)
        FileData = LineInput(1)
        FileMembershipNumber = Left(FileData, 4)
        If FileMembershipNumber = MembershipNumber Then
            FileScore = Integer.Parse(Right(FileData, 2))
            If FileScore < LowestScore Then
                LowestScore = FileScore
                LowestScoreDate = Mid(FileData, 5, 6)
            End If
        End If
    End While
    Console.WriteLine("The lowest score was " & LowestScore & " on " &
LowestScoreDate)
    FileClose(1)
End Sub

```

Q5 (d): Pascal

```

procedure OutputLowestScore();
var
    FileData : string;
    FileMembershipNumber : string;
    FileScore : integer;
    LowestScore : integer;
    LowestScoreDate : string;
    MembershipNumber : string;
    Scores : textFile;
begin
    MembershipNumber := GetNumber();
    assignFile(Scores, 'ScoreDetails.txt');
    reset(Scores);
    LowestScore := 100;
    while not eof(Scores) do
        begin
            readln(Scores, FileData);
            FileMembershipNumber := copy(FileData, 1, 4);
            if FileMembershipNumber = MembershipNumber then
                begin
                    FileScore:= StrToInt(copy(FileData, 11, 2));
                    if FileScore < LowestScore then
                        begin
                            LowestScore := FileScore;
                            LowestScoreDate := copy(FileData, 5, 6)
                        end;
                    end;
                end;
            writeln('The lowest score was ', LowestScore, ' on ', LowestScoreDate);
            close(Scores);
        end;
    end;
end;

```

Q5 (d): Python

```
def OutputLowestScore() :  
    # FileData : string  
    # FileMembershipNumber : string  
    # FileScore : integer  
    # LowestScore : integer  
    # LowestScoreDate : string  
    # MembershipNumber : string  
    # File : file handle  
    MembershipNumber = GetNumber()  
    File = open("ScoreDetails.txt", "r")  
    LowestScore = 100  
    FileData = File.readline()  
    while FileData != "":  
        FileMembershipNumber = FileData[:4]  
        if FileMembershipNumber == MembershipNumber :  
            FileScore = int(FileData[10:12])  
            if FileScore < LowestScore :  
                LowestScore = FileScore  
                LowestScoreDate = FileData[4:10]  
        FileData = File.readline()  
    print("The lowest score was ", LowestScore, " on ", LowestScoreDate)  
    File.close()
```

153. M/J 18/P22/Q6/a,b

6(a)	• lower bound • upper bound	2
------	---	----------

6(b)	Example using single temp variable: <pre> PROCEDURE Flip() //Use of Single temp value DECLARE temp : INTEGER DECLARE i : INTEGER //i is the row DECLARE j : INTEGER //j is the column FOR i ← 1 TO 5 FOR j ← 1 to 4 //swap element 1&8, 2&7, 3&6, 4&5 temp ← Picture[i,j] Picture[i,j] ← Picture[i, 9 - j] Picture[i, 9 - j] ← temp ENDFOR ENDFOR ENDPROCEDURE </pre> Alternative Solution – Use of temp array row: <pre> PROCEDURE Flip() //Use of temp array row (8 elements) DECLARE temp : ARRAY[1:8] OF INTEGER DECLARE i : INTEGER //i is the row DECLARE j : INTEGER //j is the column FOR i ← 1 to 5 FOR j ← 1 to 8 temp[j] ← Picture[i, 9 - j] //temp is row i reversed ENDFOR FOR j ← 1 to 8 Picture[i,j] ← temp[j] //copy temp back to row i ENDFOR ENDFOR ENDPROCEDURE </pre>	8
------	--	----------

6(b)	Alternative Solution – Use of new array: <pre> PROCEDURE Flip() //Flip to New array DECLARE NewPic : ARRAY[1:5, 1:8] OF INTEGER DECLARE i : INTEGER //i is the row DECLARE j : INTEGER //j is the column FOR i ← 1 to 5 FOR j ← 1 to 8 NewPic[i, 9 - j] ← Picture[i, j] //NewPic row is Pic row flipped ENDFOR ENDFOR END PROCEDURE </pre> 1 mark for each of the following (all methods): 1 Correct procedure heading and ending 2 Declaring local variables for loop counter(s) 3 Declaring a temporary storage variable for swap or new duplicate 2D array 4 A nested loop including attempt at flip operation ... 5 ... Correct number of iterations 6 Assign element to temp (single var or temp array) or to new array 7 Selection of correct source element (row, column) 8 Selection of correct destination element (row, column)	8
------	--	----------

154. M/J 18/P23/Q5/a,c

5(a)(i)	1	1
5(a)(ii)	Information is saved after the program ends // after the computer is switched off	1

5(c)	'Pseudocode' solution included here for development and clarification of mark scheme. Programming language solutions appear in the Appendix. <pre> FUNCTION GetAverageScore (MembershipNumber : STRING) RETURNS INTEGER DECLARE FileData, FileMembershipNumber : STRING DECLARE NumberOfScores, TotalScore, AverageScore : INTEGER OPENFILE "ScoreDetails.txt" FOR READ NumberOfScores ← 0 TotalScore ← 0 WHILE NOT EOF("ScoreDetails.txt") READFILE("ScoreDetails.txt", FileData) FileMembershipNumber ← LEFT(FileData, 4) IF FileMembershipNumber = MembershipNumber THEN NumberOfScores ← NumberOfScores + 1 TotalScore ← TotalScore + INT(RIGHT(FileData, 2)) ENDIF ENDWHILE AverageScore ← INT(TotalScore / NumberOfScores) CLOSEFILE("ScoreDetails.txt") RETURN (AverageScore) ENDFUNCTION </pre> 1 mark for each of the following: 1 Function heading and ending including Input and return parameter 2 Declare variables to store <code>NumberOfScores</code> and <code>TotalScore</code> as INTEGERS (commented in Python) (variable names may be different) 3 Initialisation of <code>NumberOfScores</code> and <code>TotalScore</code> to 0 4 Open file in READ mode 5 Loop until EOF() 6 Read a line from the file in a loop 7 Use of substring function to extract at least one data item 8 Compare the membership number 9 Convert score to an integer 10 Increment <code>NumberOfScores</code> and sum <code>TotalScore</code> 11 Calculate the average outside the loop 12 Close the file 13 Return the parameter	Max 10
------	--	---------------

Q5(c): Visual Basic

```
Function GetAverageScore (ByVal MembershipNumber As String) As Integer

    Dim FileData As String
    Dim FileMembershipNumber As String
    Dim NumberOfScores As Integer
    Dim TotalScore As Integer
    Dim AverageScore As Integer
    Dim ObjReader As IO.StreamReader

    ObjReader = New IO.StreamReader("ScoreDetails.txt")
    NumberOfScores = 0
    TotalScore = 0

    Do While ObjReader.Peek <> -1
        FileData = ObjReader.ReadLine()
        FileMembershipNumber = LEFT(FileData, 4)
        If FileMembershipNumber = MembershipNumber Then
            NumberOfScores = NumberOfScores + 1
            TotalScore = TotalScore + INT(RIGHT(FileData, 2))
        End If
    Loop

    AverageScore = INT(TotalScore / NumberOfScores)
    ObjReader.Close()
    Return (AverageScore)

End Function
```

Q5(c): Pascal

```
function GetAverageScore (MembershipNumber : string):integer;

var
    FileData, FileMembershipNumber: string;
    NumberOfScores, TotalScore, AverageScore : integer;
    ScoreFile : textFile;
begin
    NumberOfScores := 0;
    TotalScore := 0;
    assignFile(ScoreFile, 'ScoreDetails.txt');
    reset(ScoreFile);
    while not eof(ScoreFile) do
        begin
            readln(ScoreFile, FileData);
            FileMembershipNumber := copy(FileData, 1, 4);
            if FileMembershipNumber = MembershipNumber then
                begin
                    NumberOfScores := NumberOfScores + 1
                    TotalScore := TotalScore + StrToInt(RightStr(FileData, 2));
                end;
            end;
        end;
    end;
```



```

    AverageScore := StrToInt(TotalScore / NumberOfScores);
    GetAverageScore := AverageScore;
    CloseFile (ScoreFile);
end;
```

Q5(c): Python

```

# FileData AS STRING
# FileMembershipNumber AS STRING
# NumberOfScores AS INTEGER
# TotalScore AS INTEGER
# AverageScore AS INTEGER

def GetAverageScore(MembershipNumber):
    FileHandle = open("ScoreDetails.txt", "r")
    NumberOfScores = 0
    TotalScore = 0
    FileData = FileHandle.readline()
    while len(FileData) > 0:
        FileMembershipNumber = FileData[0:4]
        if FileMembershipNumber == MembershipNumber:
            NumberOfScores = NumberOfScores + 1
            TotalScore = TotalScore + int(FileData[-2])
            FileData = FileHandle.readline()
    AverageScore = int(TotalScore / NumberOfScores)
    Return (AverageScore)
    FileHandle.close()
```

155. M/J 18/P23/Q6/a,b

6(a)	Subscript / index	1
------	-------------------	---

6(b)	<pre> FUNCTION Clip(MaxVal : INTEGER) RETURNS BOOLEAN DECLARE i : INTEGER DECLARE j : INTEGER DECLARE ClipFlag : BOOLEAN ClipFlag ← FALSE FOR i ← 1 TO 8 FOR j ← 1 TO 8 IF Picture[i, j] > MaxVal THEN Picture[i, j] ← MaxVal ClipFlag ← TRUE ENDIF ENDFOR ENDFOR RETURN ClipFlag ENDFUNCTION </pre> 1 mark for each of the following: 1 Correct Function heading (must have <code>MaxVal</code> and return a <code>BOOLEAN</code>) and ending 2 Declare and initialise local variable for return <code>BOOLEAN</code> to <code>FALSE</code> / other mechanism to record pixel being clipped 3 Declare local variables for loop counters 4 Nested loops with correct number of iterations 5 Accessing correct element from <code>Picture</code> array 6 Comparing element with <code>MaxVal</code> 7 Changing value of element if necessary 8 Setting flag to <code>TRUE</code> / other mechanism if element is changed 9 Returning <code>BOOLEAN</code> after loop (following conversion if other mechanism used)	9
------	---	---

156. O/N 17/P21/Q3/a,b,c

3(a)	<pre> PROCEDURE BubbleSort DECLARE Temp : STRING DECLARE FirstID, SecondID : INTEGER DECLARE NoSwaps : BOOLEAN DECLARE Boundary : INTEGER Declare J : INTEGER Boundary ← 99 REPEAT NoSwaps ← TRUE FOR J ← 1 TO Boundary FirstID ← UserNameArray[J] SecondID ← UserNameArray[J + 1] IF FirstID > SecondID THEN Temp ← UserNameArray[J] UserNameArray[J] ← UserNameArray[J + 1] UserNameArray[J + 1] ← Temp NoSwaps ← FALSE ENDIF ENDFOR Boundary ← Boundary - 1 UNTIL NoSwaps = TRUE ENDPROCEDURE </pre> Mark as follows: 1. Procedure heading and ending (allow array as input parameter) 2. Variable declaration for counter / index (integer) or temp (string) 3. Outer working loop 4. Inner loop with suitable range 5. Correct comparison in a loop 6. Correct swap of complete array element in a loop 7. Set flag to indicate swap in inner loop and resetting in outer loop 8. Reducing Boundary in a loop	8
------	--	----------

3(b)	Pseudocode solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix. <pre> PROCEDURE FindRepeats DECLARE i, RepeatCount: INTEGER DECLARE FirstID, SecondID: STRING RepeatCount ← 0 FOR i ← 2 TO 100 FirstID ← LEFT(UsernameArray[i - 1], 6) SecondID ← LEFT(UsernameArray[i], 6) IF FirstID = SecondID THEN RepeatCount ← RepeatCount + 1 OUTPUT(UsernameArray[i]) ENDIF ENDFOR IF RepeatCount = 0 THEN OUTPUT "The array contains no repeated UserIDs" ELSE OUTPUT "There are " & RepeatCount & " repeated userIDs" ENDIF ENDPROCEDURE </pre> Mark as follows (all must be correct syntax for chosen language): 1. Procedure heading and ending 2. Variable declaration for INTEGER (comment in Python) and initialisation for RepeatCount (or equivalent name) 3. Loop 4. Extraction of UserID in a loop 5. Correct comparison of consecutive elements... in a loop 6. ...output correct array element (NOT original, only duplicates) in a loop 7. increment RepeatCount following a comparison in a loop 8. Correct conditional statement checking RepeatCount (or equivalent) and then two correct final OUTPUT statements	Max 8
------	---	--------------

Q3(b): Visual Basic

```
Sub FindRepeats()  
    Dim Repeats As Integer  
    Dim i As Integer  
    Dim FirstID As String  
    Dim SecondID As String  
  
    Repeats = 0  
    For i = 1 To 99  
        FirstID = Left(UserNameArray(i), 6)  
        SecondID = Left(UserNameArray(i + 1), 6)  
        If FirstID = SecondID Then  
            Console.WriteLine(UserNameArray(i + 1))  
            Repeats = Repeats + 1  
        End If  
    Next i  
  
    If Repeats = 0 Then  
        Console.WriteLine("The array contains no repeated UserIDs")  
    Else  
        Console.WriteLine("There are " & Repeats & " repeated UserIDs")  
    End If  
  
End Sub
```

Alternative:

```
Sub FindRepeats ()  
  
    Dim RepeatCount, i As Integer  
    Dim FirstID, SecondID As String  
  
    RepeatCount = 0  
    For i = 1 to 99  
        FirstID = Left(UserNameArray(i-1),6)  
        SecondID = Left(UserNameArray(i),6)  
        If FirstID = SecondID then  
            Console.WriteLine (UserNameArray(i))  
            RepeatCount = RepeatCount + 1  
        End If  
    Next i  
  
    If RepeatCount = 0 then  
        Console.WriteLine ("The array contains no repeated UserIDs")  
    Else  
        Console.WriteLine ("There are "& RepeatCount & " repeated UserIDs")  
    End If  
End Sub
```

Q3(b): Pascal

```

procedure FindRepeats ();

var
  RepeatCount, i : integer;
  FirstID, SecondID : string;

begin
  RepeatCount := 0;
  for i := 1 to 99 do
    begin
      FirstID := Copy(UsernameArray[i-1],1,6);
      SecondID := Copy(UsernameArray[i],1,6);
      if FirstID = SecondID then
        begin
          writeln (UsernameArray[i]);
          RepeatCount := RepeatCount + 1;
        end;
      end;

      if RepeatCount = 0 then
        writeln ('The array contains no repeated UserIDs')
      else
        writeln ('There are ', RepeatCount, ' repeated UserIDs')
    end;
  end;

```

Q3(b): Python

```

def FindRepeats():
    #Repeats, i Integer
    #FirstID, SecondID string
    Repeats = 0
    for i in range(0, len(UsernameArray)-1):
        FirstID = (UsernameArray[i])[:6]
        SecondID = (UsernameArray[i+1])[:6]
        if FirstID == SecondID:
            print(UsernameArray[i+1])
            Repeats = Repeats + 1
        if Repeats == 0:
            print("The array contains no repeated UserIDs")
        else:
            print("There are ", Repeats, " repeated UserIDs")

```

Alternative:

```

def FindRepeats ():

    RepeatCount = 0                                ## Defined as an integer

    for i in range (1,100):                        ## depending on next two
lines(0,99) (2,101)
        FirstID = UsernameArray[i-1]              ## Defined as string
        SecondID = UsernameArray[i]               ## Defined as string
        if FirstID[0:6] == SecondID[0:6]:        ## Using split
            print (UsernameArray[i])
            RepeatCount += 1

    if repeatCount == 0:
        print ('The array contains no repeated UserIDs')
    else:
        print ('There are ', RepeatCount, ' repeated UserIDs')

```

3(c)(i)	• Problem definition • Design • Coding / programming • Testing • Documentation • Implementation • Maintenance	3
3(c)(ii)	<u>Integrated Development Environment</u> or a suitable description	1
3(c)(iii)	Examples include: • context sensitive prompts • (dynamic) syntax checking • use of colours to highlight key words / pretty printing • Formatting • Single-stepping • Breakpoints • Report / watch window • (UML) modelling • Compiler/interpreter • Text editor	Max 2
3(c)(iv)	Run-time	1

157. O/N 17/P21/Q5/a

5(a)	<pre> PROCEDURE MakeNewfile DECLARE OldFileLine : STRING DECLARE NewFileLine : STRING OPENFILE "EmailDetails" FOR READ OPENFILE "NewEmailDetails" FOR WRITE WHILE NOT EOF("EmailDetails") READFILE "EmailDetails", OldFileLine NewFileLine ← "00" & OldFileLine WRITEFILE "NewEmailDetails", NewFileLine ENDWHILE CLOSEFILE "EmailDetails" CLOSEFILE "NewEmailDetails" ENDPROCEDURE </pre> Mark as follows: 1. Variable declaration of <code>STRING</code> for <code>OldFileLine</code> (or equivalent) 2. Open <code>EmailDetails</code> for <code>READ</code> 3. Open <code>NewEmailDetails</code> for <code>WRITE</code> 4. Correct loop checking for <code>EOF(EmailDetails)</code> 5. Reading a line from <code>EmailDetails</code> in a loop 6. Correct concatenation in a loop 7. Writing a line to <code>NewEmailDetails</code> in a loop Closing both files	8
------	--	---

158. O/N 17/P22/Q1/a(i,ii)

1(a)(i)	<table><tr><th>Data value</th><th>Data type</th></tr><tr><td>FALSE</td><td>BOOLEAN</td></tr><tr><td>03/03/2013</td><td>DATE // DATETIME</td></tr><tr><td>35</td><td>INTEGER</td></tr><tr><td>"INTEGER"</td><td>STRING</td></tr><tr><td>3.5</td><td>REAL</td></tr><tr><td>"35"</td><td>STRING</td></tr></table> One mark for each data type Mark first data type given in each case	Data value	Data type	FALSE	BOOLEAN	03/03/2013	DATE // DATETIME	35	INTEGER	"INTEGER"	STRING	3.5	REAL	"35"	STRING	6
Data value	Data type															
FALSE	BOOLEAN															
03/03/2013	DATE // DATETIME															
35	INTEGER															
"INTEGER"	STRING															
3.5	REAL															
"35"	STRING															
1(a)(ii)	1D Array // 1D List	1														

159. O/N 17/P22/Q2/a

2(a)	Mark as follows: - To search for a given value in the array - and output the position in the array if the search value is found - if search value not found then output "Not found"	Max 2
------	--	--------------

160. O/N 17/P22/Q4/a

4(a)	Pseudocode solution included here for development and clarification of mark scheme. Programming language solutions appear in the Appendix. <pre> PROCEDURE TestRandom (Repetitions : INTEGER) DECLARE Frequency : ARRAY [1 : 10] OF INTEGER DECLARE Expected : REAL / INTEGER //allow either DECLARE NextRandom : INTEGER DECLARE N : INTEGER FOR N ← 1 TO 10 Frequency[N] ← 0 ENDFOR Expected ← INT(Repetitions / 10) CALL RANDOMIZE() //Set random seed FOR N ← 1 TO Repetitions NextRandom ← INT(RND() * 10) + 1 Frequency[NextRandom] ← Frequency[NextRandom] + 1 ENDFOR OUTPUT "The expected frequency is " & Expected OUTPUT "Number Frequency Difference" FOR N ← 1 TO 10 OUTPUT N & " " & Frequency[N] & " " & Frequency[N] - Expected ENDFOR ENDPROCEDURE </pre> Mark as follows: - Procedure heading (including parameter) - Array declaration – 10 or 11 elements - Array declaration – data type - Variable declaration for a loop counter (integer) or expected frequency (integer or real) - Variable declaration for next random value (For Python solutions, mark points 1 to 4 may be gained by suitable comments) - Initialise all elements of array - To set all elements to zero - Calculate expected frequency	16
------	---	-----------

Q4 (a): Visual Basic

```

Dim random As New Random()

Sub TestRandom(ByVal repetitions As Integer)
    Dim randinrange As Integer
    Dim i As Integer
    Dim num(1 To 10) As Integer
    Dim freq As Integer
    Dim difference As Integer

    freq = repetitions / 10    'calculate expected frequency

    For i = 1 To 10            'initialise array to store total frequencies
        num(i) = 0
    Next i

    For i = 1 To repetitions 'generate random numbers & increment
        appropriate freq
        randinrange = random.Next(1, 11)
        num(randinrange) = num(randinrange) + 1
    Next i

    Console.WriteLine("The expected frequency is " & freq)    'report header
    Console.WriteLine("Number    Frequency    Difference")    'column headers

    For i = 1 To 10    'calc & display difference between expected and actual
        freq
        difference = num(i) - freq
        Console.WriteLine(i & "        " & num(i) & "        " & difference)
    Next i

End Sub

```

Other possible ways of calculating a random number in VB include:

```

    randinrange = CInt(Math.Floor((upperbound - lowerbound + 1) * Rnd())) +
lowerbound

    randinrange = math.round((Rnd()*9)+1)

    randinrange = CInt(Math.Ceiling(Rnd() * 9

```

Q4 (a): Pascal

```
procedure TestRandom(var Repetitions : integer);
var
    Frequency : array[1..10] : integer;
    Expected, NextRandom, N : integer;

begin
    Expected := Round(Repetitions/10);
    for N := 1 to 10 do
        Frequency[N] := 0;

    for N := 1 to Repetitions do
        begin
            NextRandom := random(10)+1;
            Frequency[NextRandom] := Frequency[NextRandom]+1;
        end;

    writeln ('The expected frequency is ', Expected);
    writeln ('Number Frequency Difference');

    for N := 1 to 10 do
        writeln ('    ',N,'    ',Frequency[N],'    ',Frequency[N]-
Expected);

    end;
```

Q4 (a): Python

```

# frequency is an array from 1 to 10 of type integer;
# nextNumber is an integer which stores the created random number
# expected is an integer which stores the expected frequency of each number

def TestRandom (repetitions):
    import random
    frequency = [0 for i in range(1,11)] # initialise each frequency count
    to 0

    expected = repetitions / 10

    for i in range(1, repetitions + 1):
        nextNumber = random.randint(1,10)
        frequency[nextNumber] = frequency[nextNumber]+ 1

    print ("The expected frequency is ", expected)
    print("   Number   Frequency   Difference")

    for i in range(1,11):
        print ("       ", i, "       ", frequency[i], "       ", frequency[i] -
expected)

```

Alternative:

```

def TestRandom (repetitions):
    expected = repetitions / 10      ## initialised as real/integer
                                     ## NextRandom and N defined as integers
    frequency =[0,0,0,0,0,0,0,0,0,0,0] ## defined as an array and
    initialised to zero

    for n in range (0,repetitions):
        nextNumber = randint(1, 10)
        frequency[nextNumber] += 1

    print ('The expected frequency is ', expected)

    print ('Number   Frequency   Difference')
    for n in range (1, 11):
        print (n, '       ', frequency[n], '       ', frequency[n] - expected)

```

Alternative:

```

frequency =[0]*11      ## alternate way to initialise array to zero
frequency =[]          ## empty array/list

```

Alternative:

```

for n in range (1,11):
    frequency[n-1] = 0      ##alternate way to initialise array to zero

```

Alternative:

```

for n in range (0,11):  ##alternate way to initialise array to zero
    frequency.append(0)

```

4(a)	9. Loop to generate required number of random values 10. Use of relevant <code>RANDOM()</code> function in a loop 11. Generate random integer value in the range 1 to 10 in a loop 12. Increment (array) element in a loop 13. Output expected frequency message not in any loop 14. Output column header text 15. (Loop to) output each row 16. ... including three correct values (spaces optional)	
------	--	--

161. O/N 17/P22/Q5

5	<pre> PROCEDURE RemoveDetails DECLARE FileLine: STRING DECLARE MemberToDelete: STRING OPENFILE "EmailDetails.txt" FOR READ OPENFILE "NewEmailDetails.txt" FOR WRITE INPUT MembershipNumber WHILE NOT EOF("EmailDetails.txt") READFILE "EmailDetails.txt", FileLine IF LEFT(FileLine, 4) <> MembershipNumber THEN WRITEFILE "NewEmailDetails.txt", FileLine ENDIF ENDWHILE CLOSEFILE "EmailDetails.txt" CLOSEFILE "NewEmailDetails.txt" ENDPROCEDURE </pre> Mark as follows: 1. Procedure declaration and end. No parameters. 2. Variable declaration of <code>STRING</code> for variable <code>FileLine</code> (or similar) 3. Input the <code>MembershipNumber</code> of the person who has left 4. Open <code>EmailDetails</code> for <code>READ</code> 5. Open <code>NewEmailDetails</code> for <code>WRITE</code> 6. Correct loop checking for <code>EOF(EmailDetails)</code> 7. Reading a line from <code>EmailDetails.txt</code> in a loop 8. Correct check for <code>MemberToDelete</code> in a loop 9. Writing a line to <code>NewEmailDetails.txt</code> in a loop 10. Closing both files (not in a loop)	Max 9
---	---	--------------

162. O/N 17/P23/Q3/a,b,c

3(a)	<pre> PROCEDURE BubbleSort DECLARE Temp : STRING DECLARE FirstID, SecondID : INTEGER DECLARE NoSwaps : BOOLEAN DECLARE Boundary : INTEGER Declare J : INTEGER Boundary ← 99 REPEAT NoSwaps ← TRUE FOR J ← 1 TO Boundary FirstID ← UserNameArray[J] SecondID ← UserNameArray[J + 1] IF FirstID > SecondID THEN Temp ← UserNameArray[J] UserNameArray[J] ← UserNameArray[J + 1] UserNameArray[J + 1] ← Temp NoSwaps ← FALSE ENDIF ENDFOR Boundary ← Boundary - 1 UNTIL NoSwaps = TRUE ENDPROCEDURE </pre> Mark as follows: 1. Procedure heading and ending (allow array as input parameter) 2. Variable declaration for counter / index (integer) or temp (string) 3. Outer working loop 4. Inner loop with suitable range 5. Correct comparison in a loop 6. Correct swap of complete array element in a loop 7. Set flag to indicate swap in inner loop and resetting in outer loop 8. Reducing Boundary in a loop	8
------	--	----------

3(b)	Pseudocode solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix. <pre> PROCEDURE FindRepeats DECLARE i, RepeatCount: INTEGER DECLARE FirstID, SecondID: STRING RepeatCount ← 0 FOR i ← 2 TO 100 FirstID ← LEFT(UsernameArray[i - 1], 6) SecondID ← LEFT(UsernameArray[i], 6) IF FirstID = SecondID THEN RepeatCount ← RepeatCount + 1 OUTPUT(UsernameArray[i]) ENDIF ENDFOR IF RepeatCount = 0 THEN OUTPUT "The array contains no repeated UserIDs" ELSE OUTPUT "There are " & RepeatCount & " repeated userIDs" ENDIF ENDPROCEDURE </pre> Mark as follows (all must be correct syntax for chosen language): 1. Procedure heading and ending 2. Variable declaration for INTEGER (comment in Python) and initialisation for RepeatCount (or equivalent name) 3. Loop 4. Extraction of UserID in a loop 5. Correct comparison of consecutive elements... in a loop 6. ...output correct array element (NOT original, only duplicates) in a loop 7. increment RepeatCount following a comparison in a loop 8. Correct conditional statement checking RepeatCount (or equivalent) and then two correct final OUTPUT statements	Max 8
------	---	--------------

Q3(b): Visual Basic

```
Sub FindRepeats()  
    Dim Repeats As Integer  
    Dim i As Integer  
    Dim FirstID As String  
    Dim SecondID As String  
  
    Repeats = 0  
    For i = 1 To 99  
        FirstID = Left(UserNameArray(i), 6)  
        SecondID = Left(UserNameArray(i + 1), 6)  
        If FirstID = SecondID Then  
            Console.WriteLine(UserNameArray(i + 1))  
            Repeats = Repeats + 1  
        End If  
    Next i  
  
    If Repeats = 0 Then  
        Console.WriteLine("The array contains no repeated UserIDs")  
    Else  
        Console.WriteLine("There are " & Repeats & " repeated UserIDs")  
    End If  
  
End Sub
```

Alternative:

```
Sub FindRepeats ()  
  
    Dim RepeatCount, i As Integer  
    Dim FirstID, SecondID As String  
  
    RepeatCount = 0  
    For i = 1 to 99  
        FirstID = Left(UserNameArray(i-1),6)  
        SecondID = Left(UserNameArray(i),6)  
        If FirstID = SecondID then  
            Console.WriteLine (UserNameArray(i))  
            RepeatCount = RepeatCount + 1  
        End If  
    Next i  
  
    If RepeatCount = 0 then  
        Console.WriteLine ("The array contains no repeated UserIDs")  
    Else  
        Console.WriteLine ("There are "& RepeatCount & " repeated UserIDs")  
    End If  
End Sub
```


Q3(b): Pascal

```

procedure FindRepeats ();

var
  RepeatCount, i : integer;
  FirstID, SecondID : string;

begin
  RepeatCount := 0;
  for i := 1 to 99 do
    begin
      FirstID := Copy(UsernameArray[i-1],1,6);
      SecondID := Copy(UsernameArray[i],1,6);
      if FirstID = SecondID then
        begin
          writeln (UsernameArray[i]);
          RepeatCount := RepeatCount + 1;
        end;
      end;

      if RepeatCount = 0 then
        writeln ('The array contains no repeated UserIDs')
      else
        writeln ('There are ', RepeatCount, ' repeated UserIDs')
    end;
  end;

```

Q3(b): Python

```

def FindRepeats():
    #Repeats, i Integer
    #FirstID, SecondID string
    Repeats = 0
    for i in range(0, len(UsernameArray)-1):
        FirstID = (UsernameArray[i])[:6]
        SecondID = (UsernameArray[i+1])[:6]
        if FirstID == SecondID:
            print(UsernameArray[i+1])
            Repeats = Repeats + 1
        if Repeats == 0:
            print("The array contains no repeated UserIDs")
        else:
            print("There are ", Repeats, " repeated UserIDs")

```

Alternative:

```

def FindRepeats ():

    RepeatCount = 0                                ## Defined as an integer

    for i in range (1,100):                        ## depending on next two
lines(0,99) (2,101)
        FirstID = UsernameArray[i-1]              ## Defined as string
        SecondID = UsernameArray[i]               ## Defined as string
        if FirstID[0:6] == SecondID[0:6]:         ## Using split
            print (UsernameArray[i])
            RepeatCount += 1

    if repeatCount == 0:
        print ('The array contains no repeated UserIDs')
    else:
        print ('There are ', RepeatCount, ' repeated UserIDs')

```

3(c)(i)	• Problem definition • Design • Coding / programming • Testing • Documentation • Implementation • Maintenance	3
3(c)(ii)	<u>Integrated Development Environment</u> or a suitable description	1
3(c)(iii)	Examples include: • context sensitive prompts • (dynamic) syntax checking • use of colours to highlight key words / pretty printing • Formatting • Single-stepping • Breakpoints • Report / watch window • (UML) modelling • Compiler/interpreter • Text editor	Max 2
3(c)(iv)	Run-time	1

163. O/N 17/P23/Q5/a

5(a)	<pre> PROCEDURE MakeNewfile DECLARE OldFileLine : STRING DECLARE NewFileLine : STRING OPENFILE "EmailDetails" FOR READ OPENFILE "NewEmailDetails" FOR WRITE WHILE NOT EOF("EmailDetails") READFILE "EmailDetails", OldFileLine NewFileLine ← "00" & OldFileLine WRITEFILE "NewEmailDetails", NewFileLine ENDWHILE CLOSEFILE "EmailDetails" CLOSEFILE "NewEmailDetails" ENDPROCEDURE </pre> Mark as follows: 1. Variable declaration of <code>STRING</code> for <code>OldFileLine</code> (or equivalent) 2. Open <code>EmailDetails</code> for <code>READ</code> 3. Open <code>NewEmailDetails</code> for <code>WRITE</code> 4. Correct loop checking for <code>EOF</code>(<code>EmailDetails</code>) 5. Reading a line from <code>EmailDetails</code> in a loop 6. Correct concatenation in a loop 7. Writing a line to <code>NewEmailDetails</code> in a loop Closing both files	8
------	--	----------

164. M/J 17/P21/Q1/b(i)

1(b)(i)	Boolean	1
---------	---------	----------

165. M/J 17/P21/Q5

5	Pseudocode solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix. <pre> PROCEDURE SearchFile() DECLARE FileData : STRING DECLARE MyArrayRow : INTEGER DECLARE SearchID : STRING MyArrayRow ← 0 / 1 OPEN "Loginfile.txt" FOR READ INPUT SearchID WHILE NOT EOF("Loginfile.txt") READFILE "Loginfile.txt", Filedata IF SearchID = LEFT(FileData,5) THEN LoginEvents[MyArrayRow,1] ← MID(Filedata, 6, 4) LoginEvents[MyArrayRow,2] ← RIGHT(Filedata, 14) MyArrayRow ← MyArrayRow + 1 ENDIF ENDWHILE CLOSEFILE("LoginFile.txt") ENDPROCEDURE </pre> 1 mark for each of the following: 1. Procedure heading and ending 2. Declare MyArrayRow as integer // commented in python 3. Initialising MyArrayRow 4. Input SearchID 5. Open file "LoginFile.txt" for input / read 6. Correct loop incorporating EOF() 7. Read a line from the file in a loop 8. Compare SearchID with correct data from file in a loop 9. Assign both values to LoginEvents[MyArray] in a loop 10. Increment MyArrayRow correctly in a loop 11. Close the file not in a loop	10
---	---	----

Q5 : Visual Basic

```
Sub SearchFile()  
    Dim FileData As String  
    Dim SearchID As String  
    Dim ArrayIndex As Integer  
  
    ArrayIndex = 1  
    FileOpen(1, "LoginFile.txt", OpenMode.Input)  
    SearchID = Console.ReadLine()  
  
    Do While Not EOF(1)  
        FileData = LineInput(1)  
        If SearchID = LEFT(FileData, 5) Then  
            LoginEvents(ArrayIndex, 1) = Mid(Filedata, 6, 4)  
            LoginEvents(ArrayIndex, 2) = Right(Filedata, 14)  
            ArrayIndex = ArrayIndex + 1  
        End If  
    Loop  
    FileClose(1)  
End Sub
```

Alternative:

```
Sub SearchFile()  
    Dim FileData As String  
    Dim SearchID As String  
    Dim ArrayIndex As Integer  
    Dim MyFile As System.IO.StreamReader  
  
    ArrayIndex = 1  
    MyFile = Mycomputer.FileSystem.OpenTextFileReader("Loginfile.txt")  
    SearchID = Console.ReadLine()  
  
    Do While MyFile.Peek < > -1  
        FileData = MyFile.ReadLine()  
        If SearchID = LEFT(FileData, 5) Then  
            LoginEvents(ArrayIndex, 1) = Mid(Filedata, 6, 4)  
            LoginEvents(ArrayIndex, 2) = Right(Filedata, 14)  
            ArrayIndex = ArrayIndex + 1  
        End If  
    Loop  
    MyFile.Close  
End Sub
```

Q5 : Pascal

```
Procedure SearchFile();

var FileData      : String;
var SearchID      : String;
var ArrayRow      : Integer;
Var MyFile        : Text;

Begin
  ArrayRow := 1;
  Assign(MyFile, "Loginfile.txt");
  Reset(MyFile);
  Readln(SearchID);

  While NOT EOF(MyFile) do
  Begin
    Readln(MyFile, FileData)
    IF SearchID = LeftStr(FileData,5) then
      Begin
        LoginEvents[ArrayRow,1] = Copy(FileData,6,4);
        LoginEvents[ArrayRow,2] = Rightstr(FileData,14);
        ArrayRow = ArrayRow + 1
      End;
    End;

  Close(MyFile);
End.
```

Q5 : Python

```
def SearchFile():

    # FileData : STRING
    # ArrayRow : INTEGER
    # SearchID : STRING

    ArrayRow = 0
    MyFile = open("Loginfile.txt", 'r')
    SearchID = input()
    FileData = MyFile.readline()

    While FileData != ""
        If SearchID == FileData[:5]                #First 5 characters
            LoginEvents[ArrayRow][1] = FileData[5:9] #next 4 characters
            LoginEvents[ArrayRow][2] = FileData[-14:] #last 14 characters
            ArrayRow = ArrayRow + 1
            FileData = MyFile.readline()

    myFile.close()
    return()
```

Alternative:

```
def SearchFile():

    # FileData : STRING
    # ArrayRow : INTEGER
    # SearchID : STRING

    ArrayRow = 0
    Myfile = open("Loginfile.txt", 'r')
    SearchID = input()
    For FileData in MyFile
        IF SearchID == FileData[:5]                #First 5 characters
            LoginEvents[ArrayRow][1] = FileData[5:9] #next 4 characters
            LoginEvents[ArrayRow][2] = FileData[-14:] #last 14 characters
            ArrayRow = ArrayRow + 1

    MyFile.close()
    return()
```

166. M/J 17/P22/Q5/a,b

5(a)(i)	- Any character <u>except</u> colon, space or any alpha-numeric - Reason: character is not in the login information strings	2
5(a)(ii)	DECLARE <u>LogArray</u> : ARRAY[1 : 20] OF <u>STRING</u> 1 mark per underline	2

5(b)	Pseudocode solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix. <pre> PROCEDURE LogEvents() DECLARE FileData : STRING DECLARE ArrayIndex : INTEGER OPENFILE "LoginFile.txt" FOR APPEND FOR ArrayIndex ← 1 TO 20 // IF LogArray[ArrayIndex] <> "*****" THEN FileData ← LogArray[ArrayIndex] WRITEFILE ("LoginFile.txt", FileData) ENDIF ENDFOR CLOSEFILE ("LoginFile.txt") ENDPROCEDURE </pre> 1 mark for each of the following: 1. Procedure heading and ending 2. Declare <code>ArrayIndex</code> as integer // commented in python 3. Open file 'LoginFile' for append 4. Correct loop 5. extract data from array in a loop 6. check for unused element in a loop 7. write data to file in a loop 8. Close the file outside the loop	8
------	---	----------

167. M/J 17/P23/Q1/b(i)

1(b)(i)	Boolean	1
---------	---------	----------

168. M/J 17/P23/Q5

5	Pseudocode solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix. <pre> PROCEDURE SearchFile() DECLARE FileData : STRING DECLARE MyArrayRow : INTEGER DECLARE SearchID : STRING MyArrayRow ← 0 / 1 OPEN "Loginfile.txt" FOR READ INPUT SearchID WHILE NOT EOF("Loginfile.txt") READFILE "Loginfile.txt", Filedata IF SearchID = LEFT(FileData,5) THEN LoginEvents[MyArrayRow,1] ← MID(Filedata, 6, 4) LoginEvents[MyArrayRow,2] ← RIGHT(Filedata, 14) MyArrayRow ← MyArrayRow + 1 ENDIF ENDWHILE CLOSEFILE("LoginFile.txt") ENDPROCEDURE </pre> 1 mark for each of the following: 1. Procedure heading and ending 2. Declare MyArrayRow as integer // commented in python 3. Initialising MyArrayRow 4. Input SearchID 5. Open file "LoginFile.txt" for input / read 6. Correct loop incorporating EOF() 7. Read a line from the file in a loop 8. Compare SearchID with correct data from file in a loop 9. Assign both values to LoginEvents[MyArray] in a loop 10. Increment MyArrayRow correctly in a loop 11. Close the file not in a loop	10
---	---	----

Q5 : Visual Basic

```
Sub SearchFile()  
    Dim FileData As String  
    Dim SearchID As String  
    Dim ArrayIndex As Integer  
  
    ArrayIndex = 1  
    FileOpen(1, "LoginFile.txt", OpenMode.Input)  
    SearchID = Console.ReadLine()  
  
    Do While Not EOF(1)  
        FileData = LineInput(1)  
        If SearchID = LEFT(FileData, 5) Then  
            LoginEvents(ArrayIndex, 1) = Mid(Filedata, 6, 4)  
            LoginEvents(ArrayIndex, 2) = Right(Filedata, 14)  
            ArrayIndex = ArrayIndex + 1  
        End If  
    Loop  
    FileClose(1)  
End Sub
```

Alternative:

```
Sub SearchFile()  
    Dim FileData As String  
    Dim SearchID As String  
    Dim ArrayIndex As Integer  
    Dim MyFile As System.IO.StreamReader  
  
    ArrayIndex = 1  
    MyFile = Mycomputer.FileSystem.OpenTextFileReader("Loginfile.txt")  
    SearchID = Console.ReadLine()  
  
    Do While MyFile.Peek < > -1  
        FileData = MyFile.ReadLine()  
        If SearchID = LEFT(FileData, 5) Then  
            LoginEvents(ArrayIndex, 1) = Mid(Filedata, 6, 4)  
            LoginEvents(ArrayIndex, 2) = Right(Filedata, 14)  
            ArrayIndex = ArrayIndex + 1  
        End If  
    Loop  
    MyFile.Close  
End Sub
```

Q5 : Pascal

```
Procedure SearchFile();

var FileData      : String;
var SearchID      : String;
var ArrayRow      : Integer;
Var MyFile        : Text;

Begin
  ArrayRow := 1;
  Assign(MyFile, "Loginfile.txt");
  Reset(MyFile);
  Readln(SearchID);

  While NOT EOF(MyFile) do
  Begin
    Readln(MyFile, FileData)
    IF SearchID = LeftStr(FileData,5) then
      Begin
        LoginEvents[ArrayRow,1] = Copy(FileData,6,4);
        LoginEvents[ArrayRow,2] = Rightstr(FileData,14);
        ArrayRow = ArrayRow + 1
      End;
    End;

    Close(MyFile);
  End.
```

Q5 : Python

```
def SearchFile():

    # FileData : STRING
    # ArrayRow : INTEGER
    # SearchID : STRING

    ArrayRow = 0
    MyFile = open("Loginfile.txt", 'r')
    SearchID = input()
    FileData = MyFile.readline()

    While FileData != ""
        If SearchID == FileData[:5]                #First 5 characters
            LoginEvents[ArrayRow][1] = FileData[5:9]    #next 4 characters
            LoginEvents[ArrayRow][2] = FileData[-14:]    #last 14 characters
            ArrayRow = ArrayRow + 1
            FileData = MyFile.readline()

    myFile.close()
    return()
```

Alternative:

```
def SearchFile():

    # FileData : STRING
    # ArrayRow : INTEGER
    # SearchID : STRING

    ArrayRow = 0
    Myfile = open("Loginfile.txt", 'r')
    SearchID = input()
    For FileData in MyFile
        IF SearchID == FileData[:5]                #First 5 characters
            LoginEvents[ArrayRow][1] = FileData[5:9]    #next 4 characters
            LoginEvents[ArrayRow][2] = FileData[-14:]    #last 14 characters
            ArrayRow = ArrayRow + 1

    MyFile.close()
    return()
```

169. O/N 16/P21/Q4/c,d,e

(c) (i) OPEN "PRODUCTS" FOR READ

i ← 1

WHILE NOT EOF("PRODUCTS")

 READFILE ("PRODUCTS", PCode[i])

 READFILE ("PRODUCTS", PDescription[i])

 READFILE ("PRODUCTS", Temp // PRetailPrice[i])

 PRetailPrice[i] ← TONUM(Temp)

 i ← i + 1

ENDWHILE

CLOSE "PRODUCTS"

OUTPUT "Product file contents written to arrays"

One mark per bold phrase (three READFILE() counts as a single mark)

[5]

(ii) Benefit:

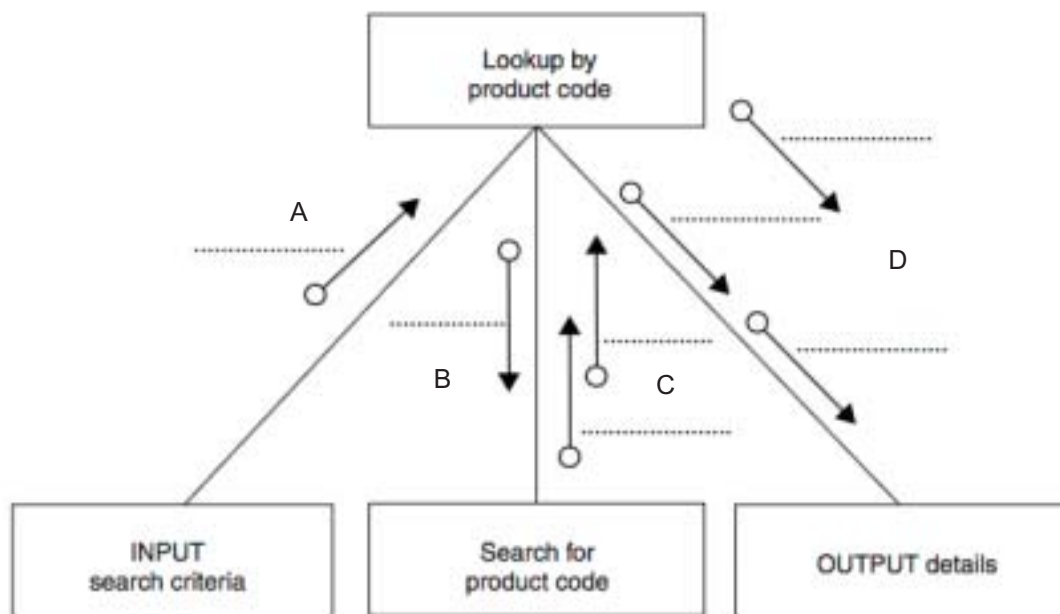
- The number of file read operations is reduced (by 2/3rds)
- It may use less storage / space in the file if strings are NOT fixed length
- All the data related to a single product is read at once / in one file operation / grouped together

Drawback:

- The program will need to use the string handling functions to isolate each of the three items of data
- Difficult to isolate data items if the format is not consistent
- More difficult to search

Max one benefit and one drawback

[2]

(d)

One mark per group (one or more names) as follows:

- A: SearchCode
- B: SearchCode // ThisIndex
- C: ThisRetailPrice, ThisDescription
- D: SearchCode, ThisDescription, ThisRetailPrice

[4]

- (e) 'Pseudocode' solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix.

```

FUNCTION ProductCodeSearch(AnyName : String) RETURNS : Integer
    DECLARE FoundPos : Integer
    DECLARE i : Integer

    i ← 1
    FoundPos ← -1

    REPEAT
        IF AnyName = PCode[i]
            THEN
                FoundPos ← i
            ELSE
                i ← i + 1
            ENDIF
    UNTIL (i = 1001) OR (FoundPos <> -1)

    RETURN FoundPos

ENDFUNCTION

```

Mark as follows:

- Function header returns INTEGER
- Initialisation of index variable
- Loop through array PCode (including exit when found)
- Comparison of AnyName with PCode[i] in a loop
- Increment index variable in a loop
- Return index if AnyName found AND return -1 if AnyName not found

[Max 6]

4(e): Visual Basic

```

Function ProductCodeSearch(ByVal SearchCode As String) As Integer
    Dim FoundCode As Integer
    Dim i As Integer

    i = 1
    FoundCode = -1

    Do
        If SearchCode = PCode(i) Then
            FoundCode = i
        Else
            i = i + 1
        End If
    Loop Until i = 1001 Or FoundCode <> -1
    Return FoundCode
End Function

```

4(e): Pascal

```
Function ProductCodeSearch (SearchCode : String): integer;

    var FoundCode, ThisIndex : integer;
        Found : Boolean;

Begin
    Found := false;
    ThisIndex := 1;
    Repeat
        If SearchCode = PCode[ThisIndex] then
            Begin
                FoundCode := ThisIndex;
                Found := true;
            End
            Else
                ThisIndex := ThisIndex + 1;
        end;
    Until (ThisIndex = 1001) OR (Found);
    If Found = false then
        FoundCode := -1
    ProductCodeSearch := FoundCode;
end.
```

4(e): Python

```
def ProductCodeSearch(SearchCode):
    # list indexes start at zero
    i = 0
    Found = "no"
    while not(i == 1001 or Found == "yes"):
        if SearchCode == PCode[i]:
            Found = "yes"
            FoundIndex = i
        else:
            i = i + 1

    if Found == "no":
        FoundIndex = -1

    return FoundIndex
```

170. O/N 16/P22/Q5/a,b(ii,iii)

- 5 (a)**
- 2D array
 - of type integer
 - with identifier PlayerScore

[Max. 2]

- (ii) 'Pseudocode' solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix.

```

DECLARE ThisPlayerName : STRING
DECLARE PlayerName : ARRAY[1:8) OF STRING
DECLARE i : INTEGER

OPENFILE "NAMES.TXT" FOR READ
i ← 1

WHILE NOT EOF("NAMES.TXT")
    READFILE "NAMES.TXT", ThisPlayerName
    PlayerName[i] ← ThisPlayerName
    i ← i + 1
ENDWHILE

CLOSEFILE "NAMES.TXT"

```

One mark for each of:

- File open in read mode
 - Loop until EOF() or count-controlled (8 iterations)
 - Read a line from the file in a loop
 - Assignment to PlayerName[1 to 8] from the file in a loop
- Close file

[Max. 4]

Q5 (b) (ii): Visual Basic

```

Dim PlayerName(8) As String
Dim i As Integer
FileOpen(1, "Names.txt", OpenMode.Input)
i = 1
Do
    PlayerName(i) = LineInput(1)
    i = i + 1
Loop Until EOF(1)
FileClose(1)

```

Alternative:

```

Dim PlayerName(8) As String
Dim i As Integer
FileOpen(1, "Names.txt", OpenMode.Input)
For i = 1 To 8
    PlayerName(i) = LineInput(1)
Next
FileClose(1)

```

Alternative:

```

Dim sr As StreamReader = New StreamReader("Names.txt")
Dim line As String
line = sr.ReadLine()
i = 1
Do While (line <> Nothing)
    PlayerName(i) = line
    i = i + 1
    line = sr.ReadLine()
Loop
sr.Close()

```

Q5 (b) (ii): Pascal

```
Var Names : TextFile;
    i : Integer;
    PlayerName : Array[1..8] Of String;
Begin
    AssignFile(Names, 'Names.txt');
    Reset(Names);
    i := 1;
    While Not Eof(Names) Do
        Begin
            Readln(Names, PlayerName[i]);
            Writeln(PlayerName[i]);
            i := i + 1;
        End;
    Close(Names);
    Readln;
End.
```

Alternative:

```
Var Names : TextFile;
    i : Integer;
    PlayerName : Array[1..8] Of String;
Begin
    AssignFile(Names, 'Names.txt');
    Reset(Names);
    For i := 1 To 8 Do
        Begin
            Readln(Names, PlayerName[i]);
            Writeln(PlayerName[i]);
        End;
    Close(Names);
    Readln;
End.
```

Q5 (b) (ii): Python

```
# PlayerName : List
# NextPlayer : String
# File : File handle
File = open("Names.txt", "r")
PlayerName = []
while (1) :
    NextPlayer = File.readline()
    if not NextPlayer :
        break
    else :
        PlayerName.append(NextPlayer)
File.close()
```

Alternative:

```
# PlayerName : List
# NextPlayer : String
# File : File handle
# i : Integer
File = open("Names.txt", "r")
PlayerName = []
for i in range(1, 9) :
    NextPlayer = File.readline()
    PlayerName.append(NextPlayer)
File.close()
```

Alternative:

```
# PlayerName : List
# NextPlayer : String
# File : File handle
# i : Integer
File = open("Names.txt", "r")
PlayerName = [" " for i in range(8)]
for i in range(1, 9) :
    PlayerName[i - 1] = File.readline()
File.close()
```

- (iii) 'Pseudocode' solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix.

```
// search for player name ....
Found ← FALSE
i ← 1

REPEAT
  IF ThisPlayerName = PlayerName[i]
    THEN
      Found ← TRUE
      PlayerNumber ← i
    ELSE
      i ← i + 1
    ENDIF
UNTIL (Found = TRUE) OR (i = 9)
```

One mark for each of:

- Initialise `i` to 1 and `Found` to `FALSE`
- Loop through array `PlayerName` (including exit when found)
- Comparison: `ThisPlayerName = PlayerName[i]` in a loop
- `Found` set to `TRUE` if `ThisPlayerName` found

[Max. 4]

Q5 (b) (iii): Visual Basic

```
Found = False
i = 1

Do
    If ThisPlayerName = PlayerName(i) Then
        Found = True
        PlayerNumber = i
    Else
        i = i + 1
    End If
Loop Until Found = True Or i = 9
```

Q5 (b) (iii): Pascal

```
Begin
Found := False;
i := 1;
Repeat
    If (ThisPlayerName = PlayerName[i]) Then
        Begin
            Found := True;
            PlayerNumber := i;
        End
    Else
        i := i + 1;
    Until (Found) Or (i = 9);
End.
```

Q5 (b) (iii): Python

```
Found = False
PlayerName = [j.strip() for j in PlayerName]
if ThisPlayerName in PlayerName :
    PlayerNumber = PlayerName.index(ThisPlayerName) + 1
    Found = True
```

Alternative:

```
Found = False
i = 1
while not Found and i < 9 :
    if ThisPlayerName == PlayerName[i].strip() :
        Found = True
        PlayerNumber = i
    else :
        i = i + 1
```

Alternative:

```
Found = False
for i in range(1, 9) :
    if ThisPlayerName == PlayerName[i].strip() :
        Found = True
        PlayerNumber = i
```

171. O/N 16/P23/Q4/c,d,e

(c) (i) OPEN "PRODUCTS" FOR READ

i ← 1

WHILE NOT EOF("PRODUCTS")

 READFILE ("PRODUCTS", PCode[i])

 READFILE ("PRODUCTS", PDescription[i])

 READFILE ("PRODUCTS", Temp // PRetailPrice[i])

 PRetailPrice[i] ← TONUM(Temp)

 i ← i + 1

ENDWHILE

CLOSE "PRODUCTS"

OUTPUT "Product file contents written to arrays"

One mark per bold phrase (three READFILE() counts as a single mark)

[5]

(ii) Benefit:

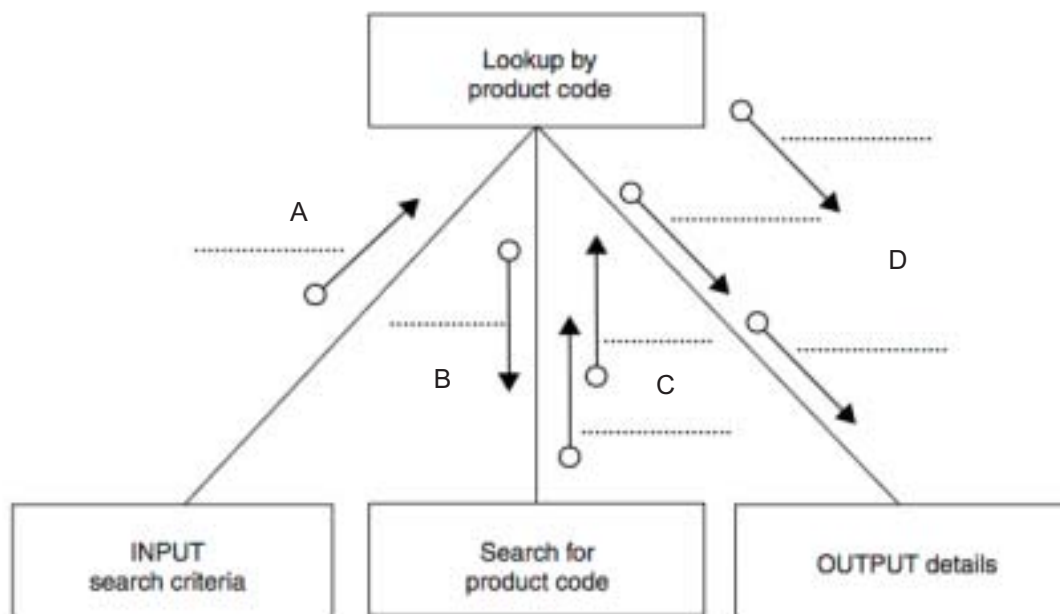
- The number of file read operations is reduced (by 2/3rds)
- It may use less storage / space in the file if strings are NOT fixed length
- All the data related to a single product is read at once / in one file operation / grouped together

Drawback:

- The program will need to use the string handling functions to isolate each of the three items of data
- Difficult to isolate data items if the format is not consistent
- More difficult to search

Max one benefit and one drawback

[2]

(d)

One mark per group (one or more names) as follows:

- A: SearchCode
- B: SearchCode // ThisIndex
- C: ThisRetailPrice, ThisDescription
- D: SearchCode, ThisDescription, ThisRetailPrice

[4]

- (e) 'Pseudocode' solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix.

```

FUNCTION ProductCodeSearch(AnyName : String) RETURNS : Integer
    DECLARE FoundPos : Integer
    DECLARE i : Integer

    i ← 1
    FoundPos ← -1

    REPEAT
        IF AnyName = PCode[i]
            THEN
                FoundPos ← i
            ELSE
                i ← i + 1
            ENDIF
    UNTIL (i = 1001) OR (FoundPos <> -1)

    RETURN FoundPos

ENDFUNCTION

```

Mark as follows:

- Function header returns INTEGER
- Initialisation of index variable
- Loop through array PCode (including exit when found)
- Comparison of AnyName with PCode[i] in a loop
- Increment index variable in a loop
- Return index if AnyName found AND return -1 if AnyName not found

[Max 6]

4(e): Visual Basic

```

Function ProductCodeSearch(ByVal SearchCode As String) As Integer
    Dim FoundCode As Integer
    Dim i As Integer

    i = 1
    FoundCode = -1

    Do
        If SearchCode = PCode(i) Then
            FoundCode = i
        Else
            i = i + 1
        End If
    Loop Until i = 1001 Or FoundCode <> -1
    Return FoundCode
End Function

```


4(e): Pascal

```
Function ProductCodeSearch (SearchCode : String): integer;

    var FoundCode, ThisIndex : integer;
        Found : Boolean;

Begin
    Found := false;
    ThisIndex := 1;
    Repeat
        If SearchCode = PCode[ThisIndex] then
            Begin
                FoundCode := ThisIndex;
                Found := true;
            End
            Else
                ThisIndex := ThisIndex + 1;
        end;
    Until (ThisIndex = 1001) OR (Found);
    If Found = false then
        FoundCode := -1
    ProductCodeSearch := FoundCode;
end.
```

4(e): Python

```
def ProductCodeSearch(SearchCode):
    # list indexes start at zero
    i = 0
    Found = "no"
    while not(i == 1001 or Found == "yes"):
        if SearchCode == PCode[i]:
            Found = "yes"
            FoundIndex = i
        else:
            i = i + 1

    if Found == "no":
        FoundIndex = -1

    return FoundIndex
```

172. M/J 16/P21/Q3/a,b(i)

3 (a)	<pre> FUNCTION EncryptString (<u>LookUp : ARRAY, PlainText : STRING</u>) RETURNS STRING DECLARE <u>OldChar, NewChar</u> : CHAR DECLARE <u>OldCharValue</u> : <u>INTEGER</u> DECLARE OutString: STRING //first initialise the return string <u>OutString</u> ← "" //initialise the return string //loop through PlainText to produce OutString FOR n ← 1 to <u>LENGTH(PlainText)</u> //from first to last character OldChar ← <u>MID(PlainText, n, 1)</u> //get next character OldCharValue ← <u>ASC(OldChar)</u> //find the ASCII value NewChar ← <u>LookUp[OldCharValue]</u> //look up substitute character <u>OutString</u> ← <u>Outstring & NewChar</u> // concatenate to OutString ENDFOR <u>RETURN OutString</u> // <u>EncryptString</u> ← OutString ENDFUNCTION One mark for each part-statement (shown underlined and bold) </pre>	10
(b) (i)	<pre> VB: <u>Dim LookUp(0 to 127 / 128) As CHAR</u> Pascal: <u>Var LookUp: Array[0..127 / 1..128] Of CHAR</u> Python: LookUp = ["" for i in range(128)] OR LookUp = [] For i in range(128) : LookUp.append("") </pre> Mark as follows: VB / Pascal: one mark per part-statement as underlined and bold Python: One mark for LookUp and [] One mark for range(128)	2

173. M/J 16/P21/Q5/a,b

5 (a) (i)	• So that the data / information is saved after the program is run / when the computer is switched off • So the data / information can be accessed next time the program is run • So the data information can be "permanently stored"	Max 1
-----------	---	-------

(ii)	Problem: • When retrieving / searching for / editing (text relating to a particular CD) • Can't tell where the artist name stops and the title begins (or any similar explanation or example) Solution 1: • Use of a separator character// or by example • Where the separator character does not occur in the original strings Solution 2: • Use a fixed number of characters for each data item • Data items are padded with e.g. <Space> character where needed Solution 3: • Convert original data items to CamelCase • ...and add a Space separator Mark as follows: Two marks for description of problem Two marks for description of solution	4
------	---	----------

(b)	'Pseudocode' solution included here for development and clarification of mark scheme. Programming language solutions appear in the Appendix. <pre> PROCEDURE InputData() DECLARE CDTitle : STRING DECLARE CDArtist : STRING DECLARE CDLocation : STRING DECLARE FileData : STRING OPENFILE "MyMusic" FOR WRITE OUTPUT "Input CD Title" INPUT CDTitle WHILE CDTitle <> "##" OUPUT "Input CD Artist" INPUT CDArtist OUPUT "Input CD Location" INPUT CDLocation FileData = CDTitle & ':' & CDArtist & ':' & CDLocation WRITEFILE "MyMusic.txt", FileData OUTPUT "Input CD Title" INPUT CDTitle ENDWHILE CLOSEFILE("MyMusic.txt ") ENDPROCEDURE </pre> One mark for each of the following: • Procedure heading and ending • Declaration of CDTitle, CDArtist and CDLocation • Open file for writing (Allow MyMusic or MyMusic.txt) • Working conditional loop structure including test for rogue value (including initial input of CDTitle) • Input of three data values (CDTitle, CDArtist and CDLocation) inside a loop • String concatenation of three variables inside a loop • Write three variables in single line to file inside a loop • Close file • Use of string separator Solutions may repeatedly OPEN – WRITE – CLOSE within the loop. In this case the first OPEN could be in WRITE or APPEND mode with all others in APPEND.	Max 8

5 (b): VB.NET**A StreamWriter() solution:**

```
Sub InputData()  
    Dim CDTitle, CDArtist, CDLocation As String  
    Dim FileHandle As IO.StreamWriter  
    FileHandle = New IO.StreamWriter("MyMusic.txt") ("MyMusic.txt")  
    Console.WriteLine("Input CD Title: ")  
    CDTitle = Console.ReadLine()  
    Do Until CDTitle = "###"  
        Console.WriteLine("Input CD Artist: ")  
        CDArtist = Console.ReadLine()  
        Console.WriteLine("Input CD Location: ")  
        CDLocation = Console.ReadLine()  
        FileHandle.WriteLine(CDTitle & ":" & CDArtist & ":" & CDLocation)  
        Console.WriteLine("Input CD Title: ")  
        CDTitle = Console.ReadLine()  
    Loop  
    FileHandle.Close()  
End Sub
```

A legacy FileOpen() solution:

```
Sub InputData()  
    Dim CDTitle, CDArtist, CDLocation As String  
    FileOpen(1, "MyMusic", OpenMode.Output)  
    Console.WriteLine("Input CD Title: ")  
    CDTitle = Console.ReadLine()  
    Do Until CDTitle = "###"  
        Console.WriteLine("Input CD Artist: ")  
        CDArtist = Console.ReadLine()  
        Console.WriteLine("Input CD Location: ")  
        CDLocation = Console.ReadLine()  
        Print(1, CDTitle & ":" & CDArtist & ":" & CDLocation)  
        Console.WriteLine("Input CD Title: ")  
        CDTitle = Console.ReadLine()  
    Loop  
    FileClose(1)  
End Sub
```

5 (b): Pascal

```
procedure InputData;
var
    CDTitle, CDArtist, CDLocation : string;
    CDFile : Textfile;
begin
    assign(CDFile, 'MyMusic');
    rewrite(CDFile);
    writeln('Input CD Title: ');
    readln(CDTitle);
    while (CDTitle <> '##') do
    begin
        writeln('Input CD Artist: ');
        readln(CDArtist);
        writeln('Input CD Location: ');
        readln(CDLocation);
        writeln(CDFile, CDTitle + ':' + CDArtist + ':' + CDLocation);
        writeln('Input CD Title: ');
        readln(CDTitle);
    end;
    close(CDFile);
end;
```

5 (b): Python

```
def InputData() :
    #CDTitle String (or CDTitle = "")
    #CDArtist String (or CDArtist = "")
    #CDLocation String (or CDLocation = "")

    FileHandle = open("MyMusic", "w")
    CDTitle = input("Input CD Title: ")
    while CDTitle != "##" :
        CDArtist = input("Input CD Artist: ")
        CDLocation = input("Input CD location: ")
        FileHandle.write(CDTitle + ":" + CDArtist + ":" + CDLocation)
        CDTitle = input("Input CD Title: ")
    FileHandle.close()
```

174. M/J 16/P22/Q3/a,b(i)

3 (a)	<pre> FUNCTION EncryptString (<u>LookUp : ARRAY, PlainText : STRING</u>) RETURNS STRING DECLARE <u>OldChar, NewChar</u> : CHAR DECLARE <u>OldCharValue</u> : <u>INTEGER</u> DECLARE OutString: STRING //first initialise the return string <u>OutString</u> ← "" //initialise the return string //loop through PlainText to produce OutString FOR n ← 1 to <u>LENGTH(PlainText)</u> //from first to last character OldChar ← <u>MID(PlainText, n, 1)</u> //get next character OldCharValue ← <u>ASC(OldChar)</u> //find the ASCII value NewChar ← <u>LookUp[OldCharValue]</u> //look up substitute character <u>OutString</u> ← <u>Outstring & NewChar</u> // concatenate to OutString ENDFOR <u>RETURN OutString</u> // <u>EncryptString</u> ← <u>OutString</u> ENDFUNCTION One mark for each part-statement (shown underlined and bold) </pre>	10
(b) (i)	<pre> VB: <u>Dim LookUp(0 to 127 / 128) As CHAR</u> Pascal: <u>Var LookUp: Array[0..127 / 1..128] Of CHAR</u> Python: LookUp = ["" for i in range(128)] OR LookUp = [] For i in range(128) : LookUp.append("") Mark as follows: VB / Pascal: one mark per part-statement as underlined and bold Python: One mark for LookUp and [] One mark for range(128) </pre>	2

175. M/J 16/P22/Q5/a,b

5 (a) (i)	• So that the data / information is saved after the program is run / when the computer is switched off • So the data / information can be accessed next time the program is run • So the data information can be "permanently stored"	Max 1
------------------	---	--------------

(ii)	Problem: • When retrieving / searching for / editing (text relating to a particular CD) • Can't tell where the artist name stops and the title begins (or any similar explanation or example) Solution 1: • Use of a separator character// or by example • Where the separator character does not occur in the original strings Solution 2: • Use a fixed number of characters for each data item • Data items are padded with e.g. <Space> character where needed Solution 3: • Convert original data items to CamelCase • ...and add a Space separator Mark as follows: Two marks for description of problem Two marks for description of solution	4
------	---	----------

(b)	'Pseudocode' solution included here for development and clarification of mark scheme. Programming language solutions appear in the Appendix. <pre> PROCEDURE InputData() DECLARE CDTitle : STRING DECLARE CDArtist : STRING DECLARE CDLocation : STRING DECLARE FileData : STRING OPENFILE "MyMusic" FOR WRITE OUTPUT "Input CD Title" INPUT CDTitle WHILE CDTitle <> "##" OUPUT "Input CD Artist" INPUT CDArtist OUPUT "Input CD Location" INPUT CDLocation FileData = CDTitle & ':' & CDArtist & ':' & CDLocation WRITEFILE "MyMusic.txt", FileData OUTPUT "Input CD Title" INPUT CDTitle ENDWHILE CLOSEFILE("MyMusic.txt ") ENDPROCEDURE </pre> One mark for each of the following: • Procedure heading and ending • Declaration of CDTitle, CDArtist and CDLocation • Open file for writing (Allow MyMusic or MyMusic.txt) • Working conditional loop structure including test for rogue value (including initial input of CDTitle) • Input of three data values (CDTitle, CDArtist and CDLocation) inside a loop • String concatenation of three variables inside a loop • Write three variables in single line to file inside a loop • Close file • Use of string separator Solutions may repeatedly OPEN – WRITE – CLOSE within the loop. In this case the first OPEN could be in WRITE or APPEND mode with all others in APPEND.	Max 8

5 (b): VB.NET**A StreamWriter() solution:**

```
Sub InputData()  
    Dim CDTitle, CDArtist, CDLocation As String  
    Dim FileHandle As IO.StreamWriter  
    FileHandle = New IO.StreamWriter("MyMusic.txt") ("MyMusic.txt")  
    Console.WriteLine("Input CD Title: ")  
    CDTitle = Console.ReadLine()  
    Do Until CDTitle = "###"  
        Console.WriteLine("Input CD Artist: ")  
        CDArtist = Console.ReadLine()  
        Console.WriteLine("Input CD Location: ")  
        CDLocation = Console.ReadLine()  
        FileHandle.WriteLine(CDTitle & ":" & CDArtist & ":" & CDLocation)  
        Console.WriteLine("Input CD Title: ")  
        CDTitle = Console.ReadLine()  
    Loop  
    FileHandle.Close()  
End Sub
```

A legacy FileOpen() solution:

```
Sub InputData()  
    Dim CDTitle, CDArtist, CDLocation As String  
    FileOpen(1, "MyMusic", OpenMode.Output)  
    Console.WriteLine("Input CD Title: ")  
    CDTitle = Console.ReadLine()  
    Do Until CDTitle = "###"  
        Console.WriteLine("Input CD Artist: ")  
        CDArtist = Console.ReadLine()  
        Console.WriteLine("Input CD Location: ")  
        CDLocation = Console.ReadLine()  
        Print(1, CDTitle & ":" & CDArtist & ":" & CDLocation)  
        Console.WriteLine("Input CD Title: ")  
        CDTitle = Console.ReadLine()  
    Loop  
    FileClose(1)  
End Sub
```

5 (b): Pascal

```
procedure InputData;
var
    CDTitle, CDArtist, CDLocation : string;
    CDFile : Textfile;
begin
    assign(CDFile, 'MyMusic');
    rewrite(CDFile);
    writeln('Input CD Title: ');
    readln(CDTitle);
    while (CDTitle <> '##') do
    begin
        writeln('Input CD Artist: ');
        readln(CDArtist);
        writeln('Input CD Location: ');
        readln(CDLocation);
        writeln(CDFile, CDTitle + ':' + CDArtist + ':' + CDLocation);
        writeln('Input CD Title: ');
        readln(CDTitle);
    end;
    close(CDFile);
end;
```

5 (b): Python

```
def InputData() :
    #CDTitle String (or CDTitle = "")
    #CDArtist String (or CDArtist = "")
    #CDLocation String (or CDLocation = "")

    FileHandle = open("MyMusic", "w")
    CDTitle = input("Input CD Title: ")
    while CDTitle != "##" :
        CDArtist = input("Input CD Artist: ")
        CDLocation = input("Input CD location: ")
        FileHandle.write(CDTitle + ":" + CDArtist + ":" + CDLocation)
        CDTitle = input("Input CD Title: ")
    FileHandle.close()
```

176. M/J 16/P23/Q3/a**3 (a)****[11]**

```

FUNCTION Decrypt (Lookup : ARRAY, CipherChar : CHAR) RETURNS CHAR
  DECLARE Found           : BOOLEAN
  DECLARE Index          : INTEGER
  DECLARE OriginalChar    : CHAR

  Index ← 1                //start with first element in the array //
                           assign the start index

  Found ← FALSE
  //now search for CipherChar in Lookup:
  WHILE Found = FALSE // Found <> TRUE // NOT Found
    // compare CipherChar with this array element:
    IF Lookup[Index] = CipherChar
      THEN
        Found ← TRUE           // Set the flag
      ELSE
        Index ← Index + 1      //Move to next array element
      ENDIF
    ENDWHILE
  //dropped out of loop so must have found CipherChar:
  OriginalChar ← CHR(Index)    // convert Index to original character
  RETURN OriginalChar

ENDFUNCTION

```

One mark for each part-statement (shown underlined and bold)

177. M/J 16/P23/Q5/a,b**5 (a) (i) Explanation:****[Max 3]**

- Easier to separate the two strings // to retrieve / search for / edit (text relating to a CD)
- Obvious where CDTitle ends and CDArtist begins

Drawback:

- Takes up more / unnecessary space in the file
- If the string is bigger than 40 characters then data will be lost // string length is limited
- The additional spaces will need to be removed before strings can be used

One mark per bullet

(ii) Problem: File mode = WRITE / file is opened for writing // by explanation

[3]

Effect: All existing file lines / contents / data will be overwritten / deleted / lost

Solution: WRITE should be changed to APPEND (allow meaningful example)

Allow first two mark points to be interchanged – read as one paragraph.

- (b) 'Pseudocode' solution included here for development and clarification of mark scheme.
Programming language example solutions appear in the Appendix.

[Max 10]

```
PROCEDURE OutputLocationList()

    DECLARE CDTitle : STRING
    DECLARE CDArtist : STRING
    DECLARE CDSearch : STRING
    DECLARE FileData : STRING
    DECLARE Total : INTEGER
    DECLARE FileData : STRING

    Total ← 0

    OPENFILE "MyMusic" FOR READ

    OUTPUT "Input Location"
    INPUT CDSearch

    WHILE NOT EOF("MyMusic")
        READFILE ("MyMusic", FileData
        IF RIGHT(FileData, 8) = CDSearch
            THEN
                CDTitle ← LEFT(FileData, 40)
                CDArtist ← MID(FileData(41, 40)
                OUPUT (CDTitle & " - " & CDArtist)
                Total ← Total + 1
            ENDIF
        ENDWHILE

    OUTPUT (Total & " CDs found"
    CLOSEFILE ("MyMusic")

ENDPROCEDURE
```

One mark for each of the following:

- Procedure heading and ending
- Declaration AND initialisation of variable used as counter (Total above)
- Prompt and input of location to search for CDSearch (or other name)
- Open file for reading (Allow MyMusic or MyMusic.txt)
- Working conditional loop structure including test for EOF

The following points must be present inside a loop

- Read a line from the file (or read complete file in one e.g. as list)
- Isolate 8 chars representing location **and** compare with CDLocation
- Extract strings representing CDTitle and CDArtist
- Output CDTitle and CDArtist (separator optional) if correct location found
- Increment Total if correct location found

The following points must be present after a loop

- Output message including number of CDs found at location (Total)
- Close file

Q5 (b): VB.NET**A StreamReader() solution:**

```
Sub OutPutLocationList()  
  
    Dim Total As Integer  
    Dim FileData As String  
    Dim ObjReader As IO.StreamReader  
    ObjReader = New IO.StreamReader("C:\MyMusic.txt")  
    Dim CDLocation As String  
    Dim CDTitle As String  
    Dim CDArtist As String  
  
    Total = 0  
    Console.WriteLine("Input location to search ")  
    CDSearch = Console.ReadLine  
    Do While ObjReader.Peek <> -1  
        FileData = ObjReader.ReadLine()  
        If Right(FileData, 8) = CDLocation Then  
            CDTitle = Left(FileData, 40)  
            CDArtist = Mid(FileData, 41, 40)  
            Console.WriteLine(CDTitle & "    " & CDArtist)  
            Total = Total + 1  
        End If  
    Loop  
    Console.WriteLine(Total & " CDs were found")  
    ObjReader.Close()  
End Sub
```

A legacy FileOpen() solution:

```
Sub OutPutLocationList()  
  
    Dim Total As Integer  
    Dim FileData As String  
    FileOpen (1, "C:\MyMusic.txt", OpenMode.Input)  
    Dim CDLocation As String  
    Dim CDTitle As String  
    Dim CDArtist As String  
  
    Total = 0  
    Console.WriteLine("Input location to search ")  
    CDSearch = Console.ReadLine  
    Do While NOT EOF(1)  
        Input(1, FileData)  
        If Right(FileData, 8) = CDSearch Then  
            CDTitle = Left(FileData, 40)  
            CDArtist = Mid(FileData, 41, 40)  
            Console.WriteLine(CDTitle & "    " & CDArtist)  
            Total = Total + 1  
        End If  
    Loop  
    Console.WriteLine(Total & " CDs were found")  
    FileClose(1)  
End Sub
```

Q5 (b): Pascal

```

procedure OutputLocationList;
var
    FileData, CDLocation, CDTitle, CDArtist : String;
    CDFile : Textfile;
    Total : Integer;
Begin
    Total := 0;
    Writeln('Input location to search ');
    Readln(CDSearch);
    AssignFile(CDFile, 'MyMusic.txt');
    Reset(CDFile);

    While not eof(CDFile) Do
    Begin
        readln(CDFile, FileData);
        If copy(FileData, 80, 8) = CDSearch Then
        Begin
            CDTitle := copy(FileData, 1, 40);
            CDArtist := copy(FileData, 41, 40);
            Writeln(CDTitle + ' : ' + CDArtist);
            Total := Total + 1;
        End;
    End;

    Writeln(Total + ' CDs were found');
    CloseFile(CDFile);

End

```

Q5 (b): Python

```

#total : Integer
#CDSearch, LineOfText, LineString : String

Def OutPutLocationList():
    FileHandle = open("MyMusicPy.TXT", "r")
    total = 0
    CDSearch = input("Enter location to search")
    LineOfText = FileHandle.readline()
    while len(LineOfText) > 0:
        LineString = LineOfText[80:87]    #extract last 8 characters
        (location)
        if LineString == CDSearch:
            total = total+1
            CDTitle = LineOfText[0:39]    #extract CD title
            CDArtist = LineOfText[40:79]  #extract CD artist
            print(CDTitle + " : " + CDArtist)
            LineOfText = FileHandle.readline() #read next line
        print("There are " + str(total) + " in that location")
    FileHandle.close()

```

178. O/N 15/P22/Q5/a(ii)

(ii) To sort / to order/put in ascending order the items (in the array)

[1]

179. M/J 15/P21/Q5/a,d,e

5 (a) • 1D Array // List [1]

• INTEGER [1]

(d) **OPENFILE** "DISCOUNT_DATES" FOR **WRITE** / **WRITING** (1)
INPUT **NextDate** (1)
WHILE NextDate <> "XXX"
 INPUT Discount
 NextLine = **CONCAT**(NextDate, " ", Discount) (1)
 WRITEFILE "DISCOUNT_DATES", NextLine
ENDWHILE (1)
OUTPUT "File now created"
CLOSEFILE [4]

(e) (i) Sensible Identifier + Data Type + Description (1 + 1 + 1)

For example:

ThisDate	STRING/DATE	date 'entered by user'
Found	BOOLEAN	flag to indicate ThisDate is 'present in the file'
NextLine	STRING	a single line 'from the text file'
NextDate	STRING/DATE	date 'from next line in the file'
NextDiscount	STRING	the discount value from NextLine
ThisMonth	INTEGER	the month part of the date (input or from file)
MyStreamReader	STREAMREADER	references DISCOUNT_DATES file

Reject 'generic' reserved words

Allow **one** instance variable to store output string(s)

Allow **one** instance of month/day/year number e.g. ThisMonth shown above [3]

(ii) *Mark as follows:*

Open file statement (1)
 File read statement for line text – NextLine (1)
 File close statement (1)

Input of the required date – ThisDate (1)

Isolate NextDate from NextLine (1)

Isolate NextDiscount from NextLine (1)

IF statement comparing the two dates (1)

Uses Boolean variable Found to flag when found (1)

Post/pre condition loop iterate through the file (1)

Test for EOF or 'found' (1)

Note: These must follow some correct logic to score ...

Output 'No discount on this date' **and** Output 'This is a discount date') (1)

Output (when date not found) 'Date not found' (1)

Accept 'any' identifier names [max 7]

APPENDIX

Programming Solutions

Question 1 (b) (ii)

Visual Basic ...

```
Dim RaceHours As Integer
Dim RaceMinutes As Integer
Dim RaceSeconds As Integer
Dim RaceTimeInSeconds As Integer
Dim PersonalBest As Integer

Console.Write("Time in hours ... ") : RaceHours = Console.ReadLine
Console.Write("Time in minutes... ") : RaceMinutes = Console.ReadLine
Console.Write("Time in seconds ... ")
RaceSeconds = Console.ReadLine
Console.Write("Personal best in seconds ... ")
PersonalBest = Console.ReadLine
RaceTimeInSeconds = RaceHours*60*60 + RaceMinutes*60 + RaceSeconds
Console.Write(RaceTimeInSeconds)
If RaceTimeInSeconds < PersonalBest Then
    Console.WriteLine("New personal best time")
Else
    If RaceTimeInSeconds = PersonalBest Then
        Console.WriteLine("Equals personal best time")
    Else
        Console.WriteLine("Below personal best")
    End If
End If
```

Python ...

```
# RaceHours          - Integer
# RaceMinutes        - Integer
# RaceSeconds        - Integer
# RaceTimeInSeconds  - Integer
# PersonalBest       - Integer

RaceHours = int(input("Time in hours ... "))
RaceMinutes = int(input("Time in minutes... "))
RaceSeconds = int(input("Time in seconds ... "))
PersonalBest = int(input("Personal best in seconds ... "))

RaceTimeInSeconds = RaceHours*60*60 + RaceMinutes*60 + RaceSeconds

if RaceTimeInSeconds < PersonalBest:
    print("New personal best time")
elif RaceTimeInSeconds == PersonalBest:
    print("Equals personal best time")
else:
    print("Below personal best")
```

Programming Solutions**Question 1 (b) (ii) – contd.*****Pascal ...***

```
var RaceHours      : Integer ;
var RaceMinutes    : Integer ;
var RaceSeconds    : Integer ;
var RaceTimeInSeconds : Integer ;
var PersonalBestTime : Integer ;

begin
  Writeln('Time in hours ... ') ; readln(RaceHours) ;
  Writeln('Time in minutes... ') ; readln(RaceMinutes) ;
  Writeln('Time in seconds ... ') ;
  readln(RaceSeconds) ;
  Writeln('Personal best in seconds ... ') ;

  Readln(PersonalBest) ;
  RaceTimeInSeconds := RaceHours*60*60 + RaceMinutes*60 + RaceSeconds ;
  Writeln(RaceTimeInSeconds) ;

  If RaceTimeInSeconds < PersonalBestTime Then
    Writeln('New personal best time')
  Else
    If RaceTimeInSeconds = PersonalBestTime Then
      Writeln('Equals personal best time')
    Else
      Writeln('Personal best time is unchanged) ;

  Readln;
End
```

Programming Solutions**Question 2 (d)****Visual Basic ...**

```
Dim Choice As Integer
Dim NoOfAttempts As Integer
CONST i = 3
Do
    Call DisplayMenu()
    NoOfAttempts = 0
    Do
        Console.WriteLine("Enter choice (1..4) ")
        Choice = Console.ReadLine
        NoOfAttempts = NoOfAttempts + 1
        Loop Until (Choice >= 1 And Choice <= 4) Or NoOfAttempts = i

        If Choice = 1 Then Call ReadFile()
        If Choice = 2 Then Console.WriteLine("Add customer code")
        If Choice = 3 Then Console.WriteLine("Search customer code")
    Loop Until Choice = 4

Sub DisplayMenu()
    Console.WriteLine()
    Console.WriteLine("1. Read customer file")
    Console.WriteLine("2. Add customer")
    Console.WriteLine("3. Search for a customer")
    Console.WriteLine("4. End")
    Console.WriteLine()
End Sub

Sub ReadFile()
    Console.WriteLine("Read file code")
End Sub
```

Python ...

```
def DisplayMenu():
    print()
    print("1. Read customer file")
    print("2. Add customer")
    print("3. Search for a customer")
    print("4. End")
    print()

def ReadFile():
    print("Read file code")

if __name__ == "__main__" :
    # Choice - Integer
    # NoOfAttempts - Integer

    Choice = 0
    while Choice !=4:
        DisplayMenu()
```

```

Choice = int(input"Enter choice (1..4) :")
NoOfAttempts = 1
while (Choice < 1 or Choice >4) and NoOfAttempts < 3:
    Choice = int(input"Enter choice (1..4) :")
    NoOfAttempts = 1
if Choice == 1:
    ReadFile()
elif Choice == 2:
    print("Add customer code")
elif Choice == 3:
    print("Print customer code")

```

Programming Solutions

Question 2 (d) – contd.

Pascal ...

```

var Choice          : Integer ;
var NoOfAttempts    : Integer ;
const i = 3 ;

procedure DisplayMenu ;
begin
    WriteLn();
    WriteLn('1. Read customer file') ;
    WriteLn('2. Add customer') ;
    WriteLn('3. Search for a customer');
    WriteLn('4. End') ;
    WriteLn() ;
End ;

Procedure ReadFile ;
begin
    WriteLn('Read file code');
End ;

begin

repeat
    DisplayMenu() ;
    NoOfAttempts := 0 ;
    repeat
        Writeln('Enter choice (1..4)') ; ReadLn(Choice) ;
        NoOfAttempts := NoOfAttempts + 1 ;
    Until ((Choice >= 1) And (Choice <= 4)) Or (NoOfAttempts = i);

    If Choice = 1 Then ReadFile() ;
    If Choice = 2 Then writeLn('Add customer code');
    If Choice = 3 Then WriteLn('Search customer code') ;
Until Choice = 4 ;

end.

```

Programming Solutions**Question 5 (ii)****Visual Basic ...**

```
Dim ThisDate As String : Dim NextDate As String
Dim FileString As String
Dim Found As Boolean

FileOpen(1, "D:DISCOUNT_DATES.txt", OpenMode.Input)
    or equivalent for a 'StreamReader' solutions

Console.WriteLine("Date to find (DD/MM/YYYY)..")
ThisDate = Console.ReadLine
Found = False
Do
    FileString = LineInput(1)
    NextDate = Left(FileString, 10)
    If NextDate = ThisDate Then
Found = True
' length is 15 when shows TRUE
If Len(FileString) = 15 Then
    Console.WriteLine("This is a discount date")
Else
    Console.WriteLine("No discount on this date")
End If
End If
Loop Until Found = True Or EOF(1)

FileClose(1)

If Found = False Then
    Console.WriteLine("Date not found")
End If
```

Python ...

```
MyFile = open("c:\DISCOUNT_DATES.txt", "r")
ThisDate = input("Next date ...(XXX to end)")

Found = 0
while Found == 0:
    NextLine = MyFile.readline()
    if not NextLine:
        break

    FileDate = NextLine[0:10]
    DiscountIndicator = NextLine[11:]

    if FileDate == ThisDate:
        Found = 1
        print (ThisDate, DiscountIndicator)

MyFile.close()
if Found == 0:
```

```
print ("This date was not found")
```

Note: Found could be Boolean to give:

```
Found = False
```

```
while not Found:
```

Programming Solutions

Question 5 (ii) – contd.

Pascal ...

```
var ThisDate    : String ;
var NextDate    : String ;
var TheFile     : Text   ;
var FileString  : String ;
var Found       : Boolean ;

begin
assign(TheFile, 'k:\DISCOUNT_DATES.txt') ;
reset(TheFile) ;

writeln('Date to find (DD/MM/YYYY)..') ;
readln(ThisDate) ;
Found := False ;

repeat
  readln(TheFile, FileString) ;
  NextDate := copy(FileString,1, 10) ;

  If NextDate = ThisDate then
    begin
      Found := True ;
      { length is 15 when shows TRUE }
      if length(FileString) = 15 then
        writeln('This is a discount date')
      else
        writeln('No discount on this date')
      end ;
    end

until Found = True or EOF(TheFile) ;

close(TheFile) ;

if Found = False then writeln('Date not found') ;
```

180. M/J 15/P22/Q5/a,d,e

5 (a) • 1D Array // List

[1]

• INTEGER

[1]

```

(d) OPENFILE "DISCOUNT_DATES" FOR WRITE / WRITING (1)
    INPUT NextDate (1)
    WHILE NextDate <> "XXX"
        INPUT Discount
        NextLine = CONCAT (NextDate, " ", Discount) (1)
        WRITEFILE "DISCOUNT_DATES", NextLine
    ENDWHILE (1)
    OUTPUT "File now created"
    CLOSEFILE

```

[4]

(e) (i) Sensible Identifier + Data Type + Description (1 + 1 + 1)

For example:

ThisDate	STRING/DATE	date 'entered by user'
Found	BOOLEAN	flag to indicate ThisDate is 'present in the file'
NextLine	STRING	a single line 'from the text file'
NextDate	STRING/DATE	date 'from next line in the file'
NextDiscount	STRING	the discount value from NextLine
ThisMonth	INTEGER	the month part of the date (input or from file)
MyStreamReader	STREAMREADER	references DISCOUNT_DATES file

Reject 'generic' reserved words

Allow **one** instance variable to store output string(s)

Allow **one** instance of month/day/year number e.g. ThisMonth shown above

[3]

(ii) *Mark as follows:*

Open file statement	(1)
File read statement for line text – NextLine	(1)
File close statement	(1)
Input of the required date – ThisDate	(1)
Isolate NextDate from NextLine	(1)
Isolate NextDiscount from NextLine	(1)
IF statement comparing the two dates	(1)
Uses Boolean variable Found to flag when found	(1)
Post/pre condition loop iterate through the file	(1)
Test for <u>EOF</u> or 'found'	(1)

Note: These must follow some correct logic to score ...

Output 'No discount on this date' and Output 'This is a discount date')	(1)
--	-----

Output (when date not found) 'Date not found'	(1)
---	-----

Accept 'any' identifier names

[max 7]

APPENDIX

Programming Solutions

Question 1 (b) (ii)

Visual Basic ...

```
Dim RaceHours As Integer
Dim RaceMinutes As Integer
Dim RaceSeconds As Integer
Dim RaceTimeInSeconds As Integer
Dim PersonalBest As Integer

Console.Write("Time in hours ... ") : RaceHours = Console.ReadLine
Console.Write("Time in minutes... ") : RaceMinutes = Console.ReadLine
Console.Write("Time in seconds ... ")
RaceSeconds = Console.ReadLine
Console.Write("Personal best in seconds ... ")
PersonalBest = Console.ReadLine
RaceTimeInSeconds = RaceHours*60*60 + RaceMinutes*60 + RaceSeconds
Console.Write(RaceTimeInSeconds)
If RaceTimeInSeconds < PersonalBest Then
    Console.WriteLine("New personal best time")
Else
    If RaceTimeInSeconds = PersonalBest Then
        Console.WriteLine("Equals personal best time")
    Else
        Console.WriteLine("Below personal best")
    End If
End If
```

Python ...

```
# RaceHours          - Integer
# RaceMinutes        - Integer
# RaceSeconds        - Integer
# RaceTimeInSeconds  - Integer
# PersonalBest       - Integer

RaceHours = int(input("Time in hours ... "))
RaceMinutes = int(input("Time in minutes... "))
RaceSeconds = int(input("Time in seconds ... "))
PersonalBest = int(input("Personal best in seconds ... "))

RaceTimeInSeconds = RaceHours*60*60 + RaceMinutes*60 + RaceSeconds

if RaceTimeInSeconds < PersonalBest:
    print("New personal best time")
elif RaceTimeInSeconds == PersonalBest:
    print("Equals personal best time")
else:
    print("Below personal best")
```


Programming Solutions**Question 1 (b) (ii) – contd.*****Pascal ...***

```
var RaceHours          : Integer ;
var RaceMinutes        : Integer ;
var RaceSeconds        : Integer ;
var RaceTimeInSeconds  : Integer ;
var PersonalBestTime   : Integer ;

begin
  Writeln('Time in hours ... ') ; readln(RaceHours) ;
  Writeln('Time in minutes... ') ; readln(RaceMinutes) ;
  Writeln('Time in seconds ... ') ;
  readln(RaceSeconds) ;
  Writeln('Personal best in seconds ... ') ;

  Readln(PersonalBest) ;
  RaceTimeInSeconds := RaceHours*60*60 + RaceMinutes*60 + RaceSeconds ;
  Writeln(RaceTimeInSeconds) ;

  If RaceTimeInSeconds < PersonalBestTime Then
    Writeln('New personal best time')
  Else
    If RaceTimeInSeconds = PersonalBestTime Then
      Writeln('Equals personal best time')
    Else
      Writeln('Personal best time is unchanged) ;

  Readln;
End
```

Programming Solutions**Question 2 (d)****Visual Basic ...**

```
Dim Choice As Integer
Dim NoOfAttempts As Integer
CONST i = 3
Do
    Call DisplayMenu()
    NoOfAttempts = 0
    Do
        Console.WriteLine("Enter choice (1..4) ")
        Choice = Console.ReadLine
        NoOfAttempts = NoOfAttempts + 1
        Loop Until (Choice >= 1 And Choice <= 4) Or NoOfAttempts = i

        If Choice = 1 Then Call ReadFile()
        If Choice = 2 Then Console.WriteLine("Add customer code")
        If Choice = 3 Then Console.WriteLine("Search customer code")
    Loop Until Choice = 4

Sub DisplayMenu()
    Console.WriteLine()
    Console.WriteLine("1. Read customer file")
    Console.WriteLine("2. Add customer")
    Console.WriteLine("3. Search for a customer")
    Console.WriteLine("4. End")
    Console.WriteLine()
End Sub

Sub ReadFile()
    Console.WriteLine("Read file code")
End Sub
```

Python ...

```
def DisplayMenu():
    print()
    print("1. Read customer file")
    print("2. Add customer")
    print("3. Search for a customer")
    print("4. End")
    print()

def ReadFile():
    print("Read file code")

if __name__ == "__main__" :
    # Choice - Integer
    # NoOfAttempts - Integer

    Choice = 0
    while Choice !=4:
        DisplayMenu()
```

```

Choice = int(input"Enter choice (1..4) :")
NoOfAttempts = 1
while (Choice < 1 or Choice >4) and NoOfAttempts < 3:
    Choice = int(input"Enter choice (1..4) :")
    NoOfAttempts = 1
if Choice == 1:
    ReadFile()
elif Choice == 2:
    print("Add customer code")
elif Choice == 3:
    print("Print customer code")

```

Programming Solutions

Question 2 (d) – contd.

Pascal ...

```

var Choice          : Integer ;
var NoOfAttempts    : Integer ;
const i = 3 ;

procedure DisplayMenu ;
begin
    WriteLn();
    WriteLn('1. Read customer file') ;
    WriteLn('2. Add customer') ;
    WriteLn('3. Search for a customer');
    WriteLn('4. End') ;
    WriteLn() ;
End ;

Procedure ReadFile ;
begin
    WriteLn('Read file code');
End ;

begin

repeat
    DisplayMenu() ;
    NoOfAttempts := 0 ;
    repeat
        Writeln('Enter choice (1..4)') ; ReadLn(Choice) ;
        NoOfAttempts := NoOfAttempts + 1 ;
    Until ((Choice >= 1) And (Choice <= 4)) Or (NoOfAttempts = i);

    If Choice = 1 Then ReadFile() ;
    If Choice = 2 Then writeLn('Add customer code');
    If Choice = 3 Then WriteLn('Search customer code') ;
Until Choice = 4 ;

end.

```

Programming Solutions

Question 5 (ii)

Visual Basic ...

```
Dim ThisDate As String : Dim NextDate As String
Dim FileString As String
Dim Found As Boolean

FileOpen(1, "D:DISCOUNT_DATES.txt", OpenMode.Input)
    or equivalent for a 'StreamReader' solutions

Console.WriteLine("Date to find (DD/MM/YYYY)..")
ThisDate = Console.ReadLine
Found = False
Do
    FileString = LineInput(1)
    NextDate = Left(FileString, 10)
    If NextDate = ThisDate Then
Found = True
' length is 15 when shows TRUE
If Len(FileString) = 15 Then
    Console.WriteLine("This is a discount date")
Else
    Console.WriteLine("No discount on this date")
End If
End If
Loop Until Found = True Or EOF(1)

FileClose(1)

If Found = False Then
    Console.WriteLine("Date not found")
End If
```

Python ...

```
MyFile = open("c:\DISCOUNT_DATES.txt", "r")
ThisDate = input("Next date ...(XXX to end)")

Found = 0
while Found == 0:
    NextLine = MyFile.readline()
    if not NextLine:
        break

    FileDate = NextLine[0:10]
    DiscountIndicator = NextLine[11:]

    if FileDate == ThisDate:
        Found = 1
        print (ThisDate, DiscountIndicator)

MyFile.close()
if Found == 0:
```

```
print ("This date was not found")
```

Note: Found could be Boolean to give:

```
Found = False
```

```
while not Found:
```

Programming Solutions

Question 5 (ii) – contd.

Pascal ...

```
var ThisDate    : String ;
var NextDate    : String ;
var TheFile     : Text   ;
var FileString  : String ;
var Found       : Boolean ;

begin
assign(TheFile, 'k:\DISCOUNT_DATES.txt') ;
reset(TheFile) ;

writeln('Date to find (DD/MM/YYYY)..') ;
readln(ThisDate) ;
Found := False ;

repeat
  readln(TheFile, FileString) ;
  NextDate := copy(FileString,1, 10) ;

  If NextDate = ThisDate then
    begin
      Found := True ;
      { length is 15 when shows TRUE }
      if length(FileString) = 15 then
        writeln('This is a discount date')
      else
        writeln('No discount on this date')
      end ;
    end

until Found = True or EOF(TheFile) ;

close(TheFile) ;

if Found = False then writeln('Date not found') ;
```

181. M/J 15/P23/Q4/b(i,iii,iv),c

- (b) (i) `OPENFILE "DISPENSERS" FOR WRITE` (1 mark)
- REPEAT** (1 mark)
- `OUTPUT "Enter dispenser code (XXXXX to end)"`
- `INPUT DispenserCode`
- `IF DispenserCode <> "XXXXX"`
- `THEN`
- `OUTPUT "Enter bank code ..."`
- `INPUT BankCode`
- `LineString ← CONCAT(DispenserCode, "▼",BankCode)` (1 mark)
- `// now write the new line to the file`
- WRITEFILE** ("DISPENSERS"), LineString (1 mark)
- `ENDIF`
- `UNTIL DispenserCode = "XXXXX"` (1 mark)
- CLOSE** ("DISPENSERS") // **CLOSEFILE** (1 mark)
- `OUTPUT "DISPENSERS file now created"` [6]
- (iii) data of the existing 15 dispensers will be lost/overwritten [1]
- (iv) Append // Illustrated with program code statement [1]

(c) Mark as follows:

- Variables declared/commented (at least X2) (1 mark)
- Input of 'ThisBank' with prompt (1 mark)

- File open statement (1 mark)
- File mode is 'Input' (1 mark)
- File close

- Loop (Not a FOR loop) (1 mark)
- Until all records considered

- Isolate LineBankCode (1 mark)
- Isolate LineDispenserCode

- Count initialised (1 mark)
- Count incremented (1 mark)

- Output – List of dispenser codes (1 mark)
- Output – dispenser count (1 mark)

[max 10]

Visual Basic ...

```

Dim DispenserRecord As String
Dim DispenserCode As String : Dim Bank As String
Dim DispenserCount As Integer
Dim ThisBank As String
FileOpen(1, "C:\DISPENSERS.txt", OpenMode.Input)

Console.WriteLine()
Console.Write("Which bank ..(Three digit code)? ")
ThisBank = Console.ReadLine

DispenserCount = 0
Do
    DispenserRecord = LineInput(1)
    DispenserCode = Left(DispenserRecord, 5)
    Bank = Mid(DispenserRecord, 7, 3)

    If Bank = ThisBank Then
        DispenserCount = DispenserCount + 1
        Console.WriteLine(DispenserCode)
    End If
Loop Until EOF(1)
FileClose(1)

Console.WriteLine()
Console.WriteLine("There are " & DispenserCount & " dispensers
for this bank")

```

Python ...

```

# DispenserLine      - String
# DispenserCode      - String
# Bank               - String
# DispenserCount     - Integer
# ThisBank           - String

MyFile = open("c:\DISPENSERS.txt", "r")

ThisBank = input("Which bank ..(Three digit code)? ")

DispenserCount = 0
while 1:
    DispenserLine = MyFile.readline()
    if not DispenserLine:
        break
    DispenserCode = DispenserLine[0:5]
    # slices chars 0,1,2,3,4
    Bank = DispenserLine[6:9] # slices chars 6,7,8

    if Bank == ThisBank:
        DispenserCount = DispenserCount + 1
        print(DispenserCode)

MyFile.close()
print
print("There are " + str(DispenserCount)
      " dispensers for this bank")

```

Pascal ...

```

var DispenserRecord   : String ;
var DispenserCode     : String ;
var Bank              : String ;
var DispenserCount    : Integer ;
var ThisBank          : String ;
var TheFile           : Text ;

begin
assign(TheFile, 'K:\DISPENSERS.txt') ;
reset(TheFile) ;

WriteLn() ;
Write('Which bank ..(Three digit code)? ') ;
Readln(ThisBank) ;
C
DispenserCount := 0 ;
repeat
    readln(TheFile, DispenserRecord) ;
    DispenserCode := Copy(DispenserRecord,1, 5) ;
    Bank := copy(DispenserRecord, 7, 3) ;

    If Bank = ThisBank Then
        begin
            DispenserCount := DispenserCount + 1 ;

```



```
        Writeln(DispenserCode)
    end ;

    until EOF(TheFile) ;
    close(TheFile) ;

writeln() ;
writeln('Dispenser count: ', DispenserCount) ;

readln ;
end.
```

182. M/J 15/P23/Q5/a

- 5 (a) (i)**
- Set of data items have a common name (1 mark)
 - Items are referenced using a subscript/index (1 mark)
 - Accept: all data items are of the same data type (1 mark) [max 2]
- (ii)** 24 [1]
- (iii)**
- The total number of amplifiers 'produced' by workers 1, 2 and 3/three workers (1 mark)
 - on day 2_ (1 mark) [2]